

Examination 解説

細川 寛晃

問題概要

- N 人の学生が数学と情報の試験を受ける
 - 数学は S_i 点、情報は T_i 点
- 数学、情報、合計点のすべてで合格基準を超えていれば合格
- Q 回のクエリに答える:
 - 「数学が X_j 点以上かつ情報が Y_j 点以上かつ合計点が Z_j 点以上の学生(合格者)は何人いるか？」

小課題1(2点)

- $N, Q \leq 3000$
- N, Q が小さい

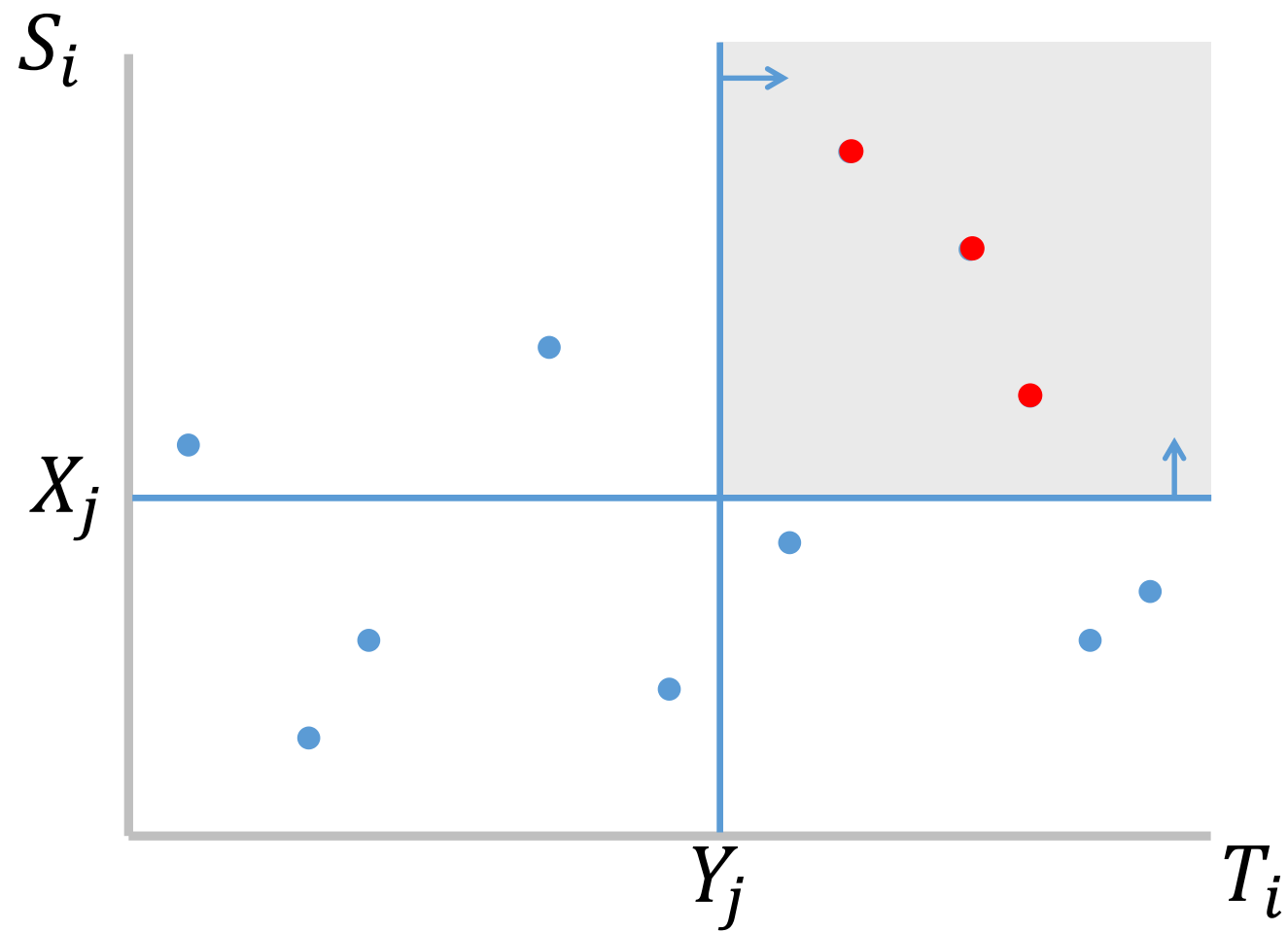
小課題1解法

- N, Q が小さいので愚直に各クエリ・各学生に対して合否判定をしても間に合う
- 判定は $O(1)$ のでできるので $O(NQ)$
- 計2点

小課題2(20点)

- $N, Q \leq 10^5$
- $S_j, T_j, X_j, Y_j \leq 10^5$
- $Z_j = 0$
- 合計点は合否に影響しない(無視できる)
 - 全員合計点は0点以上のため
- 数学が X_j 点以上、情報が Y_j 点以上の人数を数えたい

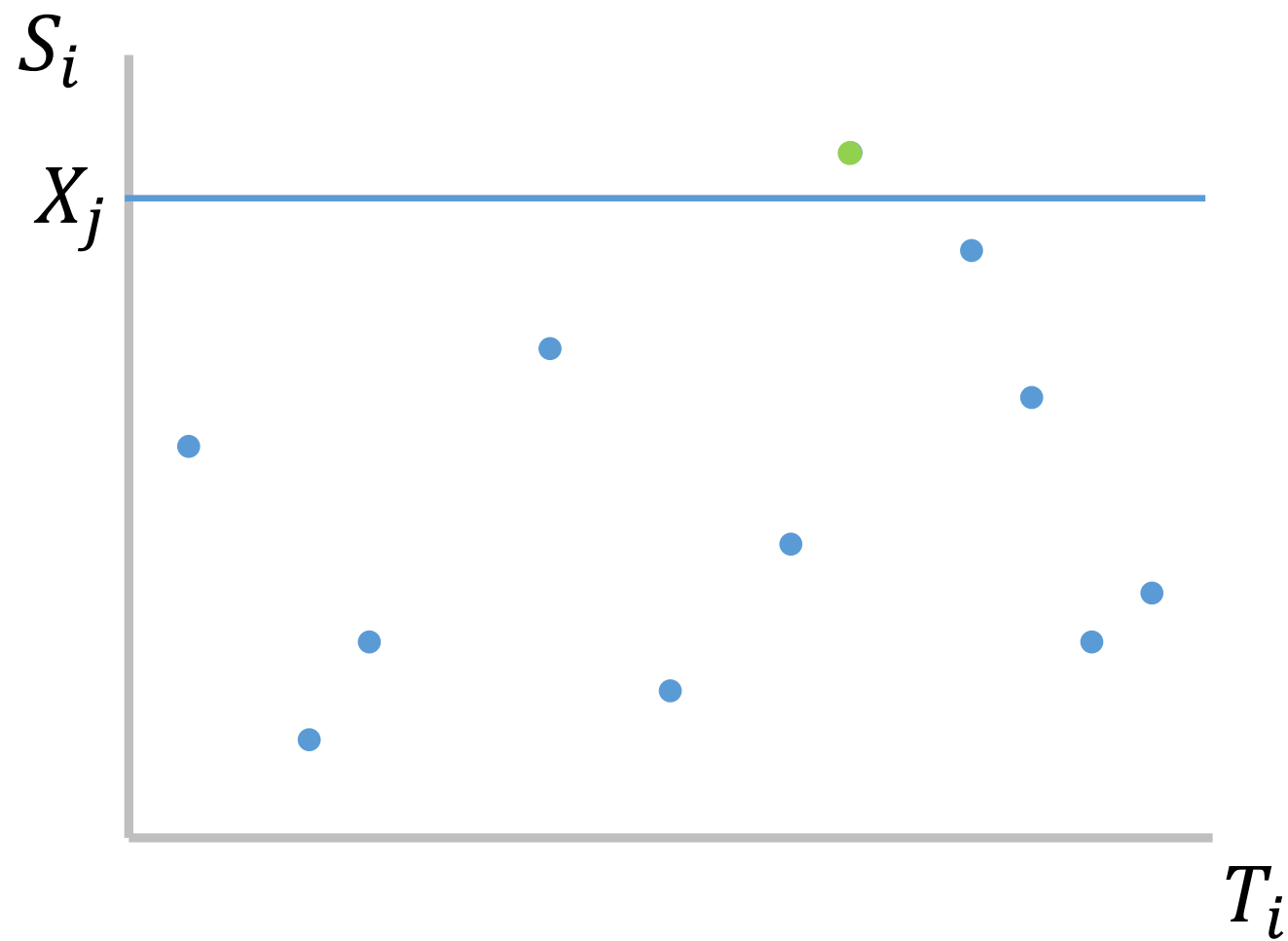
イメージ



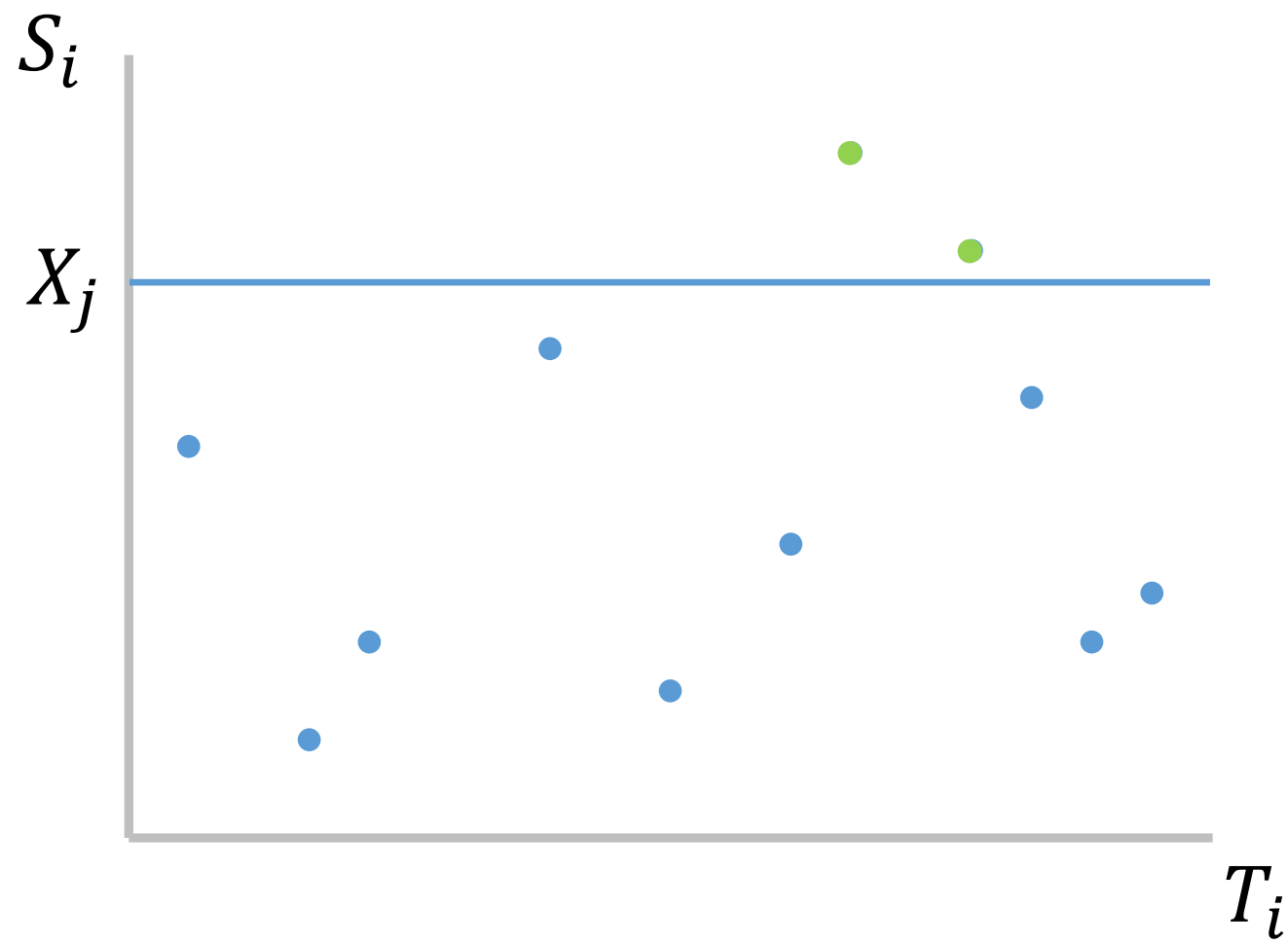
小課題2(20点)

- クエリを x_j の大きい順にソートして、数学の合格基準をだんだん下げていくことを考える

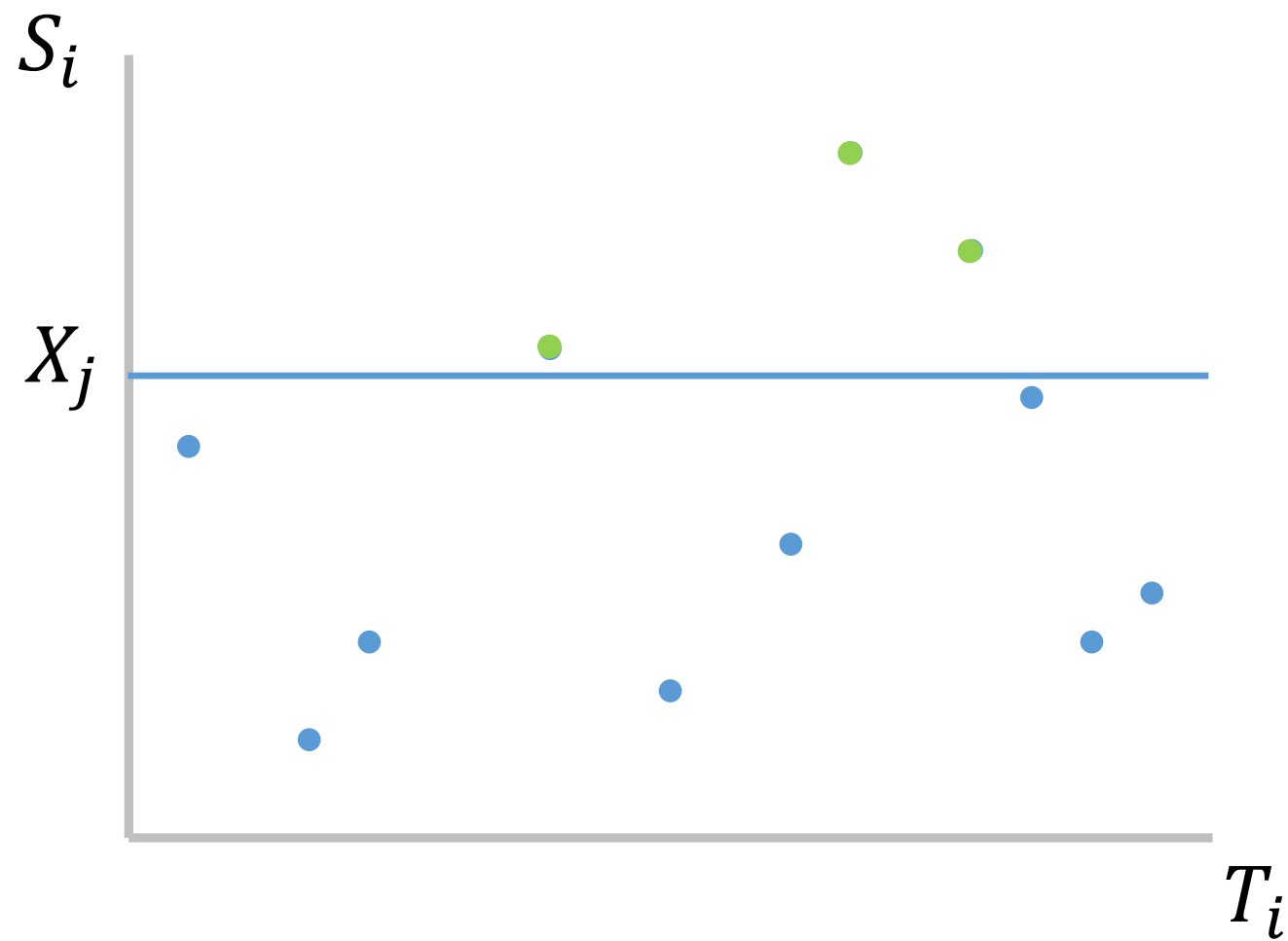
イメージ



イメージ



イメージ



小課題2(20点)

- 新たに数学の合格基準を満たした学生について、 T_i の値を記録する
 - 数列 $b_1, b_2, \dots$ (初期値はすべて0) に対し、 b_{T_i} に1を足す
 - Y_j 以上の値がいくつあるかを求める
 - $b_{Y_j}, b_{Y_j+1}, \dots$ の和を求める
- の二つの操作ができればよい
- BIT(Binary Indexed Tree)を使って高速に解ける

小課題2解法

- X_j の大きい順にクエリをソート
- 数学が X_j 点を超えた学生から順にBITの T_i のところに1を足していく
- クエリではBITで $[Y_j, \infty)$ のところの和を求めて答えればよい
- 計算量は $O((N + Q) \log \max(\text{得点}))$
- 計22点

小課題3(21点)

- $N, Q \leq 10^5$
- $S_j, T_j, X_j, Y_j \leq 10^5$
- $Z_j \leq 2 \times 10^5$
- 合計点も合否に影響してくる

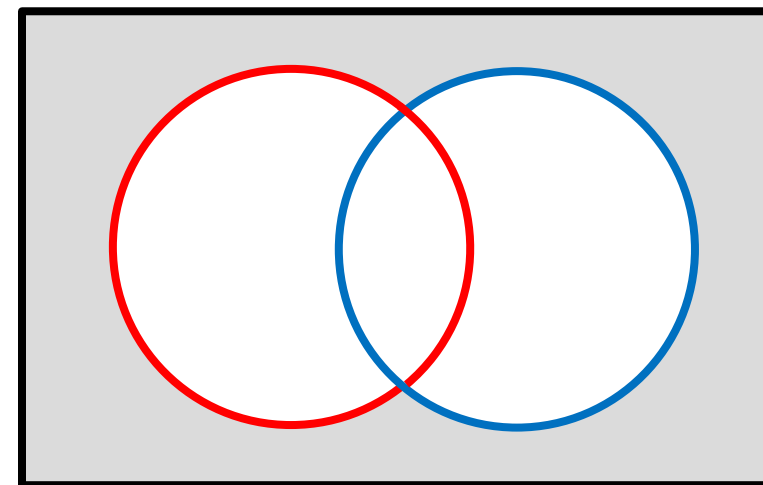
小課題3(21点)

- $X_j + Y_j \geq Z_j$ のときは合計点の制約を無視できる
→そのようなクエリには小課題2の解法をそのまま使うことにする
- 問題は $X_j + Y_j < Z_j$ のとき
- 合計点が Z_j 点以上の学生のうち、数学と情報も合格基準を満たしている人数の数を考える

少し考えてみる

合計点の合格基準を満たしている学生のうち

- ○内: 数学の合格基準を満たしている
 - ○内: 情報の合格基準を満たしている
 - ■: 数学・情報ともに合格基準を満たしていない
-
- ○・○の人数は小課題2のようにソートとBITで求めよう
 - 合計点の合格基準を下げていき、新たに合格基準を満たした学生の得点をBITに記録していく
 - ■の人数がわかれば両方合格基準を満たしている人数も求まって幸せ

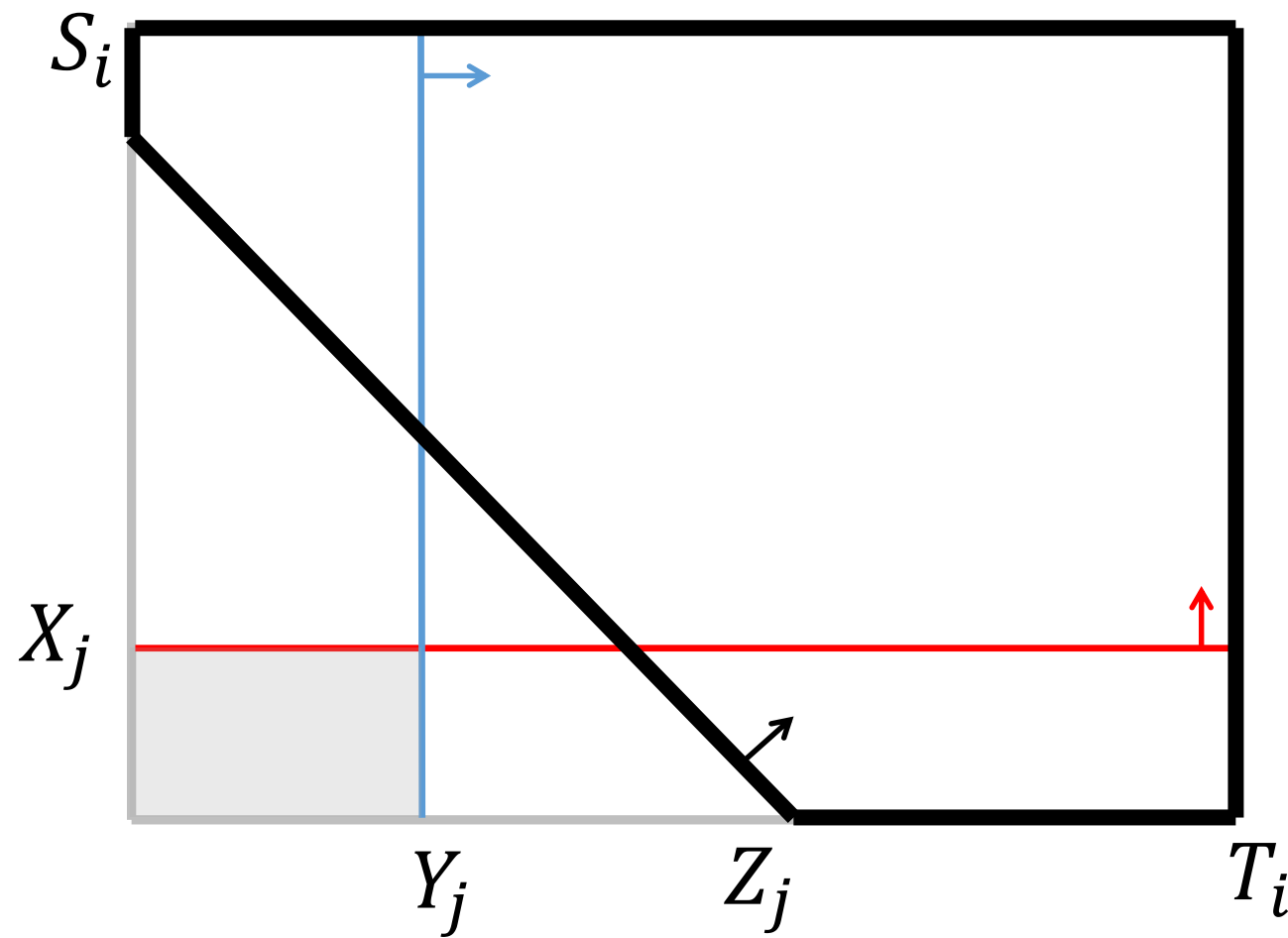


よく考えてみる

- 数学が X_j 点未満、情報が Y_j 点未満なら合計点は $X_j + Y_j$ 点未満
 - $S_i < X_j, T_i < Y_j \Rightarrow S_i + T_i < X_j + Y_j$
- ここで $X_j + Y_j < Z_j$ のときを考えているのでこのような学生はそもそも合計点の基準を満たしていない

→ ■ の人数は0人！

イメージ



■ は黒枠(合計点の合格基準を満たしている領域)内に含まれていない

小課題3解法

- $X_j + Y_j \geq Z_j$ であるクエリは小課題2と同じ
- $X_j + Y_j < Z_j$ であるクエリは Z_j の大きい順にソート
- 2つのBITを用意
 - 数学の得点を記録する用
 - 情報の得点を記録する用
- 合計点が Z_j 以上の学生について、2つのBITそれぞれに得点を記録していく

小課題3解法

- クエリでは
(数学合格者数) + (情報合格者数) - (それまでに追加した学生数)
を答えればよい
- 計算量は小課題2解法と同じ
- 計43点

小課題4(57点)

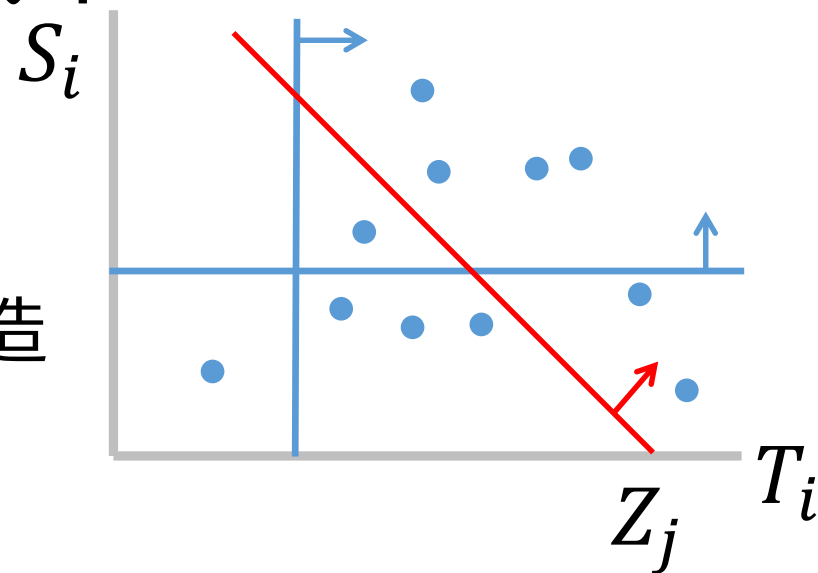
- 点数の値が大きい
- 点数について座標圧縮をすれば実質小課題3と同じ
- 計算量 $O((N + Q) \log N)$
- 計100点

別解:二次元セグメント木

- 合計点の合格基準を下げていくとき、「数学が x_j 点以上、情報が y_j 点以上である学生の数」を求められればよい
 - 下図赤線を左に動かしていく

- 次の2つのクエリが高速に処理できればよい:
 - 座標平面に点を追加
 - 矩形範囲の点の個数を求める

→セグメント木のノードに適切なデータ構造をのせればできる



方法1:セグメント木 on セグメント木

- セグ木のそれぞれのノードがセグ木をもつ
- 全体のセグ木上で、横の範囲に対応するノードまで潜る
- そのノードのセグ木上で、縦の範囲に対応するノードに対し操作する
- 子の変更を親に伝播する

0			
0		0	
0	0	0	0

0			
0		0	
0	0	0	0

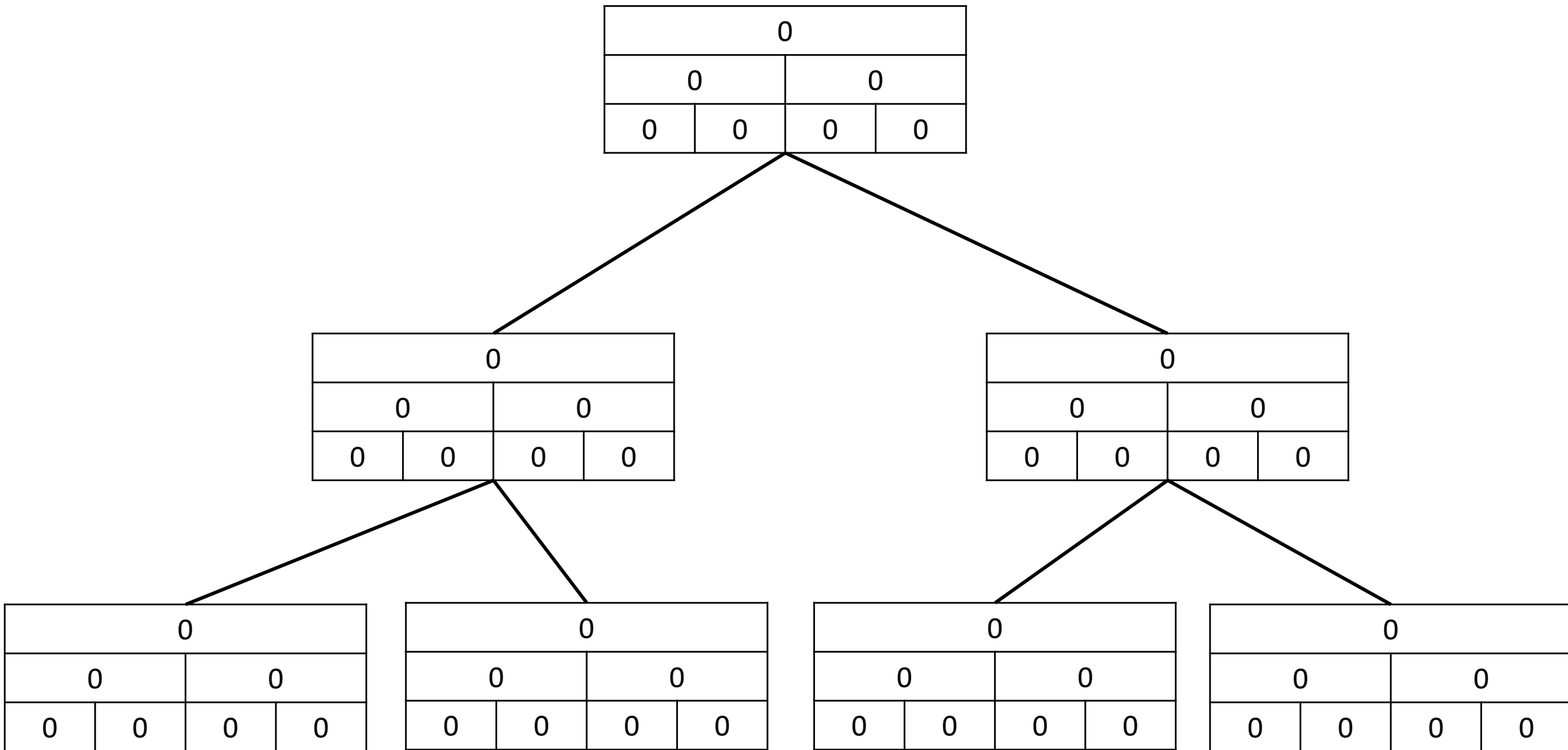
0			
0		0	
0	0	0	0

0			
0		0	
0	0	0	0

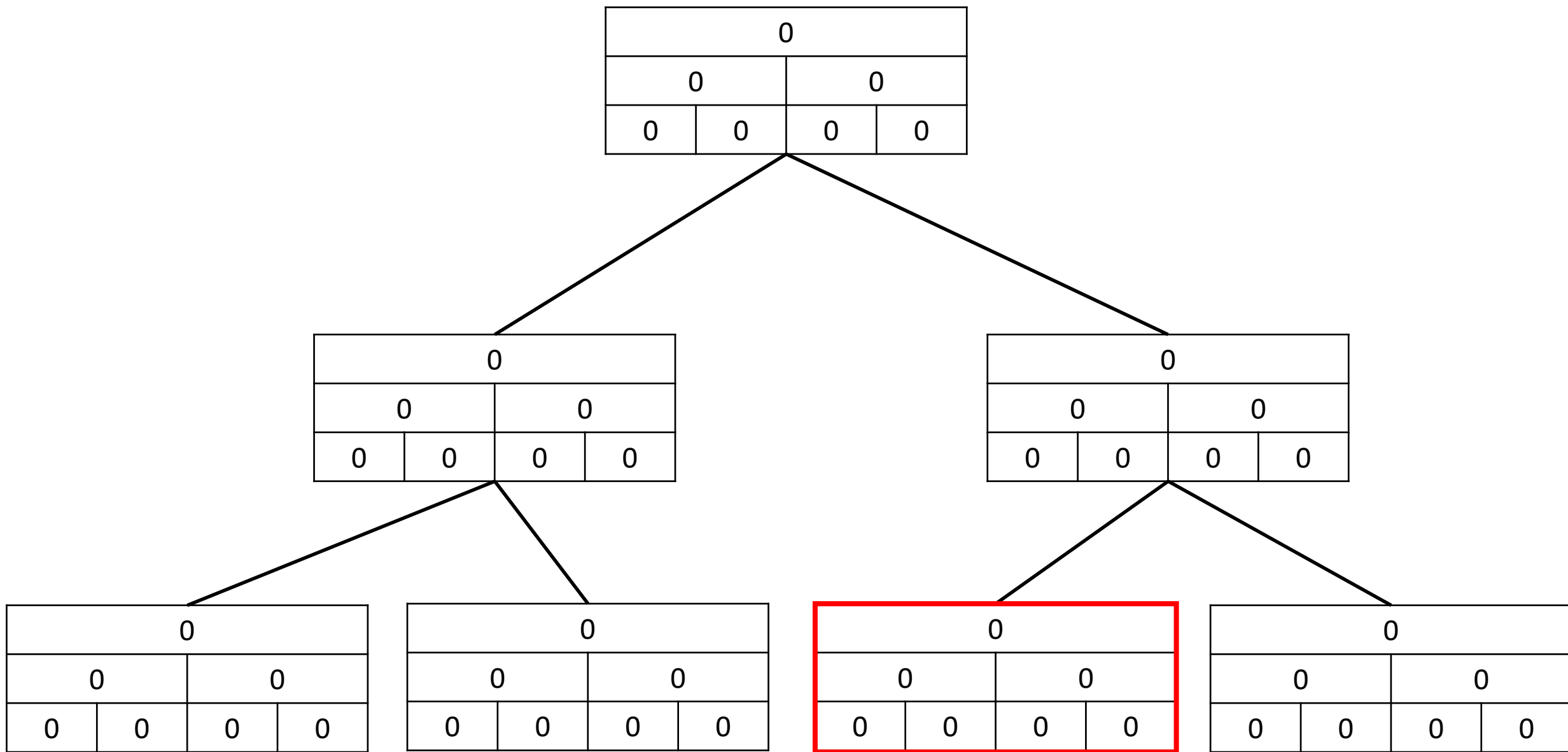
0			
0		0	
0	0	0	0

0			
0		0	
0	0	0	0

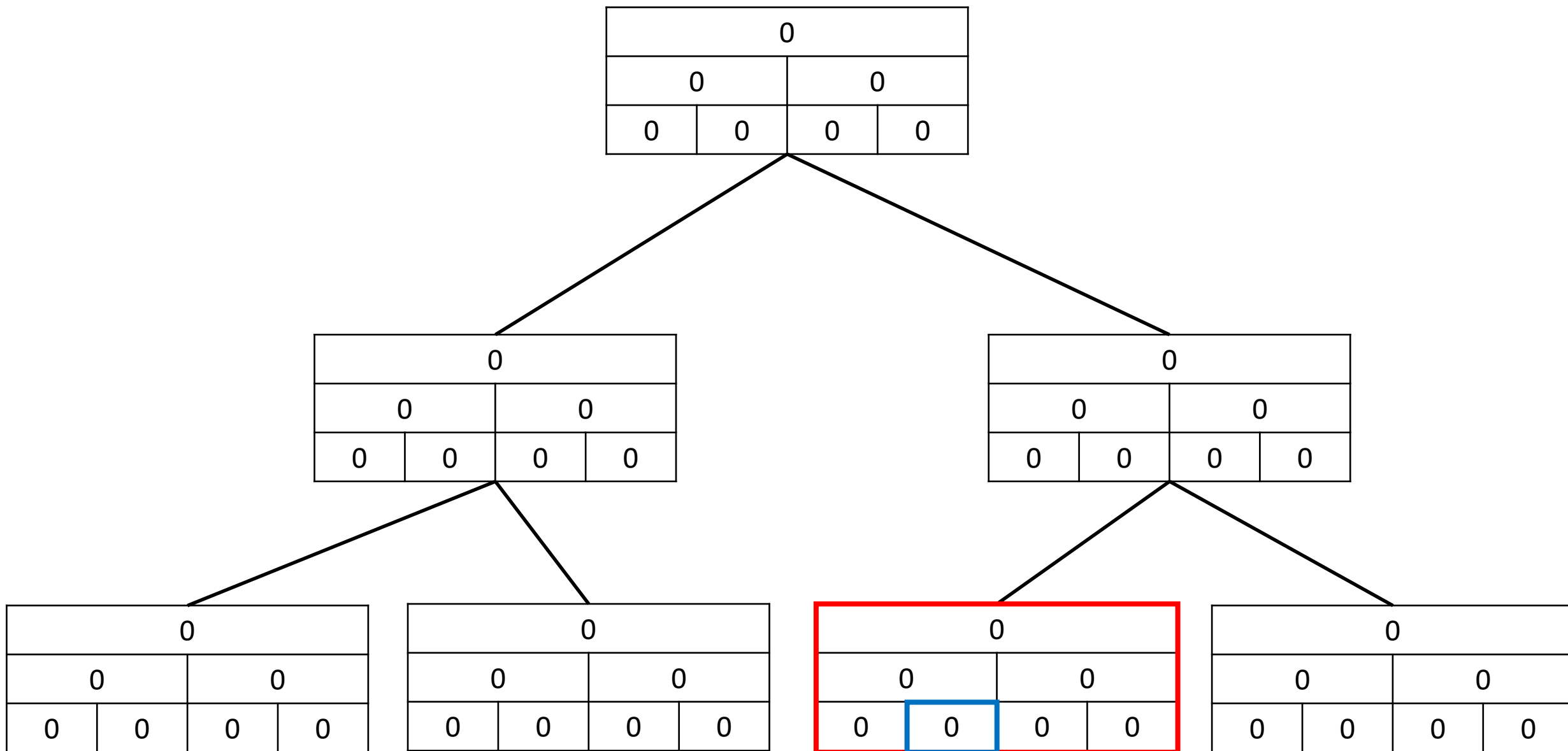
0			
0		0	
0	0	0	0



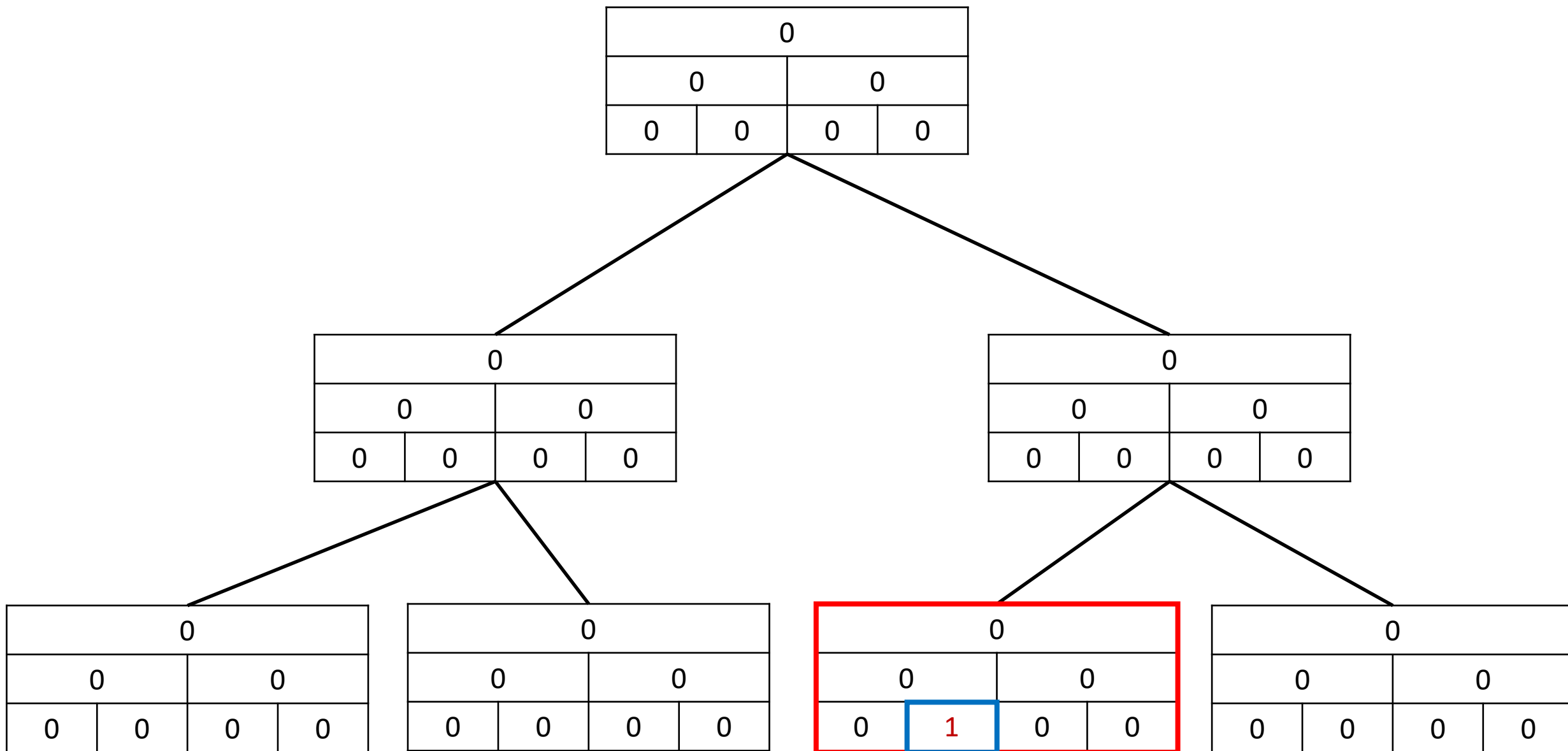
座標(2,1)に点を追加



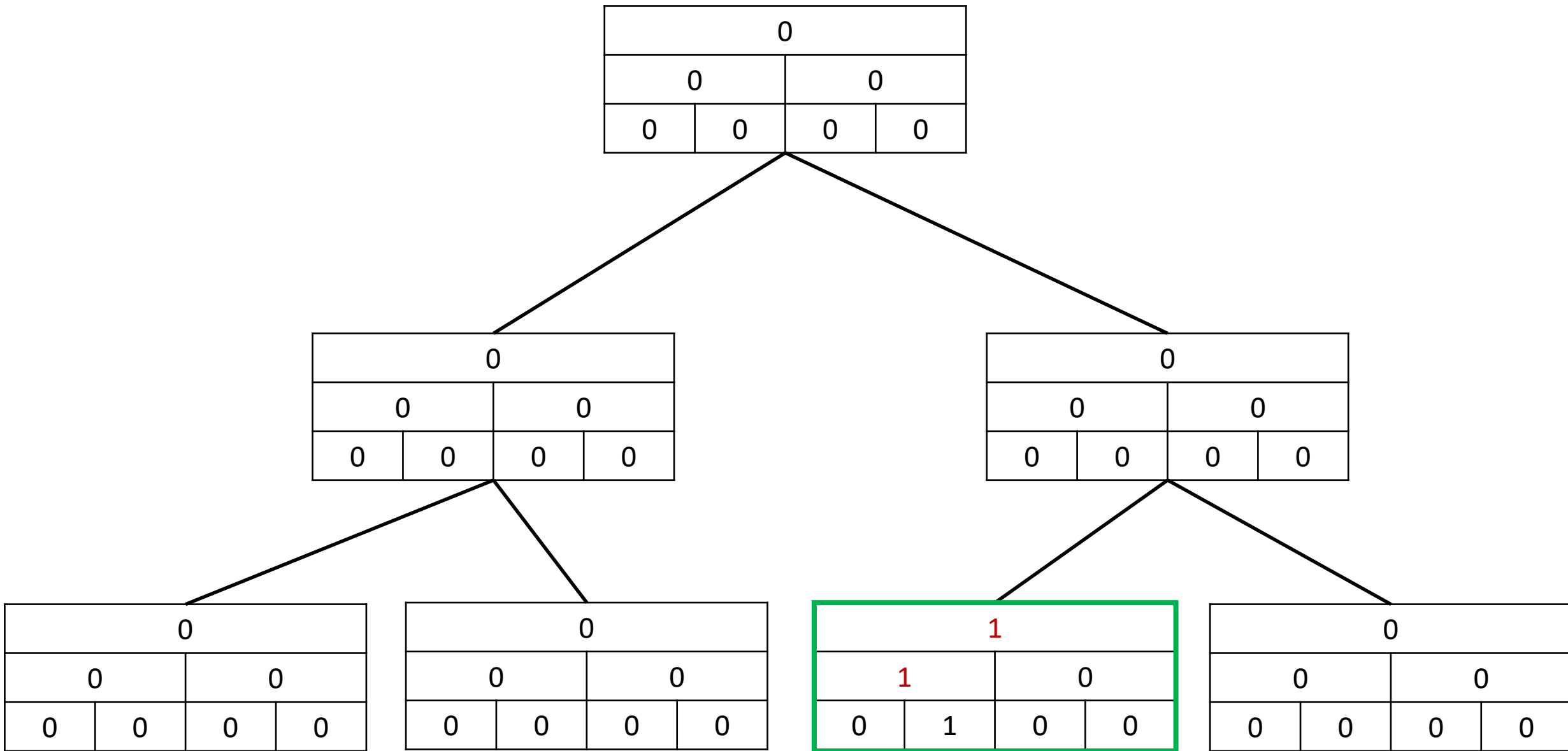
座標(2,1)に点を追加



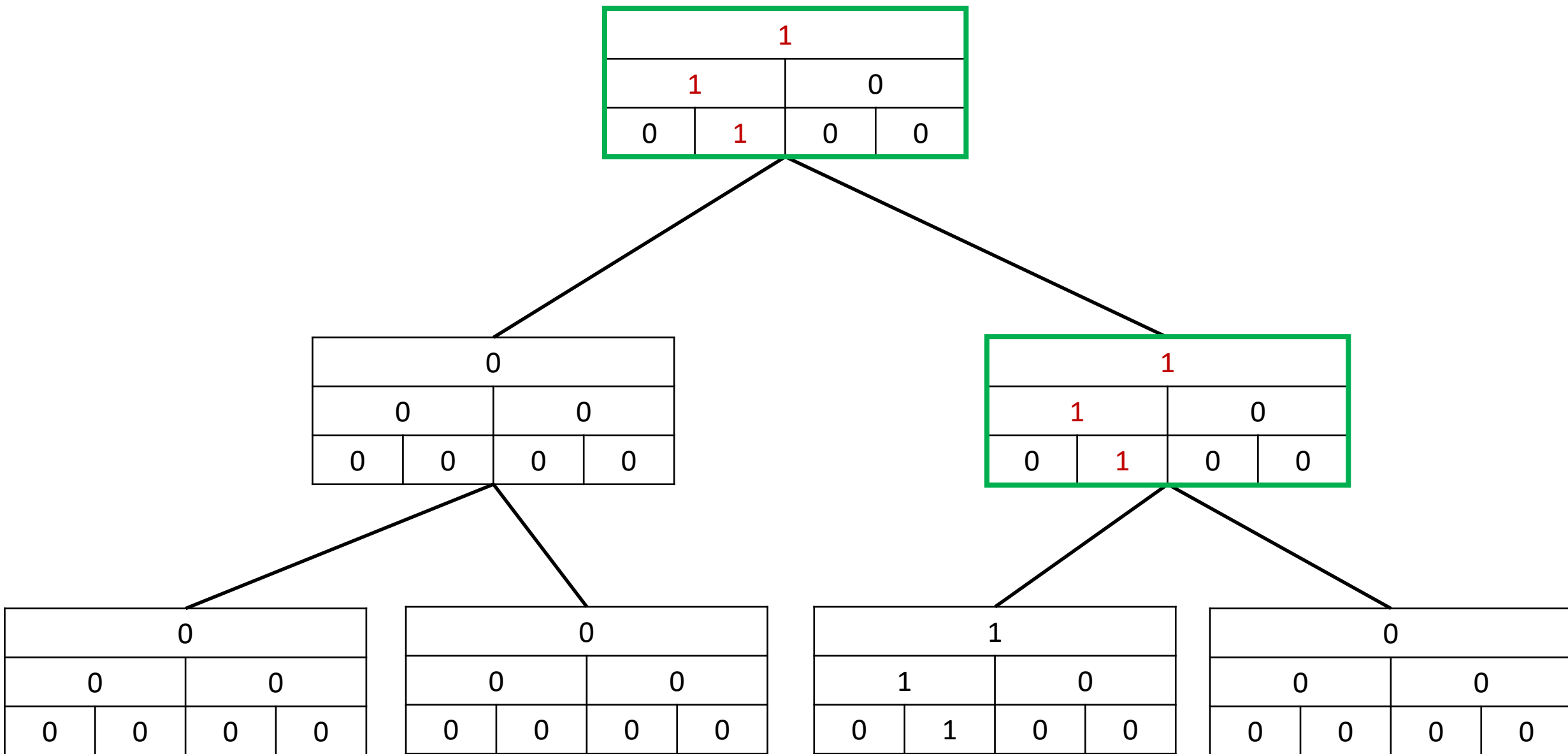
座標(2,1)に点を追加



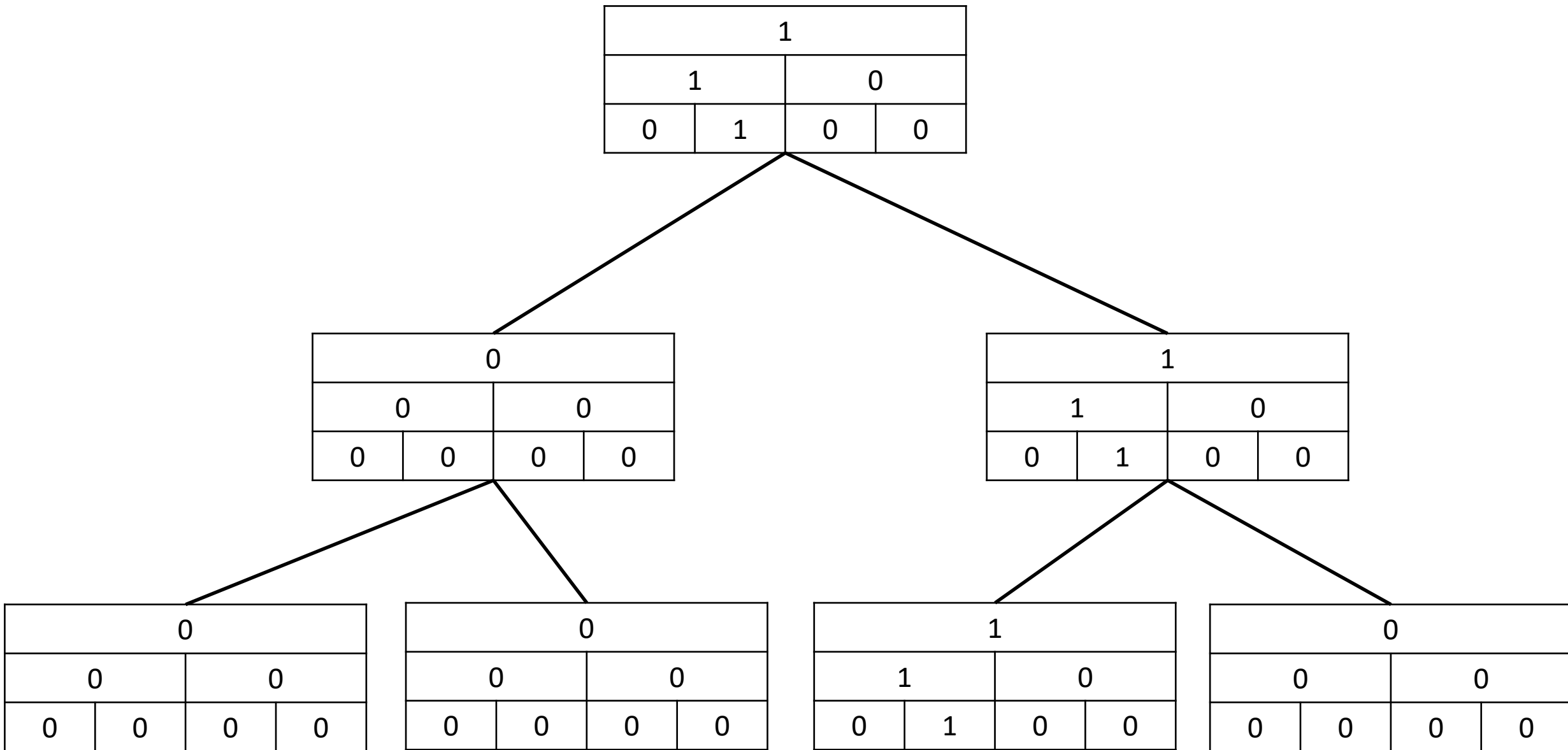
座標(2,1)に点を追加



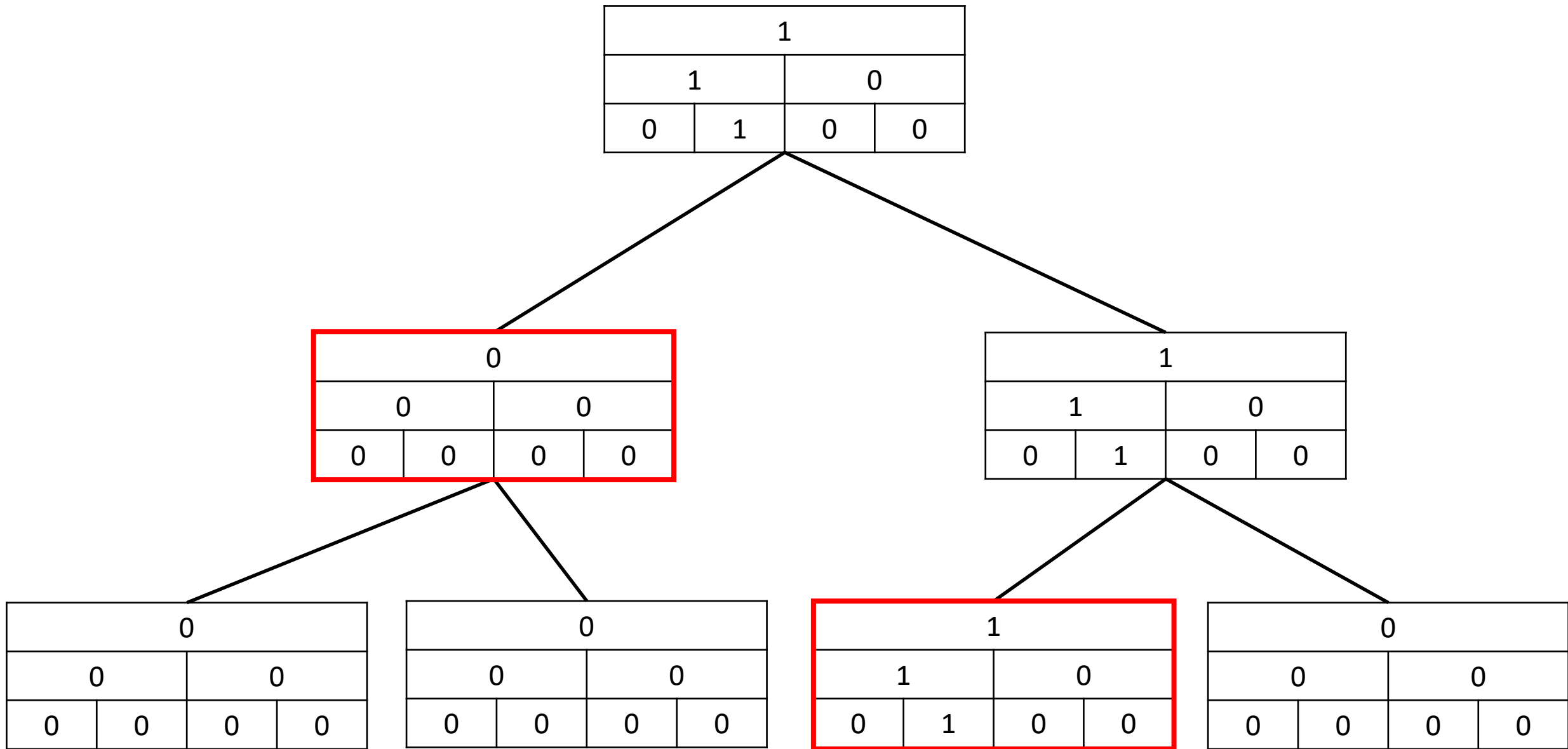
座標(2,1)に点を追加



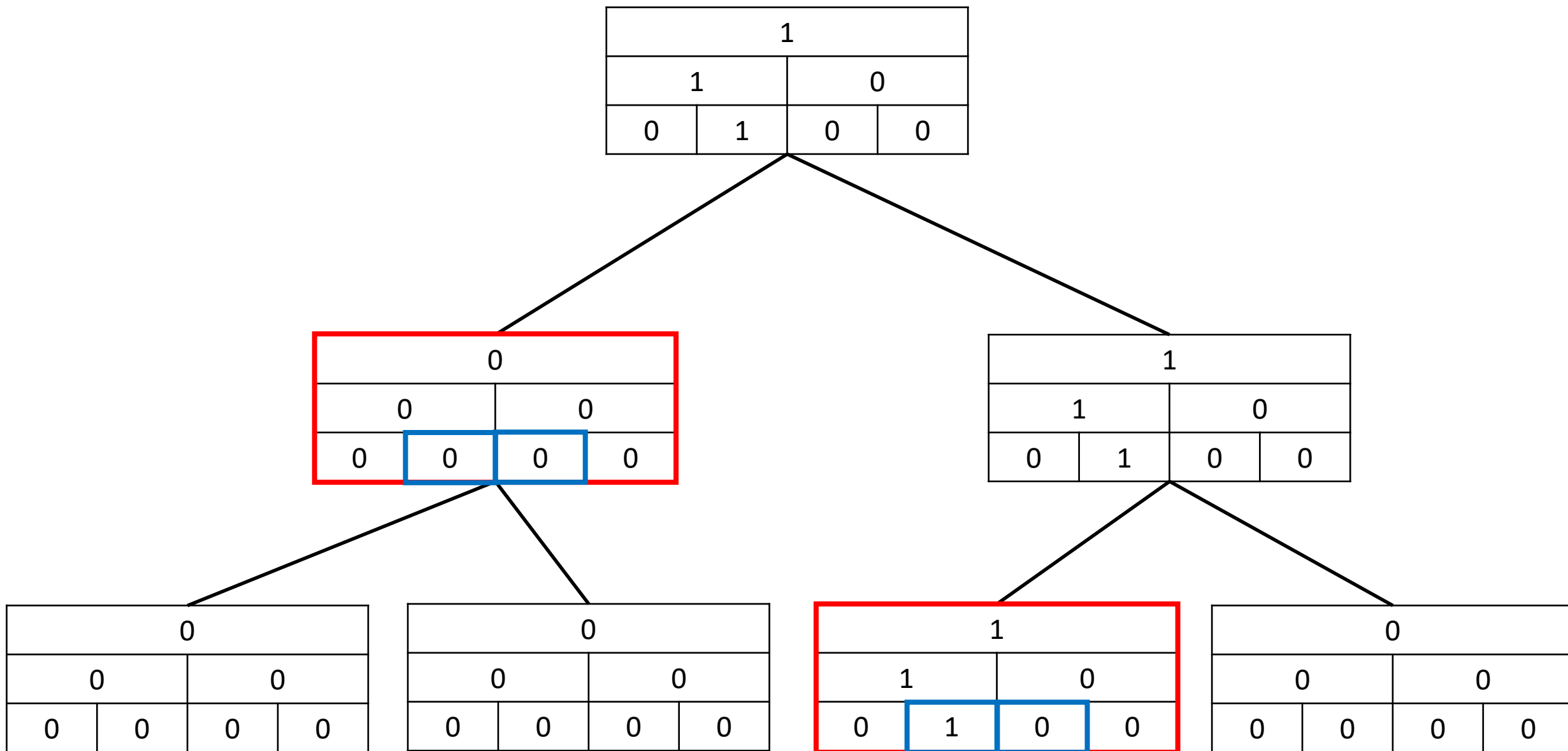
座標(2,1)に点を追加



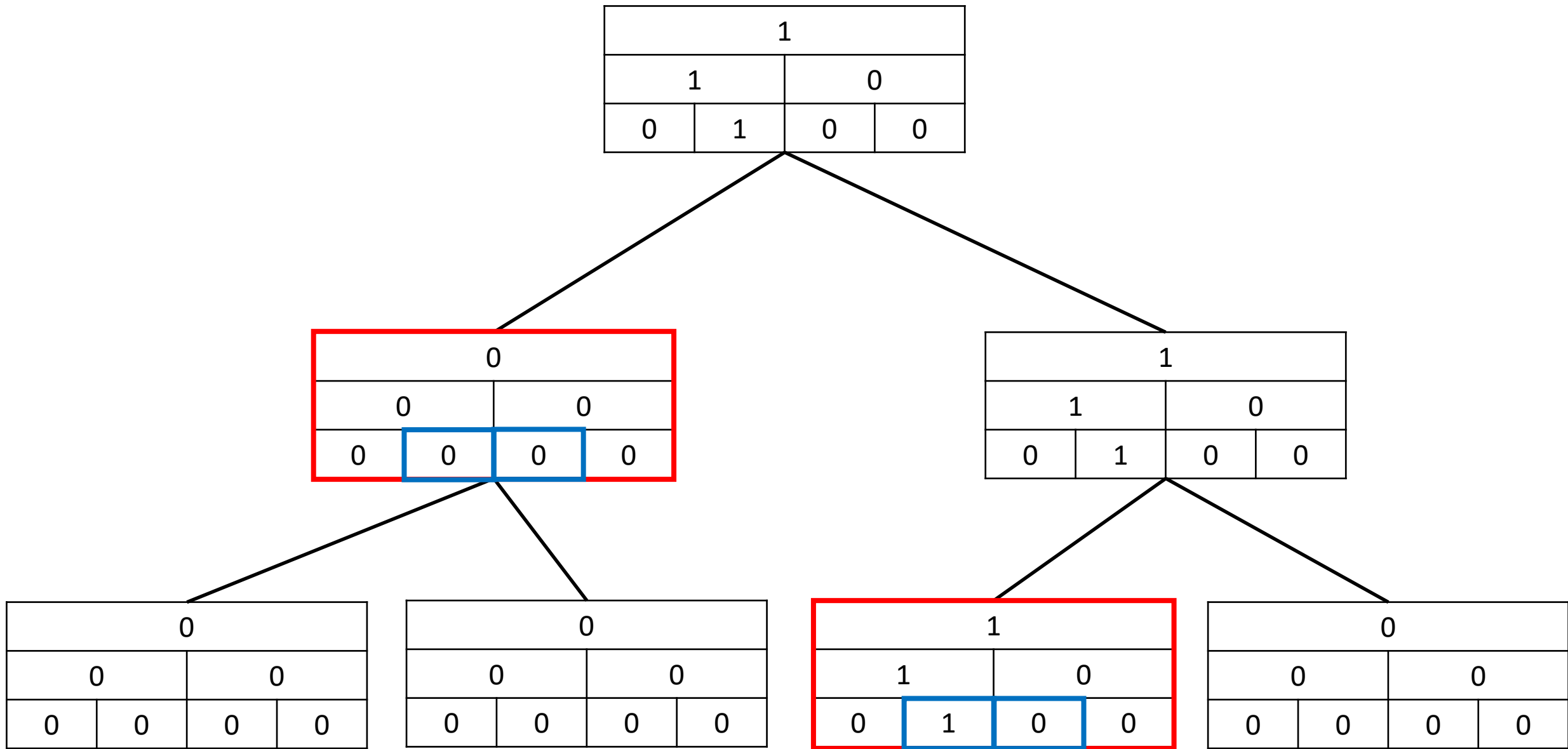
範囲 $[0,3) \times [1,3)$ 内の点の個数を求める



範囲 $[0,3) \times [1,3)$ 内の点の個数を求める



範囲 $[0,3) \times [1,3)$ 内の点の個数を求める



範囲 $[0, 3) \times [1, 3)$ 内の点の個数を求める→1

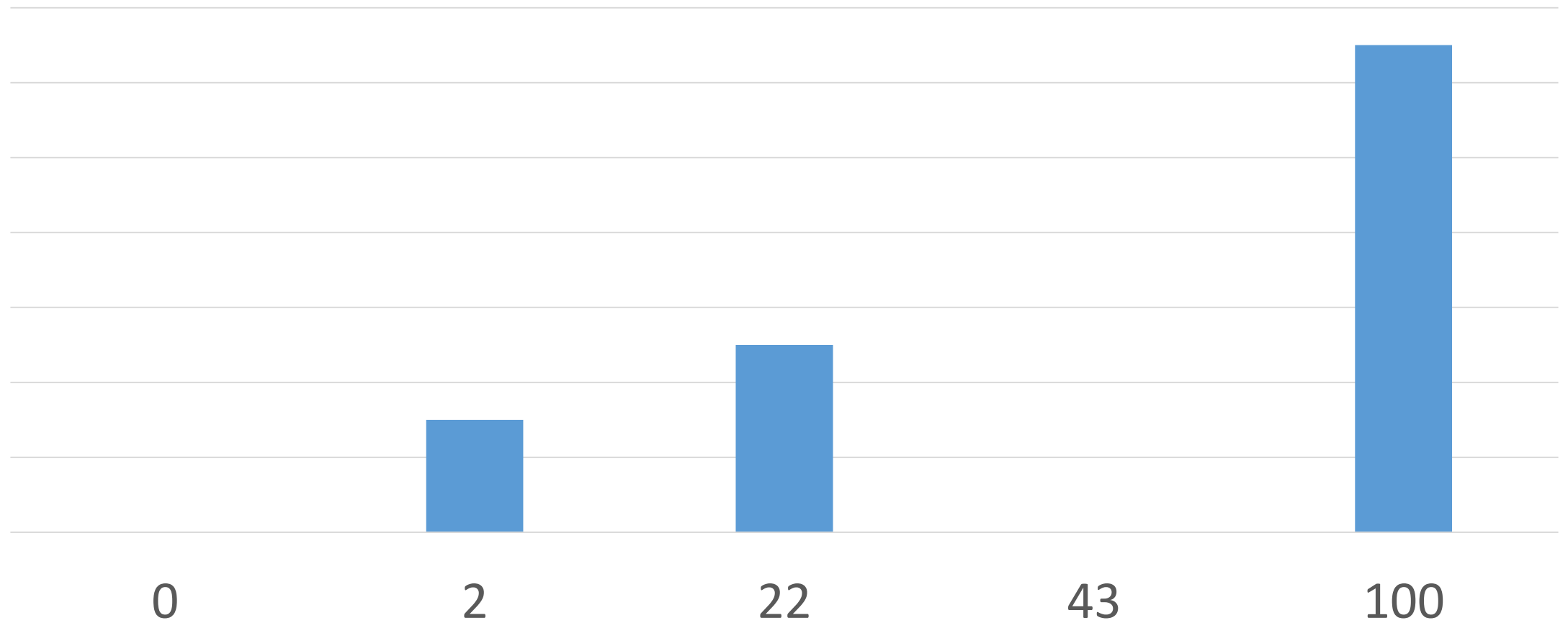
方法1:セグメント木 on セグメント木

- 1回の操作では大きなセグ木で $O(\log N)$,それぞれのノードで $O(\log N)$ かかるので全体の時間計算量は $O((N + Q)(\log N)^2)$
- ただしそのままでは空間計算量 $O(N^2)$ でMLEなので工夫する
- クエリの先読みができるので、それぞれのノードのセグメント木であらかじめ使われる値だけ座標圧縮しておけば幸せになれる

方法2: 平衡二分木 on セグメント木

- 値の挿入とある値以上の値の数の取得ができる平衡二分木を使う
 - C++ STLのsetでは後者ができない
- g++拡張のtreeを使うと楽
 - 参照:<http://hogloid.hatenablog.com/entry/2014/09/23/132440>
- 計算量 $O((N + Q) \log^2)$ くらい
 - ただし定数倍は重いので注意

点数分布



ビーバーの会合 (Meetings)

解説：井上 航

小課題1(累積7点)

- すべての木を全探索します。
- たいへんですね
- 苦行す。
- あとこの先の話をすると、 $\text{query}(u,v,w)$ を聞きたいとき、おなじものが2つあるとそれを返す関数を作っておくと楽です

小課題2(累積17点)

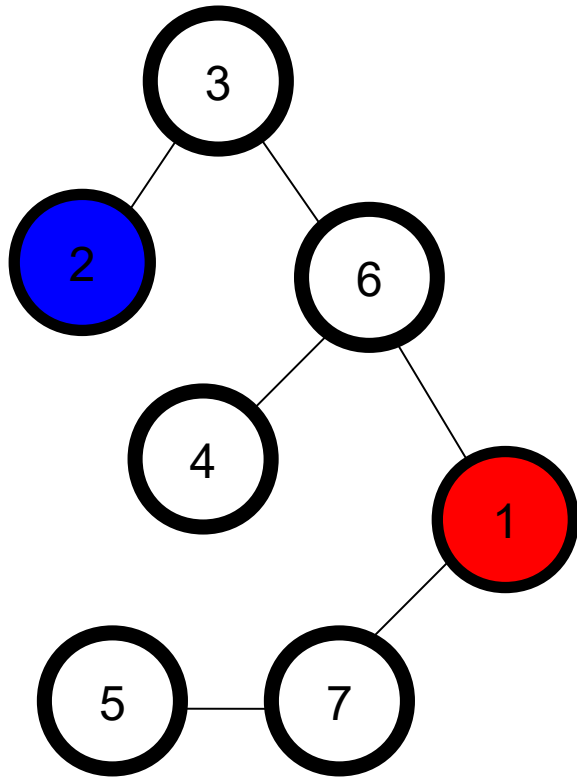
- $n \leq 50$ です
- ありうるクエリは19600通りしかありません。
- なので、クエリをメモ化すればTLEしない限りとけます。
- 頂点 u, v が直接つながれているかどうかを求めたいです。
- u, v が直接つながれている場合、すべての w に対して $\text{query}(u, v, w)$ は u か v を返します
- 逆に、直接つながれてないならその間の場所が存在して、そこを w とすると $\text{query}(u, v, w)$ は u, v 以外を返すことがあります
- よってすべての u, v に対して求めてとけました

小課題3(累積31点)

- $n \leq 300$ です
- やり方が2種類あります。ここでは、簡単な方を説明します。
- 頂点 u について、つながっている辺すべてを求めます。
- 「現在のところ、 i に対して最も近い場所」を v とします。
- `used[]` という配列を持っておきます
- すべての i に対して、`query(u,v,i)`を聞いて、それが u ではないなら`query`の答えの方が v に対して近づいています。
`v=query(u,v,i)`にします。`used[i]=true`;
- クエリの答えが u ならば u から見て i と v は別方向の頂点なので、後回しにします
- 最終的には v は u に隣接する頂点です。`bridge`を出力します。
- v を適当な`used`でない頂点において、また同じことをします

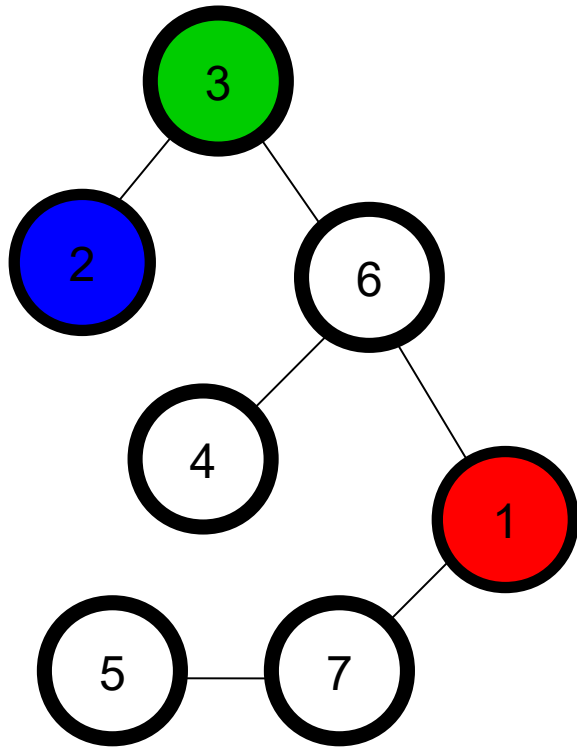
アルゴリズムの説明

- 頂点1につながる辺を求める。頂点1に近いのをとるあえず2にする

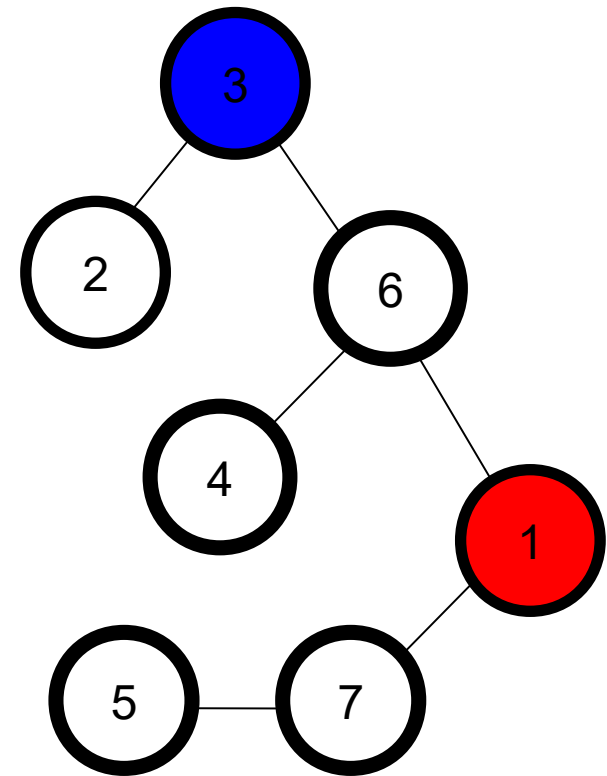


アルゴリズムの説明

- 頂点3について調べる

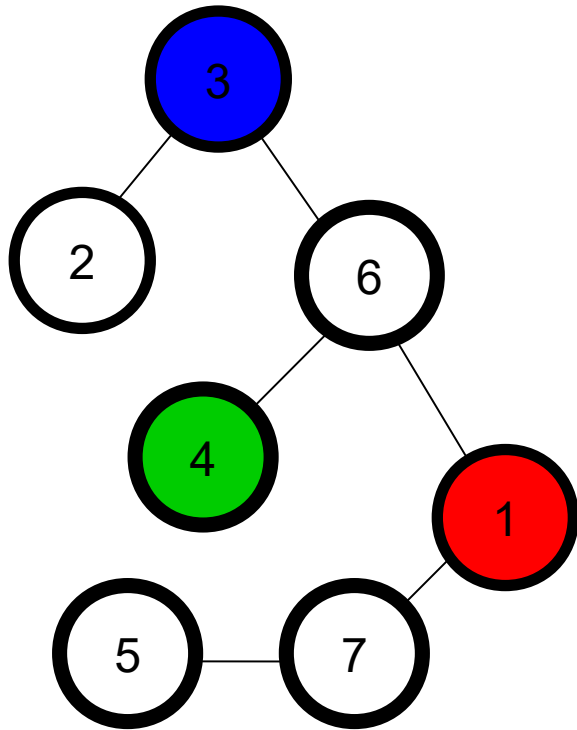


Query(1,2,3)=3
vが2から3に移動する

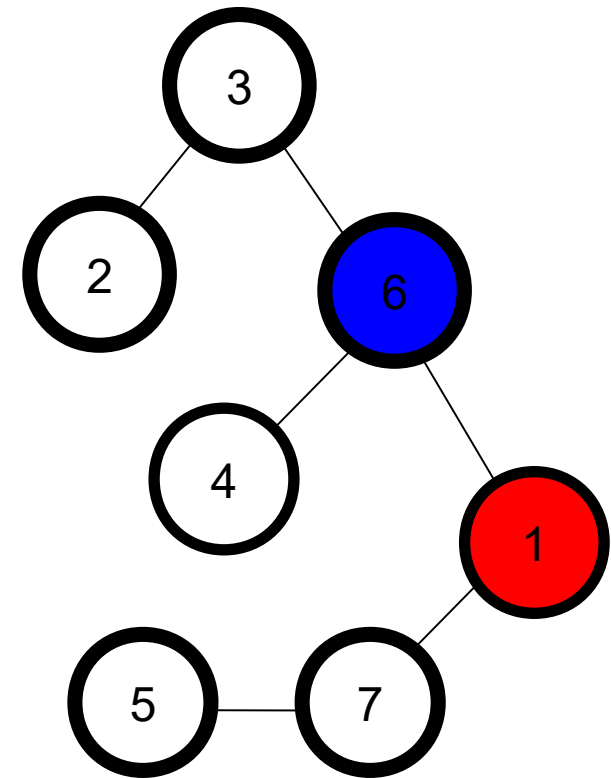


アルゴリズムの説明

- 頂点4について調べる

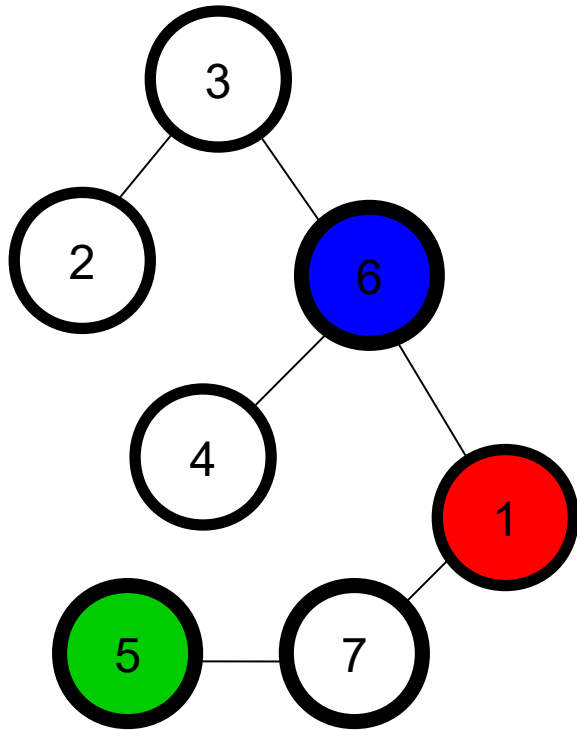


Query(1,3,4)=6
vが3から6に移動する

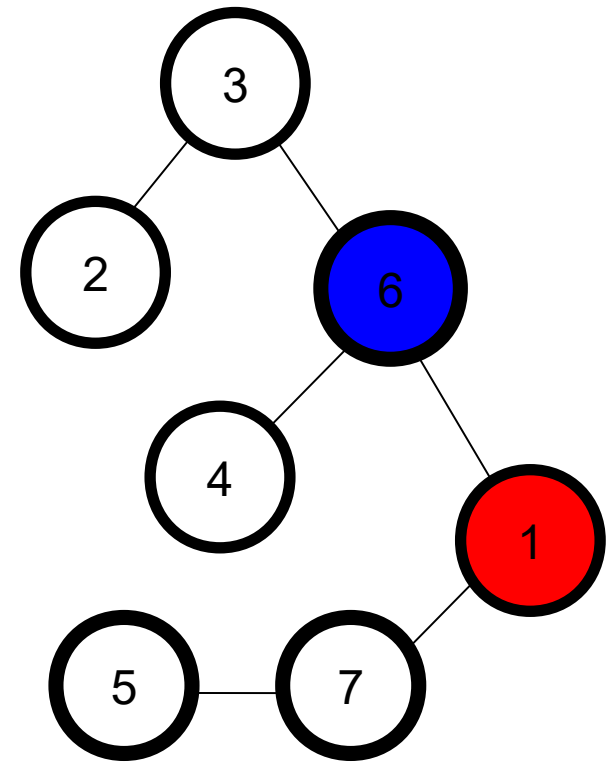


アルゴリズムの説明

- 頂点5について調べる

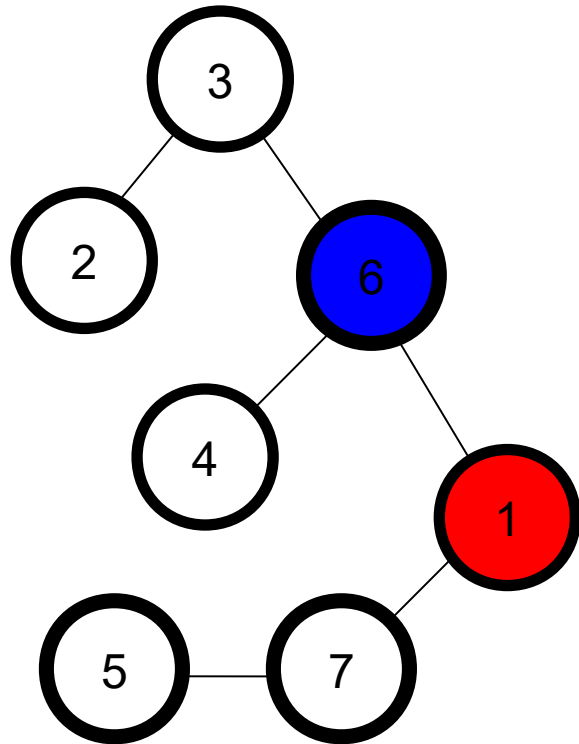


Query(1,6,5)=1
vはそのまま

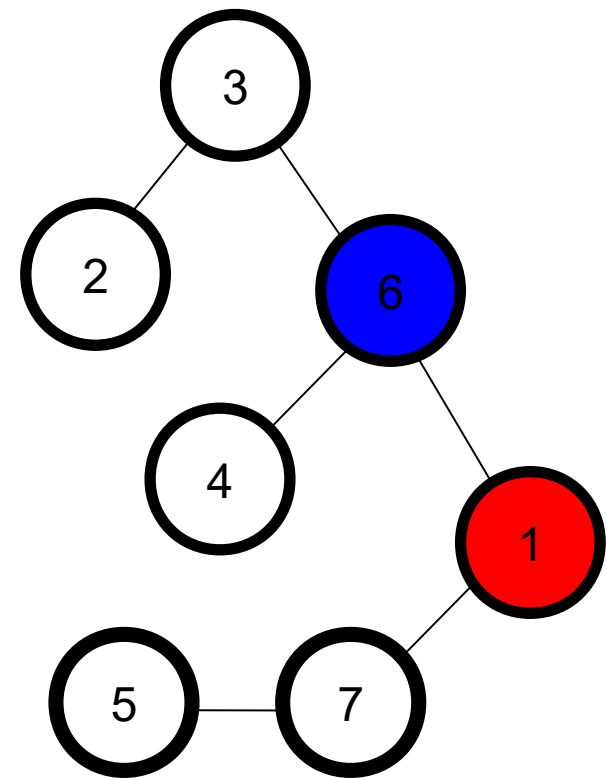


アルゴリズムの説明

- 頂点6について調べる



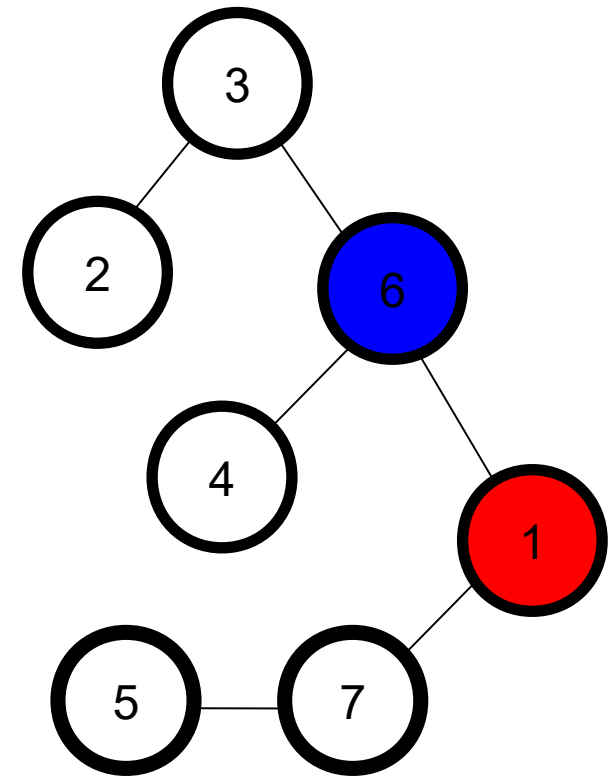
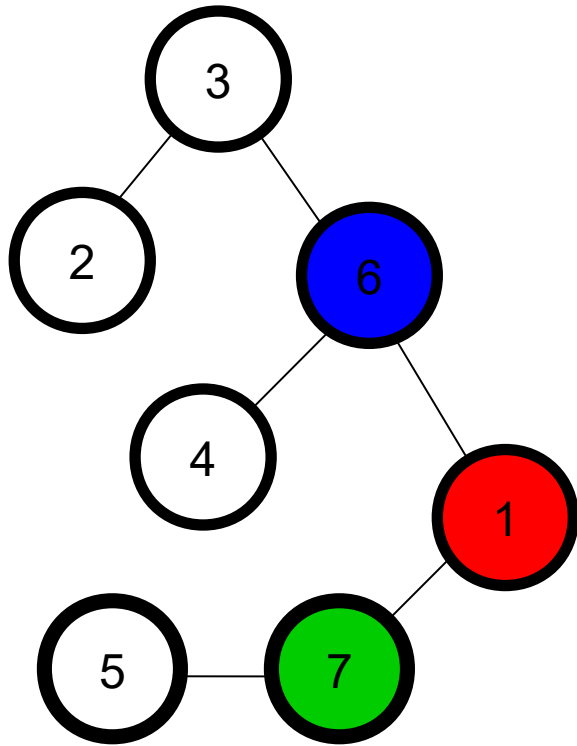
Query(1,6,6)=6
vは6



アルゴリズムの説明

- 頂点7について調べる

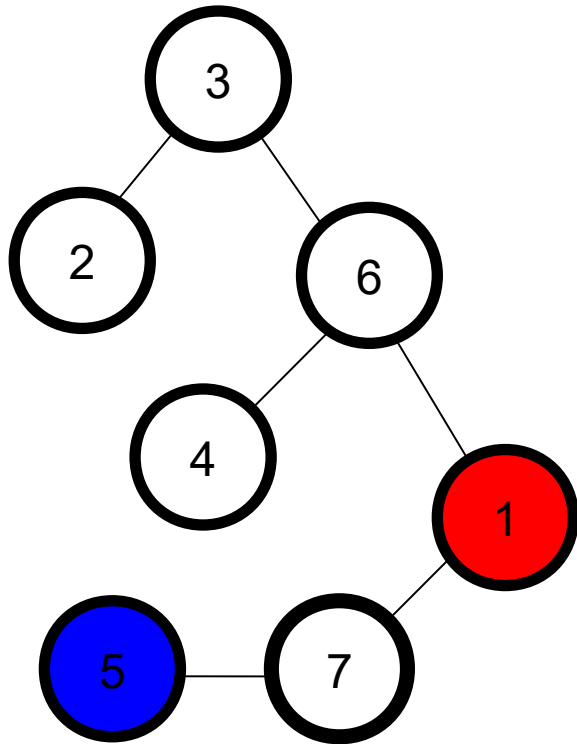
Query(1,6,7)=1
vはそのまま



1-6の橋を見つけることができた

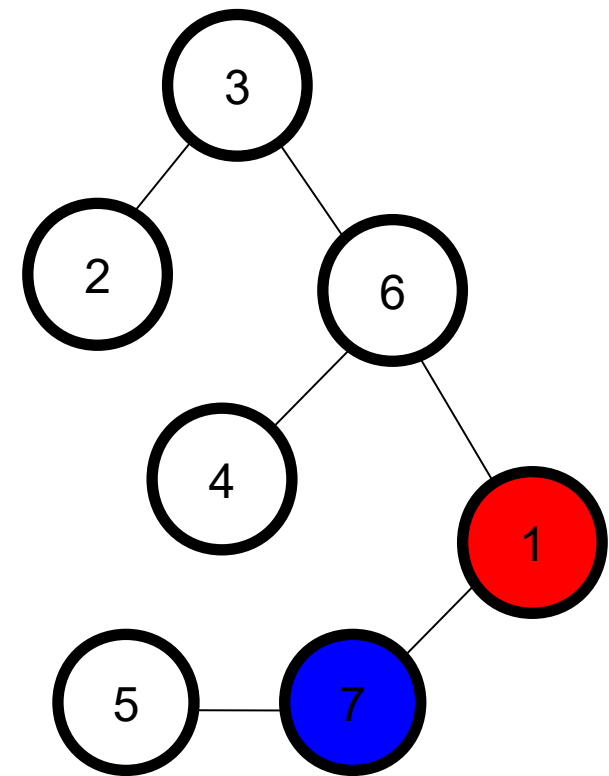
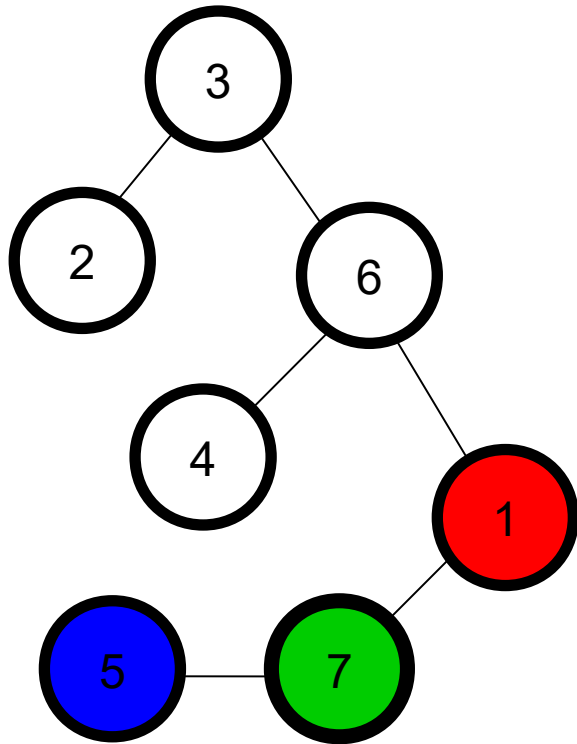
アルゴリズムの説明

- 5と7が後回しになっているのでもう一周する、 $v=5$



アルゴリズムの説明

- 頂点7について調べる



1-7の橋を見つけることができた

小課題3(続き)

- と思ったら解けていません。(最悪 $2 \cdot n^2$ 回のクエリが必要)
- だから、枝刈りをします。
- bridgeを出力するときに、 $u < v$ のものだけ出力すればよいです。
- よって、まだ見つけてない v を見つけるために、for(i)の部分は $u < i$ のものだけ調べればよいです。
- ただし、 $i < u$ のものを試していないので、最終的な v の方向に行く辺が、距離1になっているとは限りません。
- よって、距離1判定をするために、 u と隣接することがすでに分かっている頂点すべても i としてqueryをして、距離1まで縮める必要があります。
- よって、今度こそ解けました。

小課題3(別解)

すでに木が存在したときに、それに対して新たな頂点がどのようなつながり方(間に入る、とか)をするかを考えます。

- 例:すでに1-2-3 の木があったとして、頂点4を考えます。 $\text{query}(2,3,4)$ は2でした。
- $\text{Query}(1,2,4)$ が4なら木は1-4-2-3です。
- それが1なら木は4-1-2-3です。
- それが8なら木は1-8-2-3です。(このパターンに注意)

|
4

こっちの解法の良いところ

- 説明が楽
- クエリが少ない
- 満点につながる!!!

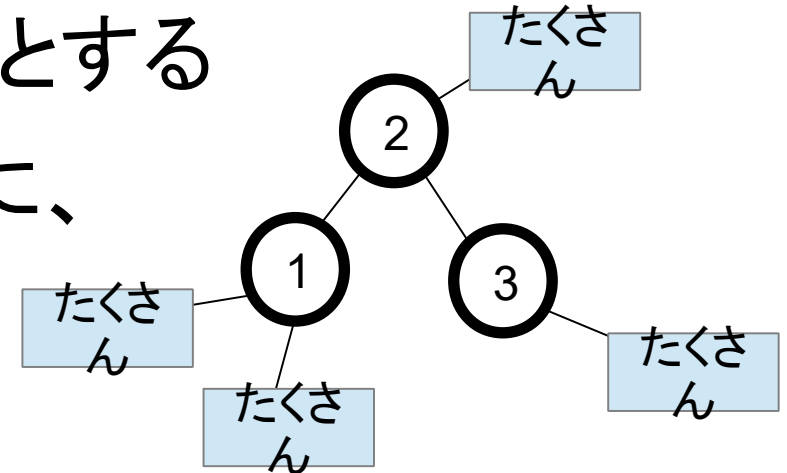


こっちの解法の悪いところ

- 実装がおもしろい

満点解法

- 小課題3の別解を改良します
- 重心分解をして、つながり方の頂点をうまく聞くと、とけます。
- 具体的に言うと、たとえば都市 v がわかっていない状況で、 $\text{query}(1,3,v)$ を聞いたとする
- それが1なら v は1の部分木側に、
- 2なら v は2の部分木側に、
- 3なら v は3の部分木側に、
- それ以外なら1-2-3の木に対して v がついていることがわかる



満点解法

- この時、 v の範囲が3分割されています。
- 「こっちにある」の方向に対して、再帰的に求めます。
- クエリを聞くときに、最悪のケースでの残った頂点の個数を最小にするように聞くとよいです。
- 具体的には、 $\text{query}(v, (\text{重心から延びる1番めに部分木が大きい方の頂点}), (\text{重心から延びる2番めに部分木が大きい方の頂点}))$ と聞くなどがあります。
- この方針だと、たぶん100点を得ます。

4万回以内の証明(かなり難しいです)

n 頂点からなる木がわかっていて 1 頂点追加する場合のコスト

追加したい頂点を q とする.

重心分解する. 重心を g とする.

g の隣接頂点を $v_1, v_2, \dots, v_d$ として, g を取り除いたときの部分木で v_i を含むものの大きさを s_i とする ($s_1 \geq \dots \geq s_d$). その部分木を S_i とおく.

q, v_i, v_j でクエリを投げると、答えは次のどれか：

- v_i または v_j : S_i か S_j の中.
- g : S_i, S_j の中にはない.
- それ以外: g と (v_i か v_j のどちらか) の間. もう 1 回クエリを投げると完全にわかる.

最後の「もう 1 回」は 1 回だけしか起きないので無視しておく.

まず d が偶数の場合だけ考えると, 次のどれかが起きることがわかる:

- 1 回のクエリを使って, 大きさが高々 s_1 の部分木に範囲を絞る.
- 2 回のクエリを使って, 大きさが高々 s_3 の部分木に範囲を絞る.
- ...
- 9 回のクエリを使って, 大きさが高々 s_{17} の部分木に範囲を絞る.

d が奇数の場合は, 次のどれかも起きうる: (部分木が 1 個になった地点で重心分解をやり直す)

- 1 回のクエリを使って, 大きさが高々 s_{3+1} の部分木に範囲を絞る.

- 2 回のクエリを使って, 大きさが高々 s_{5+1} の部分木に範囲を絞る.

-...

- 8 回のクエリを使って, 大きさが高々 s_{17+1} の部分木に範囲を絞る.

ところで、頂点数が 1 増えただけでクエリ回数が 2 回以上増えるわけではないので、この奇数の場合は無視していいことがわかる.

(例えば「8 回で $s_{17}+1$ 」より「9 回で s_{17} 」のほうがコストは大きい)

これをパソコンで計算すると、2 頂点, 3 頂点, ..., 1999 頂点についてのこのコストの和は(「もう 1 回」を含めて) 39632 になることがわかる.

($s_1 \leq n/2$, $s_k \leq n/k$ に注意)

実際には見る点の順番をrandomにすればかなりうまくいきます

得点分布

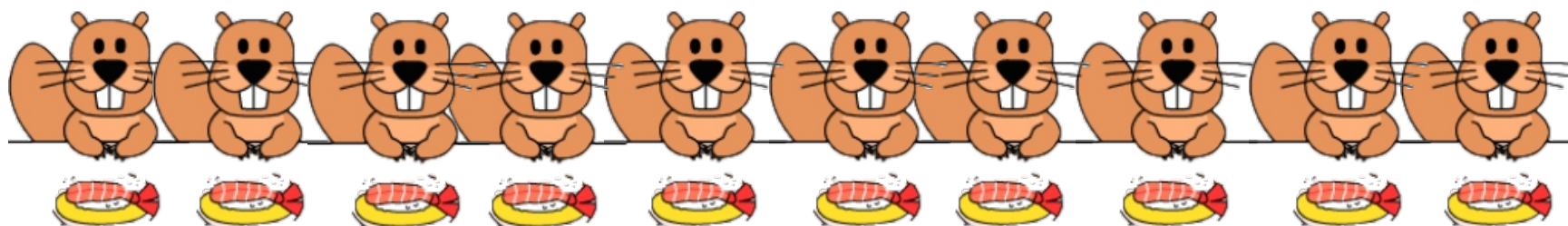
100



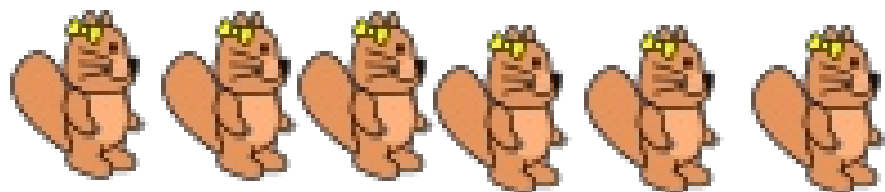
78



29

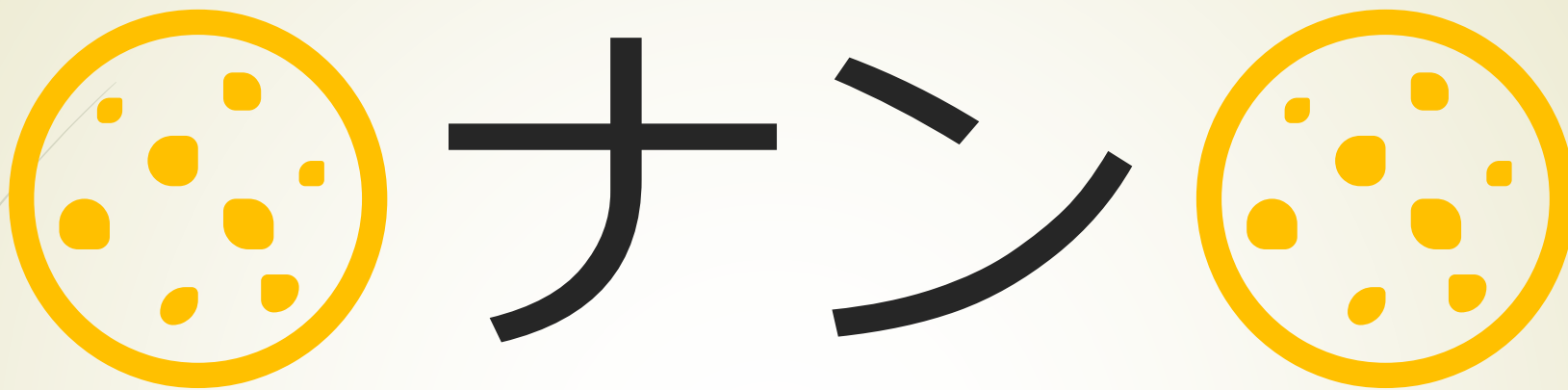


17



0





@sigma425

問題概要

- ▶ とても長いナンを N 人で分けて食べる
- ▶ ナンの長さは L [cm] で、各 1 [cm] ごとに味が違う
 - ▶ 左から味 $1, 2, \dots, L$
- ▶ 人 i が味 j のナンを食べると、1 [cm] ごとに幸福度が $V_{i,j}$ 増える
- ▶ N 人で分けるときは、切れ目 $X_1, X_2, \dots, X_{N-1}$ を選んで N 個に分けて、一つずつ分配する (左から k 番目のピースを人 P_k にわたす)
- ▶ 任意の人が、(自分ひとりでナンすべてを食べたときに得られる幸福度)/ N 以上の幸福度を得るような分配を一つ求めてください
 - ▶ このような分配はかならずあることが証明できます(ネタバレ)
- ▶ ただし、切れ目 X_i はある程度分母が小さい分数で出力する必要がある

example

入出力例

入力例 1	出力例 1
2 5	14 5
2 7 1 8 2	2 1
3 1 4 1 5	

この入力例では、1 番目の人がナンを 1 人で全部食べたときに得る幸福度が $2 + 7 + 1 + 8 + 2 = 20$ で、2 番目の人がナンを 1 人で全部食べたときに得る幸福度が $3 + 1 + 4 + 1 + 5 = 14$ である。よって、1 番目の人が $\frac{20}{2} = 10$ 以上、2 番目の人が $\frac{14}{2} = 7$ 以上の幸福度が得られる分配が公平な分配になる。

左から $\frac{14}{5}$ cm のところでナンを分け、1 番目の人に左から 2 つめのピースを、2 番目の人に左から 1 つめのピースを与えるようにすると、1 番目の人は $1 \times \frac{1}{5} + 8 + 2 = \frac{51}{5}$ の幸福度を得て、2 番目の人は $3 + 1 + 4 \times \frac{4}{5} = \frac{36}{5}$ の幸福度を得るので、公平な分配になる。

Impossible

- ▶ 不可能な場合は -1 を出力せよ
- ▶ 本当に不可能なケースがあるか?
 - ▶ ひっかけて問題文にImpossibleについて書いてある可能性もある
 - ▶ コーナーケースだけ不可能なのかもしれない
- ▶ !! 少なくともサンプルになかったらあやしい !!

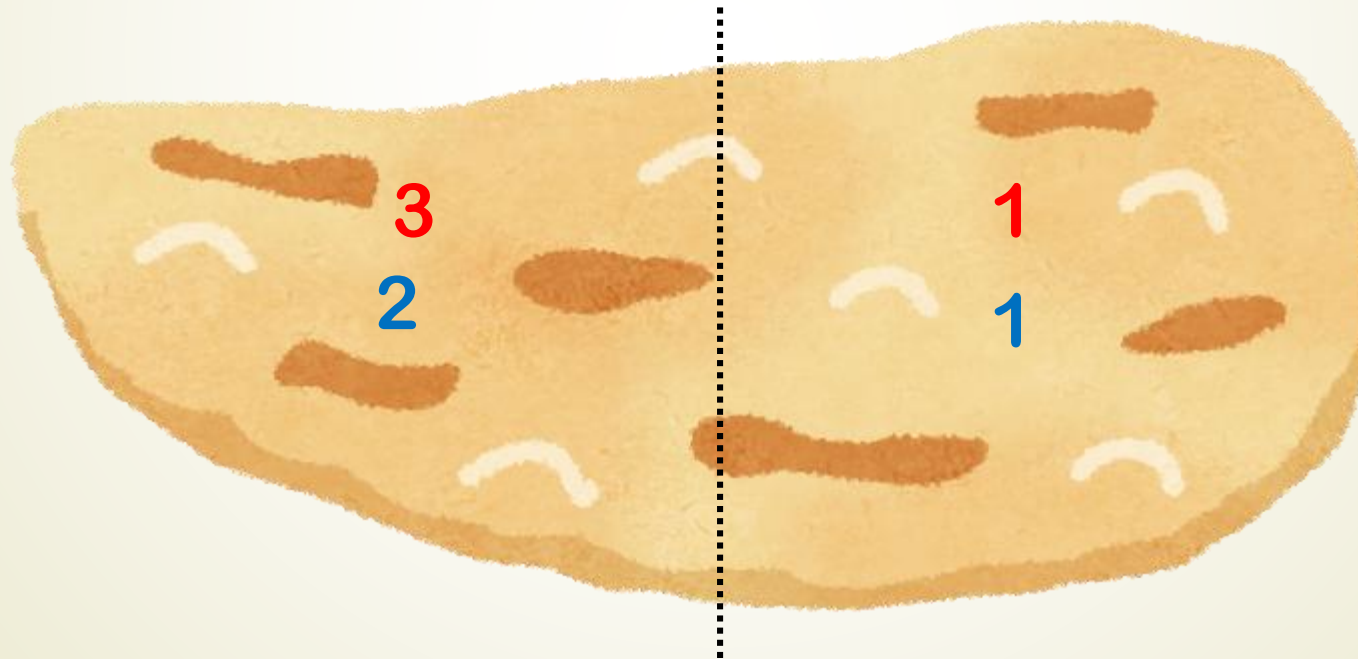
- ▶ 今回は常に可能です

$N = 2$ (5点)

- ▶ 簡単部分点
- ▶ 5点なんか取って嬉しいか?
- ▶ 嬉しい!
 - ▶ 点数が増える
 - ▶ より高い点数の解法のヒントになることもよくある
 - ▶ OI系に限らず、簡単な制約でまずは解いてみるというのは有用なアプローチだと思います
 - ▶ 木の問題をまずpathで解く、 $Q = 1$ をまず考える . . .

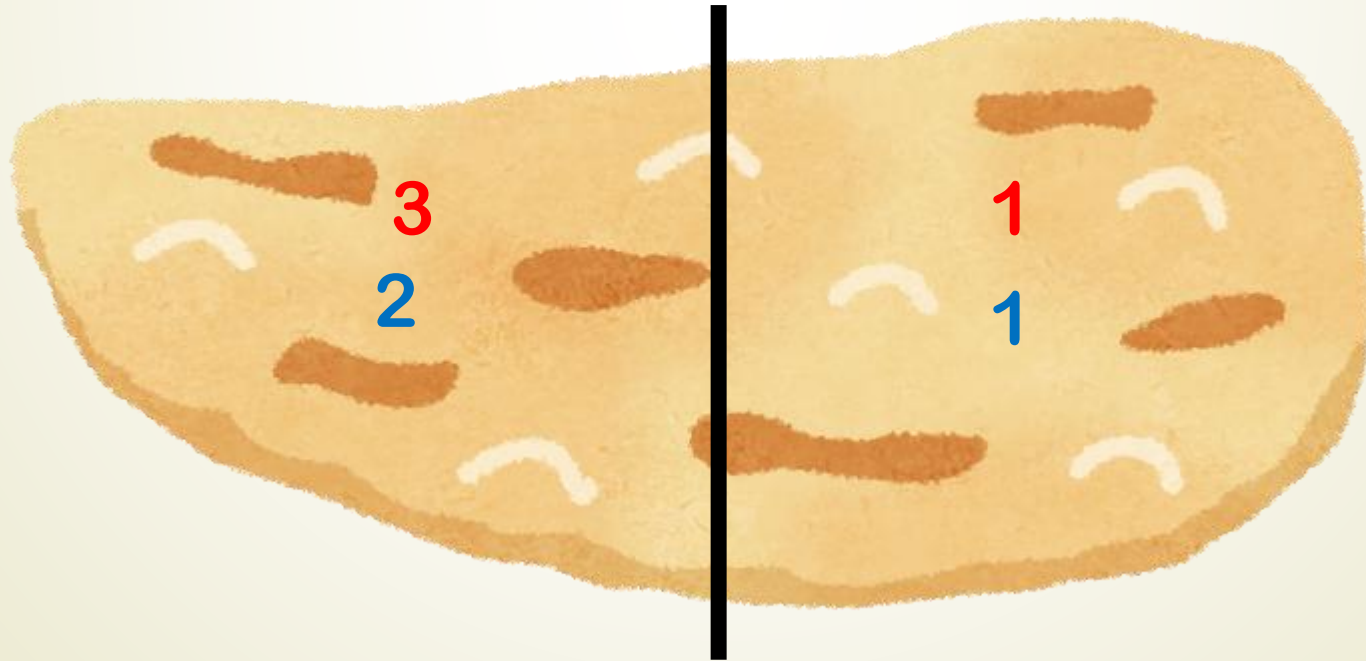
$N = 2$ (5点)

- ▶ だいたい「真ん中」あたりで切って分けたい
- ▶ 真ん中ってどこだよ



$N = 2$ (5点)

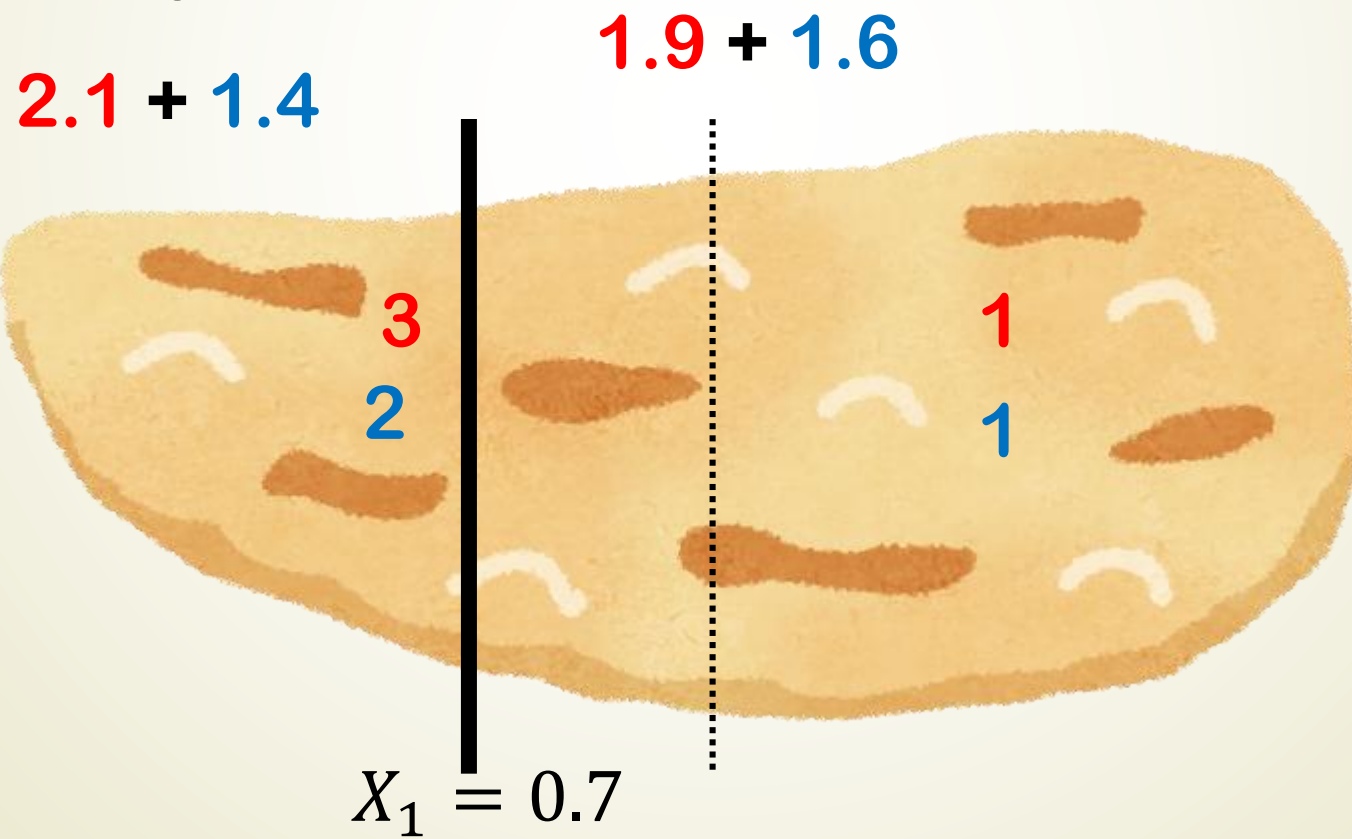
- ▶ 本当に半分に切る
- ▶ 全然ダメ
 - ▶ ナンには重みがついているので



$N = 2$ (5点)

➡ 2人の重みの総和が半分になるところで切る

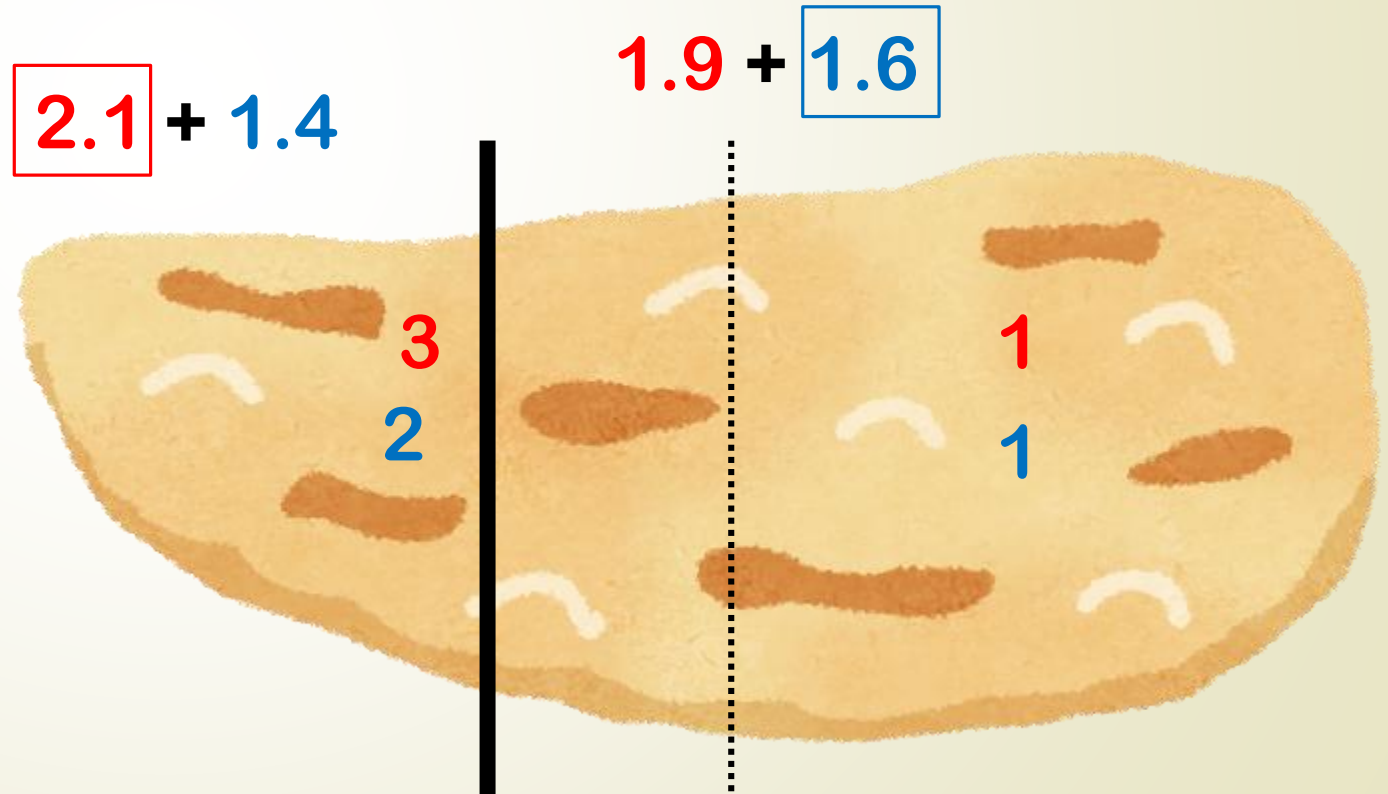
➡ OK



$N = 2$ (5点)

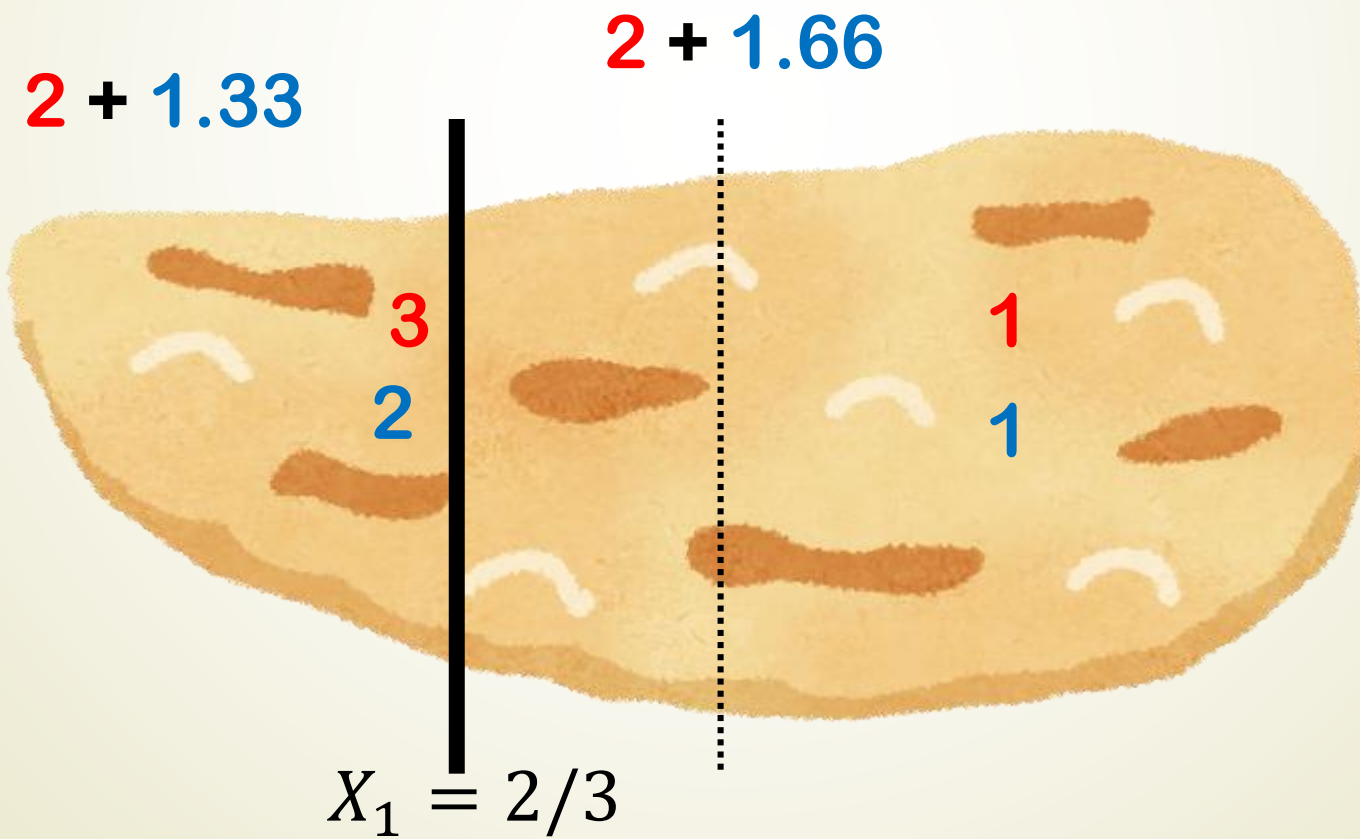
- 2人の重みの総和が半分になるところで切る
- OK
- 片方が赤でも青でも半分超えてることはありえないので

- でも、一般の N には
拡張できない



$N = 2$ (5点)

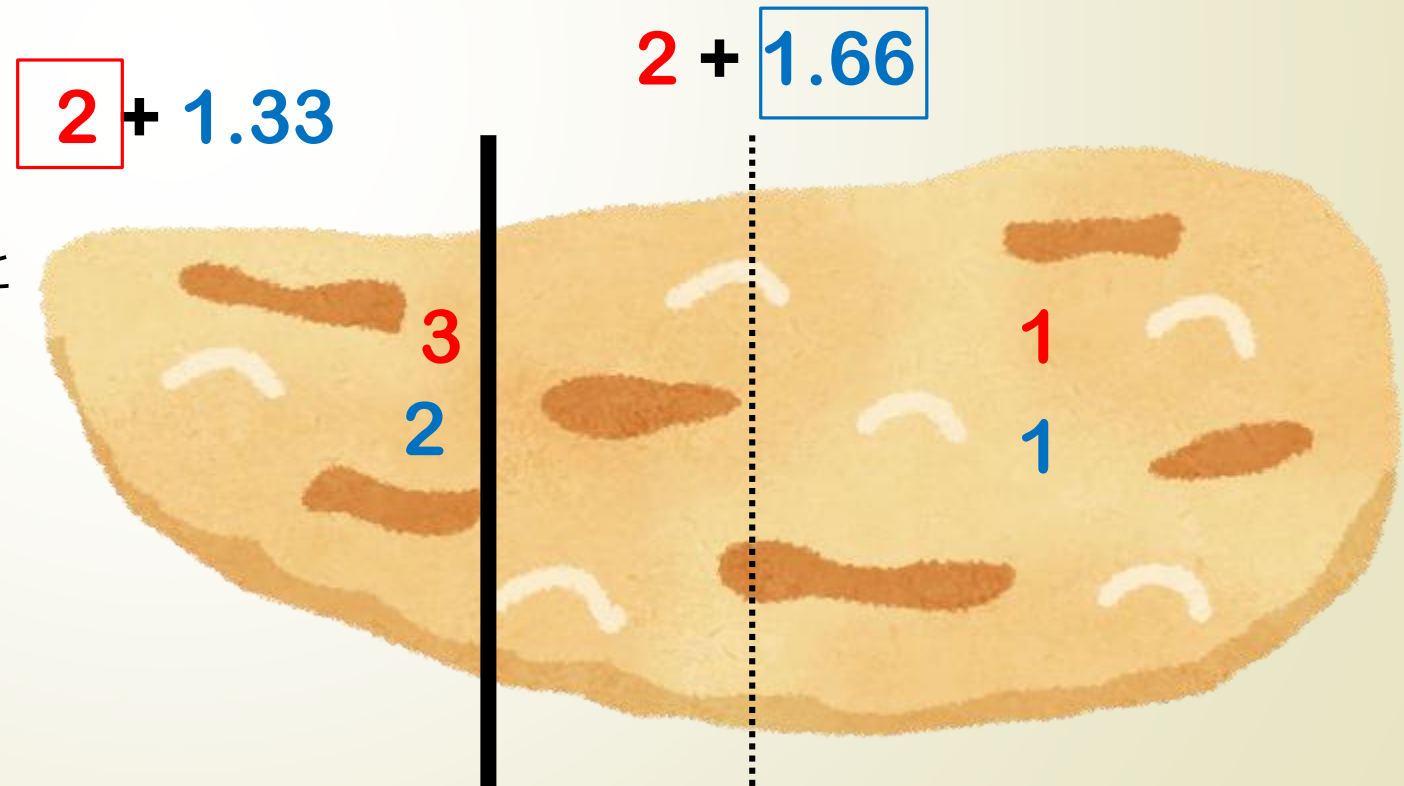
- 人1の重みが半分になるところで切る
- これでもOK



$N = 2$ (5点)

- ▶ 人1の重みが半分になるところで切る
- ▶ これでもOK
- ▶ 人1にはどっちを渡してもいいので、人2により嬉しい方をあげる

- ▶ 一般の N に拡張しようとするとうなる?



N : 一般?

- ▶ 人1の重みが $1/N$ になるところで切る (左端から)
- ▶ それを人1にあげる?
- ▶ ダメ
 - ▶ 例えば人2がそのかけらをめちゃくちゃ食べたがっていて重み $(N - 1)/N$ とかになってたらもう詰んでる

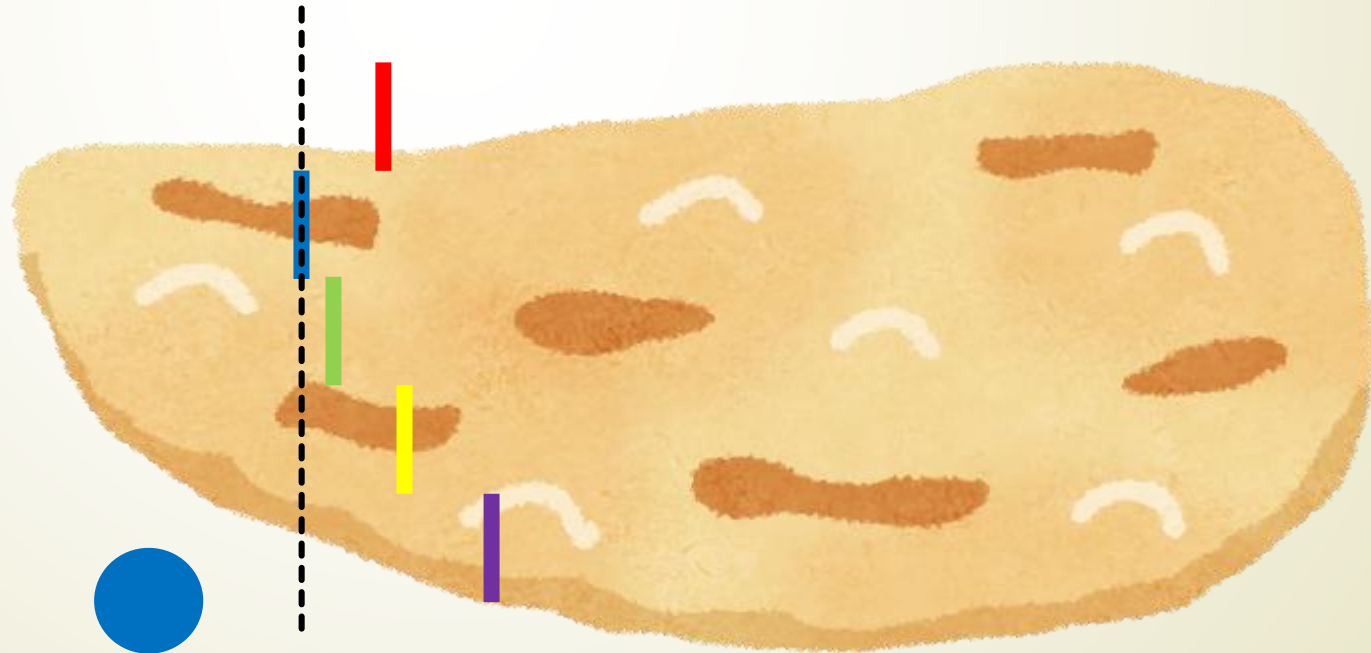


N : 一般?

- ▶ 人1の重みが $1/N$ になるところで切る (左端から)
- ▶ それを人1にあげる?
- ▶ ダメ
 - ▶ 例えば人2がそのかけらをめちゃくちゃ食べたがっていて重み $(N - 1)/N$ とかになってたらもう詰んでる
- ▶ 左端をめちゃくちゃ食べたがっている人に左端をあげると良さそう?

N : 一般?

- ▶ 左端をめちゃくちゃ食べたがっている人に左端をあげると良さそう?
- ▶ 人 i の重みが $1/N$ になるところ というのを各 i について列挙する
- ▶ それが一番左になるような i を選んで、そこで切ってそのピースを人 i にあげる

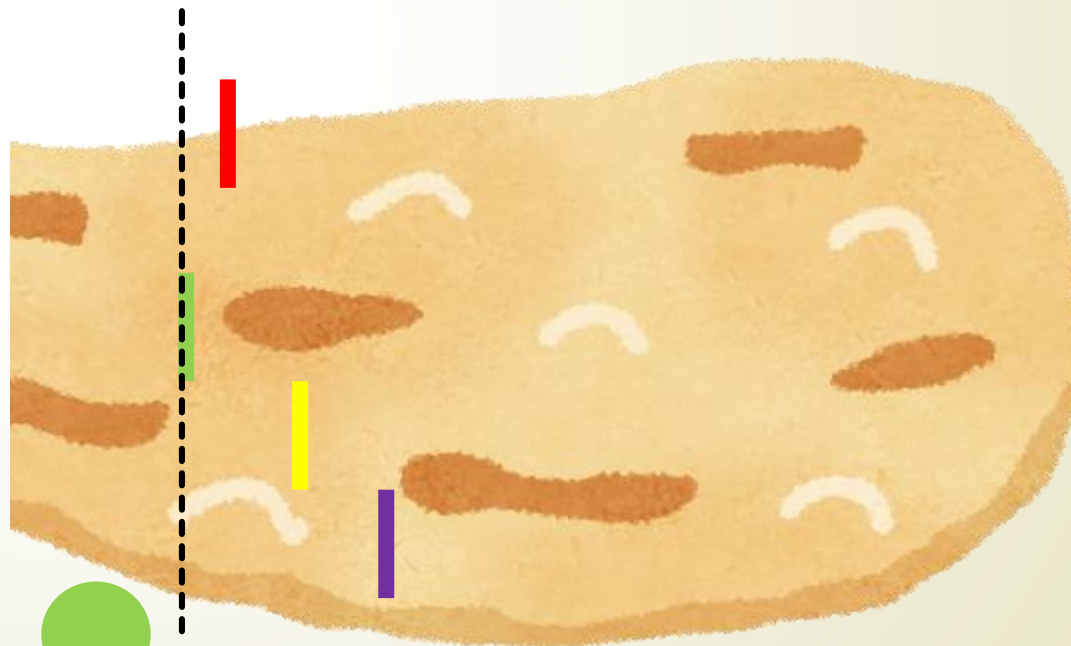


N : 一般?

- ▶ 左端をめちゃくちゃ食べたがっている人に左端を上げると良さそう?
- ▶ 人 i の重みが $1/N$ になるところ というのを各 i について列挙する
- ▶ それが一番左になるような i を選んで、そこで切ってそのピースを人 i にあげる
- ▶ そうすると、各残った人について、残ったナンの重みの割合は $(N-1)/N$ 以上になる
- ▶ なので残ったナンを使って、残った $N-1$ 人で再帰的に $(\frac{1}{N-1})$ の切れ目を求めて) 解く、ということをしていけば元の正しい分配が得られる
- ▶ 100点?

N : 一般?

- なので残ったナンを使って、残った $N - 1$ 人で再帰的に ($\frac{1}{N-1}$ の切れ目を求めて) 解く、ということをしていけば元の正しい分配が得られる
- 100点?



N : 一般?

➡ 100点?

出力の制約

公平な分配が可能な場合、出力は以下の制約を満たさなければならない。

- $1 \leq R \leq 1,000,000,000$, $1 \leq N$.

N : 一般?

➡ 100点?

出力の制約

公平な分配が可能な場合，出力は以下の制約を満たさなければならない．

- $1 \leq B_i \leq 1\,000\,000\,000 (1 \leq i \leq N)$.

N : 一般?

▶ 100点?

出力の制約

公平な分配が可能な場合，出力は以下の制約を満たさなければならない．

- $1 \leq B_i \leq 1\,000\,000\,000$ ($1 \leq i \leq N$).

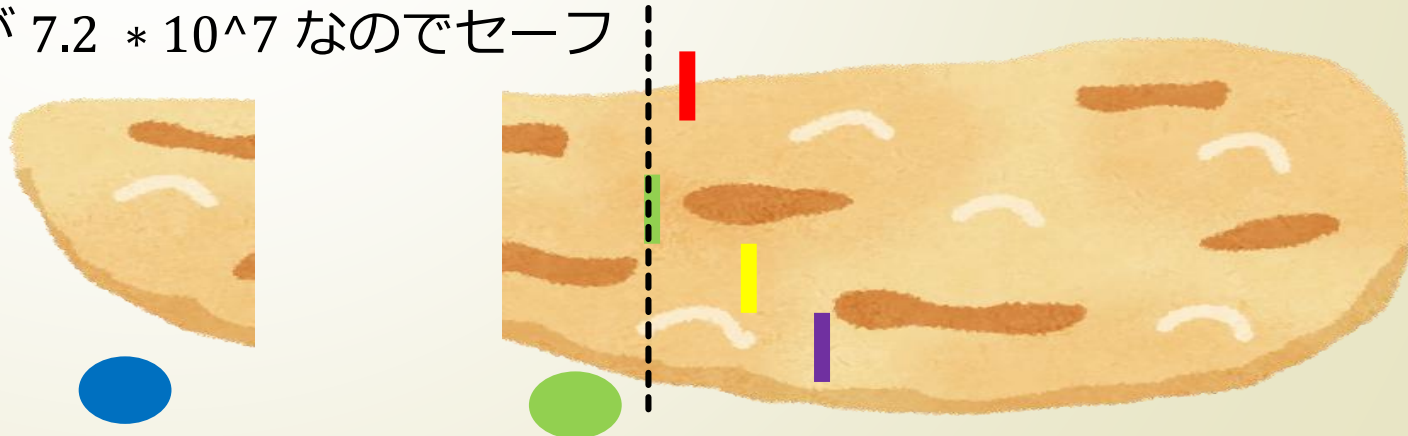
力市利利

な分配が可能の場合, 出力は

$$1 \leq B_i \leq 1\,000\,000\,000 \quad (1 \leq i \leq n)$$

小課題2 ($N \leq 6, V_{i,j} \leq 10$) 24点

- ▶ はじめに切る場所は (整数)/ N の重みを持つ部分なので、左端からの距離でいうと (整数)/($N * V_{i,j}$) みたいな感じになる
- ▶ 一段再帰すると、全体の重みの和が整数じゃなくて (整数)/($N * V$) みたいな形になるので、切る場所は (整数)/($N * V * (N - 1)$) みたいになって距離でいうと (整数)/($N * V * (N - 1) * V$) みたいになる
- ▶ 結局分母の大きさは最終的に $N! * V_{max}^{N-1}$ くらいに膨れ上がってしまう
- ▶ 小課題2についてはこれが $7.2 * 10^7$ なのでセーフ



実装上の注意

- ▶ 切れ目などを分数で持つ必要がある
- ▶ struct にして外からはできるだけ自然に使えるようにするとよいと思います

```
struct Rational{
    ll x,y;      // x/y
    Rational():x(0),y(1){}
    Rational(ll x):x(x),y(1){}
    Rational(ll x,ll y):x(x),y(y){}

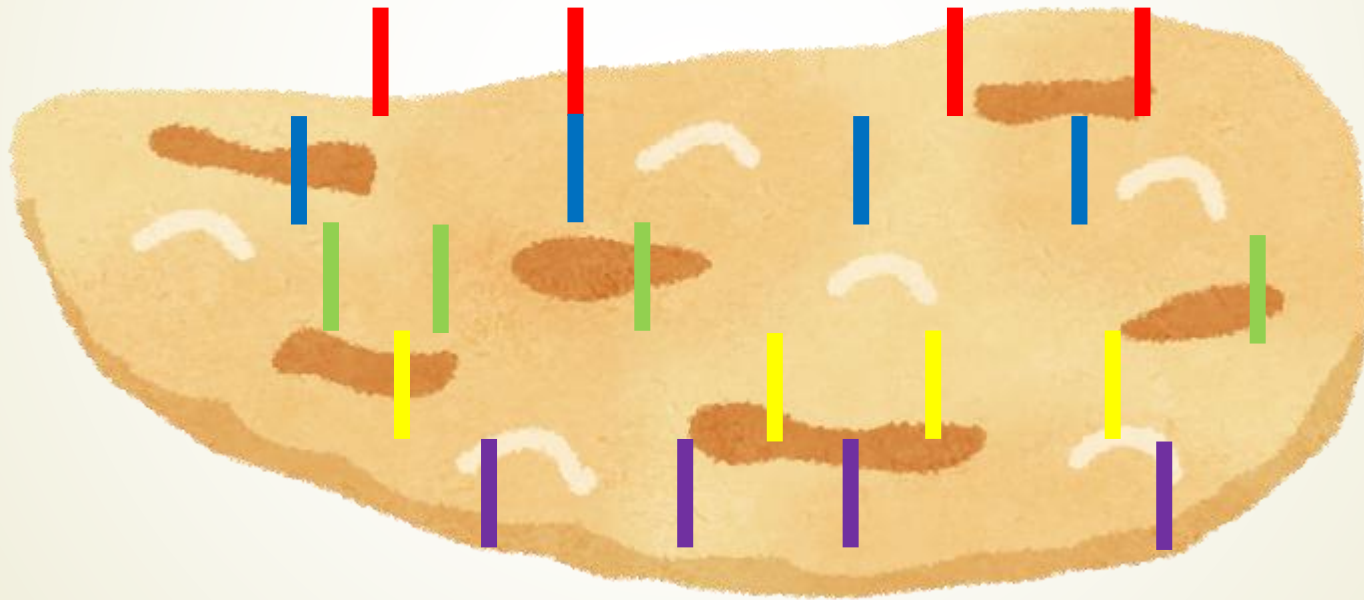
    Rational norm(ll x,ll y) const {
        ll g = gcd(x,y);
        x /= g, y /= g;
        if(y<0){
            x = -x;
            y = -y;
        }
        return Rational(x,y);
    }

    Rational operator+(const Rational& r) const {
        ll X = x * r.y + y * r.x;
        ll Y = y * r.y;
        return norm(X,Y);
    }
    bool operator<(const Rational& r) const {      //positive number
        return x*r.y < r.x*y;
    }
};
```

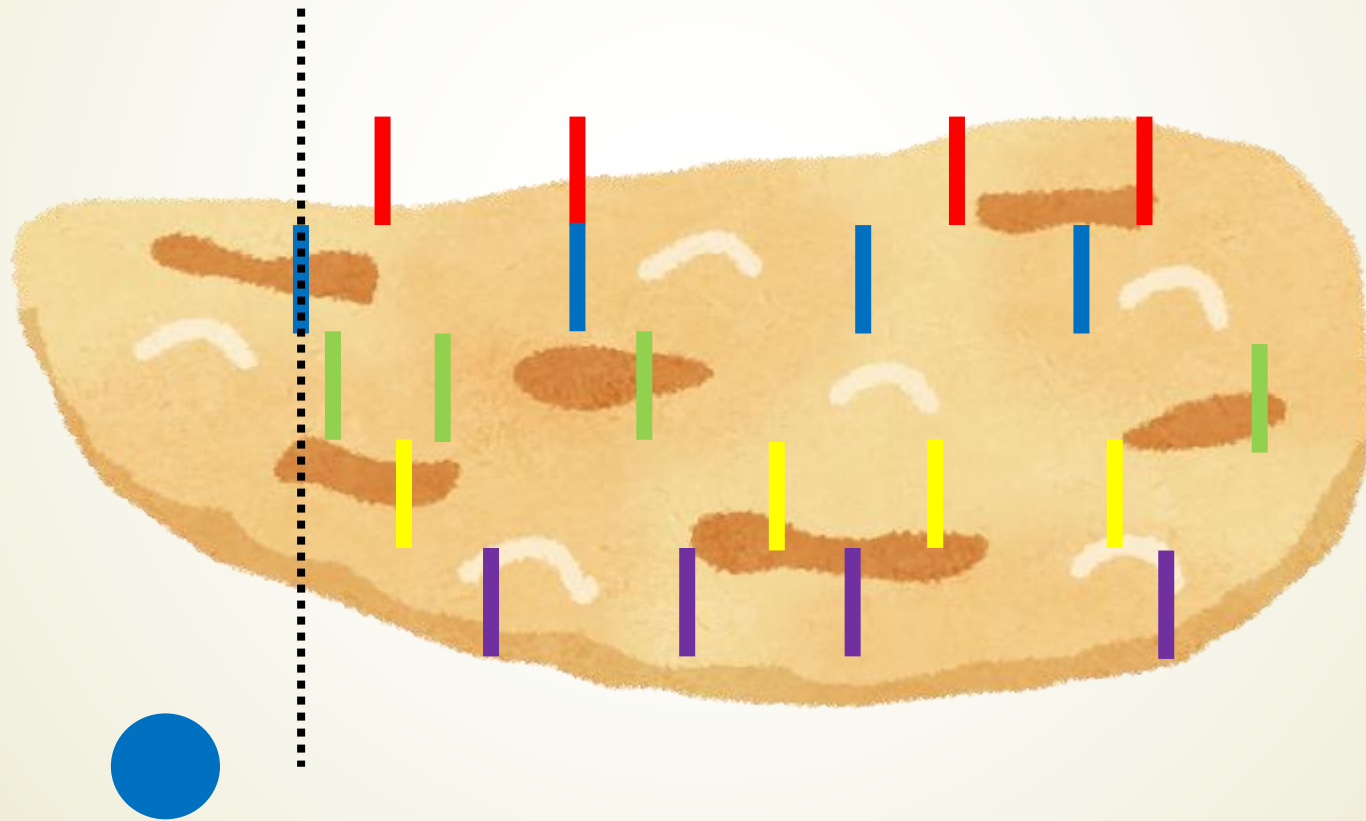
満点 ($N \leq 2000, V_{i,j} \leq 10^5$)

- ▶ はじめの一人については一緒
 - ▶ 各人に対して、 $1/N$ になるところを求めて、一番左で切れるやつにわたす
- ▶ 次は、ナンを切らなかったと思って $2/N$ で同様のことをしてみる
 - ▶ 各人に対して、 $2/N$ になるところを求めて、一番左(ただし一人目は除く)で切れるやつにわたす
- ▶ ナンを切らなかったと思って $3/N$ で同様のことをしてみる
 - ▶ 各人に対して、 $3/N$ になるところを求めて、一番左(ただし一人目,二人目は除く)で切れるやつにわたす
- ▶ :

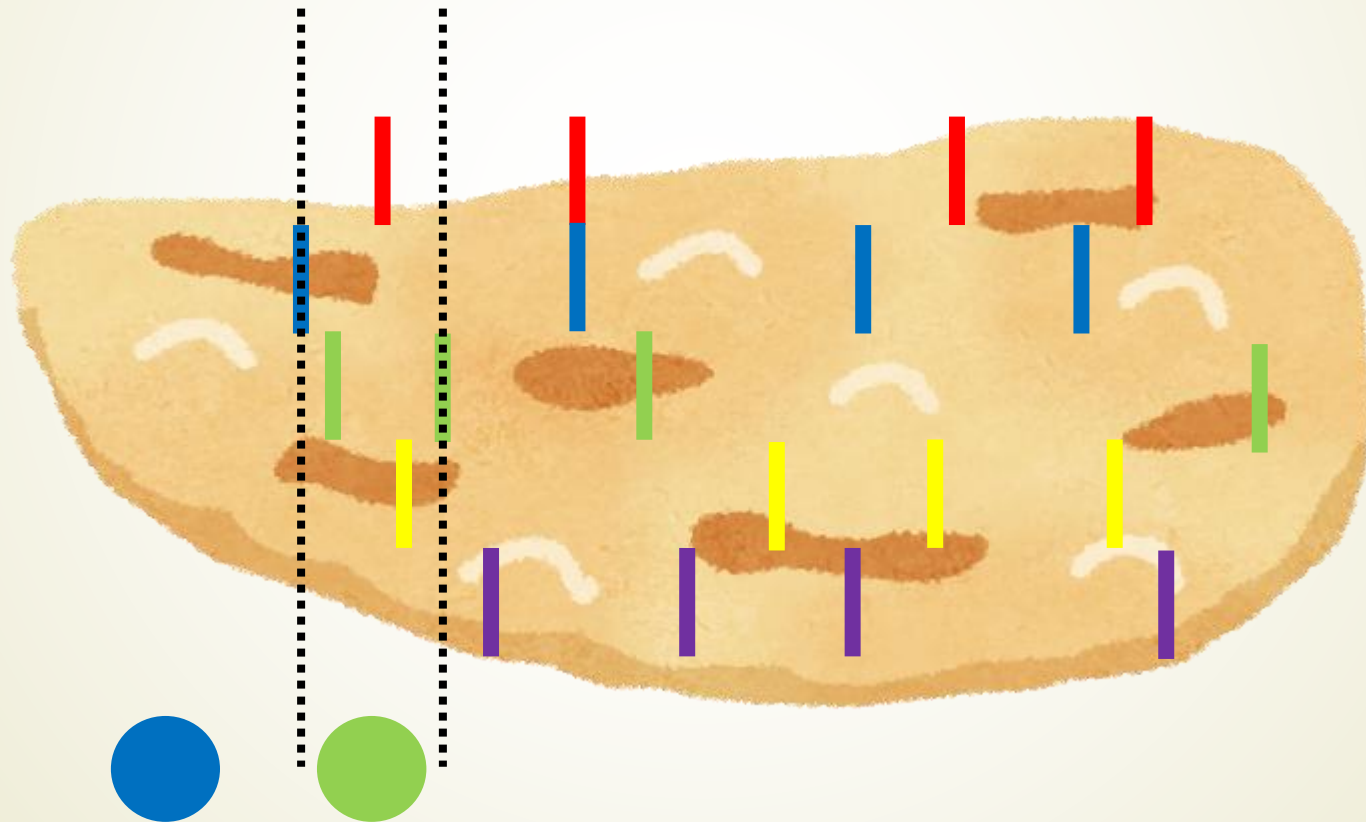
満点 ($N \leq 2000, V_{i,j} \leq 10^5$)



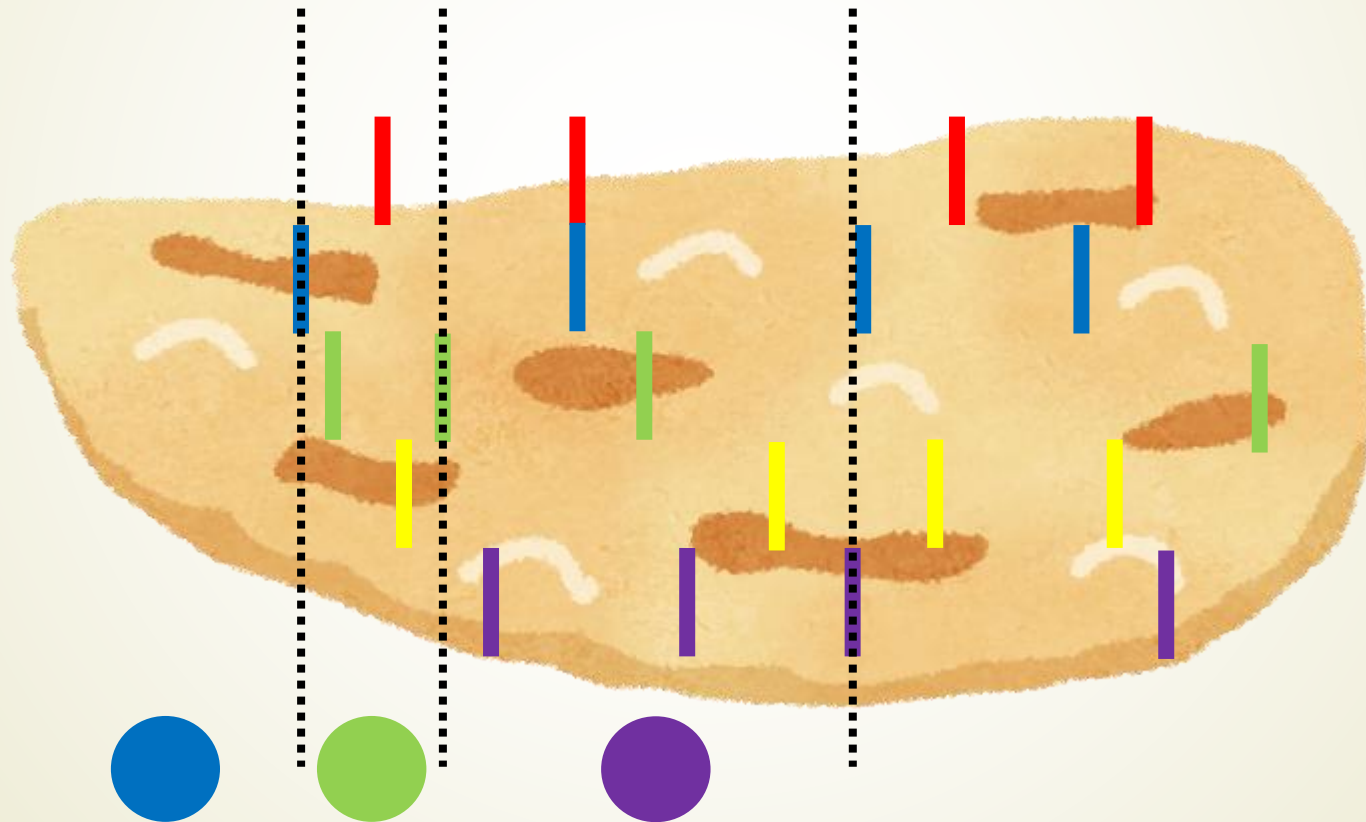
満点 ($N \leq 2000, V_{i,j} \leq 10^5$)



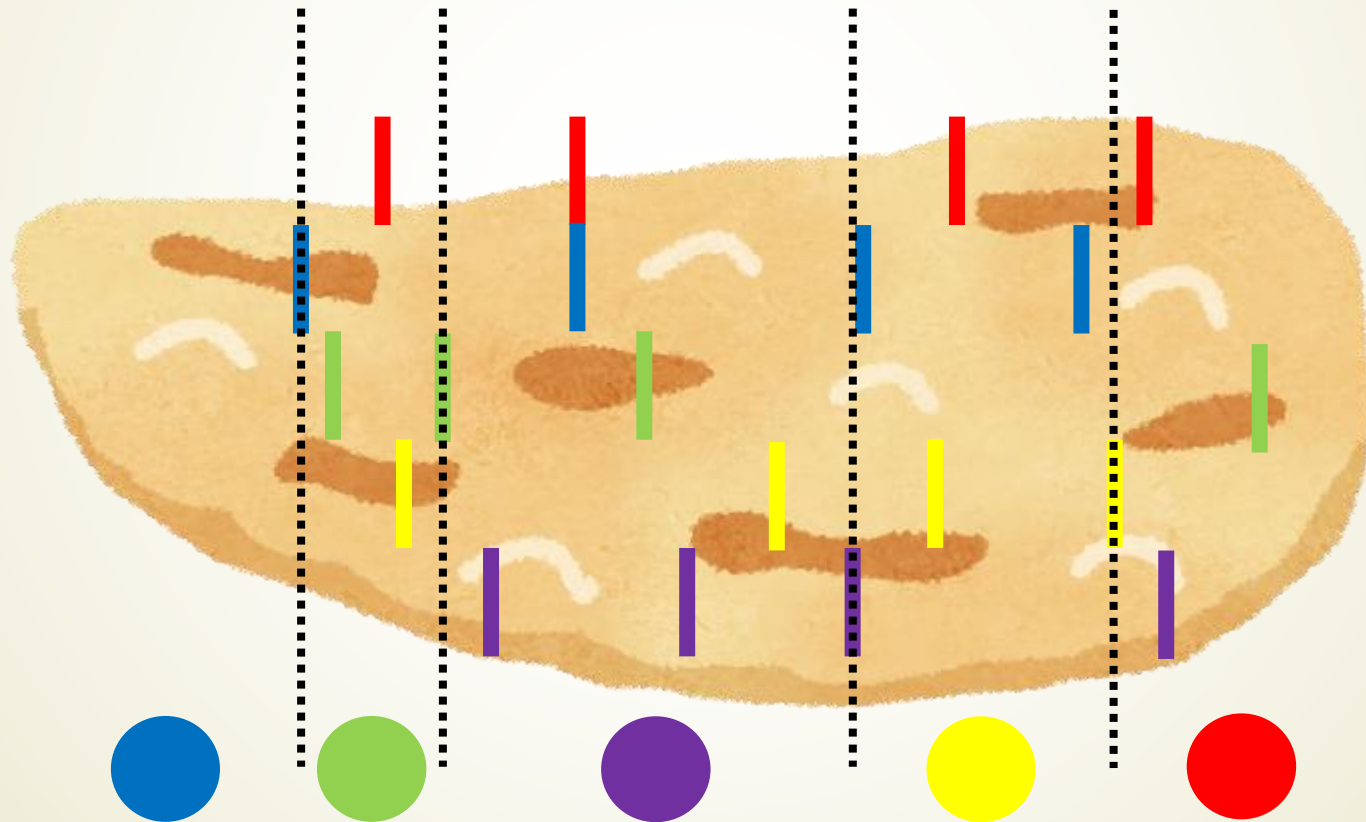
満点 ($N \leq 2000, V_{i,j} \leq 10^5$)



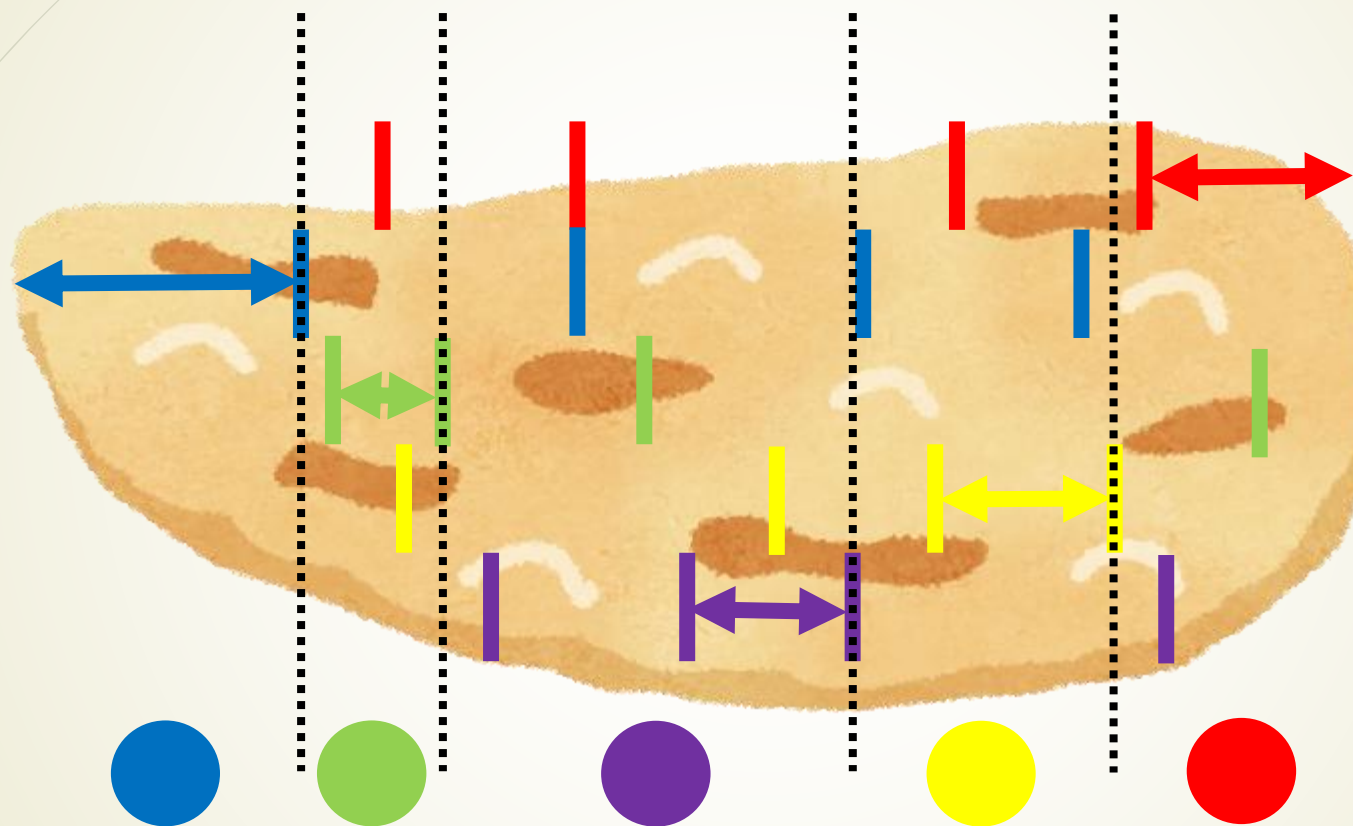
満点 ($N \leq 2000, V_{i,j} \leq 10^5$)



満点 ($N \leq 2000, V_{i,j} \leq 10^5$)



満点 ($N \leq 2000, V_{i,j} \leq 10^5$)



- それぞれ矢印で示した部分で $1/N$ を確保できていることがわかる
- 座標も 左端から (整数)/($N * V$) みたいにしかない

実装

- ▶ 小課題2 よりかなり楽
- ▶ 各人 i についてナンを左から見ていつ j/N になるか記録していけば良い
- ▶ あとは分数の比較とか
- ▶ $O(NL)$

ナン置き場



得点分布

0 点

5 点

24 点

29 点

100 点

得点分布



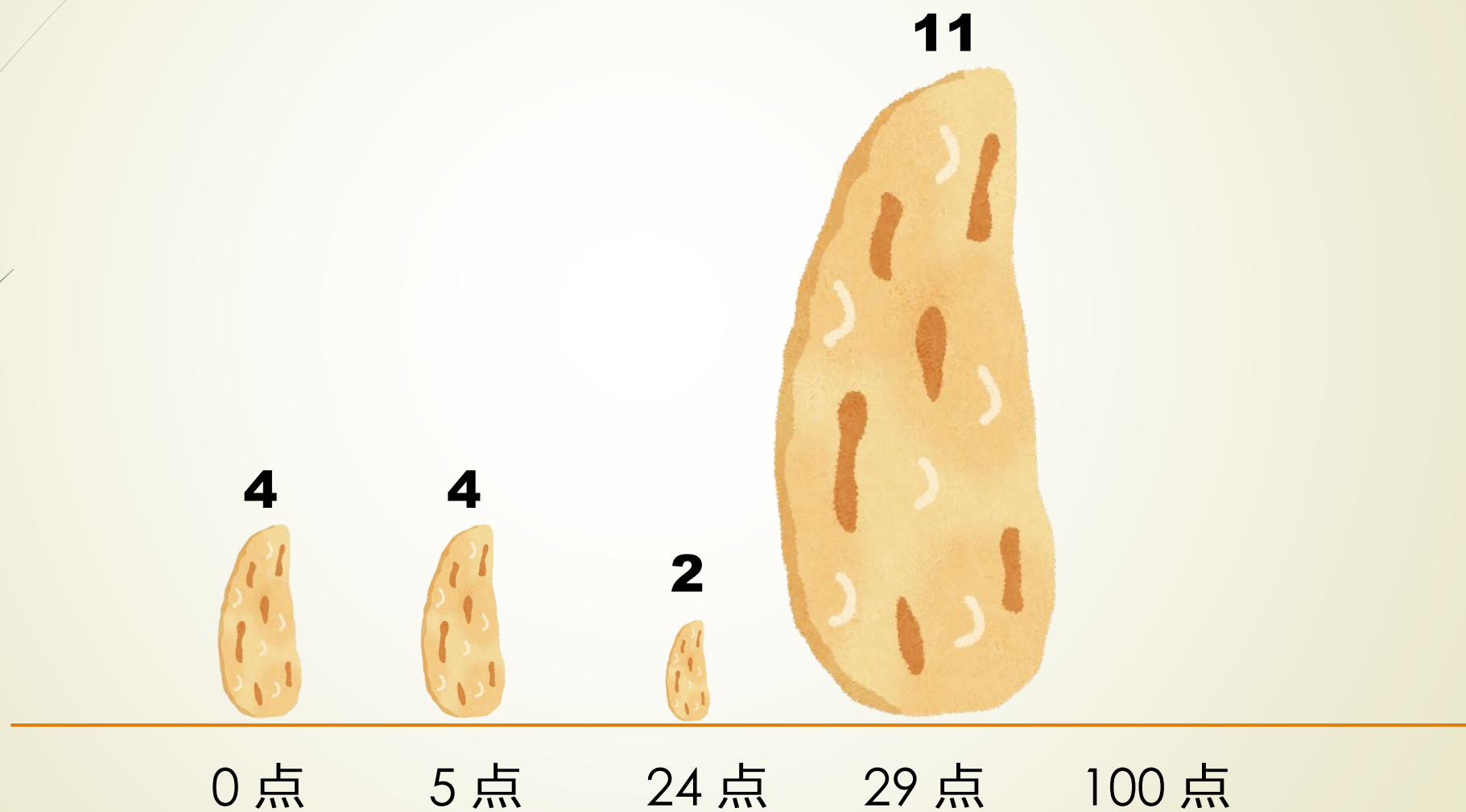
得点分布



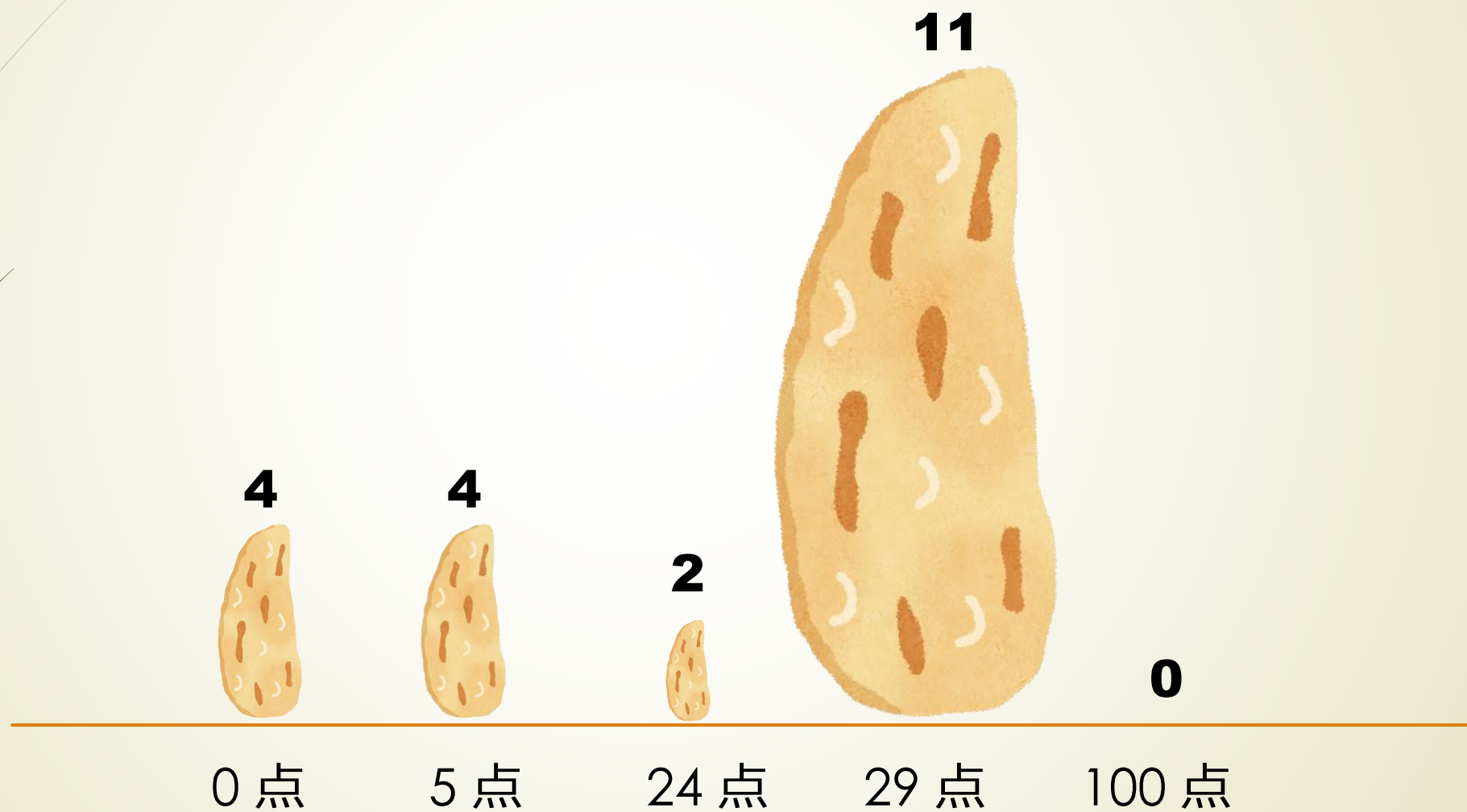
得点分布



得点分布

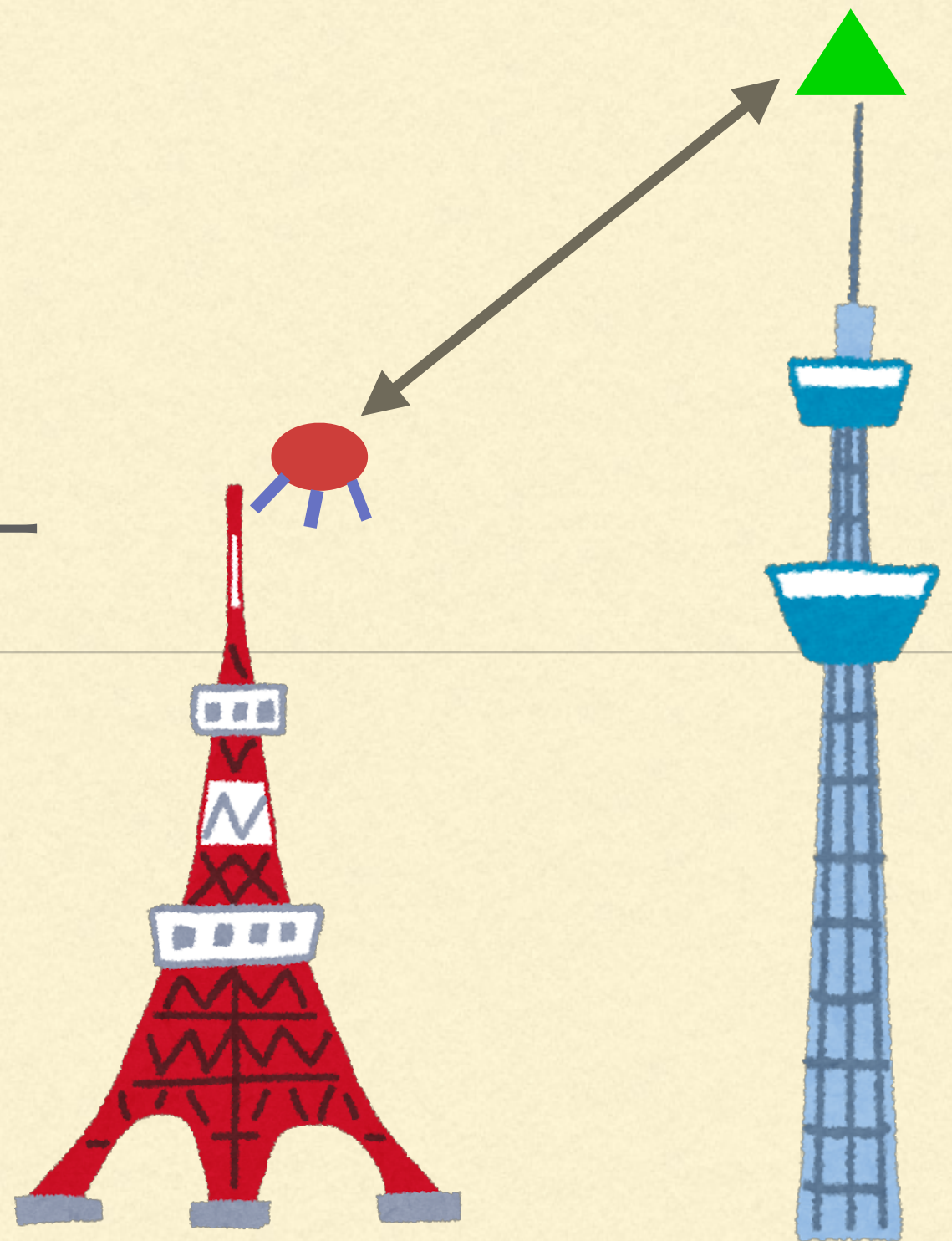


得点分布



ふたつのアンテナ

森田 晃平



問題概要

- アンテナがたくさんある
 - アンテナペア (l, r) は、
$$l + a_l \leq r \leq l + b_l, r - b_r \leq l \leq r - a_r$$
 - の時に通信してて、通信しにくさが $abs(h_i - h_j)$
 - どこが一番通信しにくい？クエリを処理
-

小課題1 (2点)

- ちゃんと起きているか / 問題文が読めたかの確認用
 - 設定が複雑なので、全然書いていいと思います
 - 誤読したら大変！
-

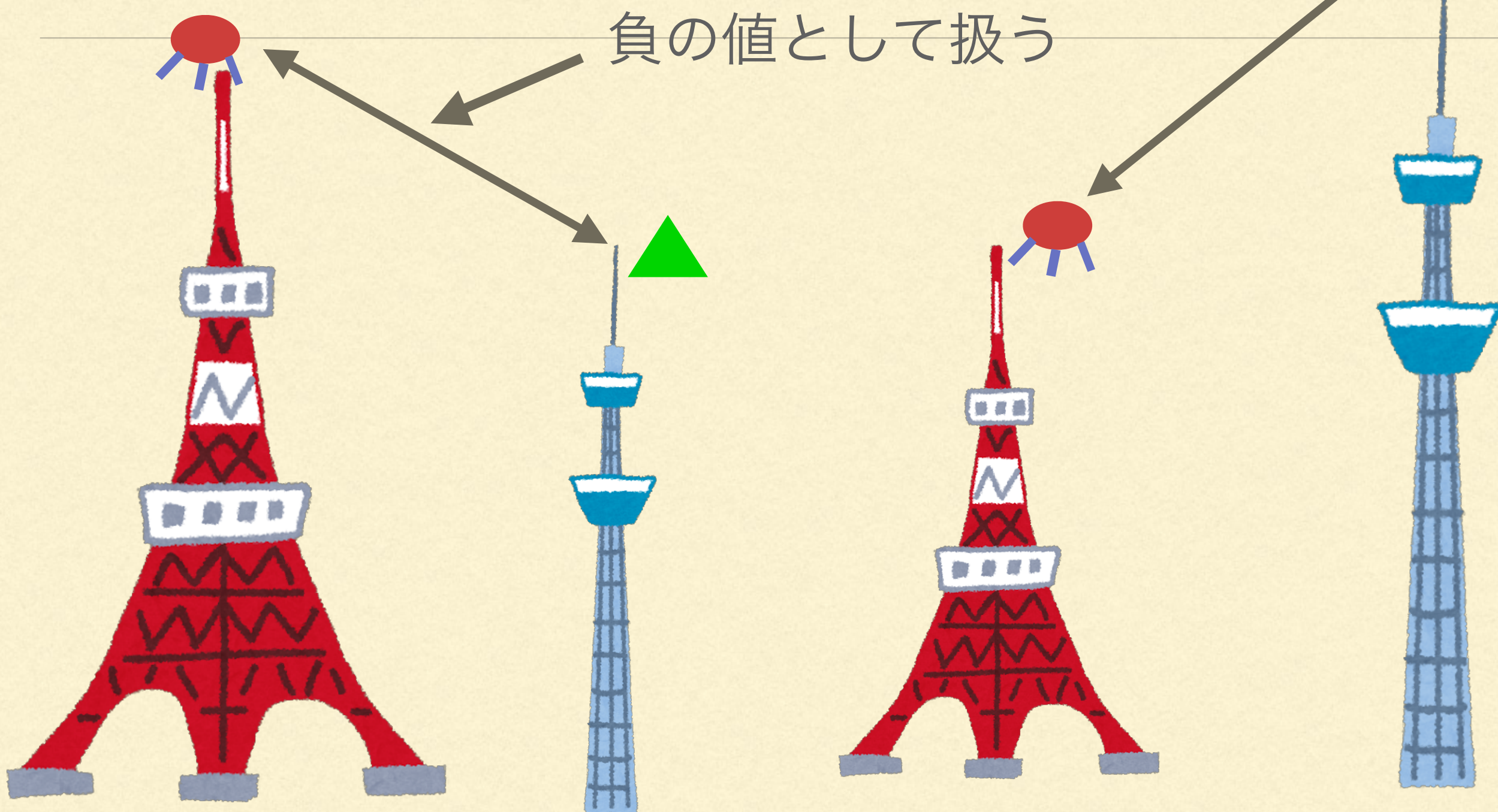
小課題2(11+2点)

- まだそんなに難しくない
 - $f(l, r)$: アンテナペア (l, r) の通信しにくさ ($or - \infty$)
 - $dp[l][r]$: クエリ (l, r) の答え
 - $dp[l][r] = \max(dp[l+1][r], dp[l][r-1], f(l, r))$
 - 前計算 $O(N^2)$, クエリ $O(1)$
-

前処理(0点)

- 小課題3に行く前に
 - $abs(h_j - h_i) = \max(h_j - h_i, h_i - h_j)(i < j)$
 - $abs(h_j - h_i)$ のmaxじゃなくて、 $h_j - h_i$ のmaxと思ってもよい
 - 配列をreverseして2回解く
-

前処理(0点)



小課題3(22+2点)

- ここから難しい
- N がデカイので、愚直に全部のアンテナのペアは試せない

小課題3(22+2点)

- r を固定して、 $(1, r), (2, r), \dots, (r - 1, r)$ を全部同時に試す $\rightarrow r$ を走査
 - $(1, 2)$ を試す
 - $(1, 3), (2, 3)$ を試す
 - $(1, 4), (2, 4), (3, 4)$ を試す
 - $(1, 5), (2, 5), (3, 5), (4, 5)$ を試す
 - ...
-

小課題3(22+2点)

- 条件を眺める
 - $r - b_r \leq l \leq r - a_r \leftarrow$ 区間クエリっぽい
 - $l + a_l \leq r \leq l + b_l \leftarrow$ 走査中にイベントが起こりそう
-

小課題3(22+2点)

- 以下のクエリが処理できればいい
 - 場所 x にアンテナ(高さ h_x)を構築
 - 場所 x のアンテナを爆破
 - $[l, r]$ に生えているアンテナのうち、高さのminは？
-

小課題3(22+2点)

- 以下のクエリが
- 場所 x
- 場所 x
- $[l, r]$ に生えている、高さのminは？



RMQ

満点(65+22+11+2点)

- とても難しい
 - 観賞用
-

満点(65+22+11+2点)

- 基本の方針は変わらない、 r で走査する
 - 走査中に逐次クエリも答えていく
 - 各 l ごとに、 $i = l+1, l+2, \dots, r$ についての $h[i] - h[l]$ というのを管理することにする(d_i)
 - クエリ(l, r)の答えは、 $d_l, d_{l+1}, \dots, d_r$ の $\max$
-

満点(65+22+11+2点)

- 配列 a_i (初期値 INF)、 d_i (初期値 $-INF$)にクエリを飛ばす
 - アンテナ爆破、建設($a_i = INF$ 、 $a_i = h[i]$)
 - 区間についてそれぞれ $d_i = \max(d_i, x - a_i)$
 - 区間の $\max(d_i)$ を求める
-

満点(65+22+11+2点)

- 配列 a_i (初期値 INF)、 d_i (初期値 $-INF$)にクエリを飛ばす
 - アンテナ爆破、建設($a_i = INF$ 、 $a_i = h[i]$)
 - 区間についてそれぞれ $d_i = \max(d_i, \textcolor{red}{x} - a_i)$
 - 区間の $\max(d_i)$ を求める



これは今までの $\max$ だけ持てばいい

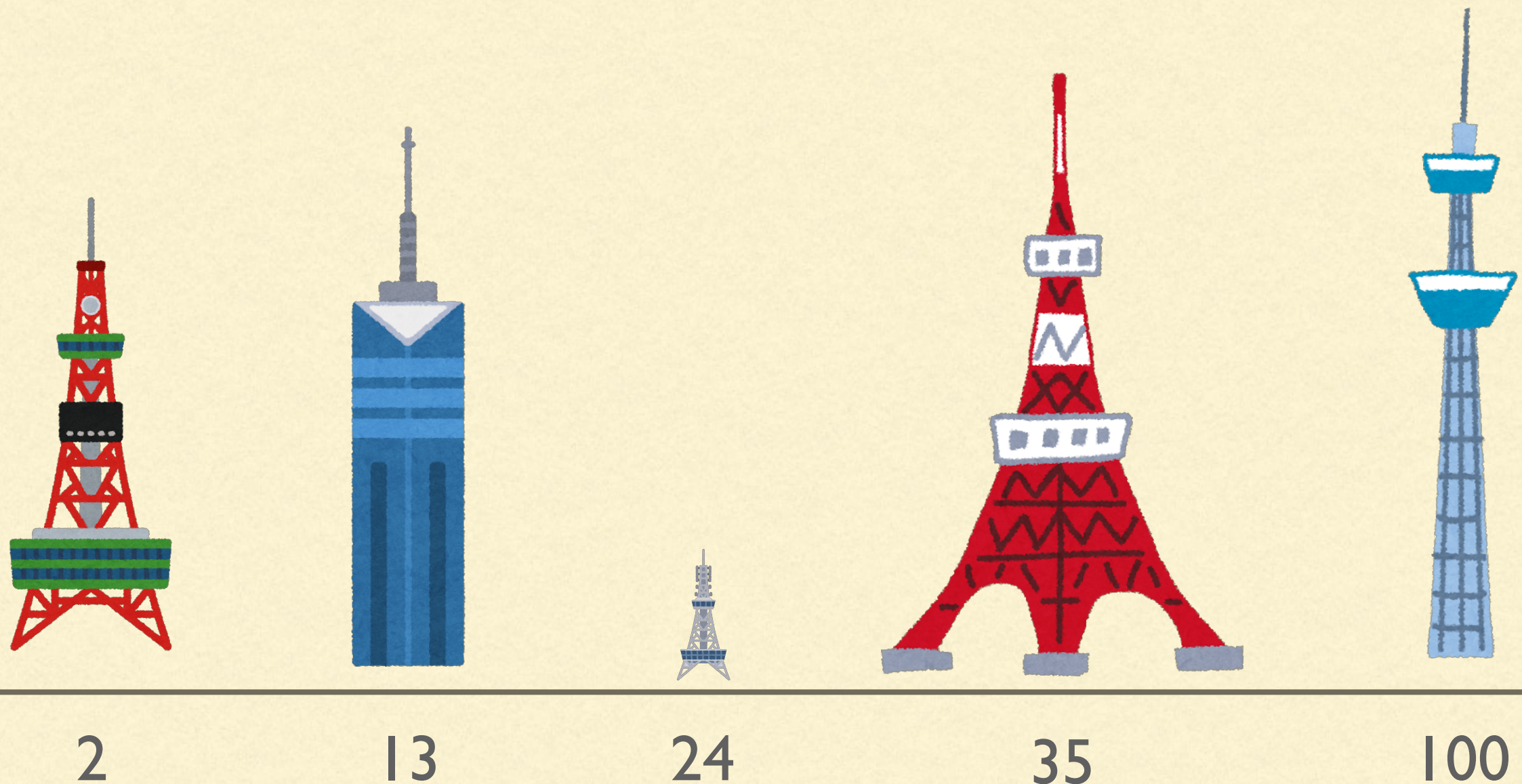
満点(65+22+11+2点)

- 配列 a_i (初期値 INF)、 c_i ($-INF$), d_i (初期値 $-INF$)にクエリを飛ばす
 - アンテナ爆破($a_i = INF$)
 - アンテナ建設($a_i = h[i]$, $c_i = -INF$)
 - 区間についてそれぞれ $c_i = \max(c_i, x)$, $d_i = \max(d_i, c_i - a_i)$
 - 区間の $\max(d_i)$ を求める
-

満点(65+22+11+2点)

- 遅延伝搬 Segment Treeで出来ます
 - c_i を伝搬する
 - ノードごとに持たせる情報
 - $c_i = \max(c_i, x)$ を遅延伝搬したもの
 - a_i のmin
 - d_i のmax
-

得点分布(0点)



Two Dishes DAY2

SORTREEW

問題概要

問題文を読みましょう

小課題1

制約: 制限時間(SやT)の制約が全て同じ

取る順序を気にしなくてよくなる。つまり、カレーと丼を制限時間内に取った個数の組から答えが求まる。また、カレーを取った個数を固定すると丼を取った個数は一意に定まり、これは二分探索で求められる。丼を固定する場合もやる。 $O(N\log M + M\log N)$

あまりに多い小課題

	N,M≤12	N,M≤2000	N,M≤200000	N,M≤1000000
P,Q=1	2	3	4	
P,Qは正			5	6
P,Qの制約なし			7	満点！

考察

空白の時間はなく、丼やカレーの中では手順が決まっているので、

ある手順で点数が取れるかどうかは

もう一種類の食べ物の手順がどこまで進んでいるかだけで決まる(これまでのとり方の順序は関係なく)

問題の変換

$N \setminus M$	0	1	2	3
0	0	1	2	3
1	0	1	2	3
2	1	2	3	4
3	2	3	4	5
4	3	4	5	6

$N \times M$ のグリッドがあり、
マス目の間に左や上から直線が伸びている。

$i-1$ 行目と i 行目の間の線を通ると P_i 点、
 $i-1$ 列目と i 列目の間の線を通ると Q_i 点、もらうことができる

左上から右下に行く時、得られる得点の最大値を求めよ。

各行と列ごとに、もう一方の食べ物を何回食べてもいいか(列や行を何回通ってもいいか)を二分探索や累積和を用いて計算することで、この表を作成することができる。

小課題2

制約: $N \leq 12, M \leq 12$

とり方を全探索する

$O(2^{(N+M)})$

小課題3

制約: $N \leq 2000, M \leq 2000$

とり方を動的計画法で求める。

よくあるグリッド上のDP

小課題3

制約: $N \leq 2000, M \leq 2000$

更新式は

$DP[n][m] = \max(DP[n-1][m] + f1(n, m), DP[n][m-1] + f2(n, m))$

$f1(n, m) =$ 上から n, m に来た時、線を通るなら $P[n]$ 、そうでないなら 0

$f2(n, m) =$ 左から n, m に来た時、線を通るなら $Q[m]$ 、そうでないなら 0

$O(NM)$

ここまでは春合宿に来れる人なら取れるはず(取ってほしい)

小課題4,5,6

主な制約: P, Q は正 (これがうれしい制約です)

とり方を動的計画法で求めたいが、制約がやばい。しかし動的計画法以外で解ける気もしないので、テーブルを試してみる

小課題4,5,6

主な制約: P,Qは正 (これがうれしい制約です)

特徴: $DP[n][m] - DP[n][m-1]$ は行ごとに見るとあまり変化しない? (サンプル1)

このテーブルでも復元はできる

(オレンジ色の線を通りながら、横線の得点とテーブルの数字を足し合わせる)

N \ M	0	1	2	3
0	0	1	2	3
1	0	1	2	3
2	1	2	3	4
3	2	3	4	5
4	3	4	5	6

N \ M	0	1	2	3
0		1	1	1
1		1	1	1
2		1	1	1
3		1	1	1
4		1	1	1

小課題4,5,6

テーブルの変化が少ないとうれしい
何がうれしいか？

→DP[n][]にDP[n-1][]のテーブルを使いまわして、
更新をサボることで高速化できることがある。

ここからテーブルの使い回しをすることにして、
より変化が少なく、元のDPテーブルに復元できるよ
うな新たなDPテーブルを考えていく

他のサンプルでも試す

特徴: $DP[n][m] - DP[n][m-1]$ はあまり変化しない(右図)

$N \setminus M$	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)	$N \setminus M$	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)
0	0	2	9	28	32	47	47	47	0	0	2	7	19	4	15	0	0
1(+16)	16	18	25	44	48	63	63	63	1(+16)	0	2	7	19	4	15	0	0
2(+10)	26	28	35	54	54	63	63	63	2(+10)	0	2	7	19	0	9	0	0
3(+1)	27	29	36	54	54	63	63	63	3(+1)	0	2	7	18	0	9	0	0
4(+16)	43	45	45	54	54	63	63	63	4(+16)	0	2	0	9	0	9	0	0
5(+10)	43	45	45	54	54	63	63	63	5(+10)	0	2	0	9	0	9	0	0

Q:こんなの気づけなくない？

A:はい

ただ、

$DP[n][m] = \max(DP[n-1][m] + f1(n, m), DP[n][m-1] + f2(n, m))$

から $DP[n][m]$ は $\max$ 内の2つの式いずれかと等しく、 $f1$ や $f2$ は n, m それぞれ固定すれば2種類しか値を取らないので、差分を取ると変化少ない気持ちにはなる(かもしれない)

他のサンプルでも試す

特徴2:

マスの左に線があるなら

$$DP[n][m] - DP[n][m-1] = Q[m]$$

偶然か？

N \ M	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)
0	0	2	7	19	4	15	0	0
1(+16)	0	2	7	19	4	15	0	0
2(+10)	0	2	7	19	0	9	0	0
3(+1)	0	2	7	18	0	9	0	0
4(+16)	0	2	0	9	0	9	0	0
5(+10)	0	2	0	9	0	9	0	0

他のサンプルでも試す

特徴2:

マスの左に線があるなら

$$DP[n][m] - DP[n][m-1] = Q[m]$$

理由: 横線はできるだけ左に
いたほうが通りやすい

(短期的には点がもらえなく
なるタイミングまで取るのを
先送りにしたほうがよい)

N \ M	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)
0	0	2	7	19	4	15	0	0
1(+16)	0	2	7	19	4	15	0	0
2(+10)	0	2	7	19	0	9	0	0
3(+1)	0	2	7	18	0	9	0	0
4(+16)	0	2	0	9	0	9	0	0
5(+10)	0	2	0	9	0	9	0	0

テーブルの変形

つまり、 $DP[n][m] - DP[n][m-1] - Q[m]$ も変化が少なそう？

$N \setminus M$	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)	$N \setminus M$	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)
0	0	2	7	19	4	15	0	0	0	0	0	0	0	0	0	0	0
1(+16)	0	2	7	19	4	15	0	0	1(+16)	0	0	0	0	0	0	0	0
2(+10)	0	2	7	19	0	9	0	0	2(+10)	0	0	0	0	-4	-6	0	0
3(+1)	0	2	7	18	0	9	0	0	3(+1)	0	0	0	-1	-4	-6	0	0
4(+16)	0	2	0	9	0	9	0	0	4(+16)	0	0	-7	-10	-4	-6	0	0
5(+10)	0	2	0	9	0	9	0	0	5(+10)	0	0	-7	-10	-4	-6	0	0

テーブルの変形

変化は少ないが、
0以上というよさそうな性質
が崩れるので、
左に線がないマスには
 $DP[n][m] - DP[n][m-1]$ を入れて
みる(これでも復元できる)

N \ M	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)
0	0	0	0	0	0	0	0	0
1(+16)	0	0	0	0	0	0	0	0
2(+10)	0	0	0	0	0	9	0	0
3(+1)	0	0	0	18	0	9	0	0
4(+16)	0	0	0	9	0	9	0	0
5(+10)	0	2	0	9	0	9	0	0

テーブルの変形

変化は少ないが、
0以上というよさそうな性質
が崩れるので、
左に線がないマスには
 $DP[n][m] - DP[n][m-1]$ を入れて
みる(これでも復元できる)

$N \setminus M$	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)
0	0	0	0	0	0	0	0	0
1(+16)	0	0	0	0	0	0	0	0
2(+10)	0	0	0	0	0	9	0	0
3(+1)	0	0	0	18	0	9	0	0
4(+16)	0	0	0	9	0	9	0	0
5(+10)	0	2	0	9	0	9	0	0

テーブルの変形

変化は少ないが、
0以上というよさそうな性質
が崩れるので、
左に線がないマスには
 $DP[n][m] - DP[n][m-1]$ を入れて
みる(これでも復元できる)

N \ M	0	1 (+2)	2 (+7)	3 (+19)	4 (+4)	5 (+15)	6 (+14)	7 (+8)
0	0	0	0	0	0	0	0	0
1(+16)	0	0	0	0	0	0	0	0
2(+10)	0	0	0	0	0	9	0	0
3(+1)	0	0	0	18	0	9	0	0
4(+16)	0	0	0	9	0	9	0	0
5(+10)	0	2	0	9	0	9	0	0

テーブルの更新

さて、どうやって更新するか(これが難しい)

テーブルの更新

縦の更新と横の更新を独立に見たいので、極端な例を試してみる

N \ M	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0	0	2	6	12	20	30	42
1(+1)	1	2	6	12	20	30	42
2(+3)	4	4	6	12	20	30	42
3(+5)	9	9	9	12	20	30	42
4(+7)	16	16	16	16	20	30	42
5(+9)	25	25	25	25	25	30	42

N \ M	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

極端な例

(おさらい) テーブルの n 行 m 列の式 $\text{Diff}[n][m]$ は

左に線がある:

$$\text{Diff}[n][m] =$$

$$\text{DP}[n][m] - \text{DP}[n][m] - Q[m] = 0$$

左に線がない:

$$\text{Diff}[n][m] =$$

$$\text{DP}[n][m] - \text{DP}[n][m]$$

$N \setminus M$	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

わかったこと

縦の線が消えた時:

消えた線の右下のdiffに、
その線の点(Q[m])が加算
される

理由: 差は埋まらず、単
純にテーブルの式が変わ
るため

N \ M	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

わかったこと

横の線が消えた時:

消えた線の右下を(n,m)とする

$DP[n][m] - DP[n][m-1] \geq P[n]$ のとき、

$DP[n][m]$ は変わらないが、

$DP[n][m-1]$ はk増えるので

$Diff[n][m]$ はk減少する。

$N \setminus M$	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

わかったこと

横の線が消えた時
(おおざっぱに):

$DP[n][m] - DP[n][m-1] < k$ のとき、

$DP[n][m]$ は横から遷移するようになる。

さらに、 $DP[n][m]$ の値も余った分だけ増えて、右のほうに伝搬していく

$N \setminus M$	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

わかったこと

横の線が消えた時

(くわしく):

消えた線の右下を(n,m)とする

$DP[n][m] - DP[n][m-1] < P[n]$ のとき、

$DP[n][m] = DP[n][m-1] + f(n, m)$ となる、

$DP[n][m-1]$ は k 増え、 $DP[n][m]$ も
 $Q[m] - diff[n][m]$ 増える

$Diff[n][m]$ は 0 になる。

$N \setminus M$	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

わかったこと

横の線が消えた時

(くわしく):

さらに左から遷移している
DPの値が全て増えるので、
次に上から遷移しているマ
スで今までの議論を $Q[m]$ の
代わりに $Q[m]-diff[n][m]$ を
用いて同様に行う。

$N \setminus M$	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

わかったこと

計算量は？

まず、値の加算は縦の線の数ぶんだけ、つまり $O(M)$ 回

値の減算は伝搬があるが、減算する場所が $O(M)$ 箇所しかなく、1度の減算で N 回減算すると $N-1$ 箇所消えるのでならずと

値の減少は $O(N+M)$ 回

$N \setminus M$	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

わかったこと

計算量は？

減算する場所の管理は
setなどを使うと全体で
 $O(M \log M)$ くらいでできる

全体で $O((N+M) \log(N+M))$
とかで抑えられる

$N \setminus M$	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

小課題4,5

伝搬を1回分しか処理できない(実装や考察不足など)

→P,Q=1の小課題だけ正解

diffを取らず、セグ木の区間加算等logを余分につける

→N,M $\leq$ 200000だけ正解

小課題7 満点解法

P,Qに制約がない場合を考える

線が消えるときの、diffに対する操作がちゃんとできればいい

今の理解では操作が怪しくなるので、操作について考え直す

わかったこと(みなおし)

縦の線が消えた時
($Q[m] > 0$):

消えた線の右下のdiffに、
その線の点($Q[m]$)が加算される

理由: 上からの遷移はどうしても縦の線を通るが、左からの遷移は縦の線を通らないため

$N \setminus M$	0	1 (+2)	2 (+4)	3 (+6)	4 (+8)	5 (+10)	6 (+12)
0		0	0	0	0	0	0
1(+1)		1	4	6	8	10	12
2(+3)		0	2	6	8	10	12
3(+5)		0	0	3	8	10	12
4(+7)		0	0	0	4	10	12
5(+9)		0	0	0	0	5	12

満点解法

diffに対する操作に対する影響は(得点が正なら)同じ
得点が負になる場合を考える

よく考えると、今まで減っていた部分がかわりに増えて、増えていた部分が減ることになることがわかる。

満点解法

操作を考える(行)

i 行目の $P[i] > 0$ の横線が消える:

線の右下にあるマスに減算(伝搬する)

i 行目の $P[i] < 0$ の横線が消える:

線の右下にあるマスに加算

満点解法

操作を考える(列)

j 列目の $Q[j] > 0$ の縦線が消える:

線の右下にあるマスに加算

j 列目の $Q[j] < 0$ の縦線が消える:

線の右下にあるマスに減算(伝搬する)

満点解法

非対称に見えた操作の原因は行か列かの違いではなく、正か負かの違いにあった。

(これはdiffの下限が0であることに起因している?)

満点解法

以下のクエリを投げて処理していく

Add(x,v):diff[x]にvを加算

Decrease(x,v):

diff[x]<vのとき:diff[x]=0として $y > x$ かつdiff[y]となる最小のyに対し、Decrease(y,v-diff[x])をする

diff[x]>=vのとき:diff[x]にvを減算

満点解法

操作を抽象的に考える(行)

i 行目の $P[i] > 0$ の横線が消える:

$\text{Decrease}(x, P[i])$

i 行目の $P[i] < 0$ の横線が消える:

$\text{Add}(x, -P[i])$

x は線の右下の座標

満点解法

操作を抽象的に考える(列)

j 列目の $Q[j]>0$ の縦線が消える:

Add($x, Q[j]$)

j 列目の $Q[j]<0$ の縦線が消える:

Decrease($x, -Q[j]$)

x は線の右下の座標

満点解法

これで全ての更新を処理することができ、線の情報から $DP[n][m]-DP[n][m-1]$ のテーブルに復元して、更に左下を経由するルートを使って $DP[N][M]$ を復元できる。

計算量は加算と減算がそれぞれならしで $O(N+M)$ 回でできるので、 $O((N+M)\log(N+M))$ で抑えられる

実装上の注意

マスに対する操作と線に対する操作があるので、気をつけないと添字に+1が大量発生し、困る

行ごとに加算と減算をそれぞれ分けてやること(加算を先にやらないとWA)

得点分布

15点 11人

10点 5人

0点 2人

3点 1人

54点 1人

65点 1人

ふたつの

坂部 圭哉

ふたつの



坂部 圭哉

ふたつの

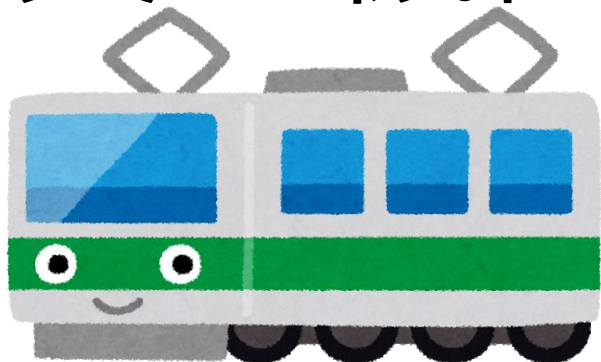
交通機関

交通機関

坂部 圭哉

ふたつの

交通機関



交通機関



坂部 圭哉

IOI 2019

IOI 2019



IOI 2019



IOI 2019



IOI 2019



IOI 2019



IOI 2019



アゼルバイジャン

IOI 2019



Azerbaijan

IOI 2019



AzerBaijan

問題概要

- 連結な重み付き無向グラフ
- 一部の辺の情報は **Azer** に
残りの情報は **Baijan** に与えられる
- **Azer** は、頂点 **0** からそれぞれの頂点までの距離を知りたい
- **58,000** ビットまで双方向に情報を送ることができる

小課題1 ($A = 0$)

- $A = 0$



小課題1 ($A = 0$)

- $A = 0$
- **Azer**の辺が無い

小課題1 ($A = 0$)

- $A = 0$
- **Azer**の辺が無い
- 全辺が**Baijan**にある

小課題1 ($A = 0$)

- $A = 0$

- **Azer**の辺が無い

- 全辺が**Baijan**にある

→ **Baijan**は都市0からの
距離(答え)が分かる

小課題1 ($A = 0$)

1. Baijanが距離を求める
2. 距離をAzerに送る

1. Baijan が距離を求める

- $N \leq 2,000$

- Dijkstra 法

時間 $O((N + B)\log B)$

2. 距離を **Azer** に送る

- $N \leq 2,000 < 2^{11}$
- (重み) $\leq 500 < 2^9$

2. 距離を **Azer** に送る

- $N \leq 2,000 < 2^{11}$
- (重み) $\leq 500 < 2^9$
- (距離) $< 2^{20}$

2. 距離をAzerに送る

- $N \leq 2,000 < 2^{11}$
- (重み) $\leq 500 < 2^9$
- (距離) $< 2^{20}$

→ (頂点, 距離)を送る

$$1,999(11 + 20) = 61,969 \text{ bit}$$

2. 距離をAzerに送る

- $N \leq 2,000 < 2^{11}$
 - (重み) $\leq 500 < 2^9$
 - (距離) $< 2^{20}$
- 頂点番号順に距離だけ
 $1,999 \times 20 = 39,980$ bit

2. 距離をAzerに送る

- 実装

BaijanはInitB内で
SendBを連続で呼び出す
Azerは累計bit数を覚えて
20bitごとに整数に変換

小課題2 ($B \leq 1,000$)

- $B \leq 1,000$



小課題2 ($B \leq 1,000$)

- $B \leq 1,000$
- **Baijan**の辺が少ない

小課題2 ($B \leq 1,000$)

- $B \leq 1,000$
- Baijanの辺が少ない
→ Baijanの辺の情報を
全て送れないだろうか

小課題2 ($B \leq 1,000$)

- $S[j] \leq 2,000 < 2^{11}$

- $T[j] \leq 2,000 < 2^{11}$

- $D[j] \leq 500 < 2^9$

→ $(S[j], T[j], D[j])$ を送る

$$1,000(11+11+9) = 31,000 \text{ bit}$$

小課題3 ($A + B = N - 1$)

- $A + B = N - 1$



小課題3 ($A + B = N - 1$)

- $A + B = N - 1$

- 木



小課題3 ($A + B = N - 1$)

- $A + B = N - 1$

- 木

- $B \leq N - 1$, 多くない

→ **Baijan**の全辺を
送れないだろうか

小課題3 ($A + B = N - 1$)

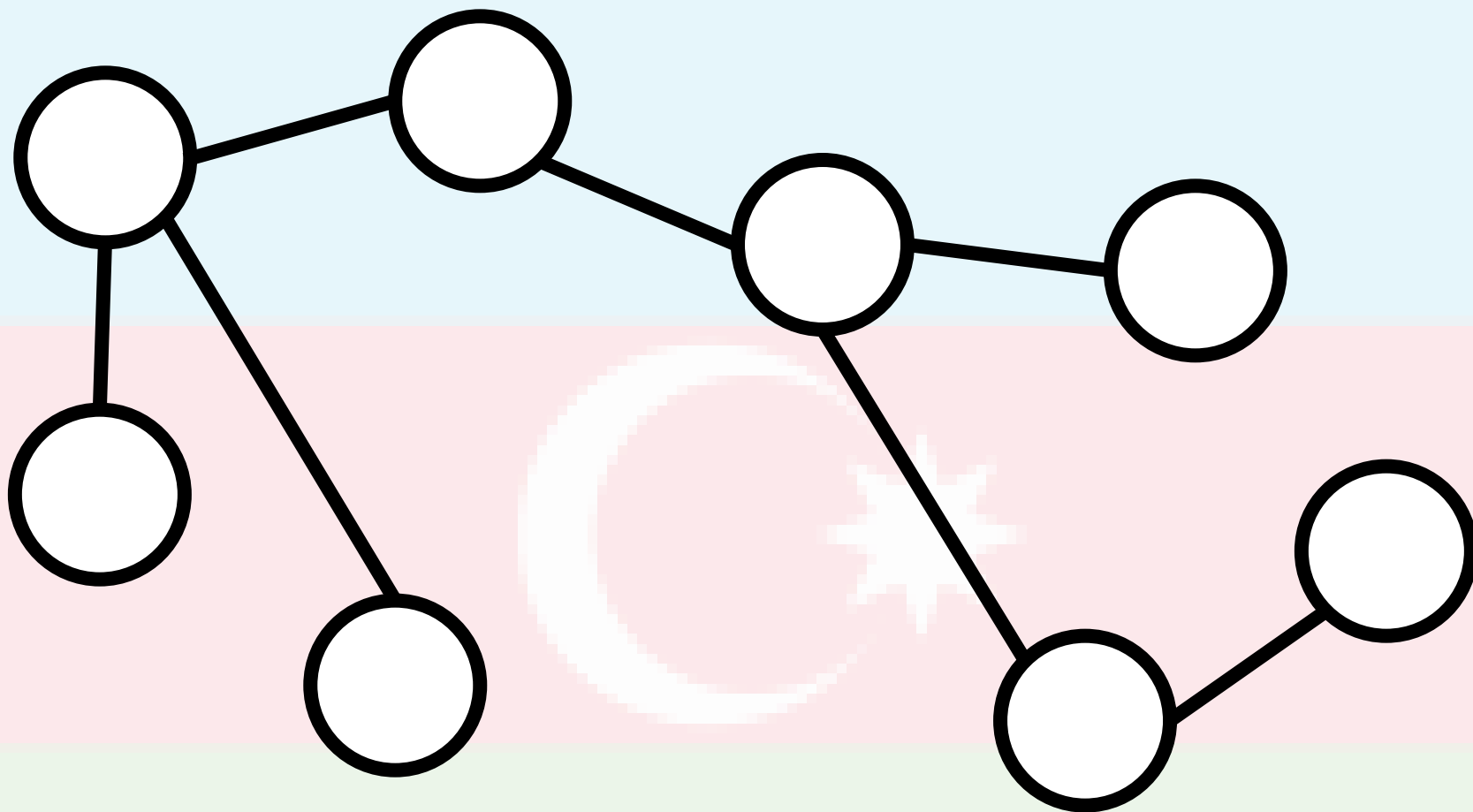
もし愚直に送ると...

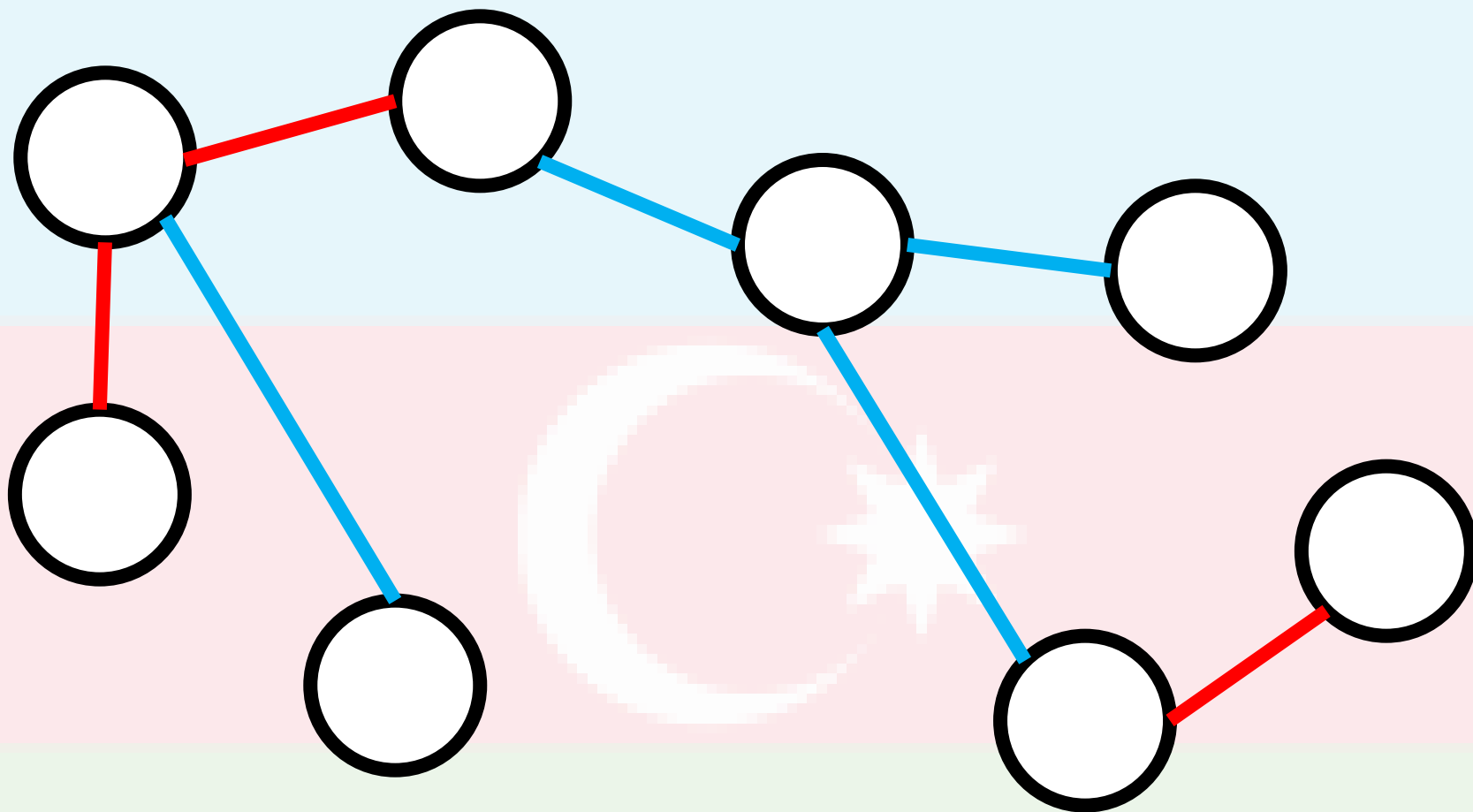
$$1,999(11+11+9)=61,969 \text{ bit}$$

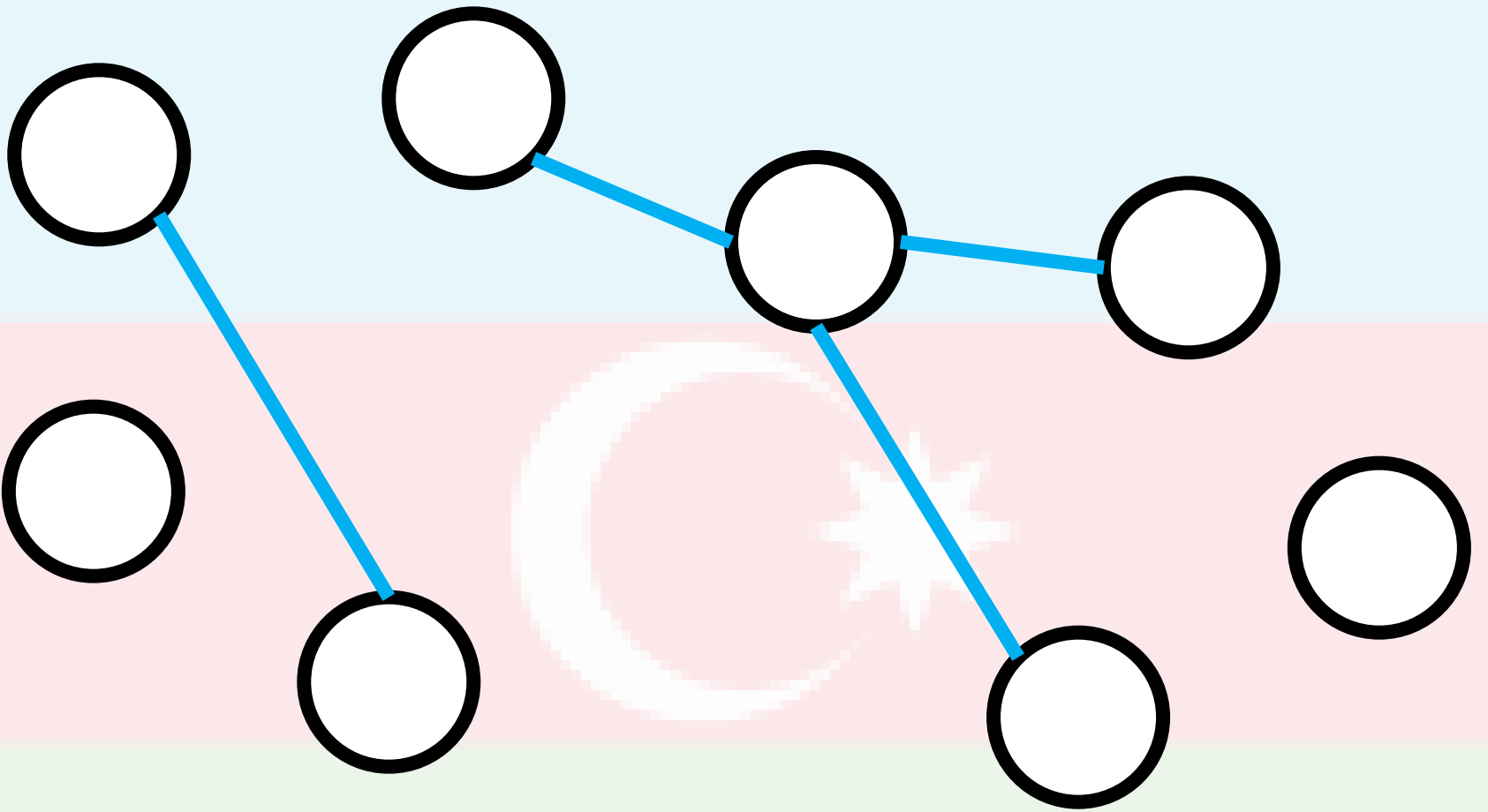
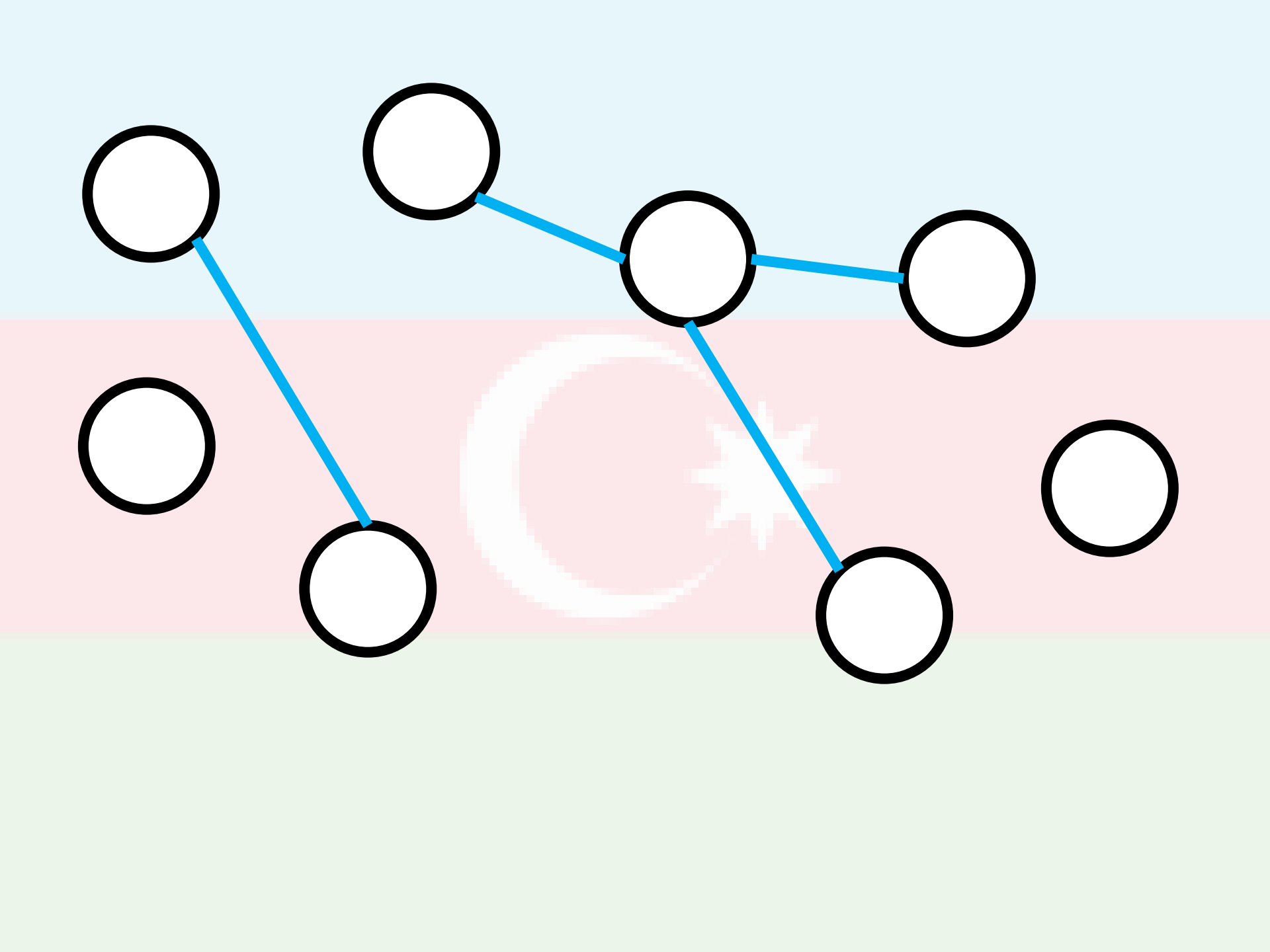
小課題3 ($A + B = N - 1$)

木の性質を考える









小課題3 ($A + B = N - 1$)

Baijanの持っている辺は



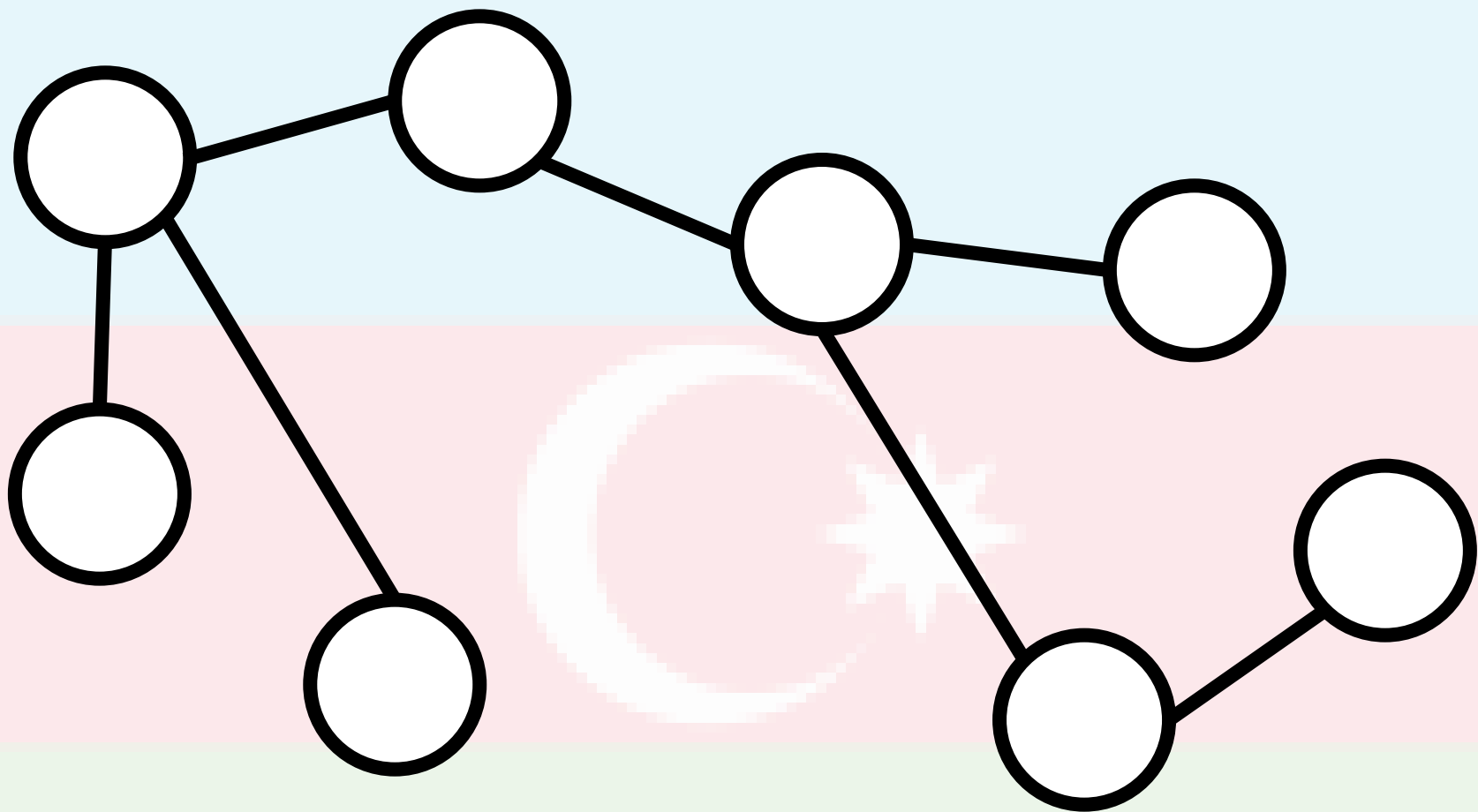
小課題3 ($A + B = N - 1$)

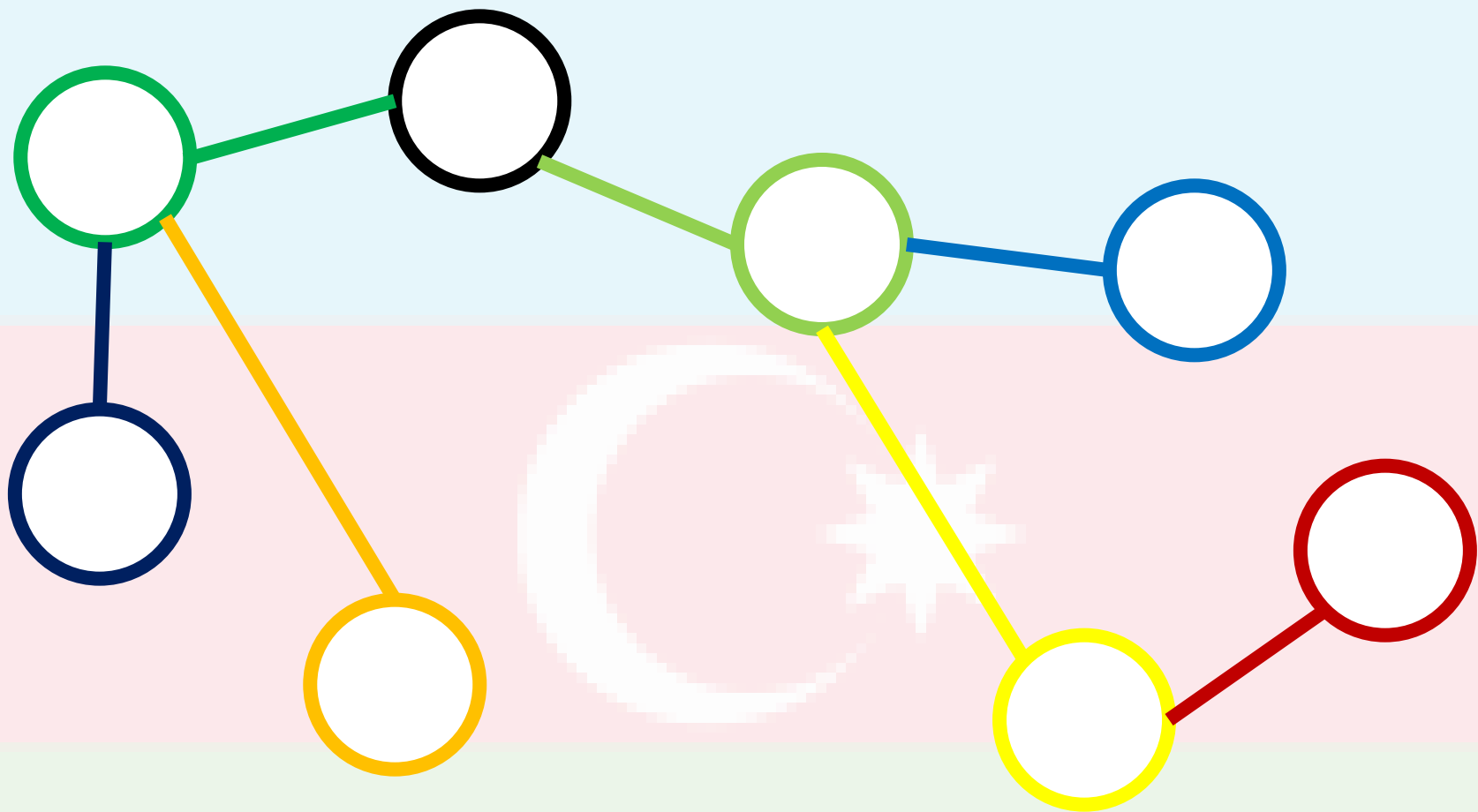
Baijanの持っている辺は

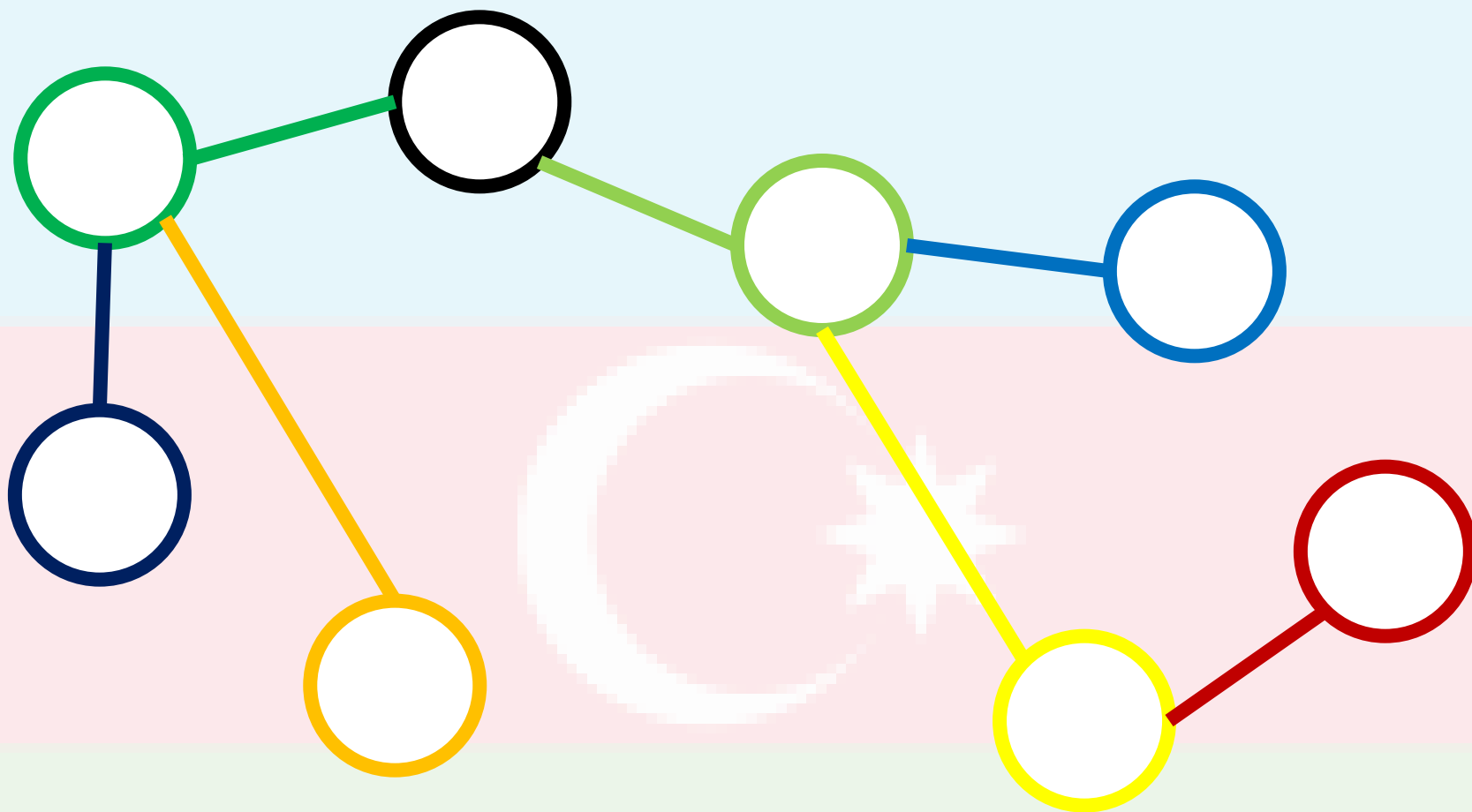
木
森

...木の集合体

もう少し木の性質を考える







1頂点を除き、頂点と辺が対応
(この対応はDFS等で得られる)

小課題3 ($A + B = N - 1$)

1. Baijanの辺を
頂点と対応づける
2. k 回目には、
頂点 k に対応する辺の情報
(反対の頂点, 重み)を送る

$$1,999(11 + 9) = 39,980 \text{ bit}$$

小課題4 ($N \leq 900$)

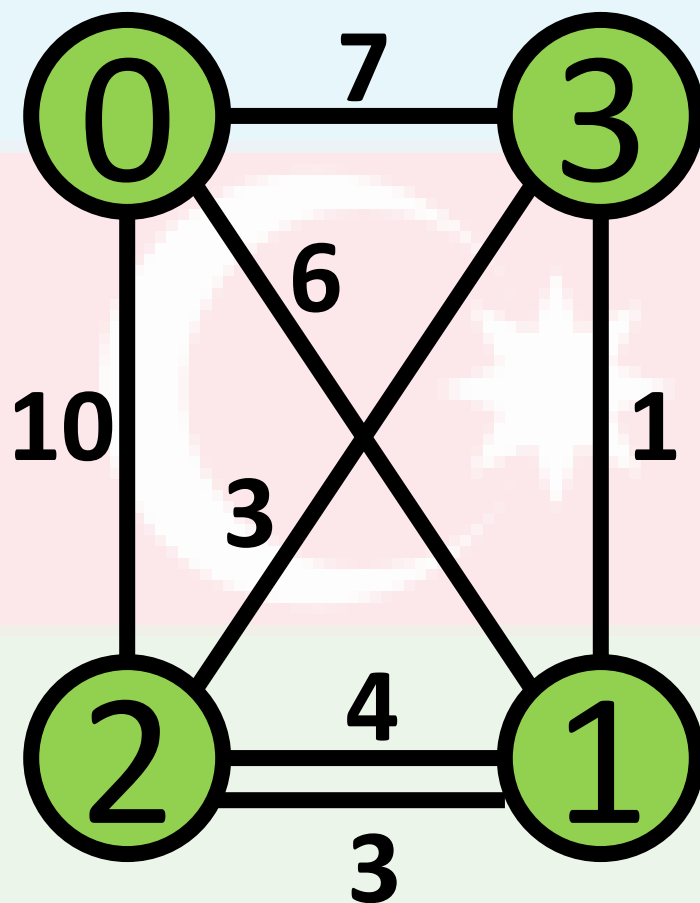
- 特に嬉しい性質なし



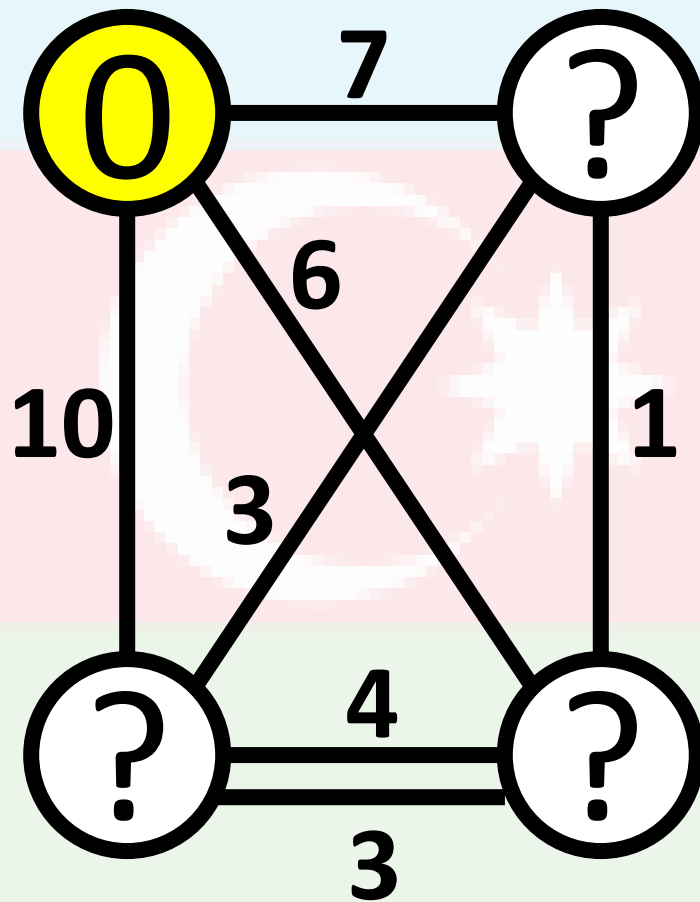
小課題4 ($N \leq 900$)

- 嬉しい性質なし
- 「一点から各頂点までの距離を知りたい」
- Dijkstra法を応用できる？

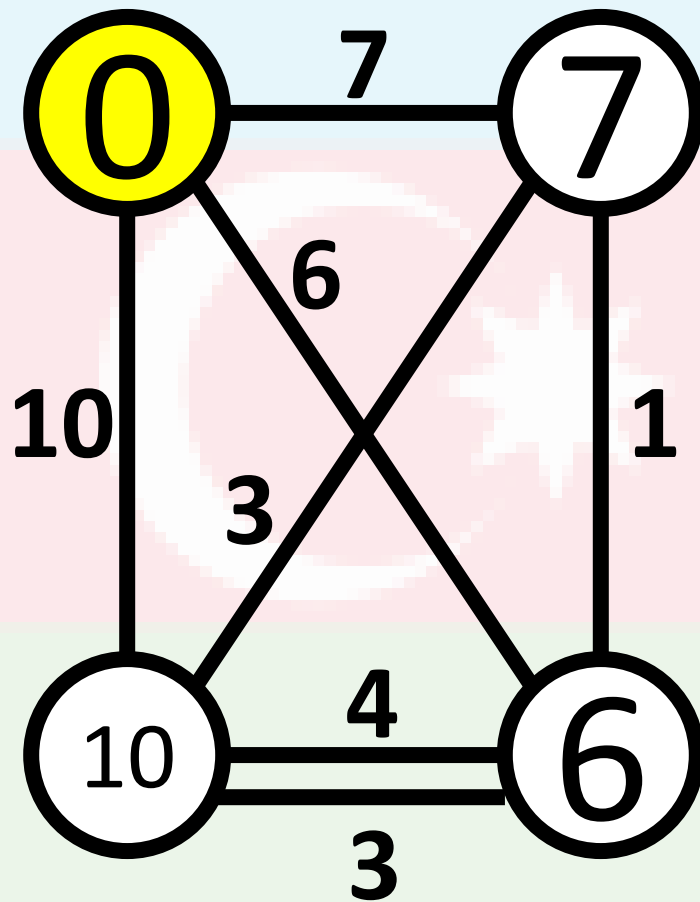
Dijkstra法



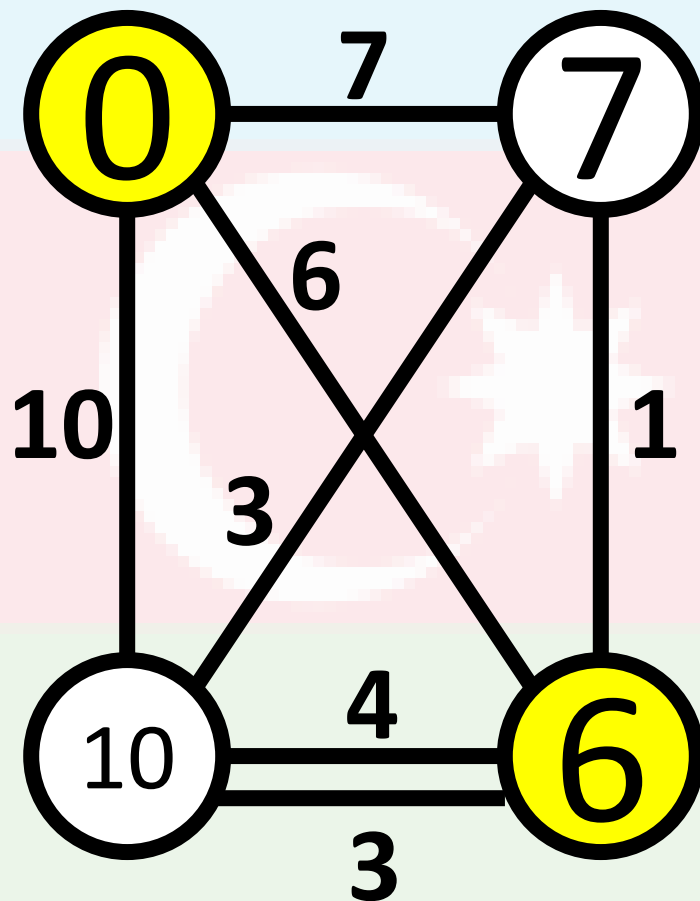
Dijkstra法



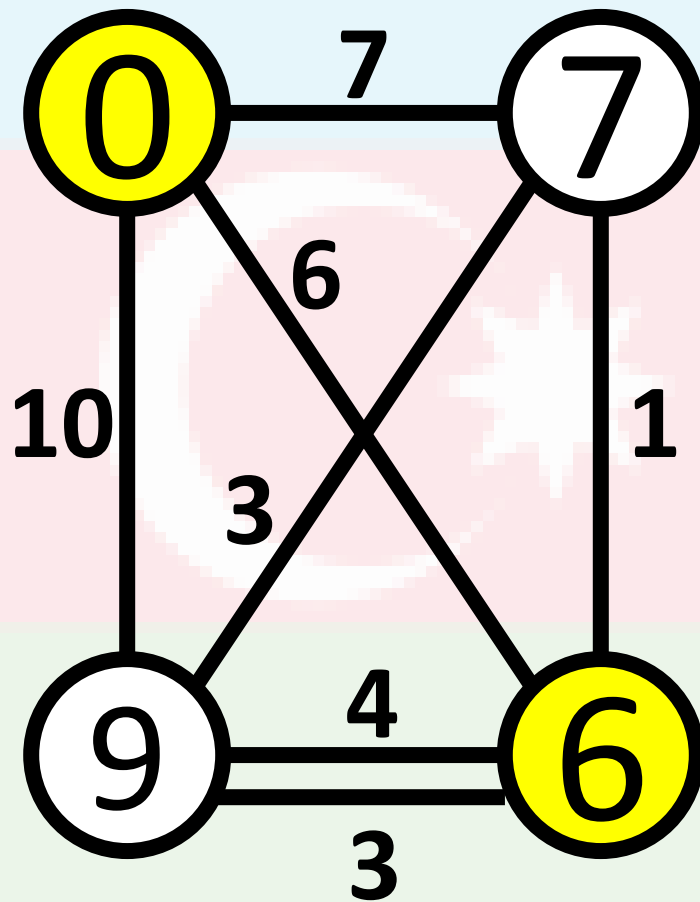
Dijkstra法



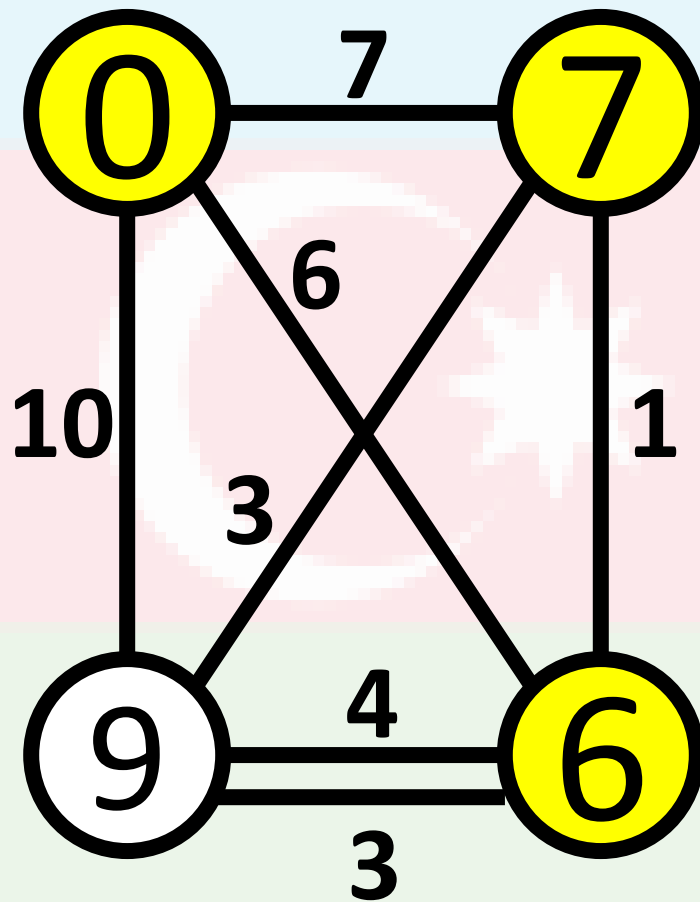
Dijkstra法



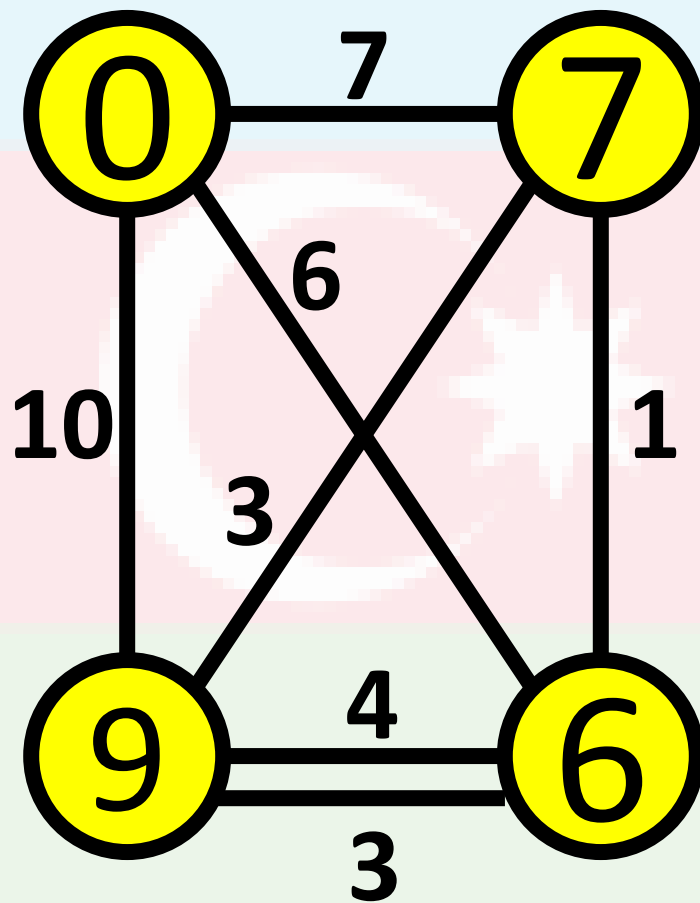
Dijkstra法



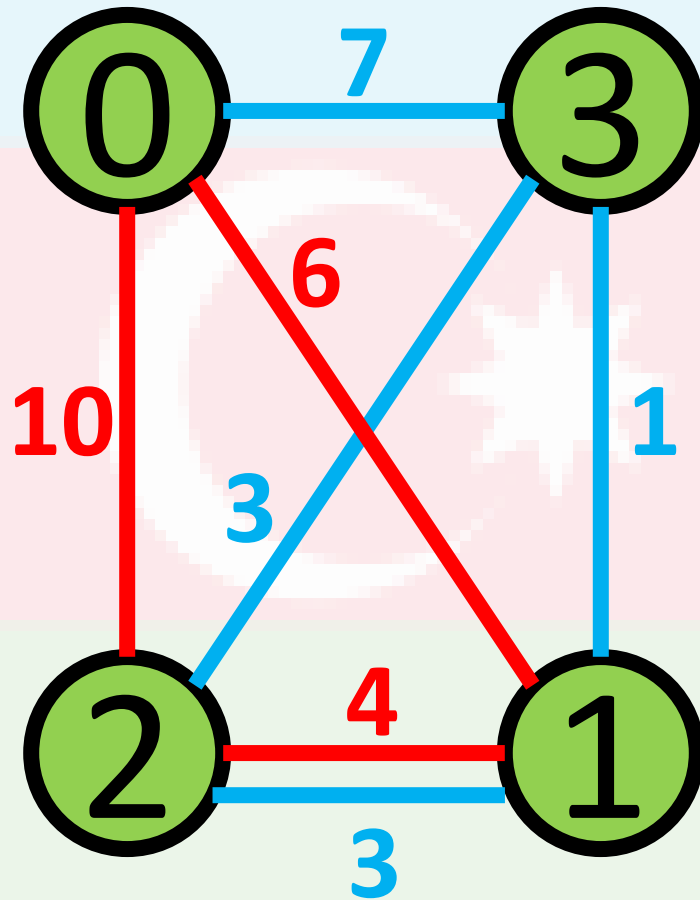
Dijkstra法



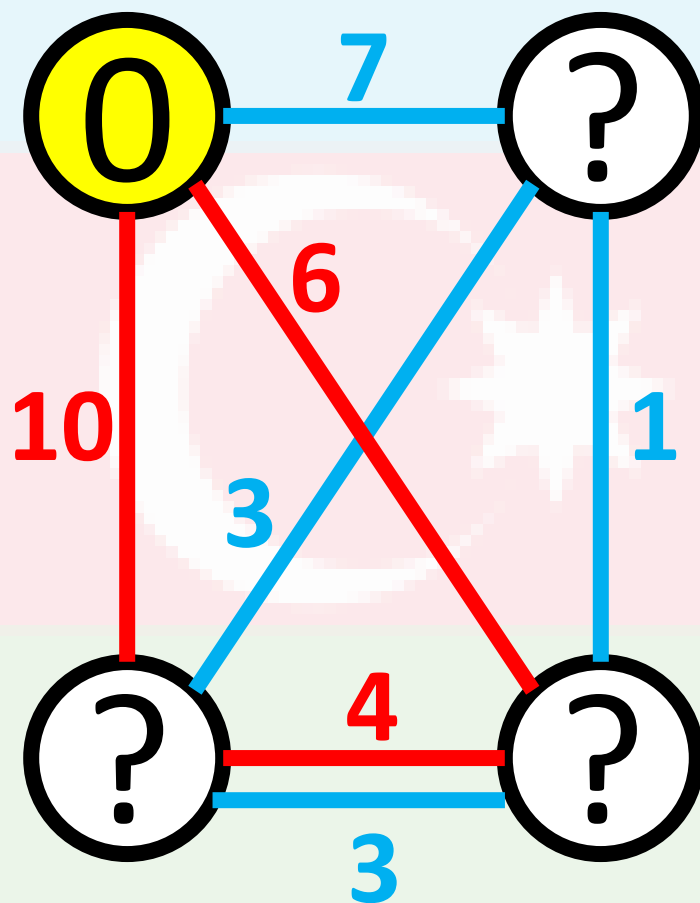
Dijkstra法



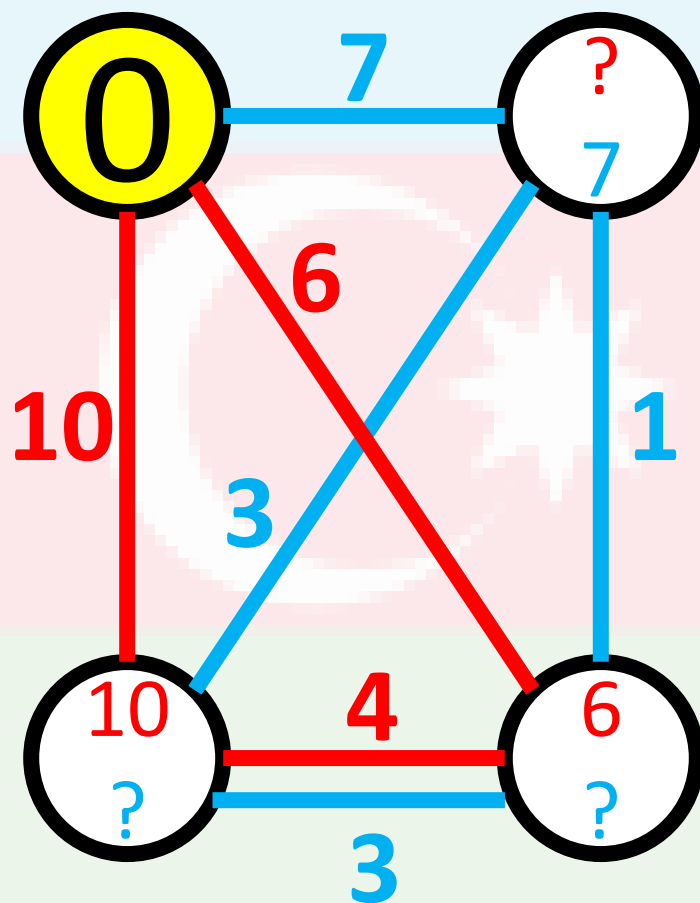
辺が2種類あるとき



辺が2種類あるとき

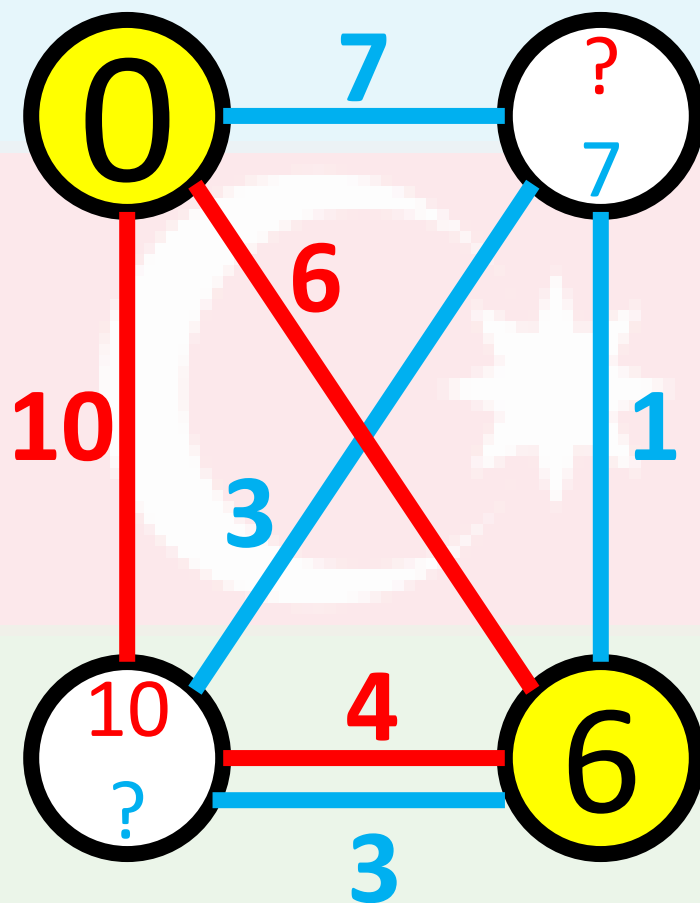


辺が2種類あるとき



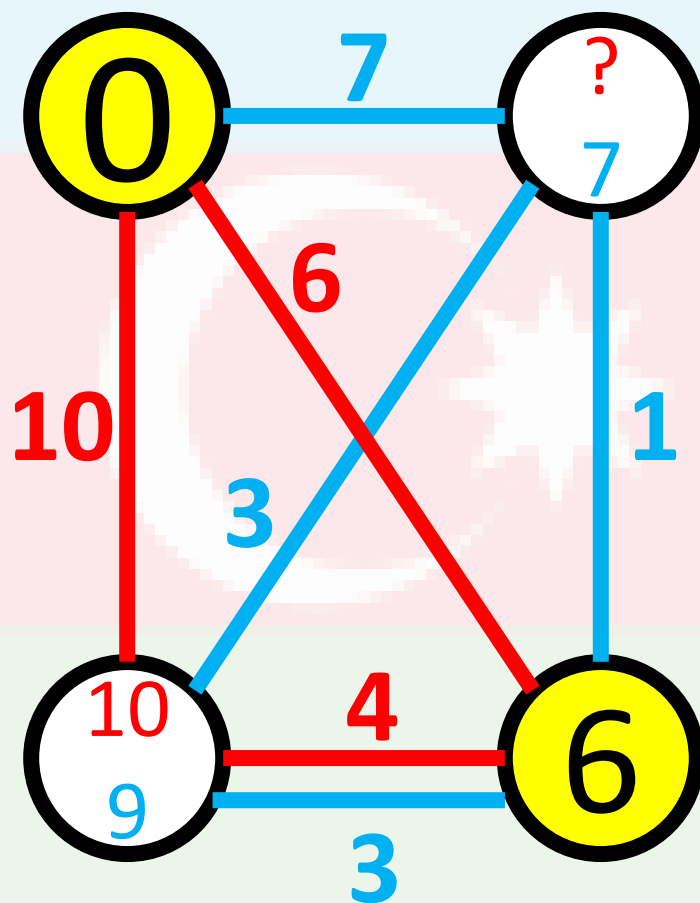
辿る辺の種類で最小値を2種類持つ

辺が2種類あるとき



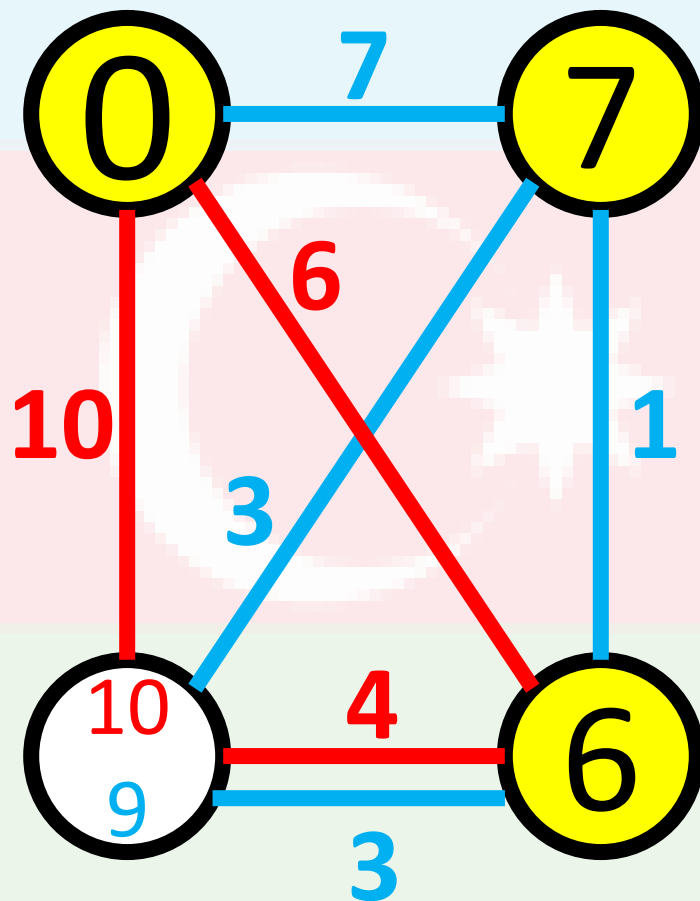
辿る辺の種類で最小値を2種類持つ

辺が2種類あるとき



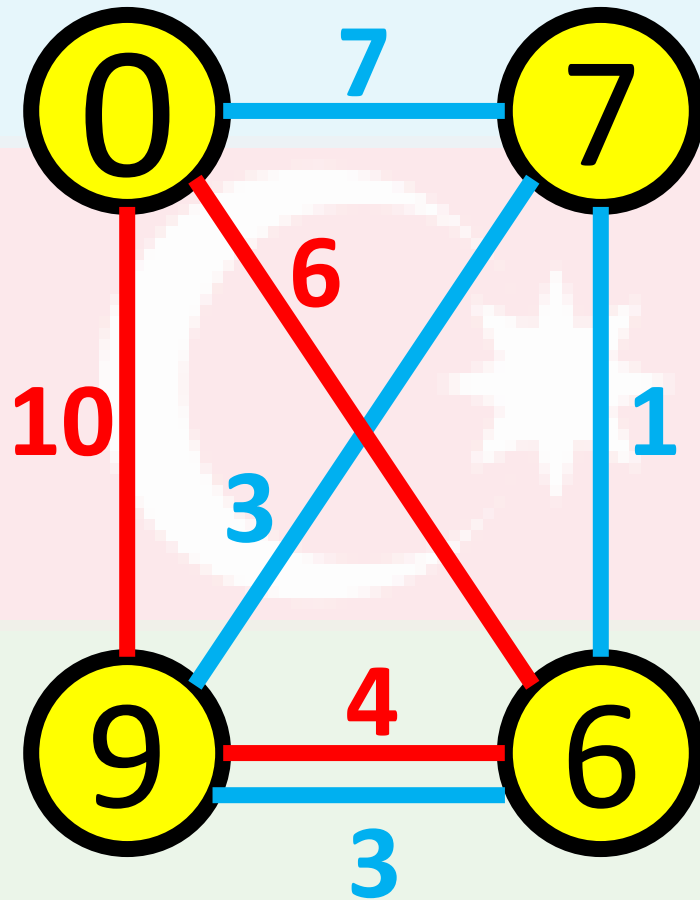
辿る辺の種類で最小値を2種類持つ

辺が2種類あるとき



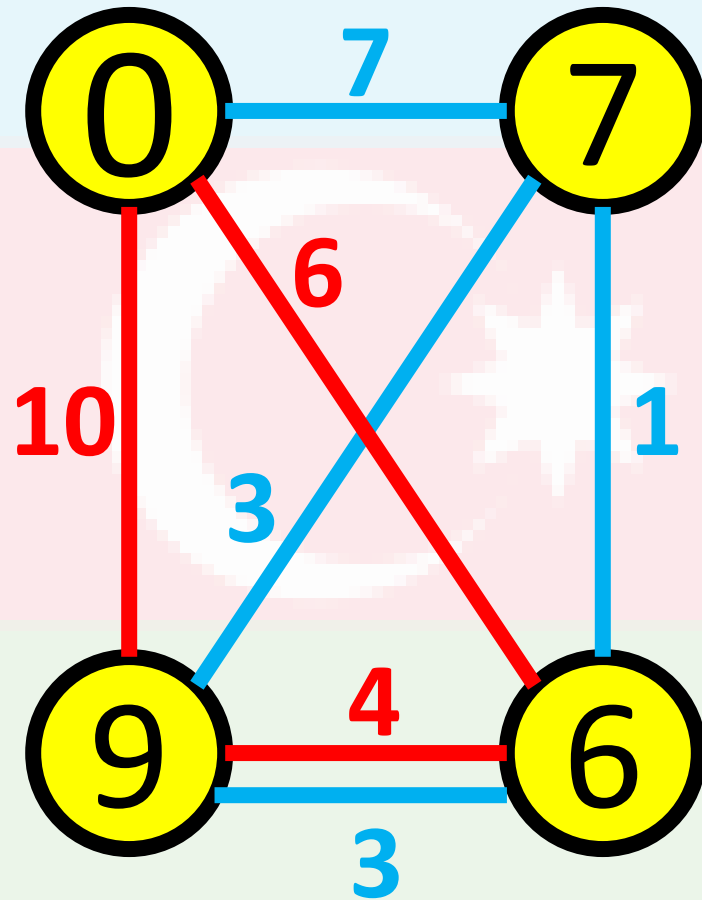
辿る辺の種類で最小値を2種類持つ

辺が2種類あるとき



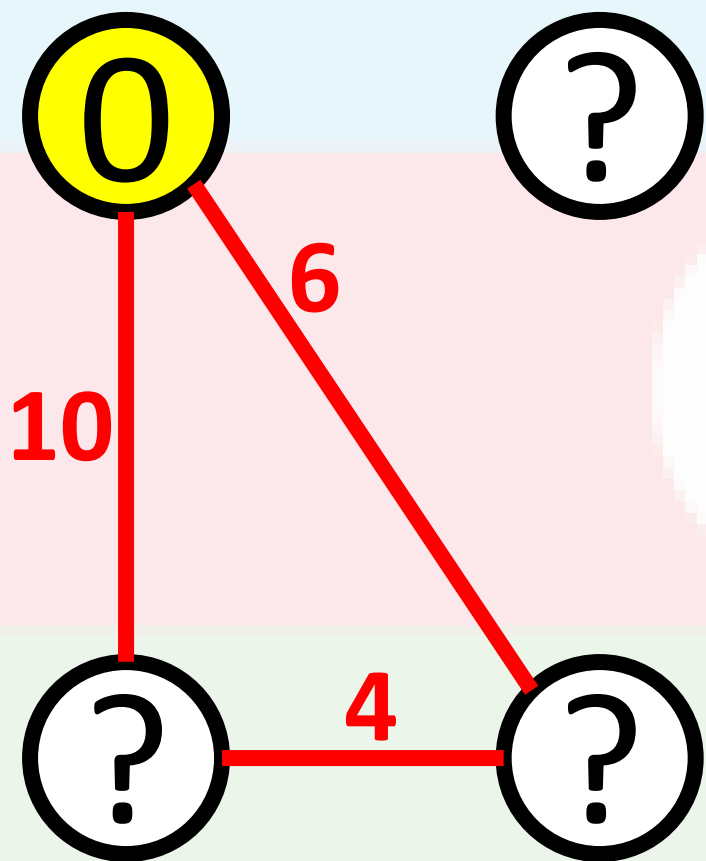
辿る辺の種類で最小値を2種類持つ

辺が2種類あるとき

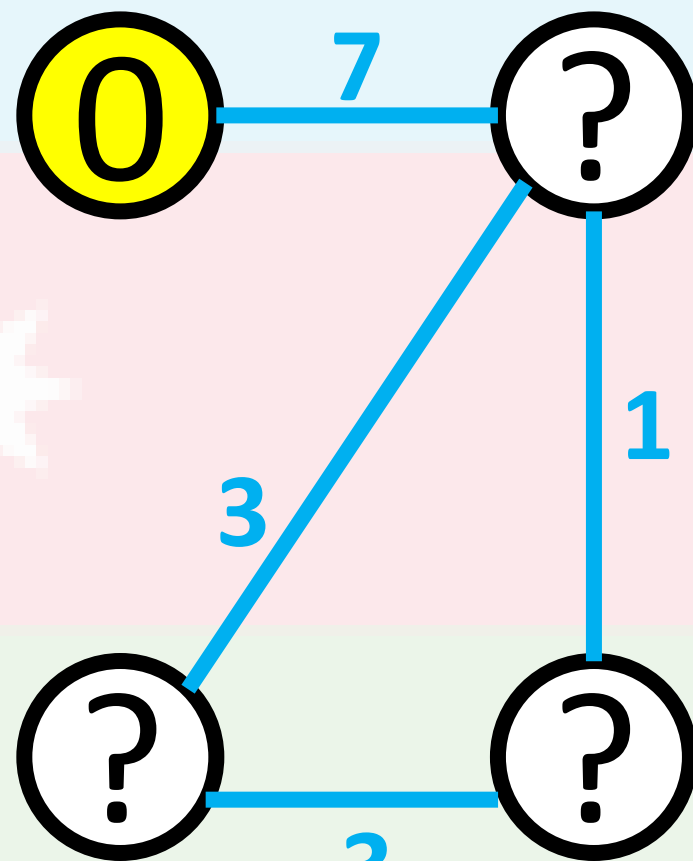


これを2人でできないか?

2人でやってみる

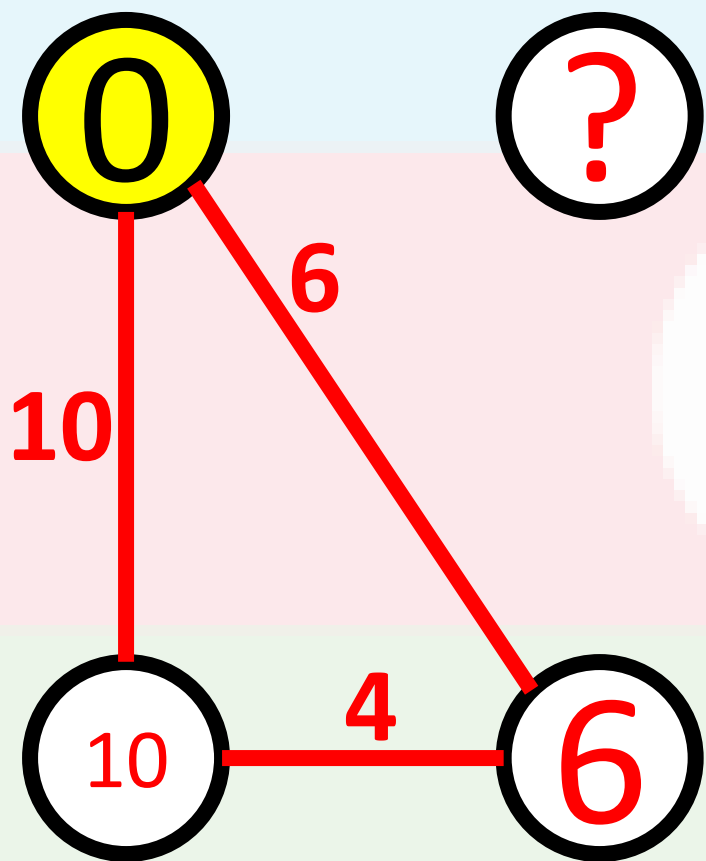


Azer

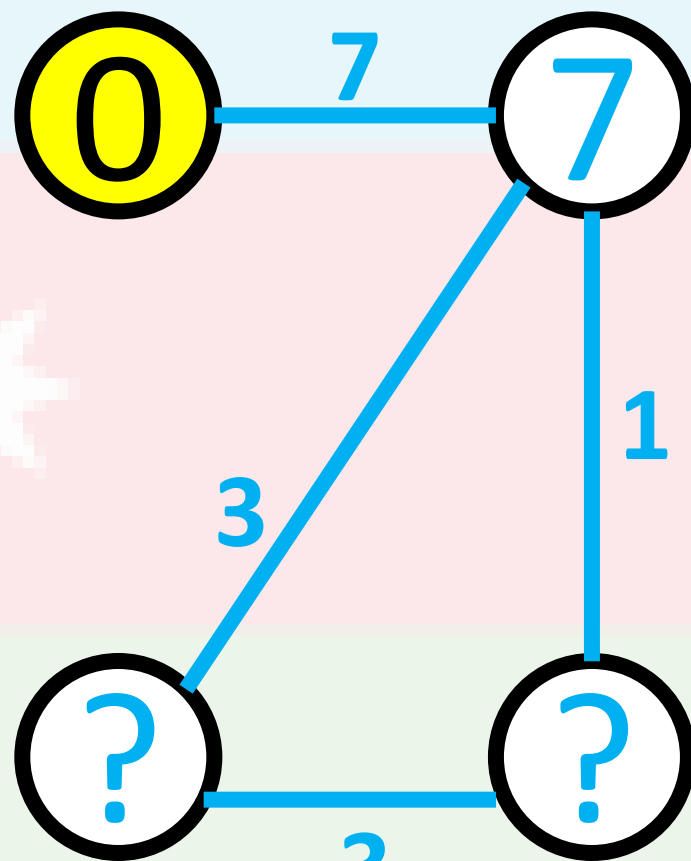


Baijan

2人でやってみる

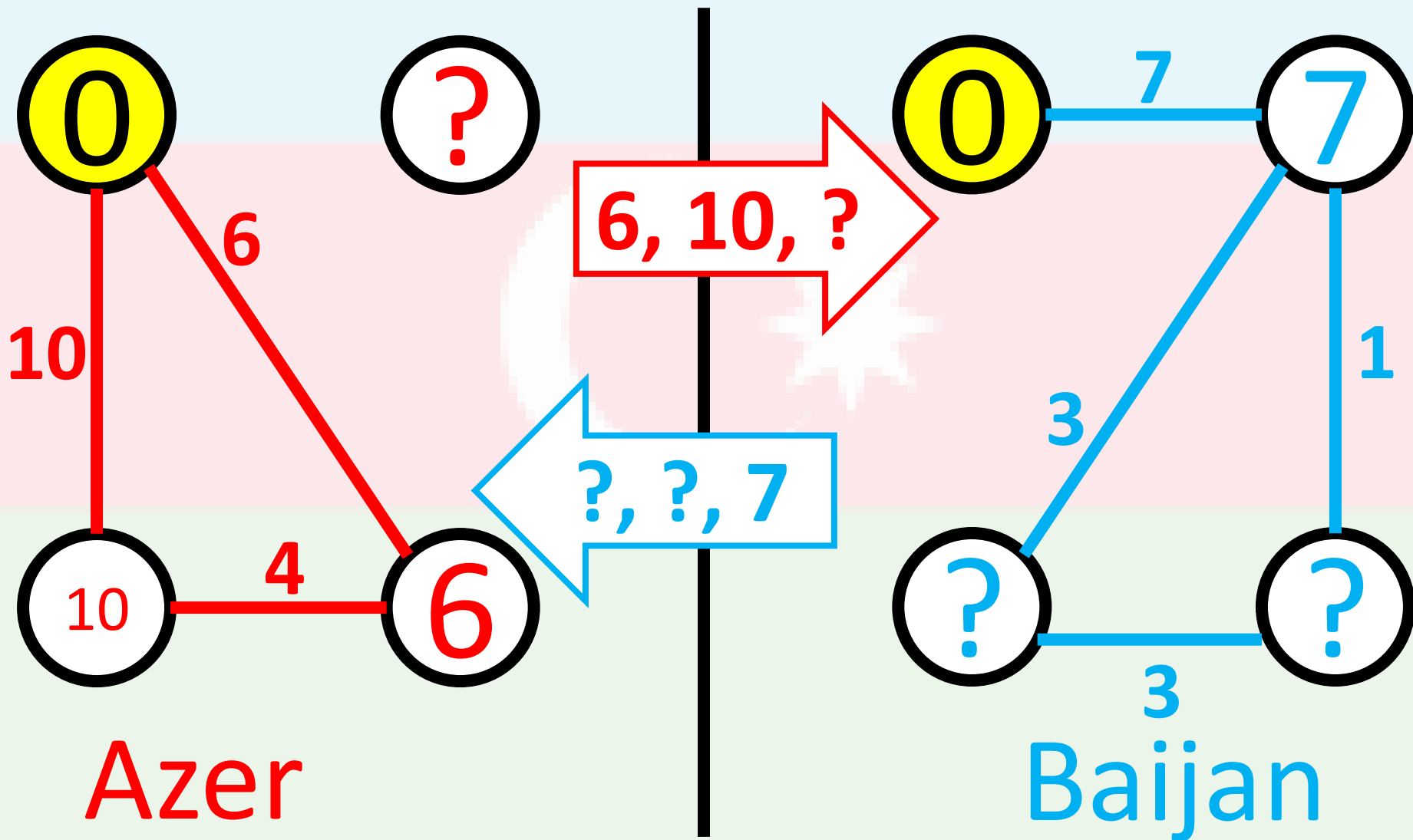


Azer

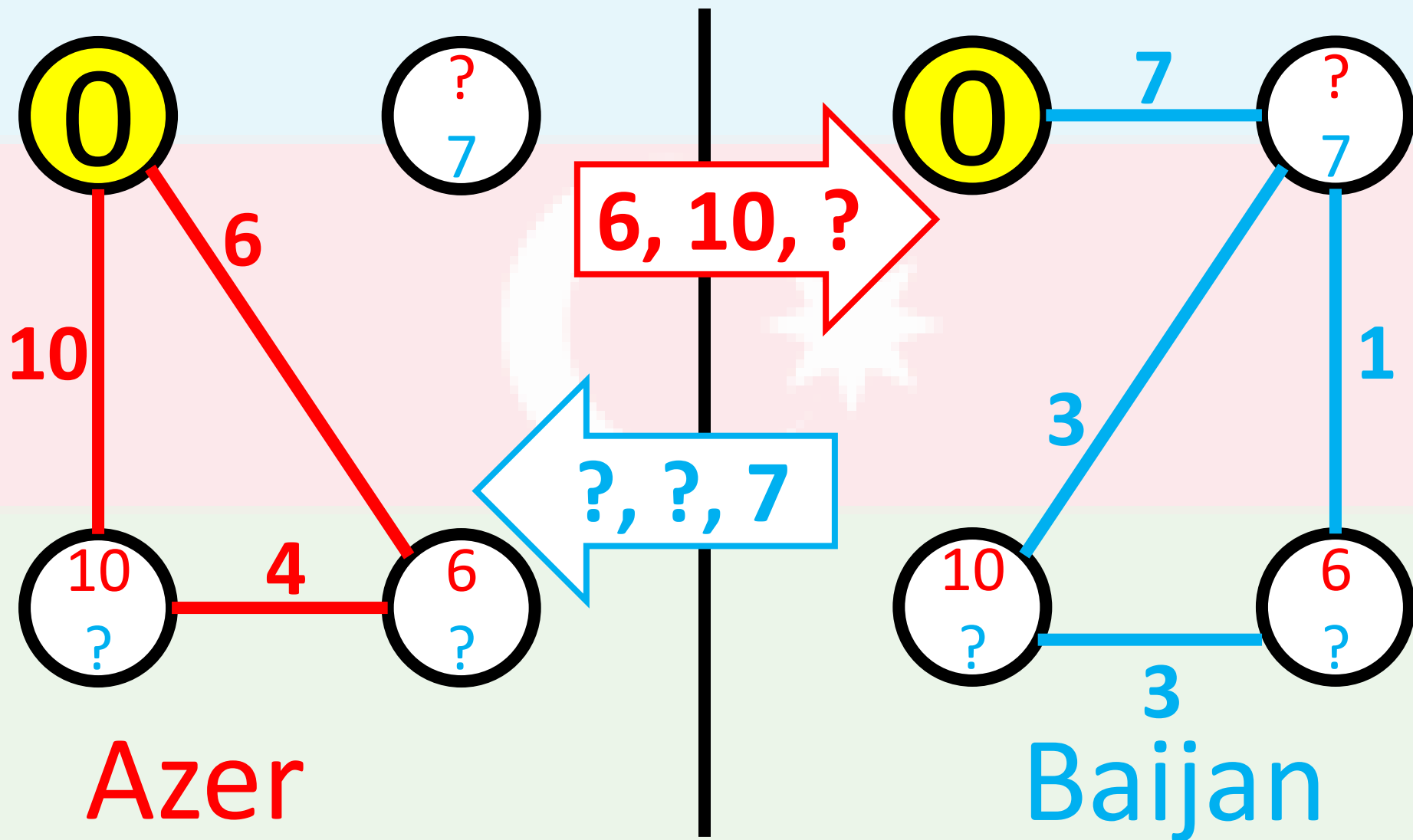


Baijan

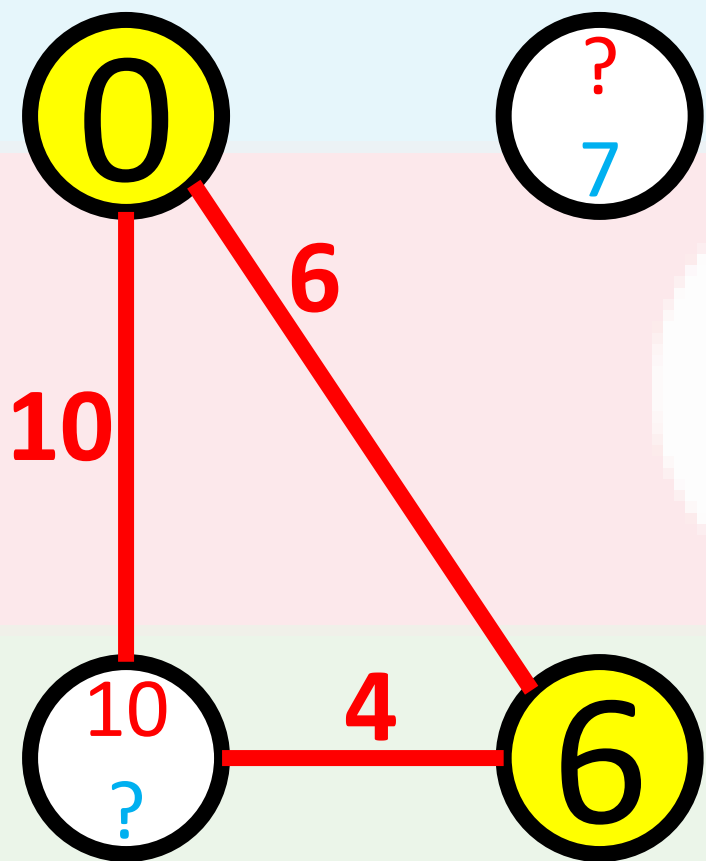
2人でやってみる



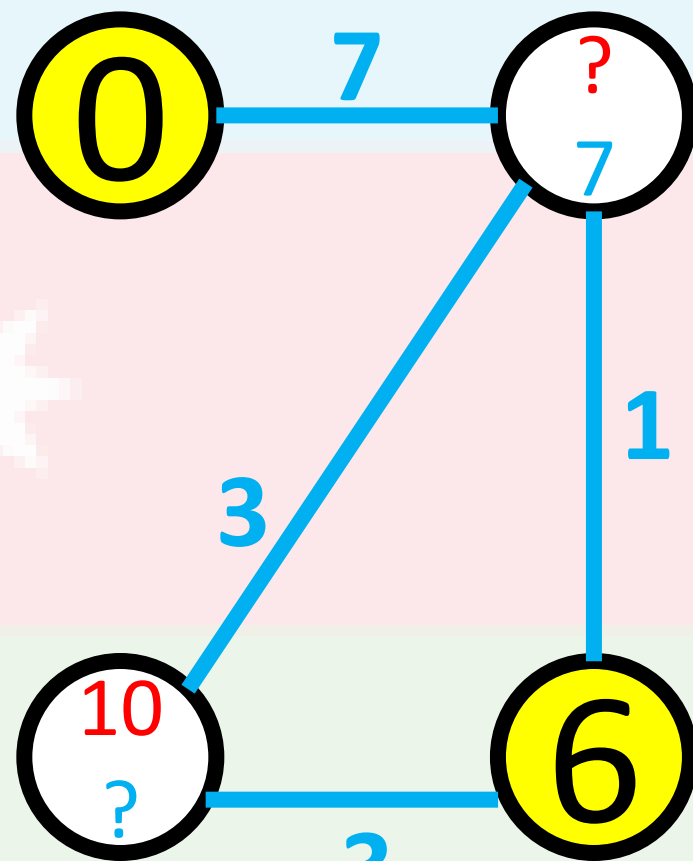
2人でやってみる



2人でやってみる

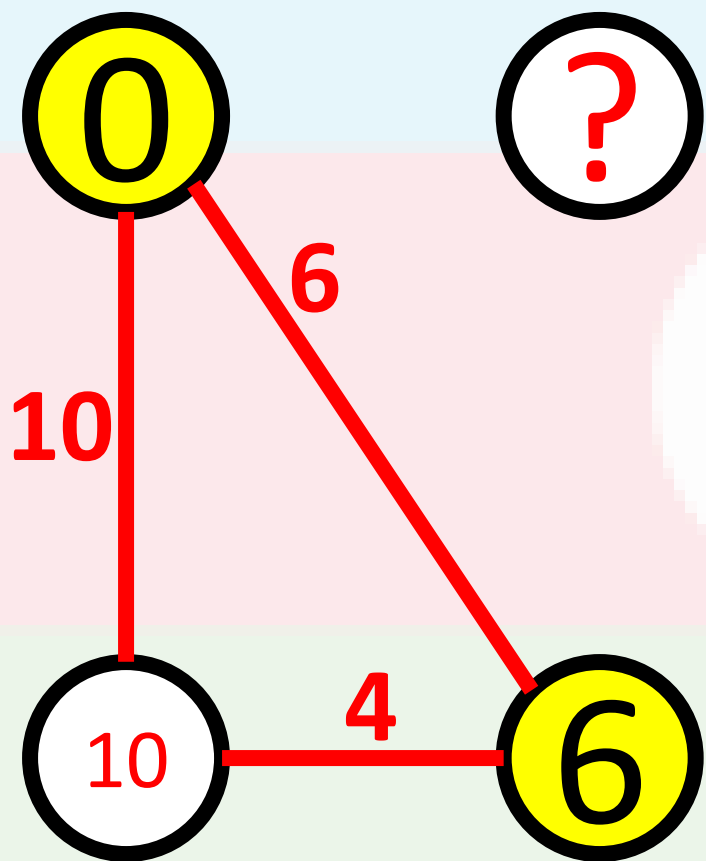


Azer

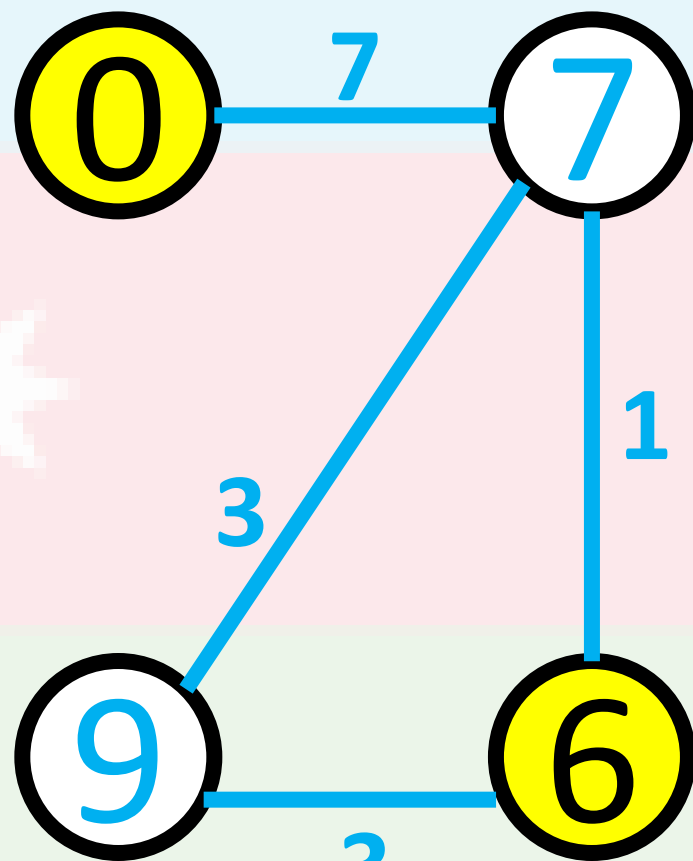


Baijan

2人でやってみる

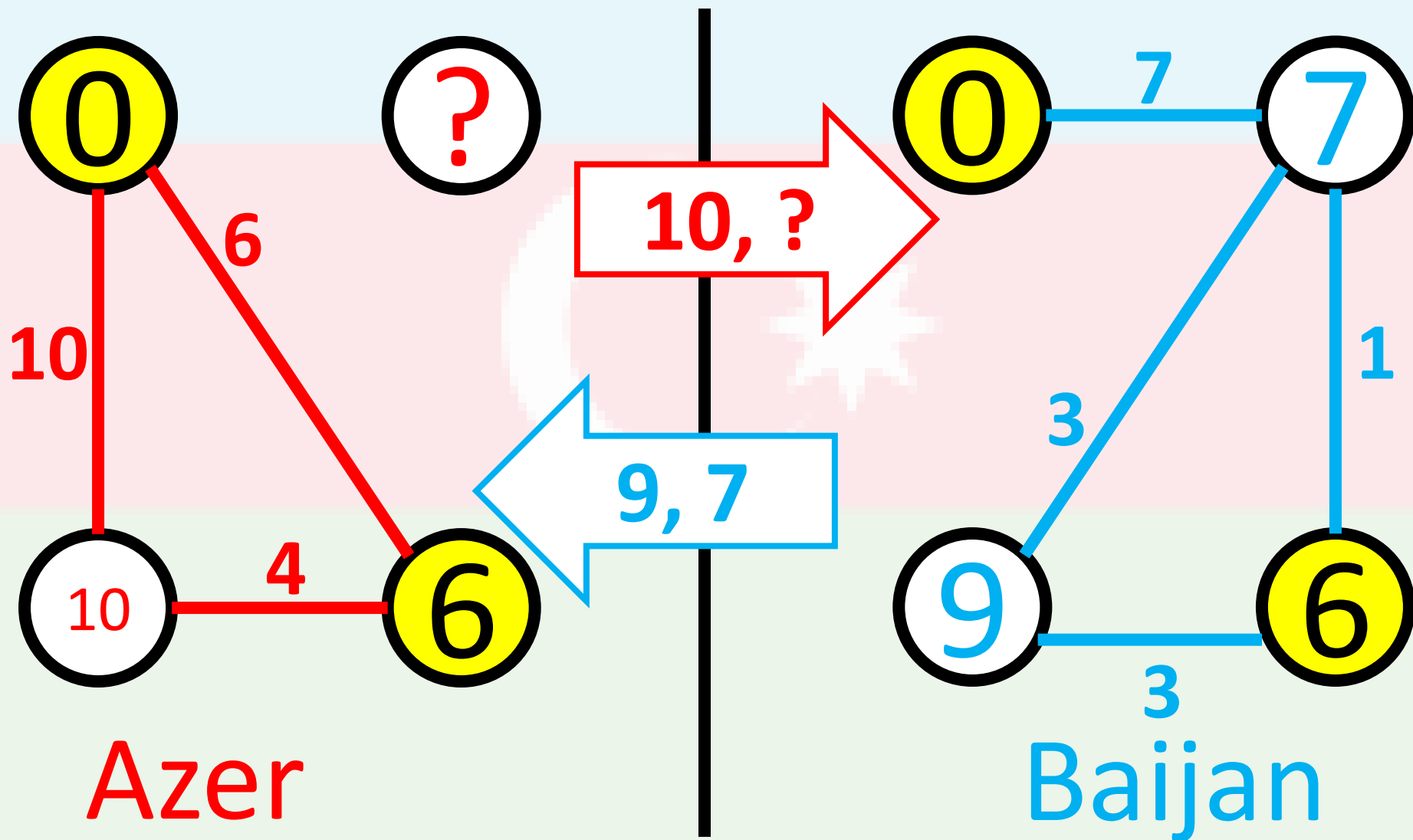


Azer

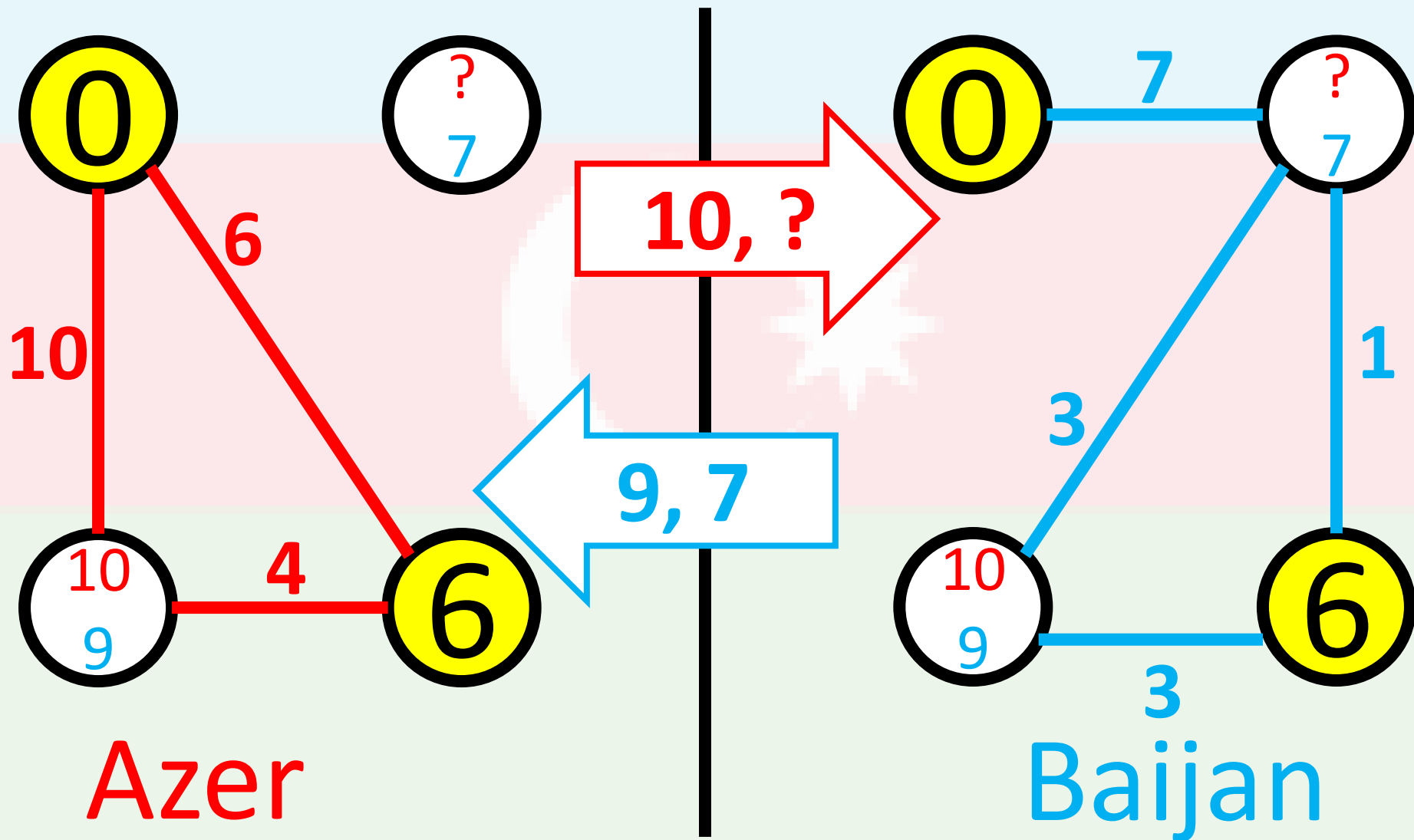


Baijan

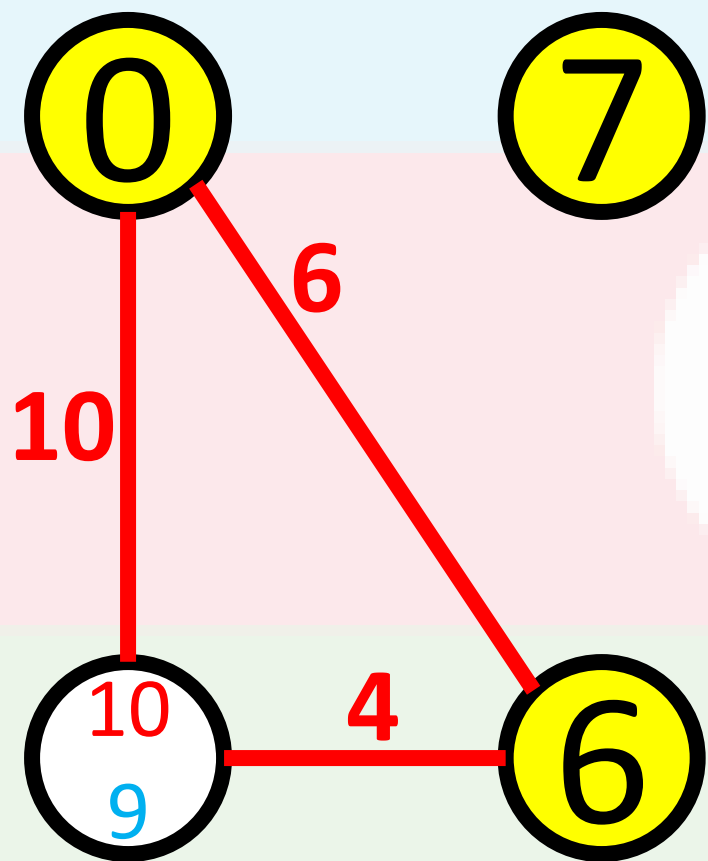
2人でやってみる



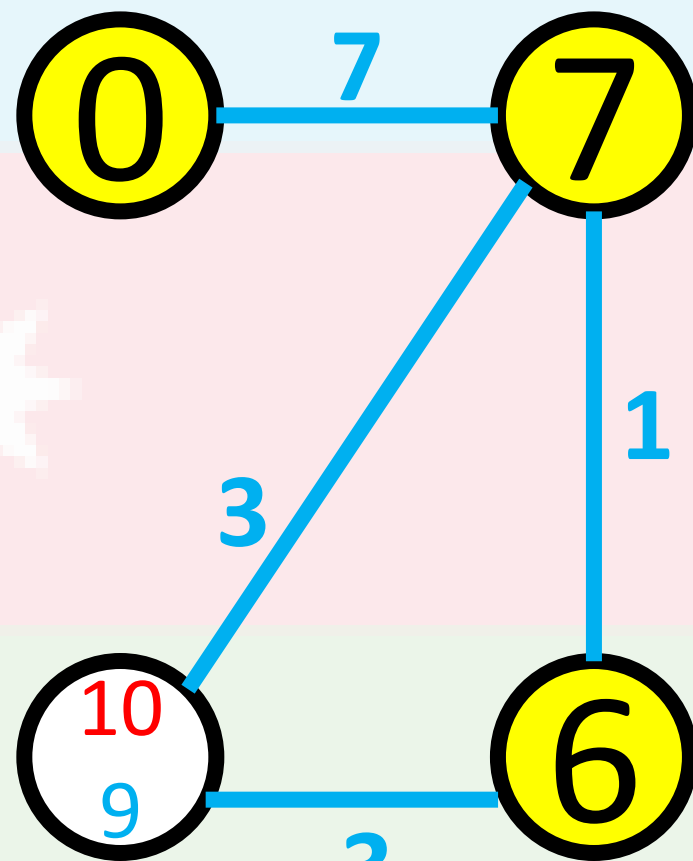
2人でやってみる



2人でやってみる

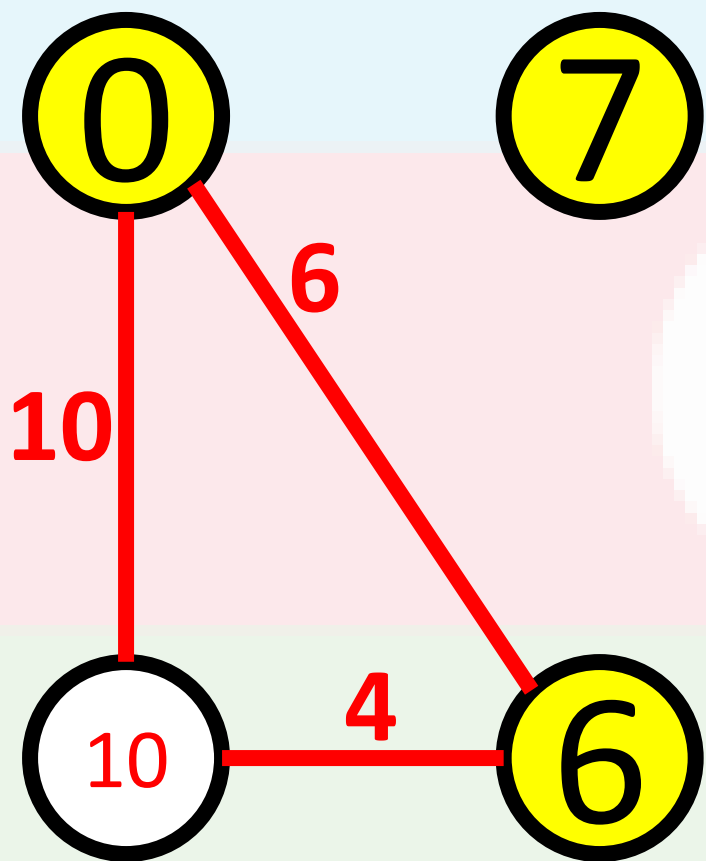


Azer

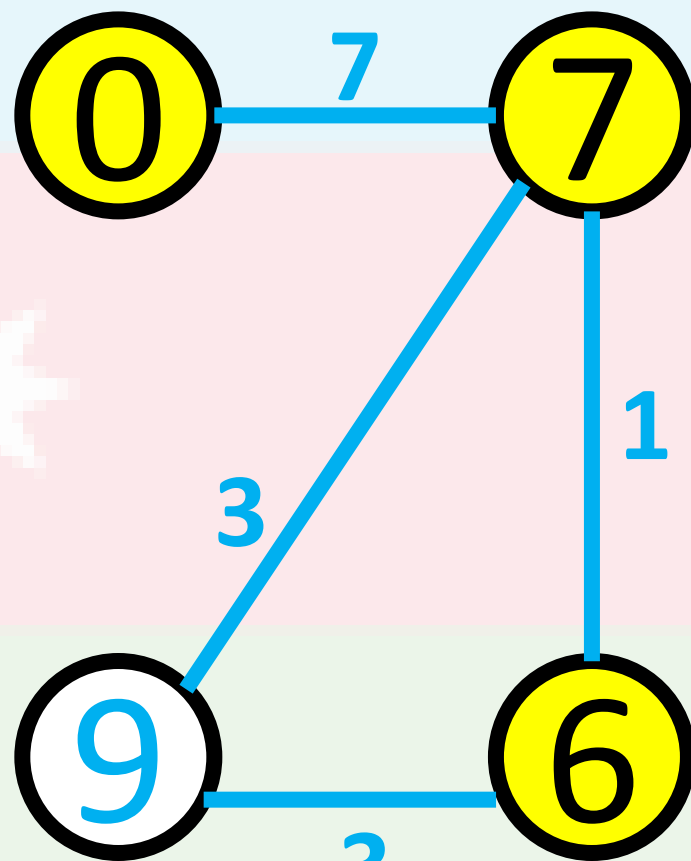


Baijan

2人でやってみる

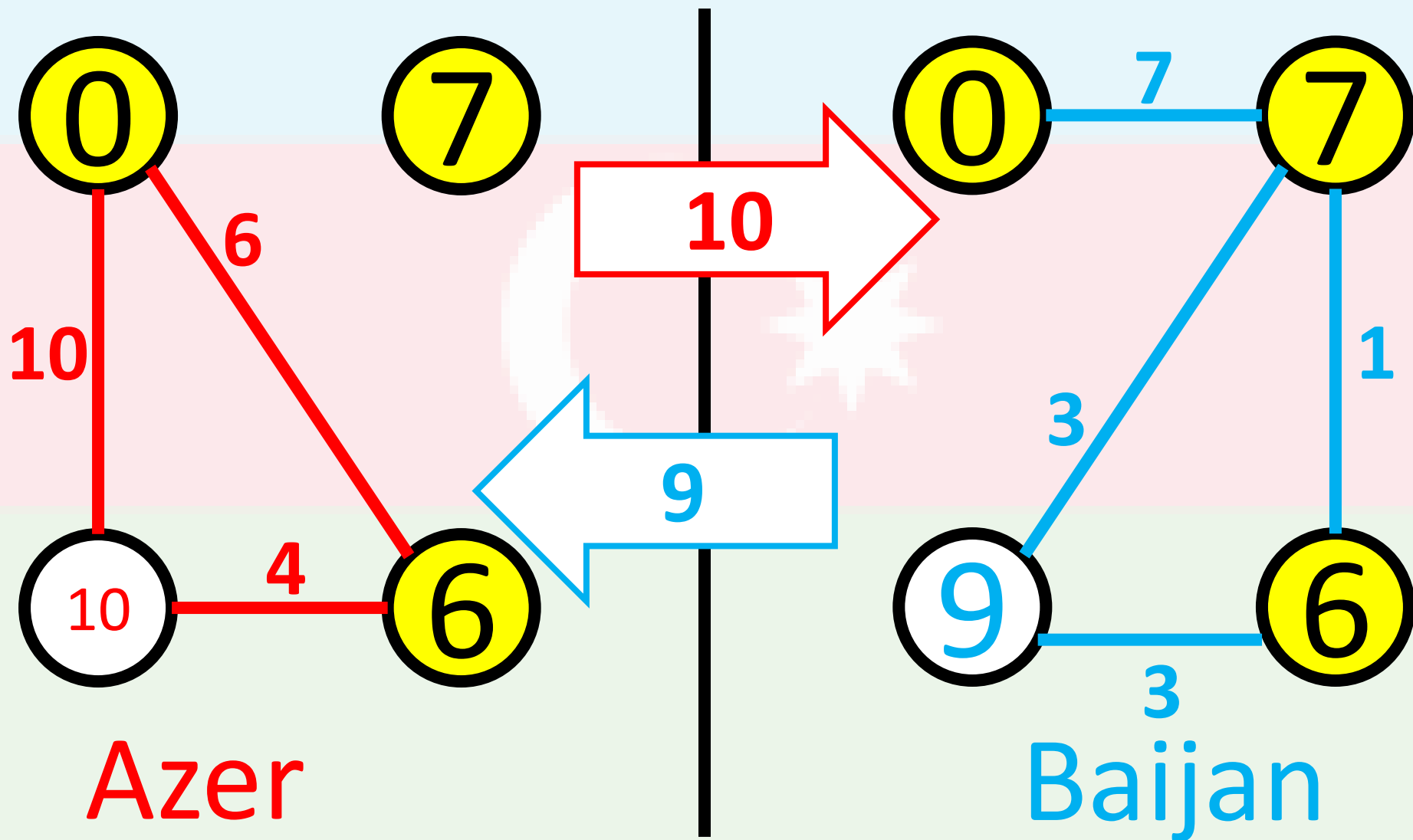


Azer

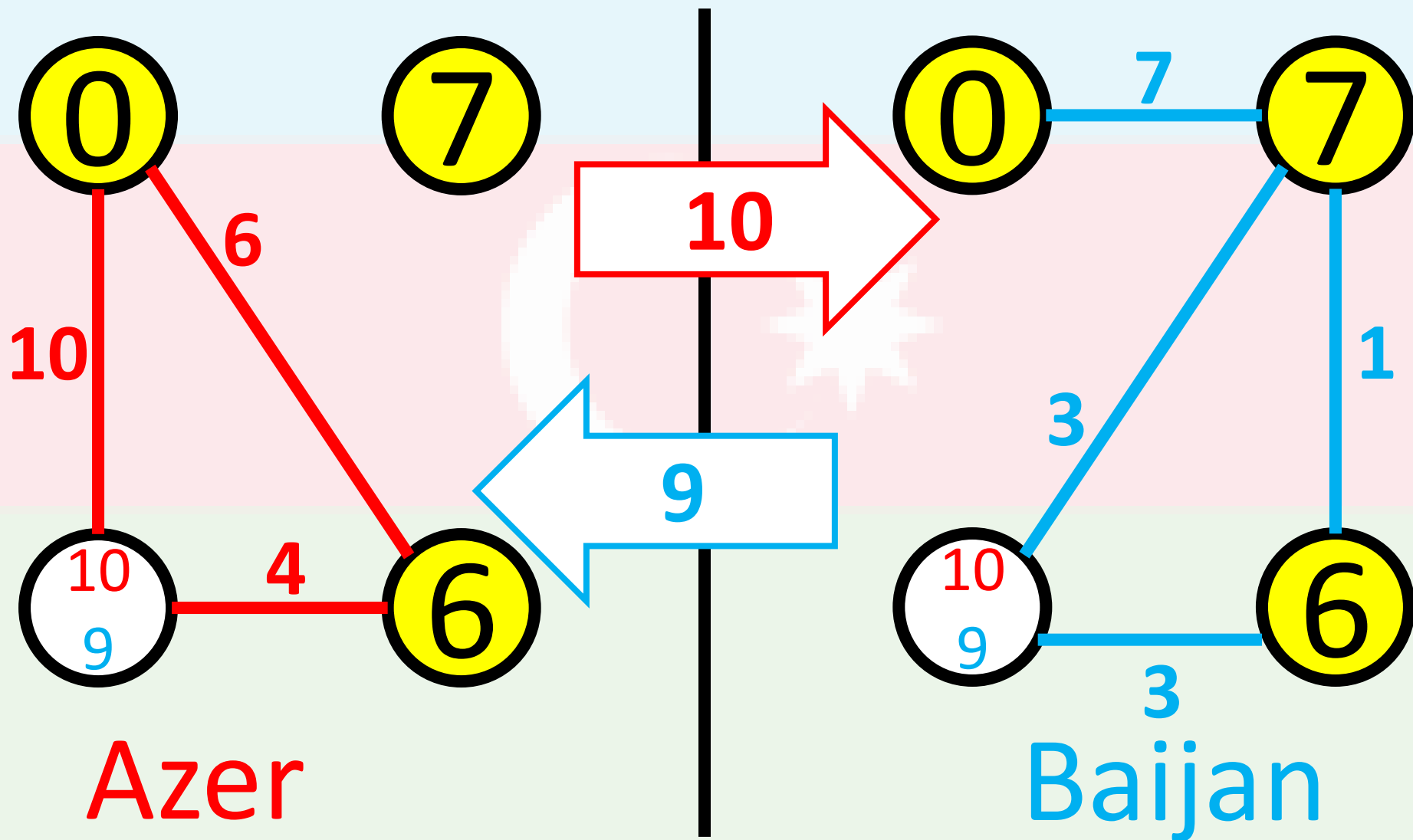


Baijan

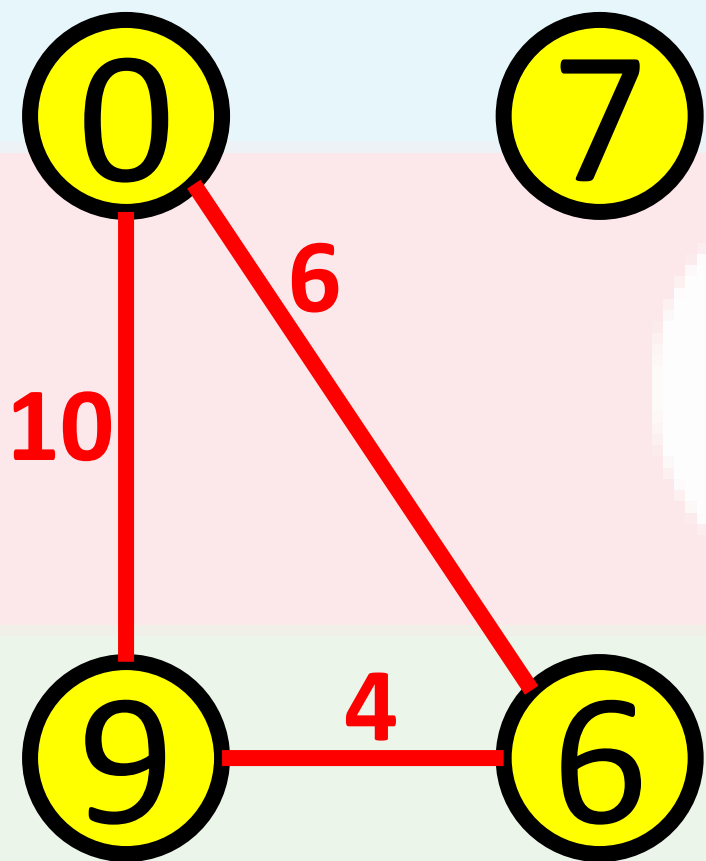
2人でやってみる



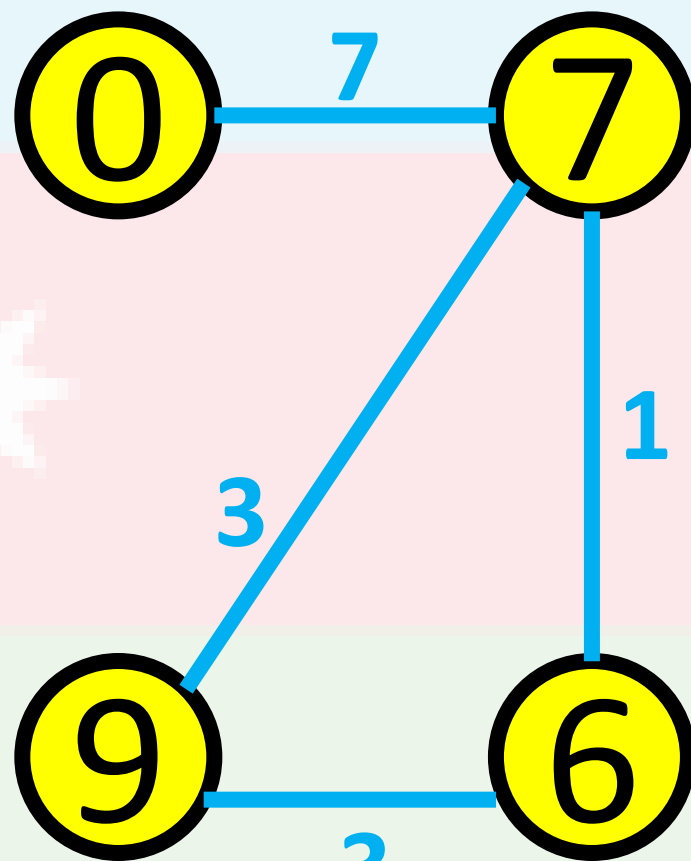
2人でやってみる



2人でやってみる



Azer



Baijan

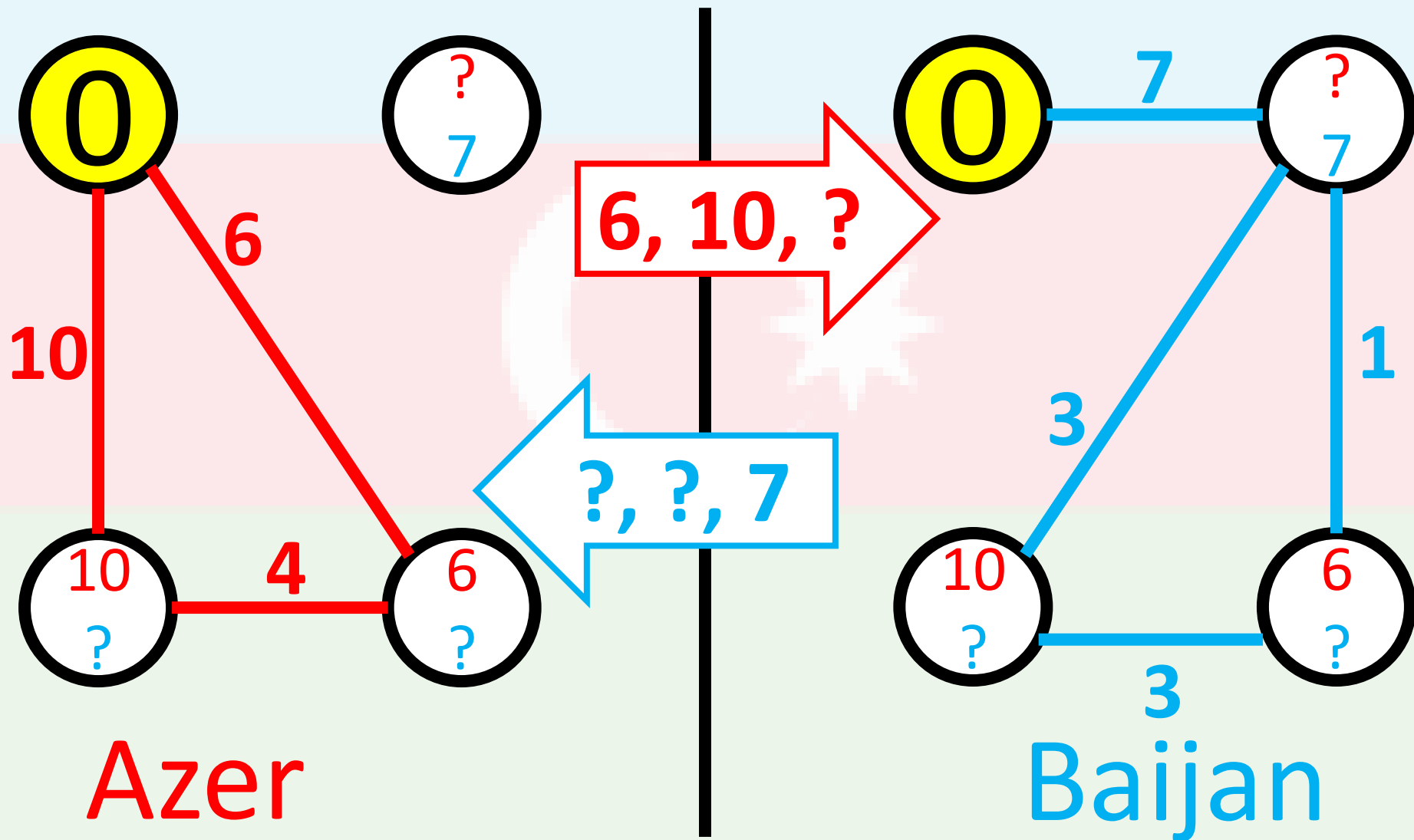
2人でやってみた

- 各段階で $O(N)$ の通信
→ 全体で $O(N^2)$ で ×

2人でやってみた

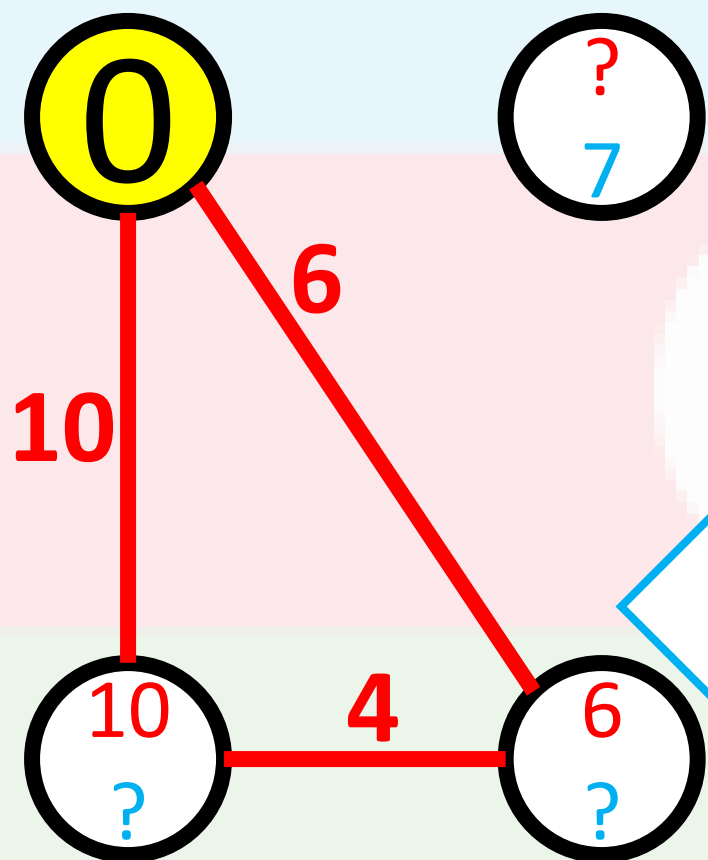
- 各段階で $O(N)$ の通信
→ 全体で $O(N^2)$ で ×
- 値が更新されるタイミングだけにすれば?
→ 最悪 $O(A + B)$ で ×

振り返り



振り返り

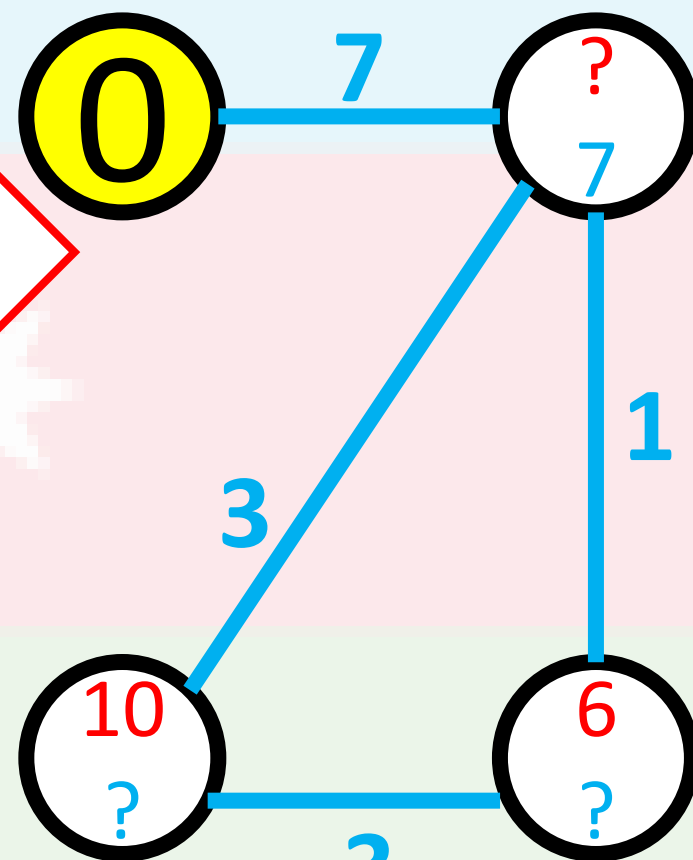
これらの情報は
使われない



Azer

6, 10, ?

?, ?, 7



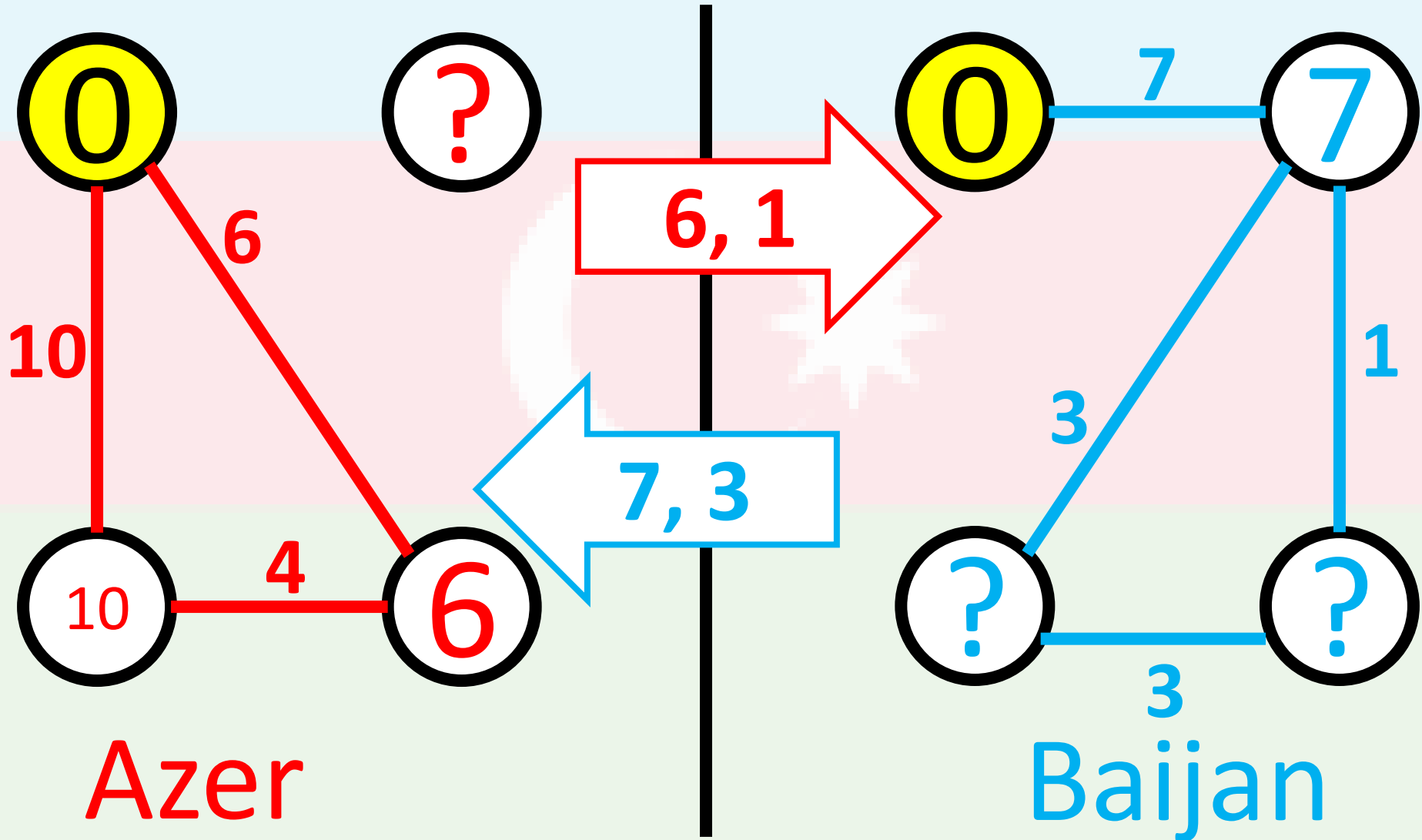
Baijan

考察

- 各段階で最小距離とその頂点番号を送る
→ 各段階で $O(1)$
全体で $O(N)$

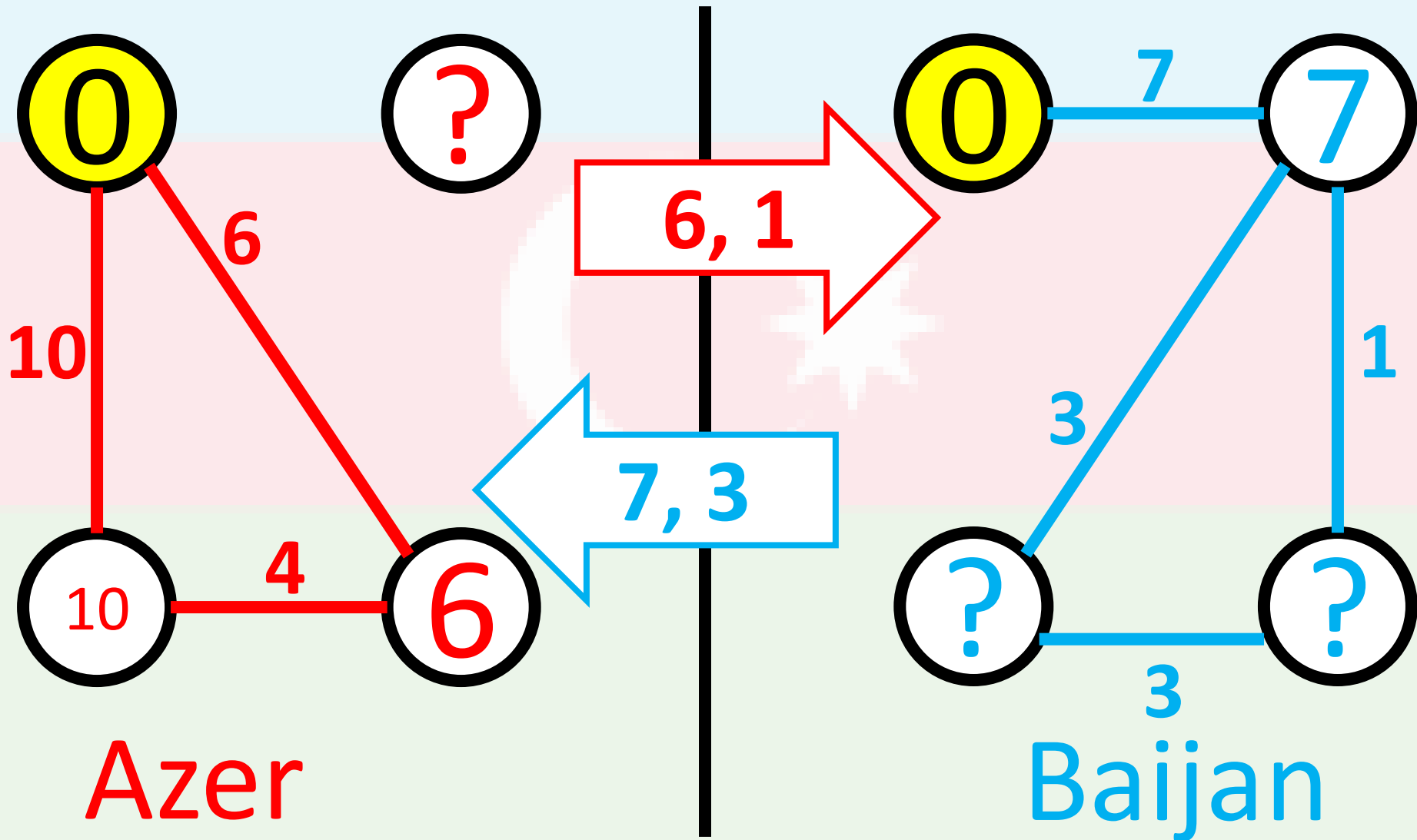
考察

最小距離と頂点番号を送る



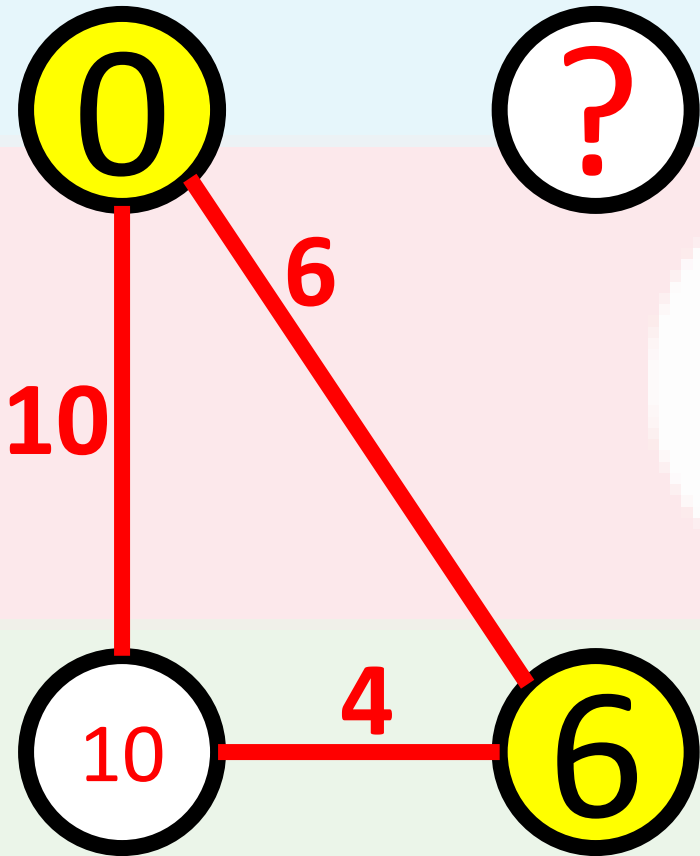
考察

7より6の方が小さい

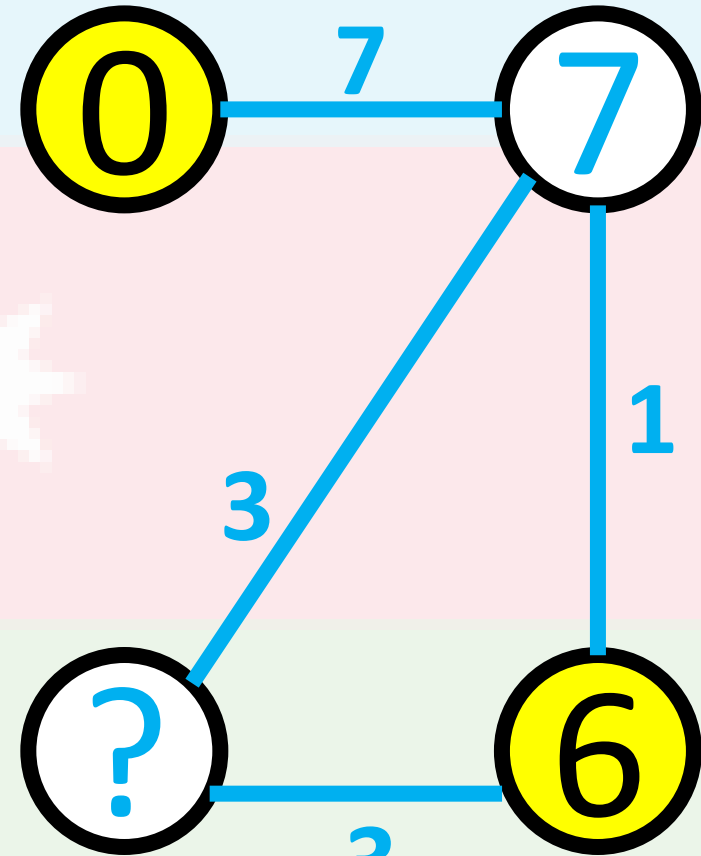


考察

7より6の方が小さい



Azer



Baijan

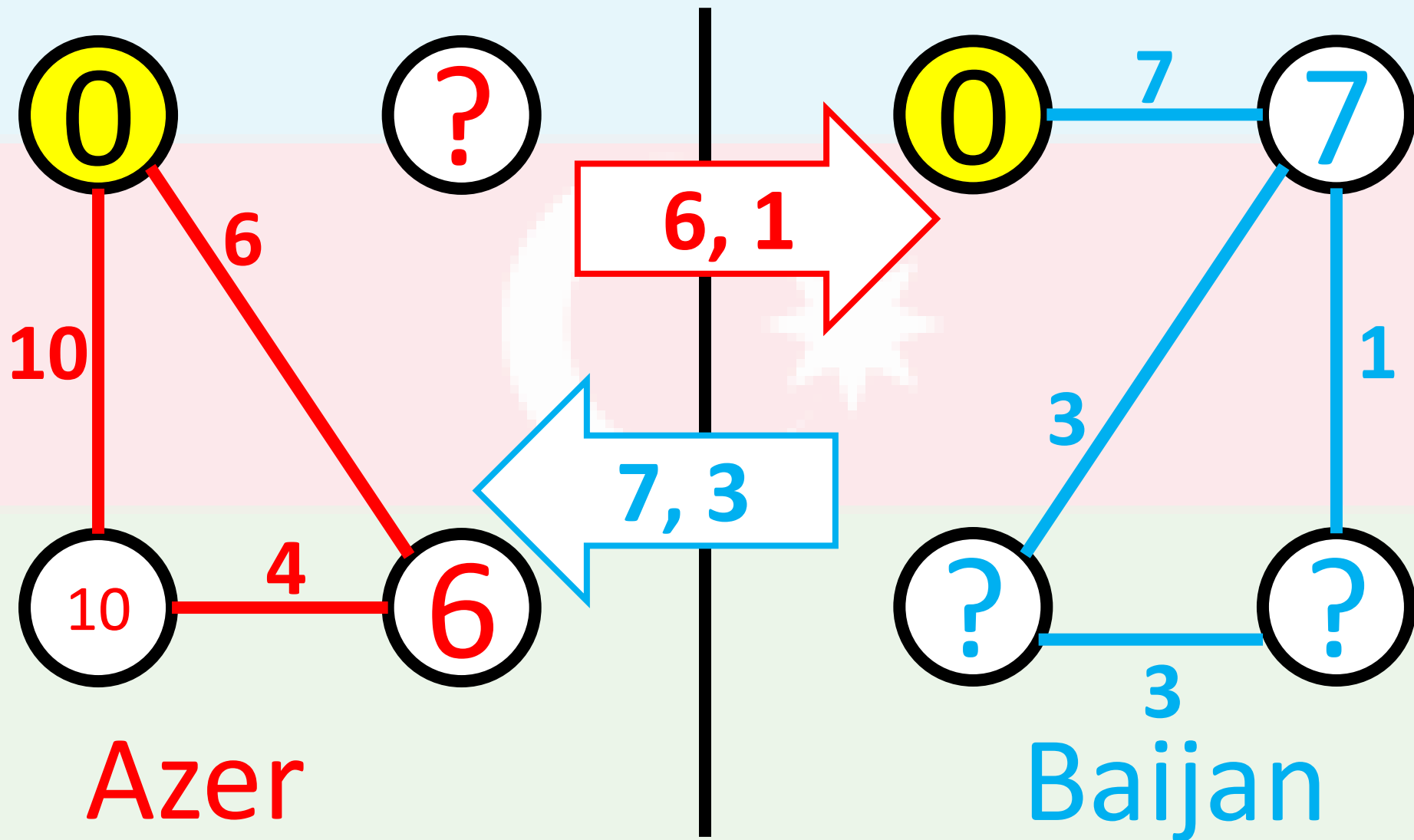
小課題4 ($N \leq 900$)

- 1頂点あたり送るのは
(距離, 頂点) $\times 2$
(20 + 11) $\times 2 = 62$ bit

小課題4 ($N \leq 900$)

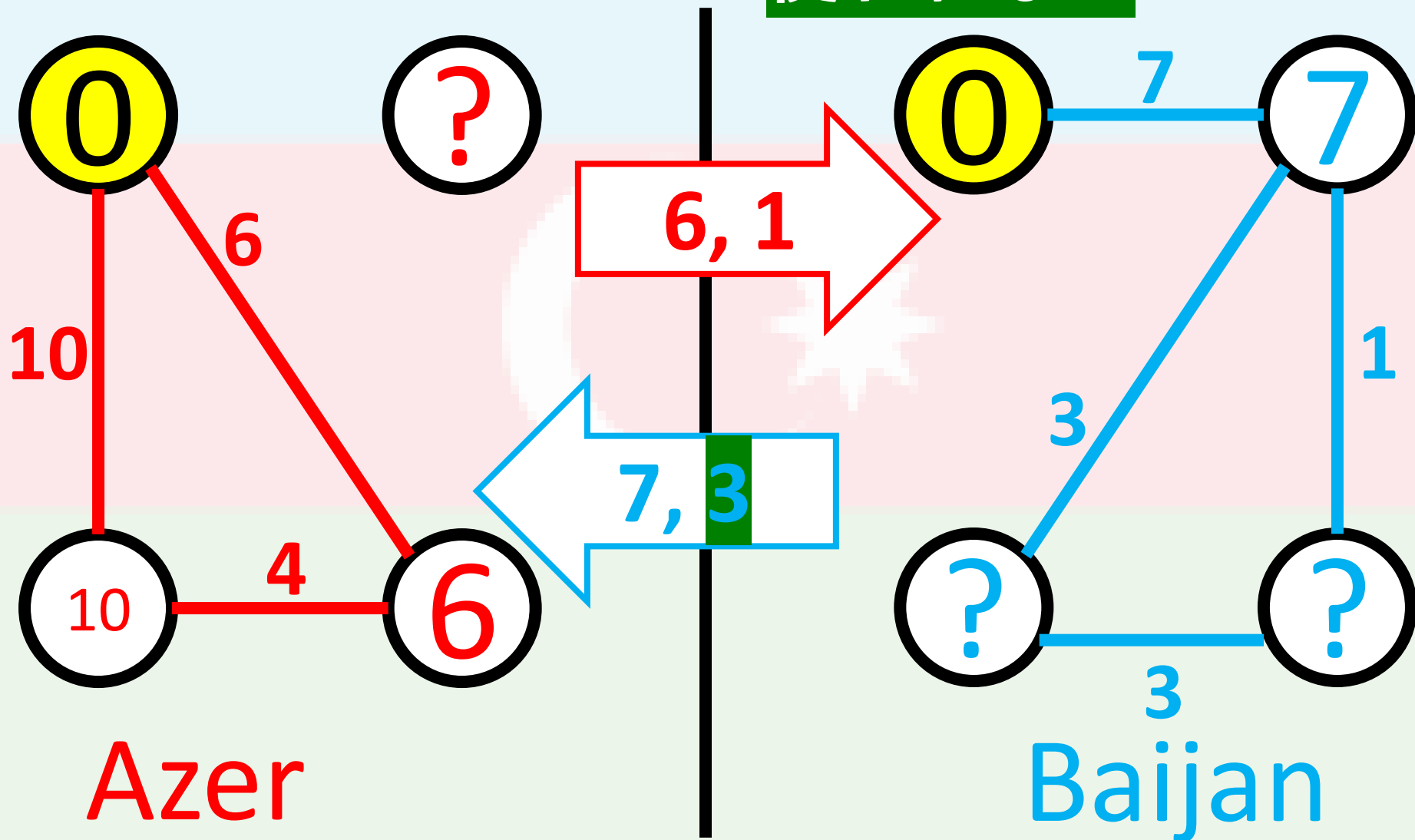
- 1頂点あたり送るのは
(距離, 頂点) $\times 2$
 $(20 + 11) \times 2 = 62 \text{ bit}$
- $N \leq 58,000 / 62 \doteq 935$
で解ける

振り返り



振り返り

この情報は
使われない

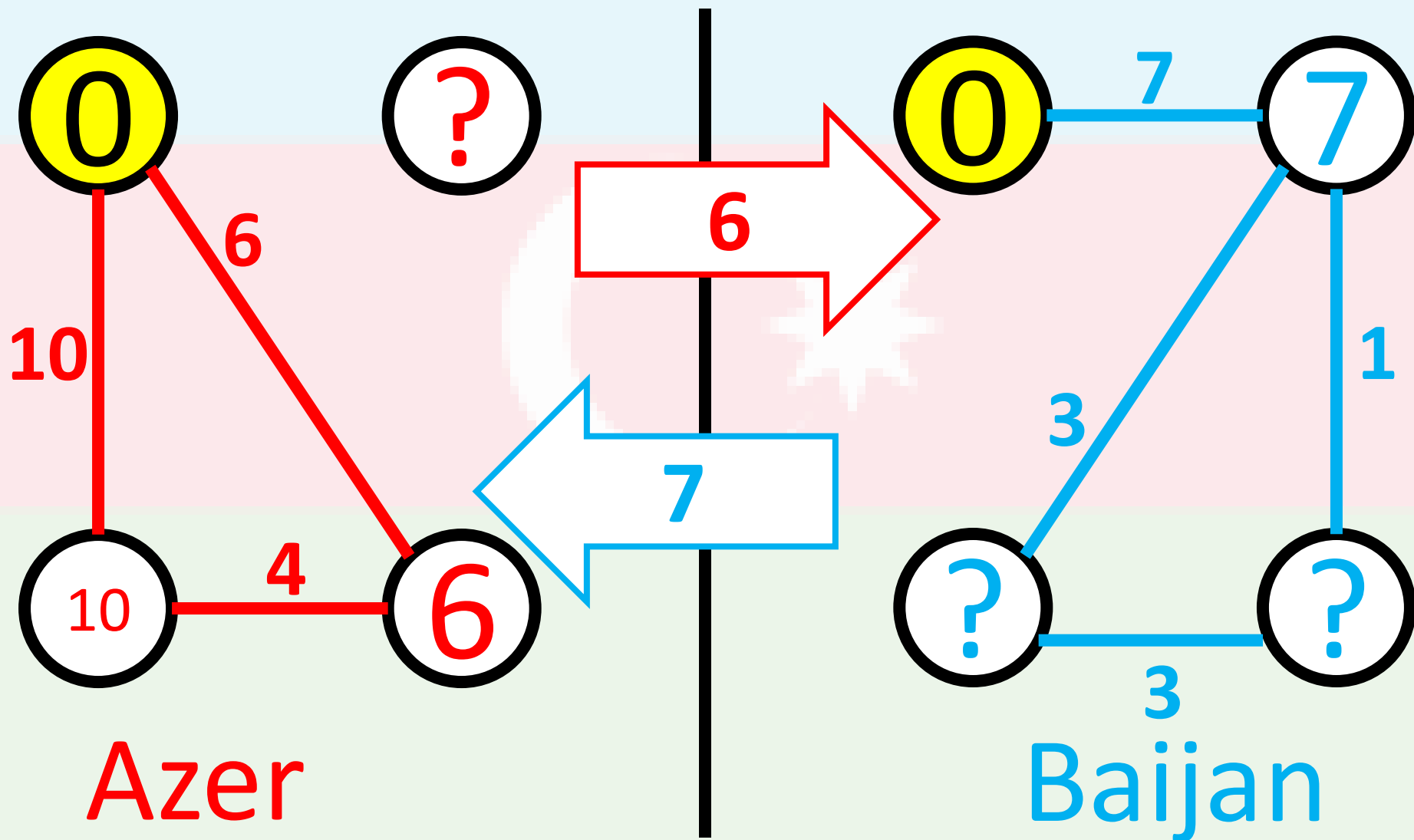


改善①

- 頂点番号は、距離が小さい方だけ送る

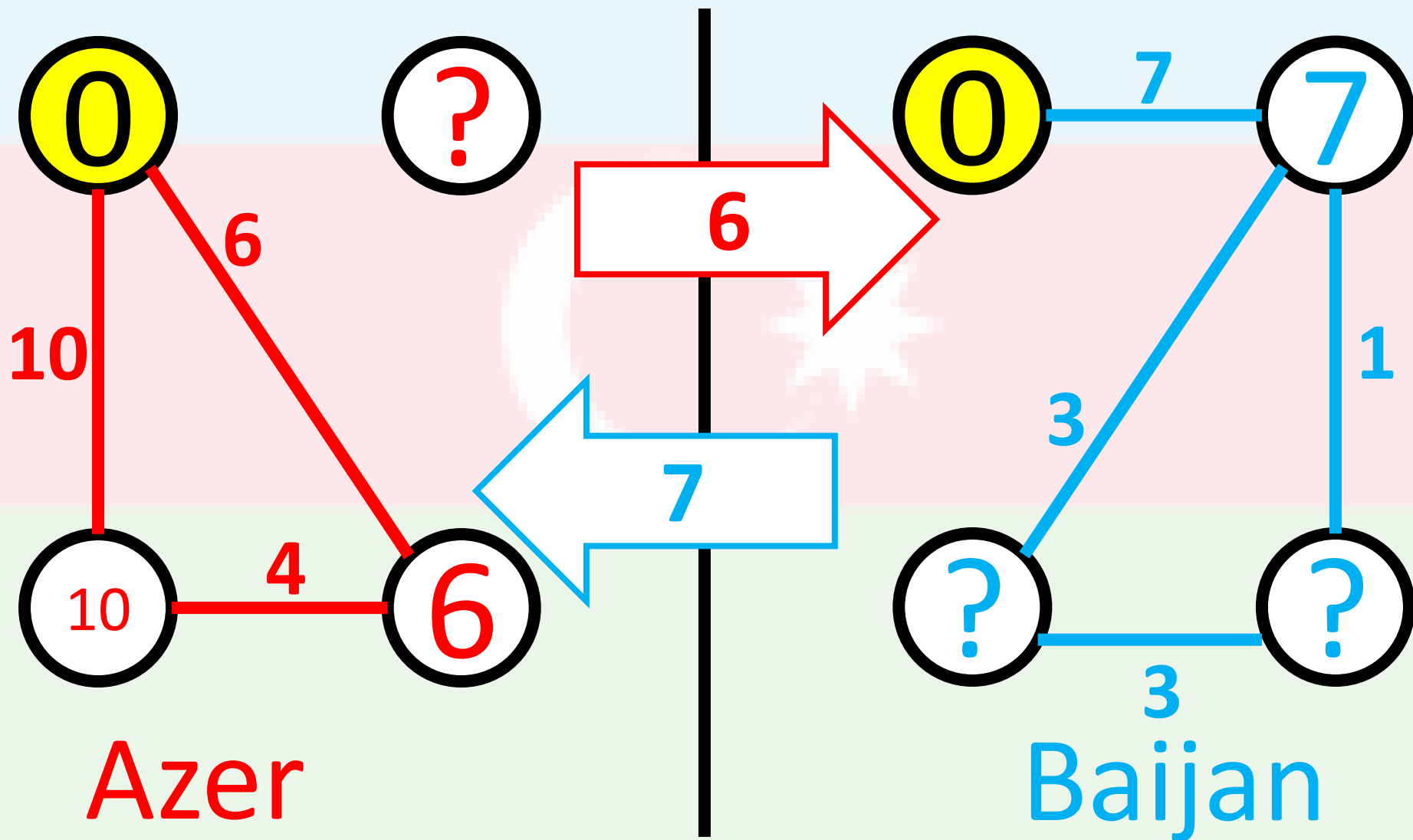
改善①

距離情報だけ送る



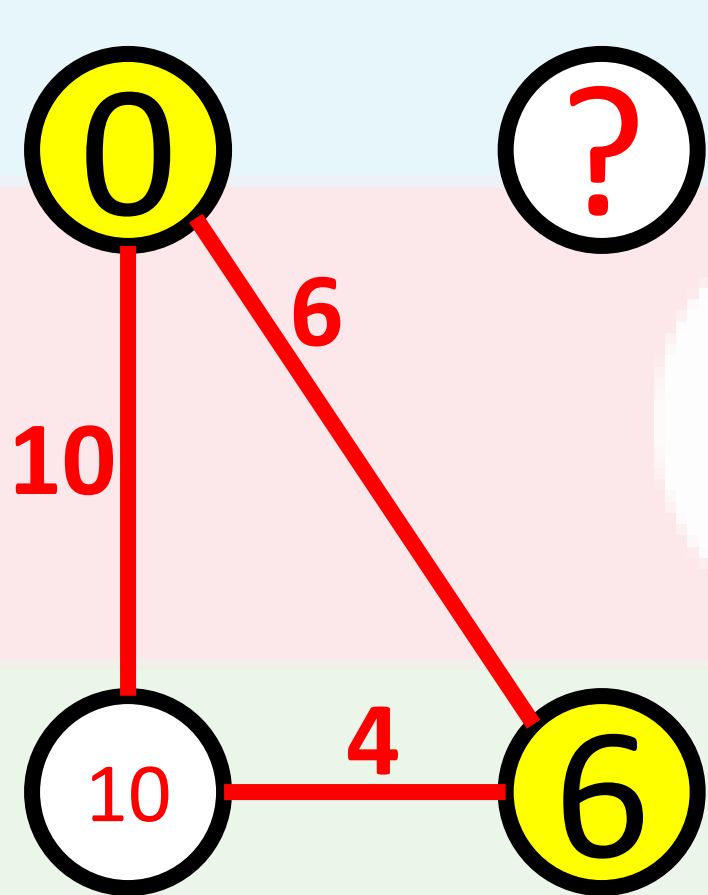
改善①

7より6の方が小さい

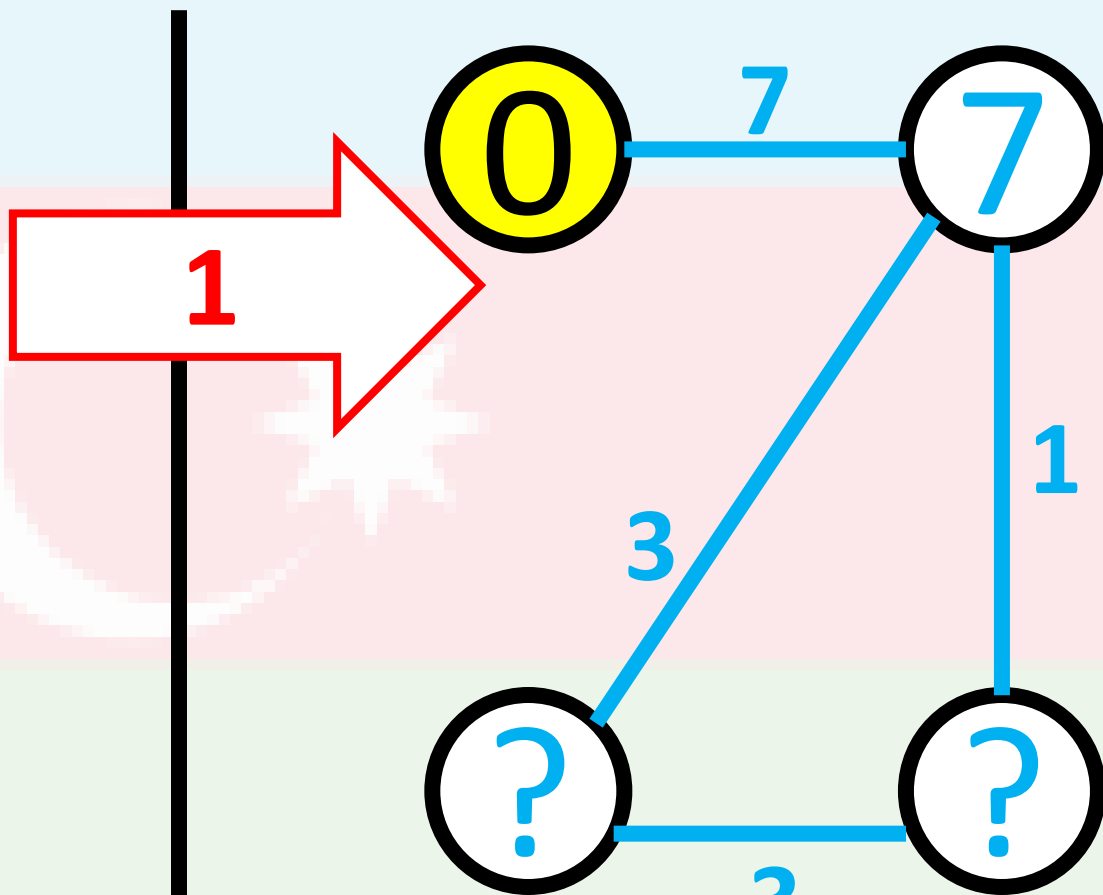


改善①

頂点番号は後から送る



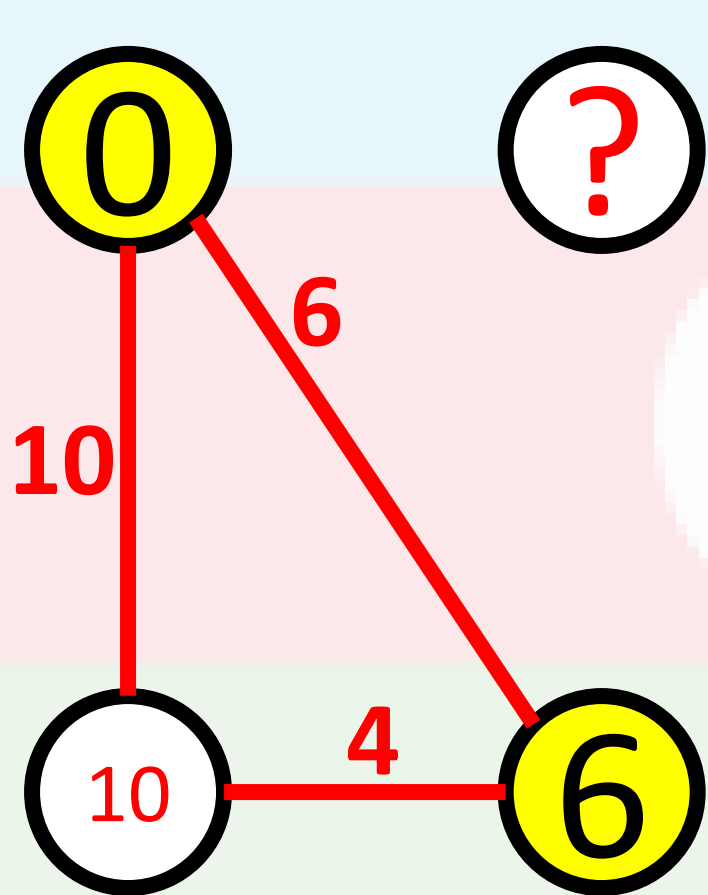
Azer



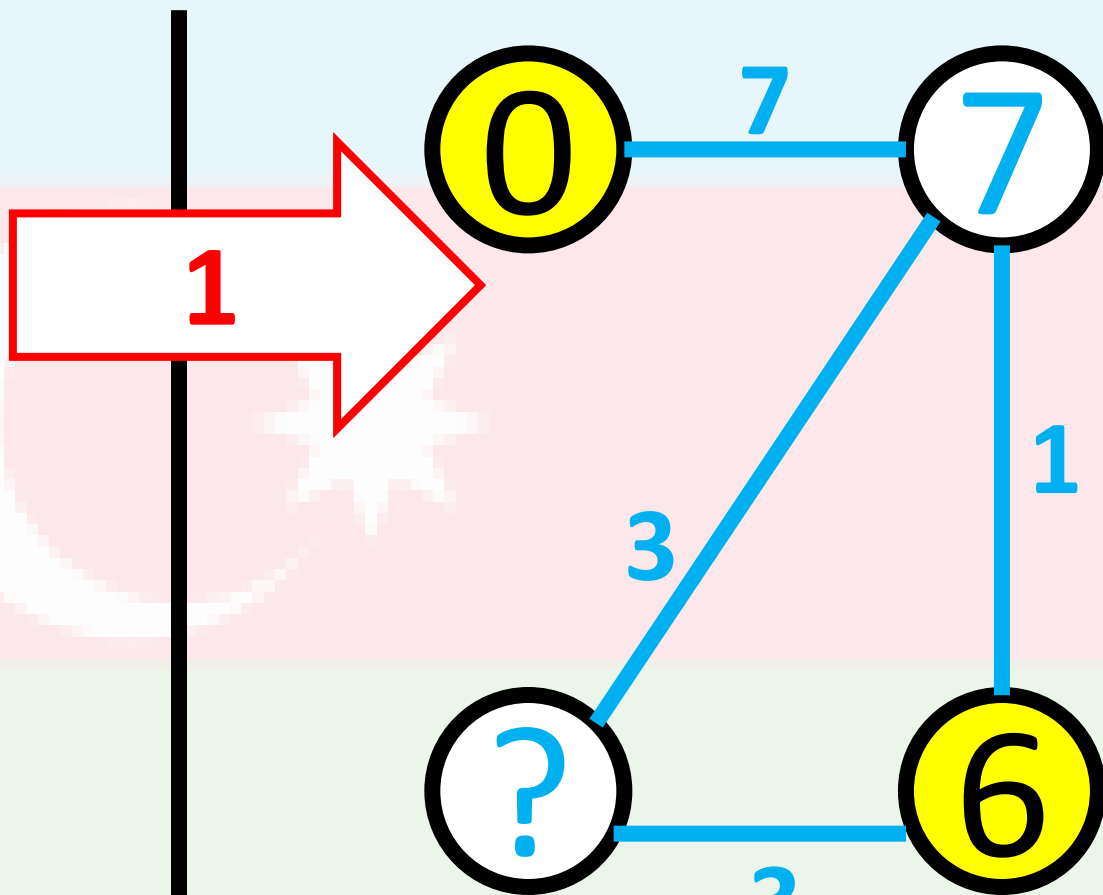
Baijan

改善①

頂点番号は後から送る



Azer



Baijan

改善①

- 1頂点あたり送るのは
(距離) $\times 2 +$ (頂点)
 $20 \times 2 + 11 = 51\text{bit}$

改善①

- 1頂点あたり送るのは
(距離) $\times 2 +$ (頂点)
 $20 \times 2 + 11 = 51\text{bit}$
- $N \leq 58,000 / 51 \doteq 1,137$
小課題5($N \leq 1,100$)が通る

考察

- Dijkstra法では
距離が小さい順に求まる

考察

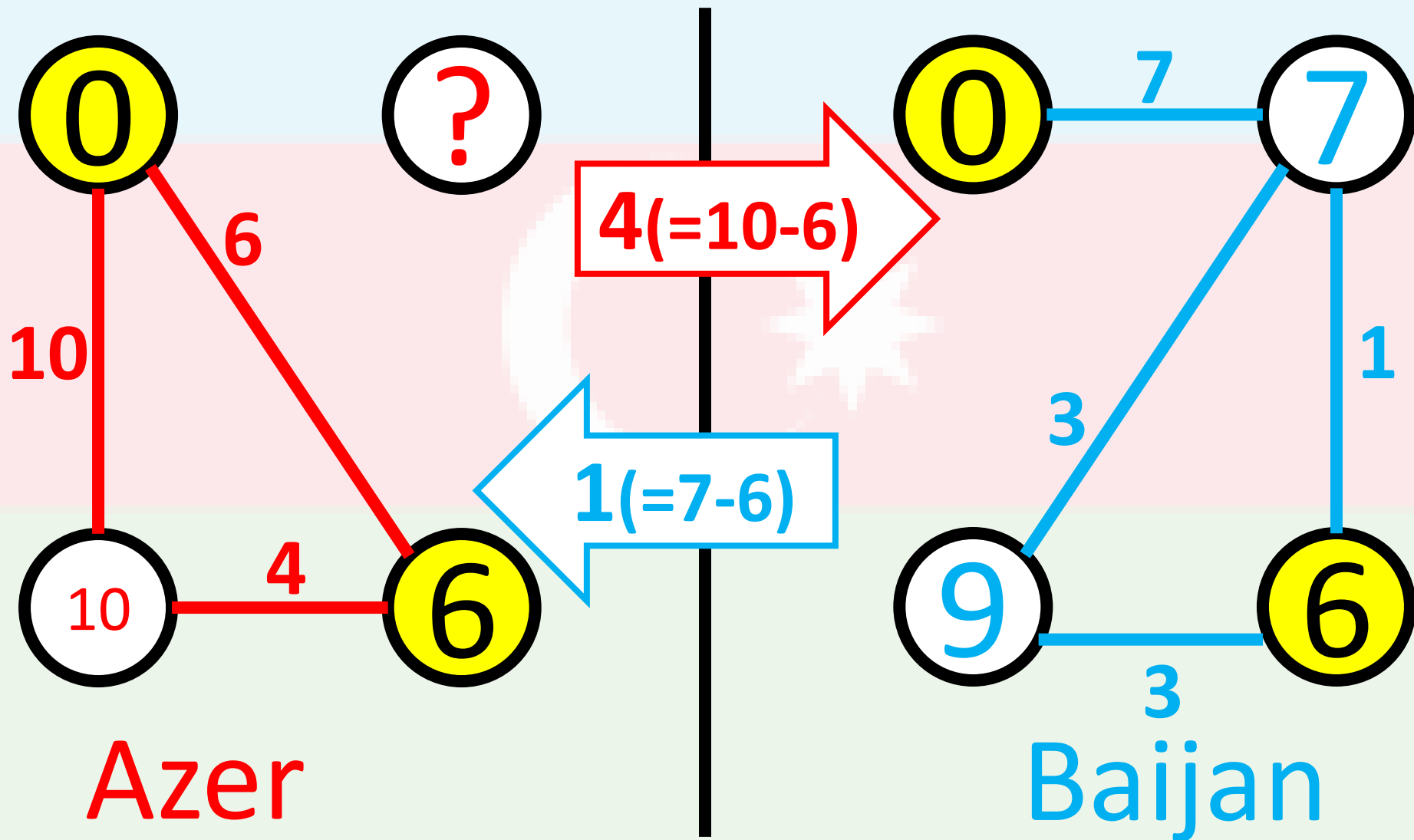
- Dijkstra法では
距離が小さい順に求まる
- ある段階で距離 z が求まる
→ 次段階での距離は z 以上
- 逆に $z + (\text{重みの最大})$ 以下

改善②

- 真の距離の代わりに
前段階での距離との差
を送る
- これは 重みの最大(=500)
以下なので 9 bit

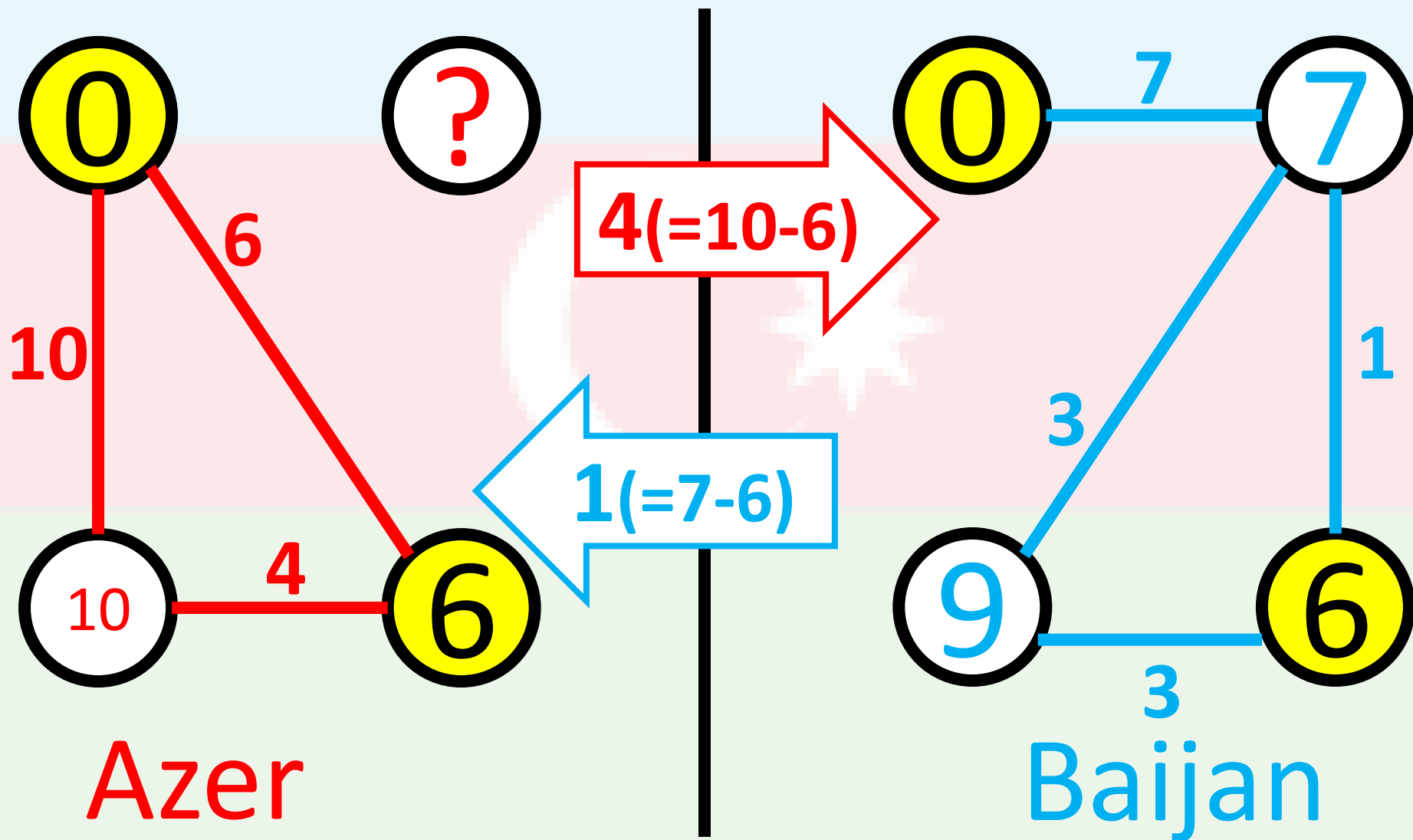
改善②

前段階での距離(6)
との差を送る



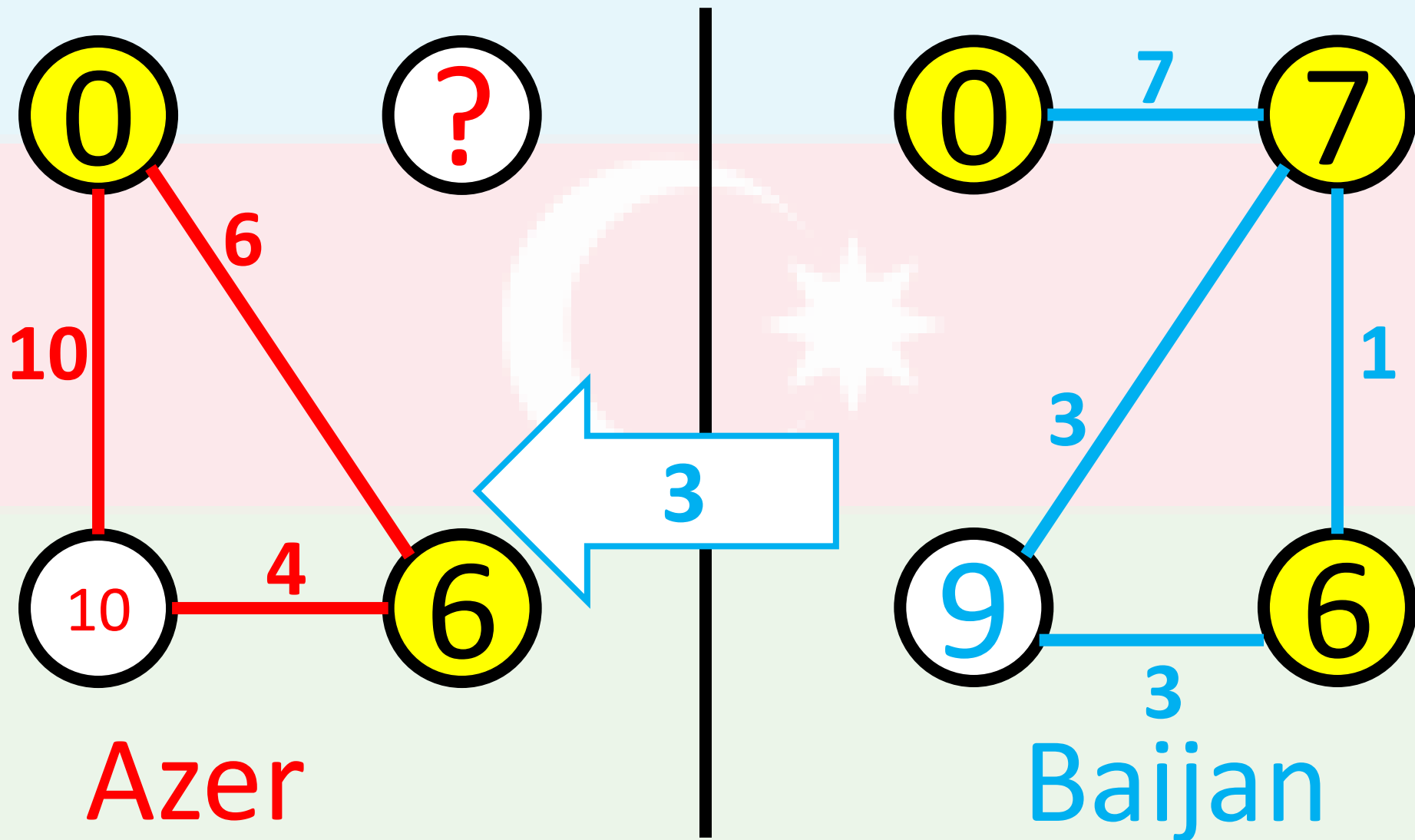
改善②

4より1の方が小さい



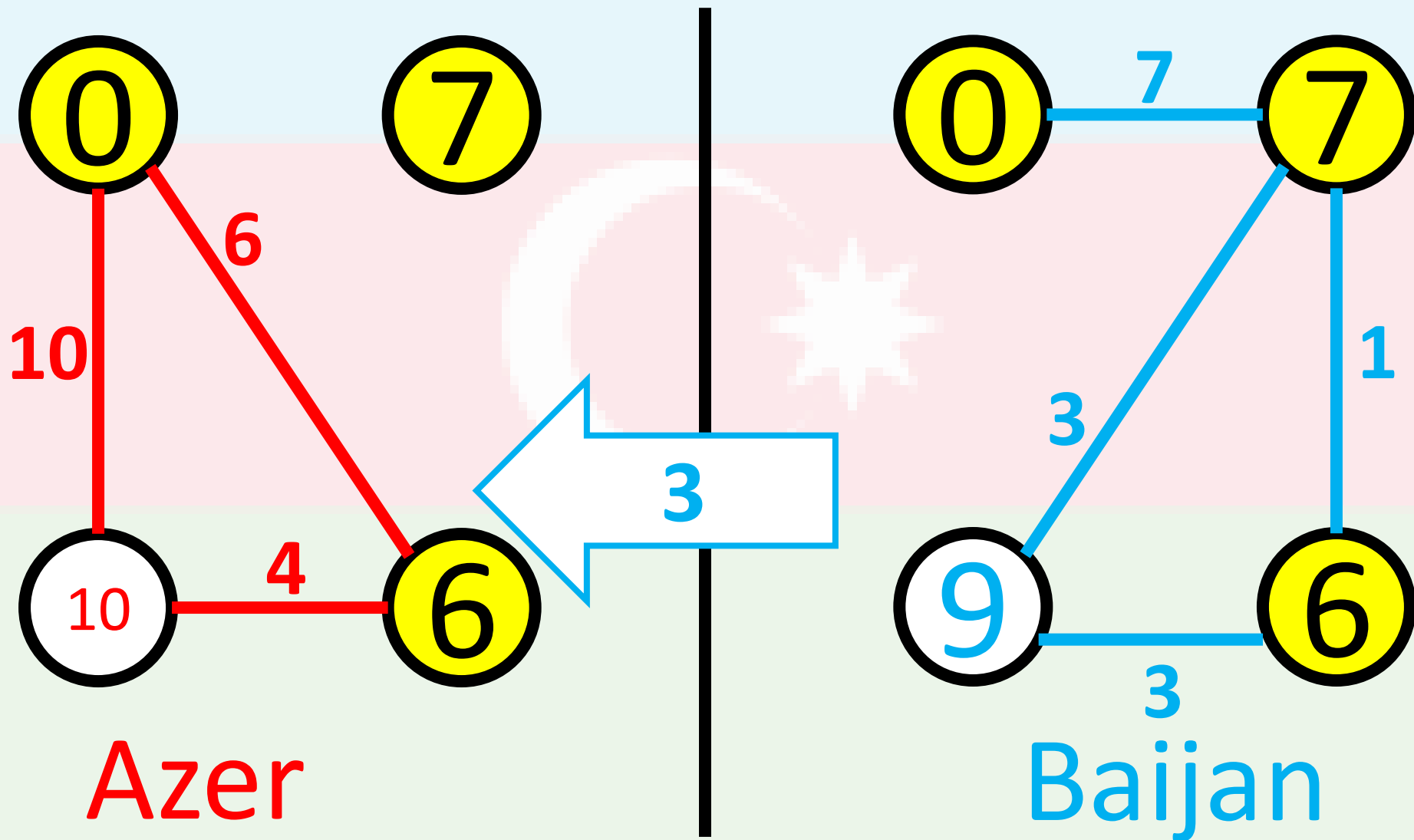
改善②

頂点番号を送る



改善②

頂点番号を送る



改善②

- 1頂点あたり送る情報は
 $9 \times 2 + 11 = 29\text{bit}$

- $N \leq 58,000 / 29 \doteq 2,000$
満点

※改善①をせずに②だけすると
1頂点あたり40bitで小課題6が通る

得点分布

0



44



6



52



8



68



14



100



22



Designated Cities

maroon

問題概要

- N 頂点の木がある
- 各辺はそれぞれの向きにコストが定まっている
- 一個頂点を選ばと、各辺について、選んだ頂点に向かう向きのコストが0になる
- 何頂点か選んでコストの合計の最小化せよ、という問題にたくさん答える

小課題1

- $N \leq 16$
- 2^N 通り選ぶ頂点を全探索してDFSする
- $O(N^2 2^N)$

小課題2

- $Q=1$, 選ぶ頂点は1個
- とりあえず頂点0を選んだ際のコストの総和を求める
- 頂点 v でのコスト総和がわかっているとき、 v に隣接する頂点 u でのコストの総和は、 v との差分を考えると、辺 $e=(u,v)$ のコストからわかる
- よってdfsしていけば全頂点について求められる
- $O(N)$

小課題3

- $Q=1$, 選ぶ頂点は2個
- 2頂点選んだときのコストの総和は、(それぞれの頂点を独立に選んだときのコストの総和+2頂点間のパスの重み(双方向)の総和)/2という形になっている
- 木の直径を求めるDPと同じ要領でDPできる
- $O(N)$

小課題4

- $N \leq 2000$
- 選ぶ頂点が1個の場合はできるのでそれ以外の場合を考える
- 木の葉の個数以上選べるときは、葉を全部選べばコスト0
- 木の葉の個数未満しか選べないときは、選ぶ頂点は全て葉である(葉でないものを選んだ場合は適当な葉に向けて動かすと得できる)

小課題4

- 選ぶ葉を1個固定する、これは $O(N)$ 通り
- 選んだ頂点を根にして考える
- 根に向かう頂点のコストはすべて0になっている
- 辺のコストは葉に向かうものだけ考えればよい
- 葉を一つ指定すると、根からその葉へ向かうパスのコストが0になる
- 葉をいくつか選んで、(根—葉)を結ぶパス上の辺のコストの総和を最大化する問題になる

小課題4

- 実は、選ぶ葉は貪欲に選んでよい(次に選ぶと増えるコストが最大、というものを選んでいく)
- 証明は簡単
- 最初の段階でコスト最大の(根-葉)パスを任意にとり、その葉をLとおく
- このとき、Lを使わない解があるとする、Lを使うように変形して損しないことがわかる

小課題4

- 次に選ぶと増えるコストが最大、というものを選ぶ
- やり方は主に2つ
- SegTreeで頑張る
- それぞれの葉について使ったときに増えるコストをSegTreeで管理する
- あるパスを使ったときの、それぞれの葉における増加コストの変化は、範囲加算等で表現できる

小課題4

- priority_queueで頑張る
- まずpriority_queueに、最大コストのパスをpush
- 以下を繰り返す
- priority_queueから最大コストのパスを取り出して、使う　パスを使うことによって、木がいくつかの部分木に分かれることになる
- 分かれる各部分木の最大パスをpriority_queueにpushする
- こっちのほうが簡単

小課題4

- どちらの方法でも $O(N \log N)$ でできる
- 決め打つ葉を $O(N)$ 通り試すので、合計 $O(N^2 \log N)$

小課題5

- これいる？
- 僕はいらないと思います
- 満点解法に繋がらないし面倒なだけなので省略
- 一応簡単に書くと、先の解法でpriority_queueにpushされるコストの集合を考えて、根として選ぶ頂点を(葉に限定しないで)一つ隣の頂点に移動した場合の変化を見る
- 変化の回数が1回の移動あたり定数回なので解ける

小課題6

- 葉を全部試すのが大変
- 実は、2個選ぶときの解で選ぶ頂点のうち一つを適当にとってくる(R とする)と、2個以上選ぶときの解であって R を含むものが必ず存在

小課題6

- 証明は帰納法を回す
- K 個選ぶ解であって R を含むものが存在すると仮定
- $K=2$ は自明
- $K+1$ 個選ぶ解を任意にとる
- ある解に対して、選んだ2頂点間のパス上に存在する点を内点と呼ぶことにする
- K 個選ぶ解と $K+1$ 個選ぶ解で、内点が共有されている場合、そこを起点に先程の貪欲を走らせれば同じ解が得られるはず

小課題6

- 内点が共有されていない場合、 $K+1$ 個の解で選んだ頂点のうち適切なものを取ると、それを K 個の解に付け足すと $K+1$ 個の解と比べて損しない解が得られる
- よって示された

小課題6

- 根として固定する葉が1つでいいので、 $O(N \log N)$ で通る

得点分布

●0点	1人
●6点	3人
●7点	1人
●13点	8人
●16点	1人
●22点	1人
●30点	1人
●39点	4人
●100点	1人

ランプ (Lamps)

解説：保坂

JOI 春合宿 Day 3 2019/03/22



例 1

$$\begin{array}{cccccccc} 1 & 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ & & & \downarrow & & & & \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{array}$$


例 1

1	1	0	1	1	1	0	0
TOGGLE							
0	0	1	0	1	1	0	0
ON							
0	1	1	0	1	1	0	0
					TOGGLE		
0	1	1	0	1	0	1	1
					OFF		
0	1	1	0	1	0	0	1



単純な解法

全状態を探索する

- 0, 1 の状態は 2^N 通り
- 各状態からの遷移は $O(N^2)$ 通り

初期状態から各状態までの距離が BFS で $O(2^N N^2)$ 時間でわかる



特殊な場合

初期状態が全部 0 のとき

0	0	0	0	0	0	0	0	0	0	0	0	0
ON			ON					ON				
1	1	0	1	1	1	1	0	0	1	1	0	

- 「連続する 1 の区間の個数」回の操作でできる (各区間で ON または TOGGLE をする)
- それ未満ではできない？



下からの評価

“01” あるいは “10” と並んでいる場所は，1 回の操作で高々 2 箇所しか増えない

- 区間の左右の端の部分でしか増えないため
- 全体の左右の端には 0 が並んでいるとする

連続する 1 の区間を k 個作るには，“01” あるいは “10” と並ぶ場所を $2k$ 箇所作らないといけないので， k 回の操作は必要



例 2

1	0	1	0	0	1	0	0	1	0	1	0	0
						↓						
0	0	0	0	1	1	1	0	0	1	0	1	1



例 2

1	0	1	0	0	1	0	0	1	0	1	0	0
OFF												
0	0	0	0	0	1	0	0	1	0	1	0	0
				ON								
0	0	0	0	1	1	1	0	1	0	1	0	0
								TOGGLE				
0	0	0	0	1	1	1	0	0	1	0	1	1

かんたん



例 3

0	0	1	1	0	0	0	1	0	0	1	0	0	0	0	1	1	0
			OFF														
0	0	1	0	0	0	0	1	0	0	1	0	0	0	0	1	1	0
						ON											
0	0	1	0	0	1	1	1	0	0	1	0	0	0	0	1	1	0
								ON									
0	0	1	0	0	1	1	1	0	1	1	1	0	0	0	1	1	0
					TOGGLE												
1	1	0	1	1	0	0	0	1	0	0	0	1	0	0	1	1	0
															TOGGLE		
1	1	0	1	1	0	0	0	1	0	0	0	1	0	0	1	0	1

むずかしい

OFF, ON による下準備と TOGGLE 分けて考える
とよい？



例 1 ふたたび

1	1	0	1	1	1	0	0
TOGGLE							
0	0	1	0	1	1	0	0
ON							
0	1	1	0	1	1	0	0
					TOGGLE		
0	1	1	0	1	0	1	1
					OFF		
0	1	1	0	1	0	0	1

これでもいいけど



例 1 ふたたび

```

1 1 0 1 1 1 0 0
  OFF
1 0 0 1 1 1 0 0
-----
  TOGGLE
0 1 1 0 1 1 0 0
                        TOGGLE
0 1 1 0 1 0 1 1
                        OFF
0 1 1 0 1 0 0 1
  
```

こうしてもいいし



例 1 ふたたび

```

1 1 0 1 1 1 0 0
  OFF
1 0 0 1 1 1 0 0
-----
  TOGGLE
0 1 1 0 1 1 0 0
                        ON
0 1 1 0 1 1 1 0
                        TOGGLE
0 1 1 0 1 0 0 1
  
```

こうもできるし



例 1 ふたたび

1	1	0	1	1	1	0	0
				OFF			
1	0	0	1	1	1	0	0
					ON		
1	0	0	1	1	1	1	0
				TOGGLE			
0	1	1	0	1	1	1	0
					TOGGLE		
0	1	1	0	1	0	0	1

こうしてもいい



操作の順番

OFF, ON を先にやるとしてよい

すべての OFF, ON をすべての TOGGLE より先に行う最適解が存在する

直感的な理由：OFF, ON は操作前の列の情報を消してしまう感じ，TOGGLE は保つ感じだから



操作の順番

OFF, ON を先にやるとしてよい

すべての OFF, ON をすべての TOGGLE より先に行う最適解が存在する

操作の結果を変えず回数を増やさずに, OFF, ON を先に持ってこれることを示す



操作の順番

TOGGLE した部分の一部に OFF [ON] をする場合

?	?	?	?	?	?	?	?
TOGGLE							
?	i	i	i	i	i	i	?
OFF							
?	i	0	0	0	i	i	?

代わりに先に ON [OFF] をする

?	?	?	?	?	?	?	?
ON							
?	?	1	1	1	?	?	?
TOGGLE							
?	i	0	0	0	i	i	?



操作の順番

そうでない場合

```

  ?  ?  ?  ?  ?  ?  ?  ?
    -----
      TOGGLE
  ?  ?  ?  ?  ?  ?  ?  ?
    -----
      OFF
  ?  ?  0  0  0  0  0  ?
  
```

(TOGGLE する区間を縮めて) 入れ替える

```

  ?  ?  ?  ?  ?  ?  ?  ?
    -----
      OFF
  ?  ?  0  0  0  0  0  ?
  TOGGLE
  ?  ?  0  0  0  0  0  ?
  
```



操作の順番

そうでない場合

```

?  ?  ?  ?  ?  ?  ?  ?
      TOGGLE
?  ?  ?  ?  ?  ?  ?  ?
      OFF
?  0  0  0  0  0  0  ?
    
```

(TOGGLE する区間を縮めて) 入れ替えるか消す

```

?  ?  ?  ?  ?  ?  ?  ?
      OFF
?  0  0  0  0  0  0  ?
    
```



方針

操作の様子を以下の方針で考える

1. 操作 OFF, ON たちが終了した時点で、各ランプが「OFF にされた」「ON にされた」「最初のまま」のいずれになるか決める
2. 初期状態から 1. で決めた状態にするために必要な操作 OFF, ON の最小回数を求める
3. 1. で決めた状態から目標状態にするために必要な操作 TOGGLE の最小回数を求める



方針

操作の様子を以下の方針で考える

1. 操作 OFF, ON たちが終了した時点で、各ランプが「OFF にされた」「ON にされた」「最初のまま」のいずれになるか決める
2. 初期状態から 1. で決めた状態にするために必要な操作 OFF, ON の最小回数を求める
3. 1. で決めた状態から目標状態にするために必要な操作 TOGGLE の最小回数を求める



OFF, ON の回数

「OFF にされた」「ON にされた」「最初のまま」
を 0, 1, ? で表すとする

?0111001111?00110111000?? みたいにするには?

? 0 1 1 1 0 0 1 1 1 1 ? 0 0 1 1 0 1 1 1 0 0 0 ??



OFF, ON の回数

? で区切って, 0 の連続や 1 の連続はまとめて,

- 0 : 1 回
- 01 : 2 回
- 010 : 2 回
- 0101 : 3 回
- 01010 : 3 回
- 010101 : 4 回



方針

操作の様子を以下の方針で考える

1. 操作 OFF, ON たちが終了した時点で、各ランプが「OFF にされた」「ON にされた」「最初のまま」のいずれになるか決める
2. 初期状態から 1. で決めた状態にするために必要な操作 OFF, ON の最小回数を求める
3. 1. で決めた状態から目標状態にするために必要な操作 TOGGLE の最小回数を求める



TOGGLE の回数

1001010110 から 0100101111 に TOGGLE だけで
変えるには？

0000000000 から 1101111001 に TOGGLE だけで
変えるには？ と同じ (XOR をとった)
小課題 3 の解法で説明した通り



方針

操作の様子を以下の方針で考える

1. 操作 OFF, ON たちが終了した時点で、各ランプが「OFF にされた」「ON にされた」「最初のまま」のいずれになるか決める
2. 初期状態から 1. で決めた状態にするために必要な操作 OFF, ON の最小回数を求める
3. 1. で決めた状態から目標状態にするために必要な操作 TOGGLE の最小回数を求める



動的計画法

0, 1, ? の列をすべて試すと 3^N 通りでダメ

OFF, ON の回数も TOGGLE の回数も，列を左から一度見ていくだけで求まる

OFF, ON の回数や TOGGLE の回数を求めるのに必要なものを状態として，左から 0, 1, ? の列を決めていく DP ができそう



動的計画法

状態

- 今何文字目まで見たか
- 直前は 0, 1, ? のどれであるか
- (0 の連続や 1 の連続をまとめたとき) 今作っている 0101... [1010...] の部分の長さ

遷移

- 次を 0, 1, ? のどれにするか



動的計画法

回数計算

- OFF, ON の回数：0101... を閉じるときに足す
- TOGGLE の回数：目標との XOR が 0 から 1 になるときに足す

$O(N^2)$ 時間・ $O(N^2)$ メモリ (小課題 2 が解ける)



効率の良い回数計算

状態

- 今何文字目まで見たか
- 直前は 0, 1, ? のどれであるか
- (0 の連続や 1 の連続をまとめたとき) 今作っているのは 0101... か 1010... か

$5N$ 状態くらい

$O(N)$ 時間・ $O(N)$ メモリ (満点)



操作の手順をもっと制限

実は……

OFF, ON をする区間は重ならなくてよい

以下を満たす最適解が存在する

- すべての OFF, ON をすべての TOGGLE より先に行う
- OFF, ON をする区間は互いに重ならない
- OFF, ON をする区間は隣接しない



操作の手順をもっと制限

OFF や ON をする区間が重なっているのは……

```

?  ?  ?  ?  ?  ?  ?  ?
  _____
      OFF
?  0  0  0  0  0  ?  ?
  _____
      ON
?  0  0  1  1  1  1  ?
  
```

意味がない

```

?  ?  ?  ?  ?  ?  ?  ?
  _____
      OFF
?  0  0  ?  ?  ?  ?  ?
  _____
      ON
?  0  0  1  1  1  1  ?
  
```



操作の手順をもっと制限

OFF や ON をする区間がくっついているのは……

```

  ?  ?  ?  ?  ?  ?  ?  ?
    -----
    OFF
  ?  0  0  ?  ?  ?  ?  ?
            -----
                ON
  ?  0  0  1  1  1  1  ?
  
```

TOGGLE を使うとくっついている部分を消せる

```

  ?  ?  ?  ?  ?  ?  ?  ?
    -----
    OFF
  ?  0  0  0  0  0  0  ?
            -----
        TOGGLE
  ?  0  0  1  1  1  1  ?
  
```



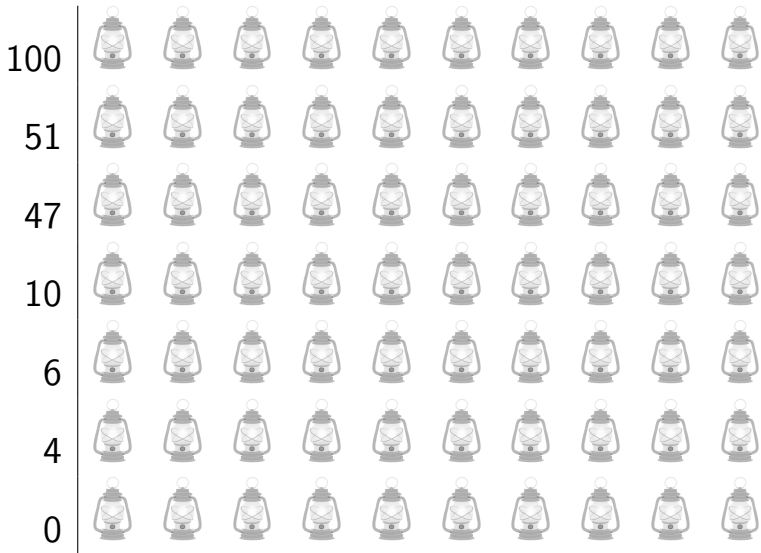
操作の手順をもっと制限

0, 1, ? の列としては, $?...?0...0?...?0...0?...?1...1$
 のように 0 と 1 が隣接しないものだけを考えればよい

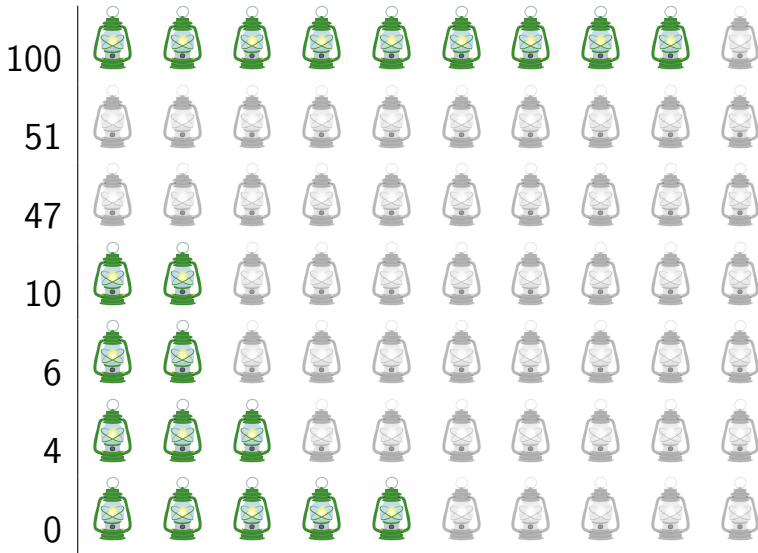
DP の状態を $3N$ 個くらいにできる



得点分布



得点分布



時をかけるビ太郎

Bitaro, who Leaps through Time

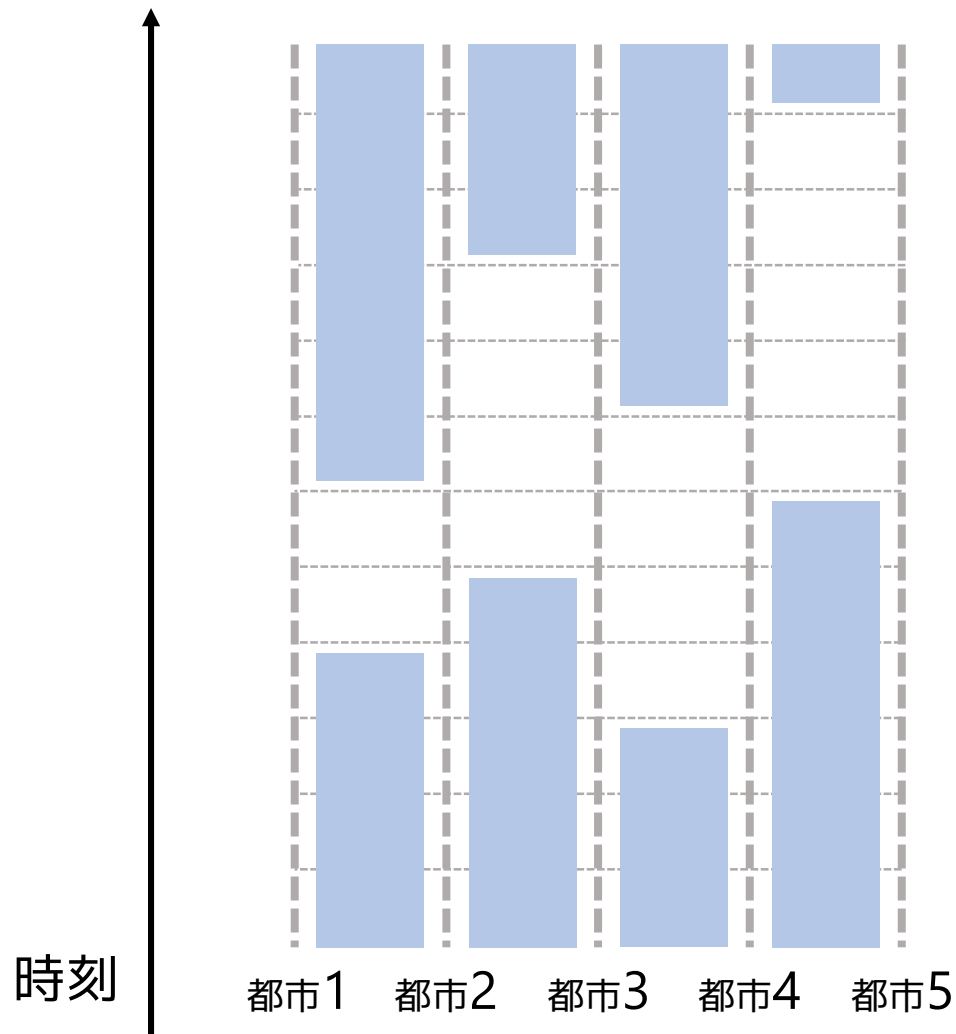
JOI2018/2019 Spring Training Camp Day3

wafrelka

問題概要

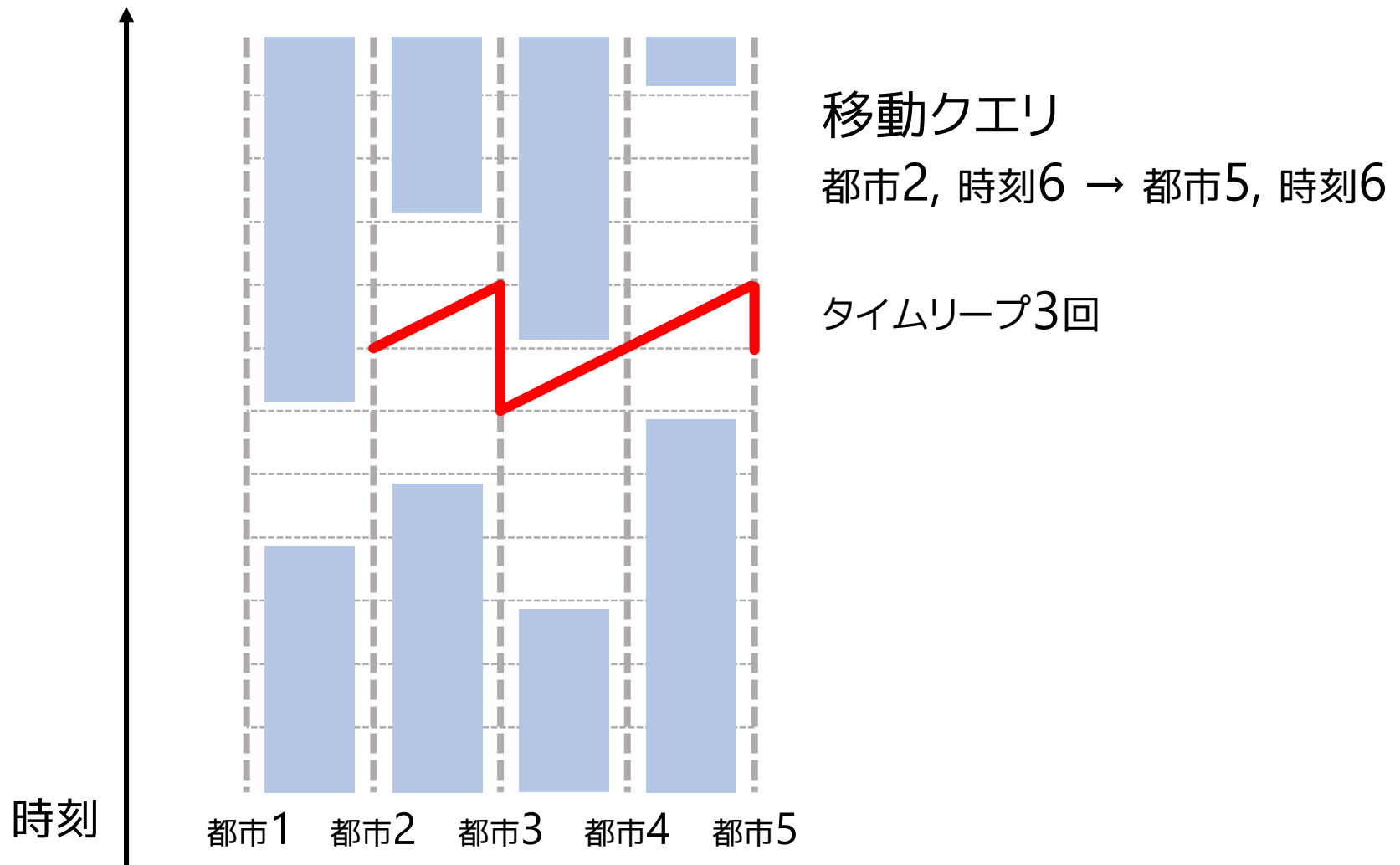
- N 個の都市が一行に並んでいる
- $N - 1$ 本の道路がある
 - 道路 i を使うと都市 i と都市 $i + 1$ を 1 秒で行き来できる
 - 道路 i は時刻 L_i から時刻 R_i の間しか開いていない
- ビ太郎は都市にいるときタイムリープができる
 - 1 回タイムリープをすると 1 秒だけ時間を遡れる
- 以下のクエリを処理してください
 - 道路 i が開いている時刻を S_j から E_j までに変更する
 - ビ太郎が時刻 B_j の時点で都市 A_j にいるとき
時刻 D_j の時点で都市 C_j にいるように移動・タイムリープをするときの最小のタイムリープ回数

問題概要 – サンプル

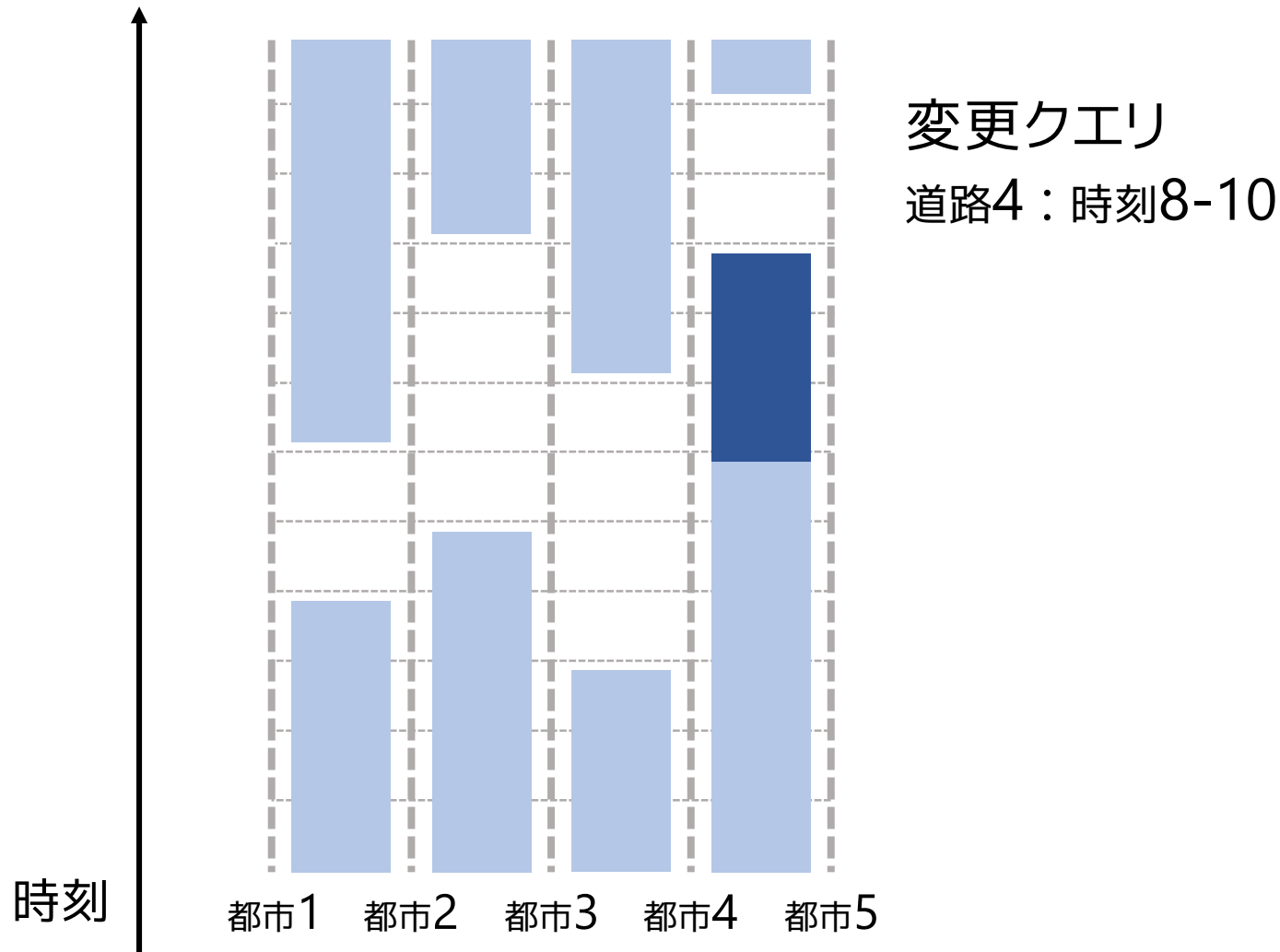


$$\{(L_i, R_i)\} = \{(3, 5), (4, 8), (2, 6), (5, 10)\}$$

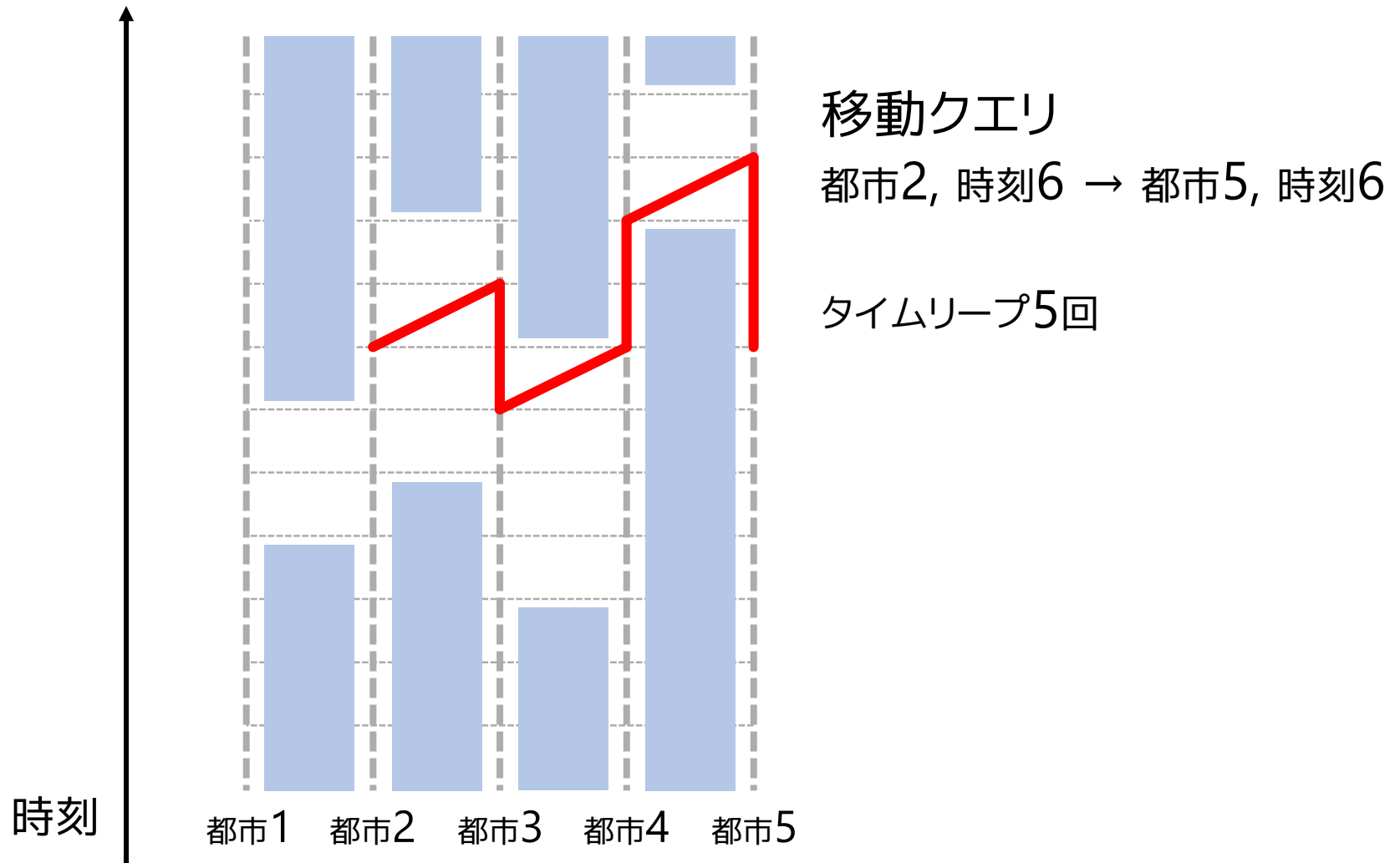
問題概要 – サンプル



問題概要 – サンプル



問題概要 – サンプル



小課題 1

- 制約: $N \leq 1000, Q \leq 1000$
- 移動クエリごとにシミュレーションかな？

考察 – ビ太郎の行動

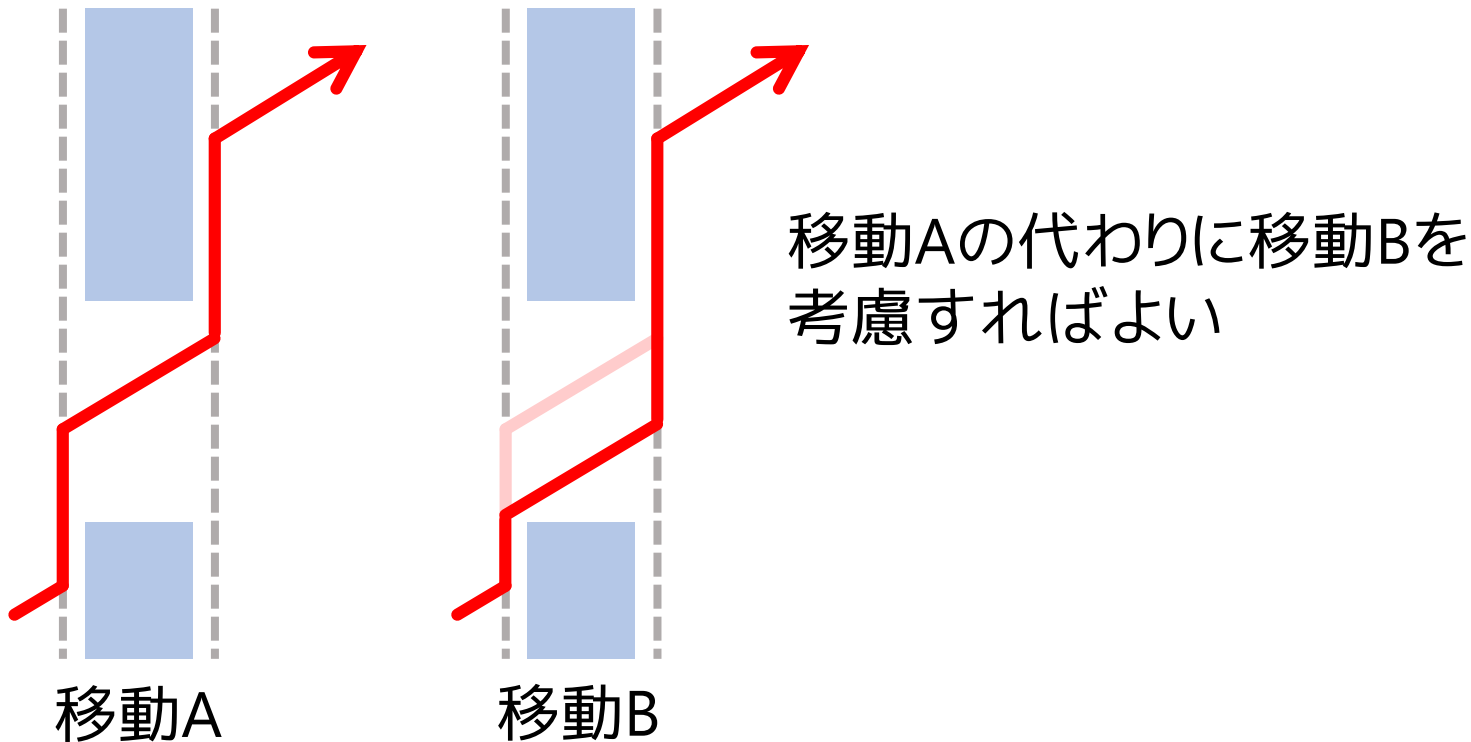
- ビ太郎の行動は 3 種類
 - 1 秒かけて隣の都市へ移動する
 - 都市で X 秒だけ待機する
 - 都市で Y 回タイムリープする
- これらの組み合わせを考えればよい

考察 – ビ太郎の行動

- タイムリープ回数が同じならば
同じ都市へ行く複数の行動のうち
到着時刻がいちばん早いものが嬉しい
- 目的地とは反対側の都市へ移動する必要はない
- → 都市 A_i から都市 B_i ($A_i < B_i$) に移動するときは
都市 $A_i, A_i + 1, \dots, B_i$ の出発時刻だけ考えればよい

考察 – ビ太郎の行動

- 各都市での待機・タイムリープは必要最小限でよい
- 各都市の出発・到着時刻も整数値のみ考えればよい



部分点解法 1

- 移動クエリごとにシミュレーションする
 - いまいる都市および現在時刻を記録しておく
 - 目的地に向かって移動をする
 - 道路が開いていないときは必要最低限の待機・タイムリープを行う
-
- $O(NQ)$ で小課題 1 が解けて 4 点が得られる

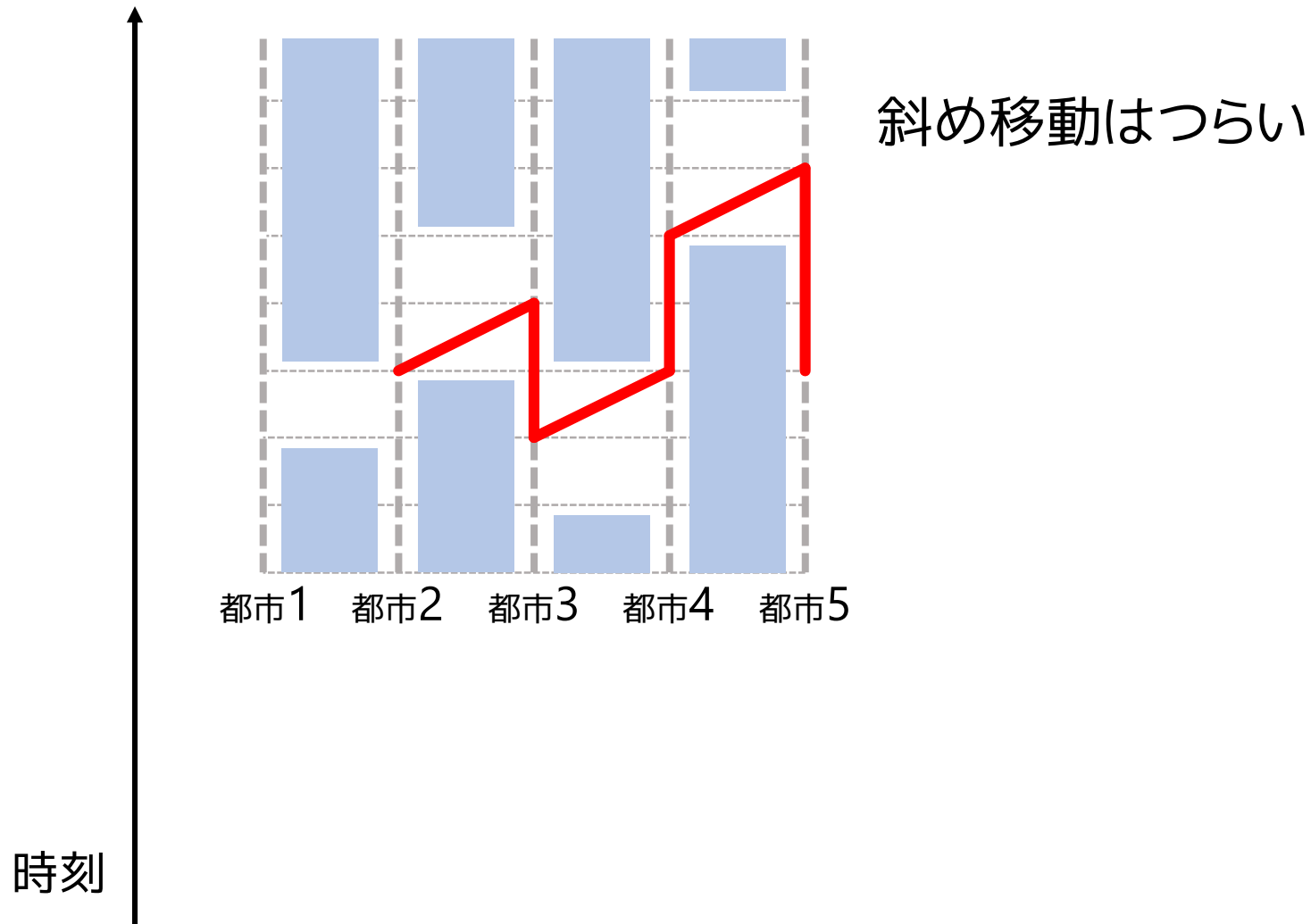
小課題 2

- 制約: $N \leq 300000, Q \leq 300000$
- 変更クエリがない

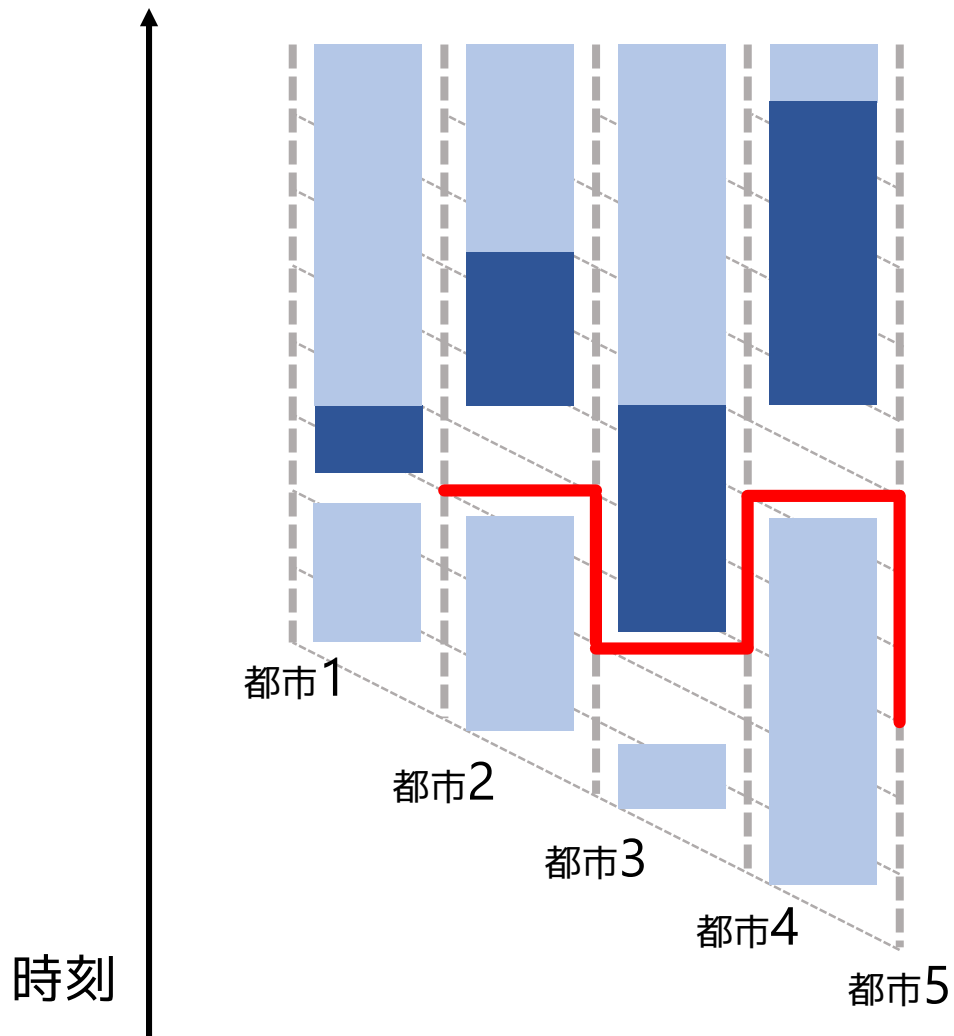
問題の変形 – 移動クエリ

- 以降は都市 A_i から都市 B_i ($A_i < B_i$) への移動クエリのみを考える
 - $A_i > B_i$ の場合は都市の順番をひっくり返して考えれば OK
 - $A_i = B_i$ の場合は明らか

問題の変形 – 時間軸



問題の変形 – 時間軸



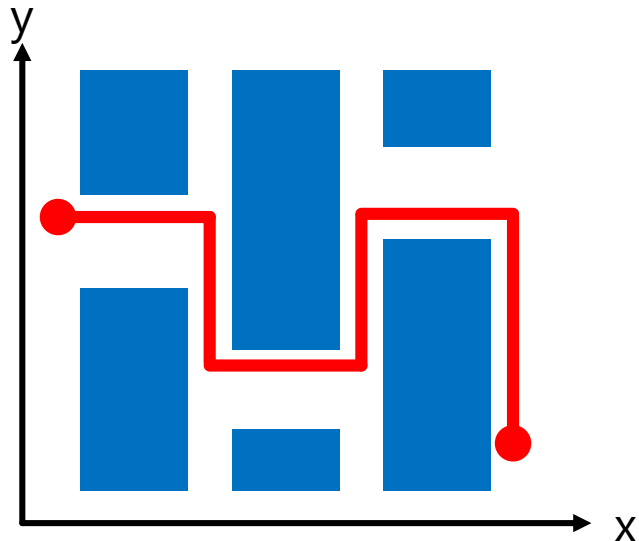
時間軸を歪めて
都市 i の時刻が
 i だけ進んでいることにする

移動コスト : 0

移動クエリ: $(B_i - A_i, D_i - C_i)$ 道路 $i: (L_i - i, R_i - (i + 1))$

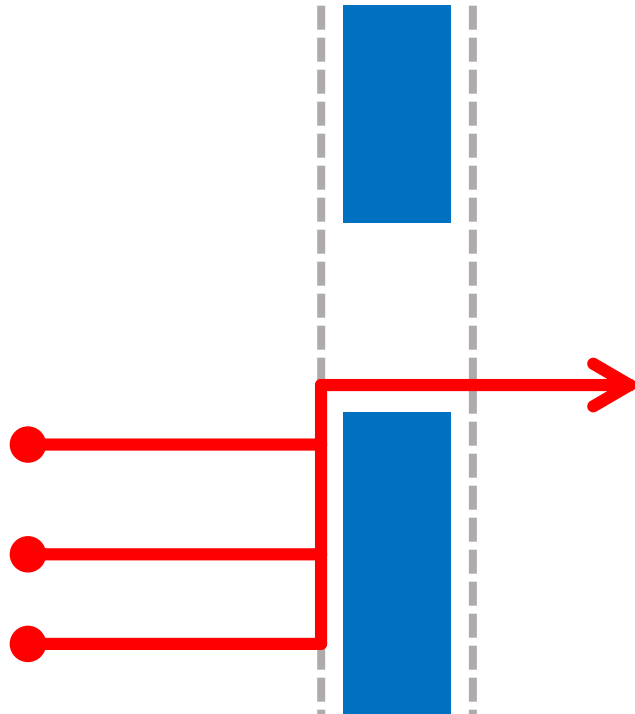
問題の変形

- 以下のような問題になった
- 長方形のブロックがいくつか並んでいる
- 以下のクエリを処理してください
 - ブロックの位置を変更する
 - 指定された始点から終点まで移動するときの
下方向移動コストの最小回数を計算する



考察 – 共通化

- あるブロックにぶつかるような経路はすべてそれ以降は同じような経路をたどる



最初にぶつかるブロック

- とある頂点から右に移動していったとき
最初にぶつかるブロックは $O(N)$ で計算できる
- `std::set` を使って左側からスキャンするなどで
頂点 K 個に対しても $O(N + K \log K)$ で計算できる
 - $x = 1$ から $x = N$ までスキャンしていく
 - $x = k$ に注目しているとき
 $x = k$ でブロックにぶつかる頂点をセットから削除して
 $x = k$ な頂点をセットに追加する
 - $x = k$ で上 (下) ブロックにぶつかる頂点は
セットから y の値が最大 (最小) な頂点を取り出すのを
繰り返すことで効率的に削除できる

ダブリング

- 各移動クエリの始点 + ブロックの角に関してそこから右側に進んだとき最初にぶつかるブロックを求める
- すると各始点から右側に進んだとき 2^k 回目にぶつかるブロックが求められる (ダブリング)
- ブロックを回避するために必要なコストも同時に計算できる

部分点解法 2

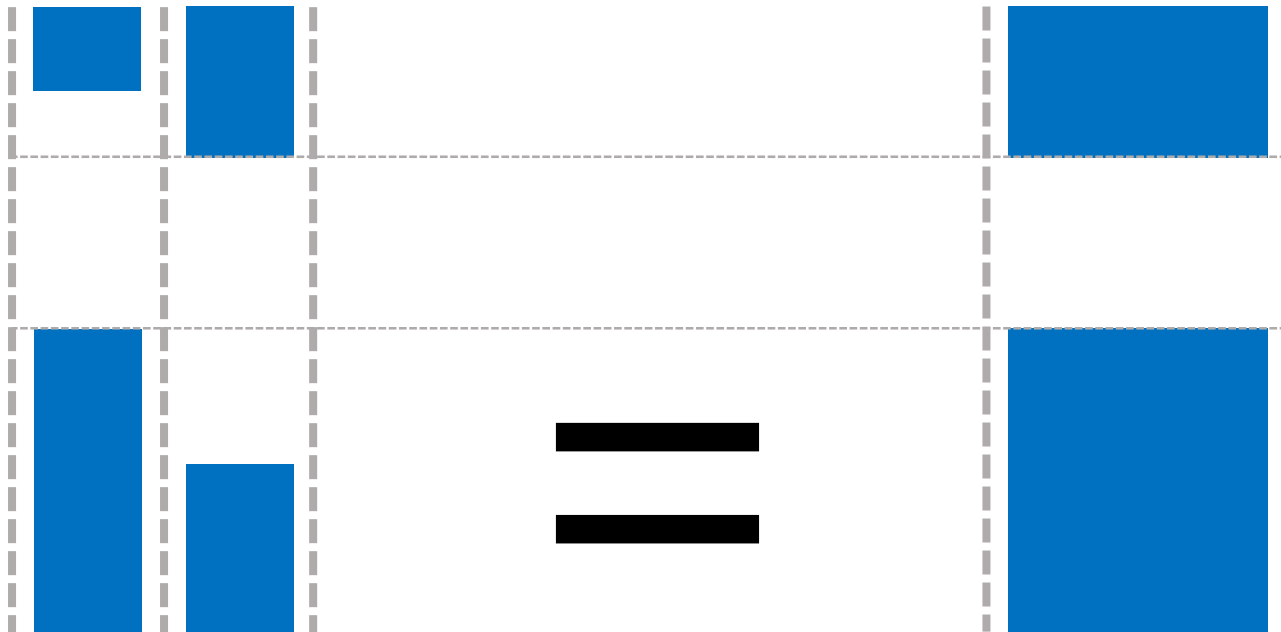
- 各移動クエリの始点 + 各ブロックの角に関してダブリングを使うことで 2^k 回目にぶつかるブロックとそこまでのコストを求めておく
 - $O((N + Q) \log(N + Q))$
- 各移動クエリに対して終点にたどり着く前に最後にぶつかるブロックとそこまでのコストを計算する
 - $O(\log N)$
- 変更クエリがなければ全体として $O((N + Q) \log(N + Q))$ 計算できる

小課題 3

- 制約: $N \leq 300000, Q \leq 300000$
- 変更クエリあり
- 部分点解法 2 は無理かな

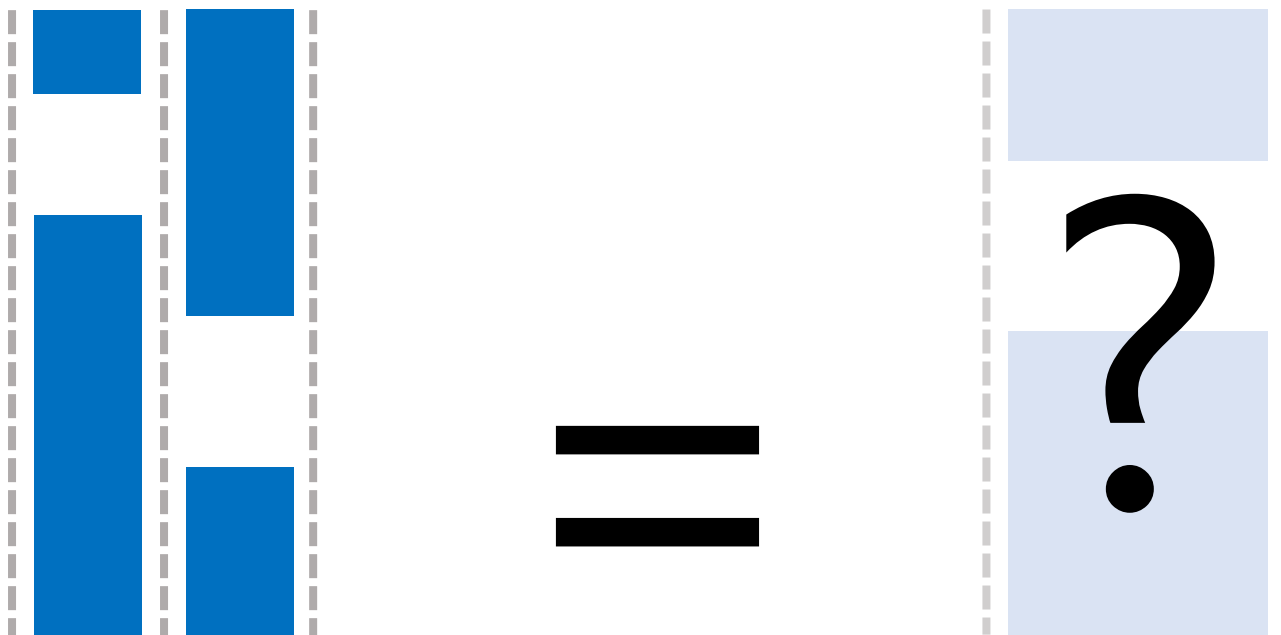
考察 – セグメントツリー？

- 左側の区間 2 つは右側の区間と同一視できる
- 通過後の座標およびコストが一致する
- セグメントツリーの機運かな？



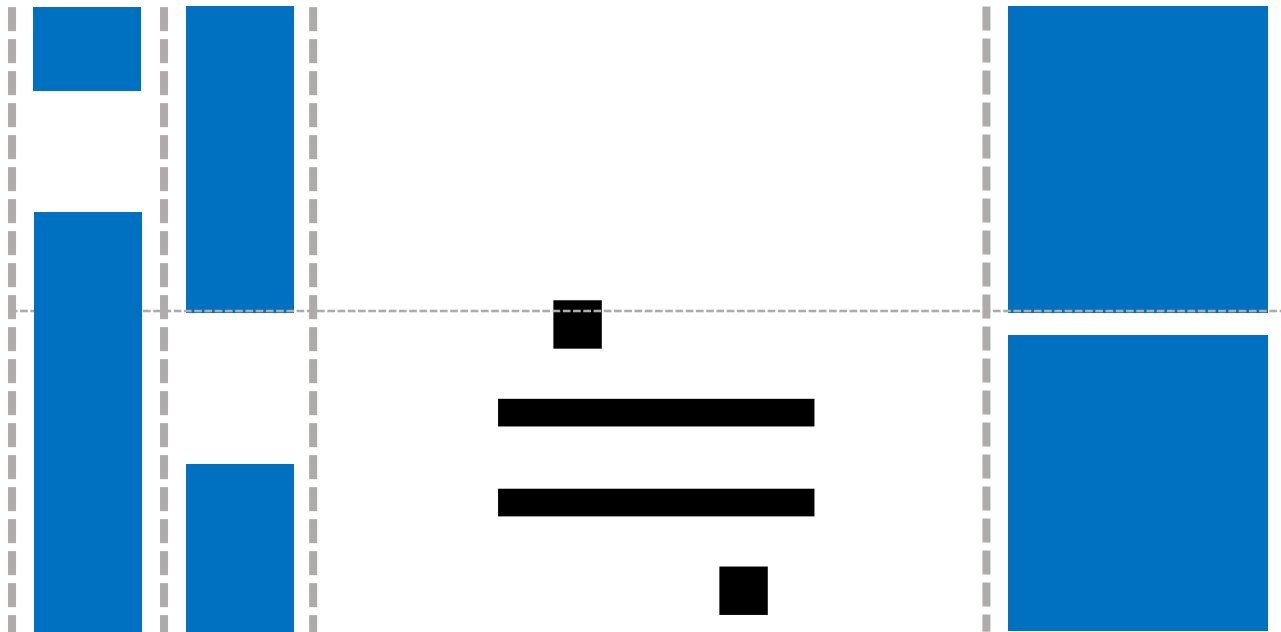
考察 – セグメントツリー？

- 左側の区間 2 つに対応する単体の区間は**存在しない**



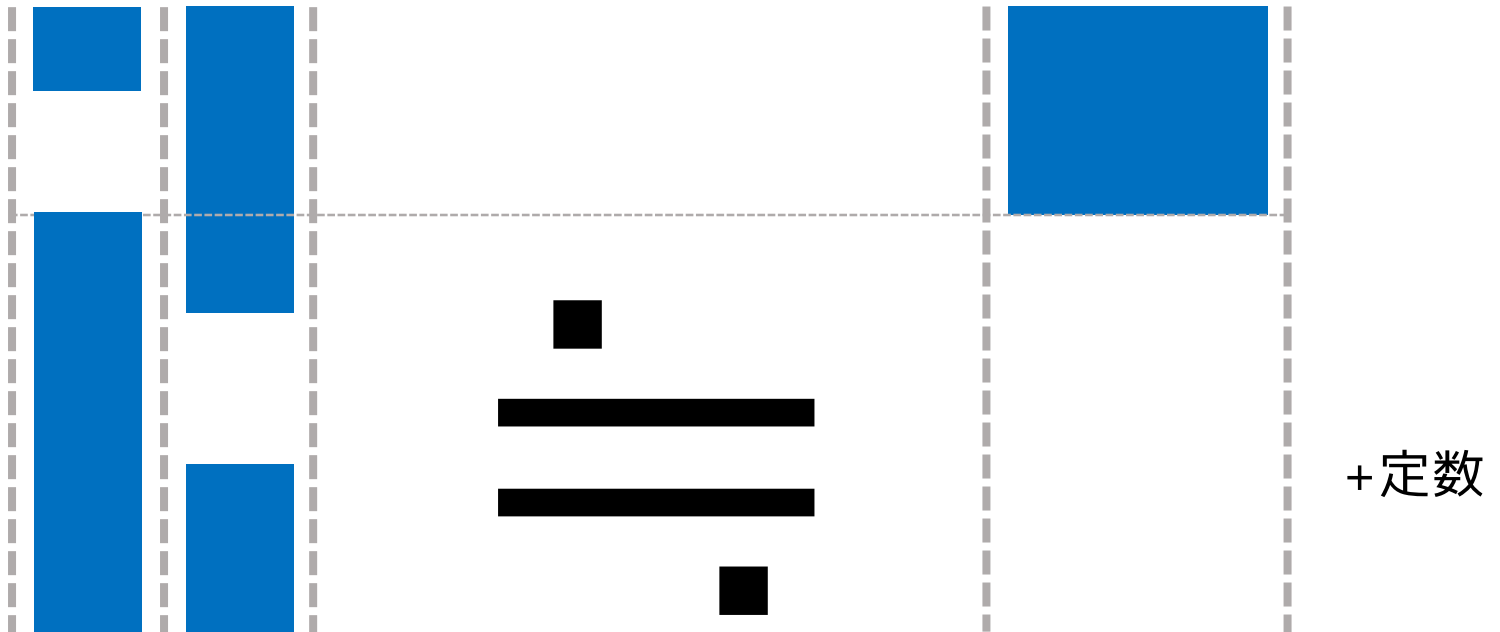
考察 – セグメントツリー？

- 通過後の座標が一致するものはある



考察 – セグメントツリー？

- 通過後のコストがおおよそ一致するものもある
(定数だけずれる)



考察 – マージ

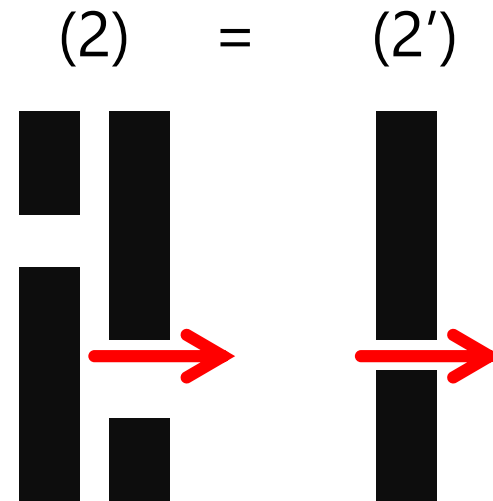
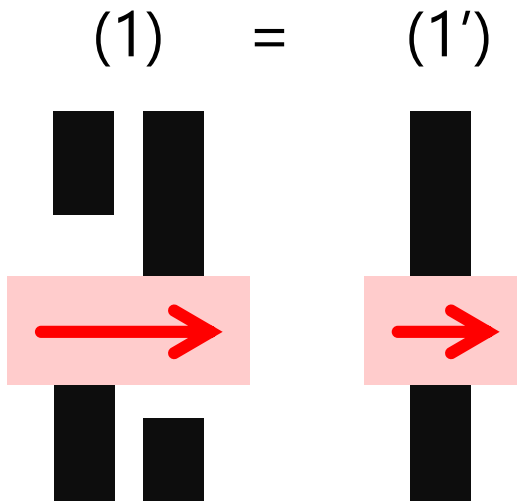
- 座標とコストを同時に一致させるようなものは存在しないことがあるのでうまくマージできない
- → 通過後の座標とコストを分解して考える

考察 – マージ

- 各区間は以下の 2 つのどちらかだと見なせる
 - a. 通過後の座標・コストがある 1 つのブロック対を通過したときと一致する
 - b. 通過後の座標は一意に定まりコストはとある上ブロックを通過したときと定数を除いて一致する
- **キーポイント**
これらの区間を合成してもふたたび a, b のどちらかの種類の区間になる

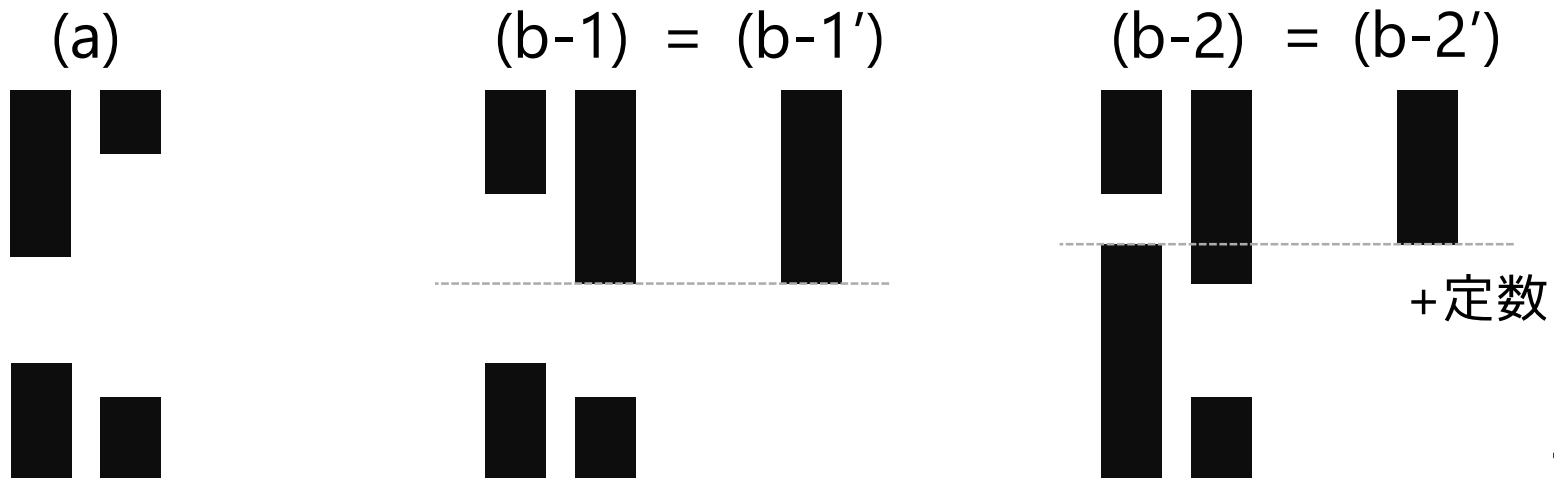
マージ – 座標 (かんたん)

- (1) 空いている場所について共通部分があるなら合成結果は共通部分だけ空いているブロック対
- (2) 共通部分がないなら通過後の座標は右側のブロック対の端点のどちらか



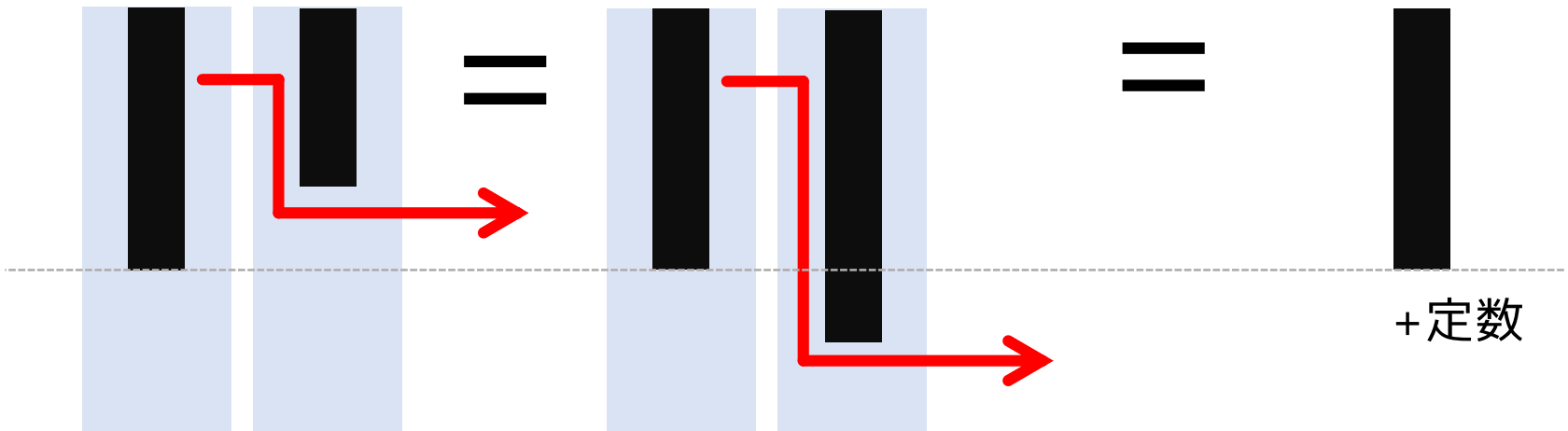
マージ - コスト (むずかしい)

- 左側の区間がタイプ a (座標・コスト一致) のとき
- 右側の区間のコスト担当の上ブロックについて
- (a) 左側の上ブロックより小さいとき
合成後は左側の上ブロックと一致
- (b) そうでないとき
合成後はとある上ブロック + 定数と一致



マージ - コスト (むずかしい)

- 左側の区間がタイプ b (座標・コスト不一致) のとき
- 左側の区間を通ると座標は一意に決まるので
右側の区間を通るコストも一意に決まる
- もちろん各区間で足される定数コストも足す



マージ – 一貫性

- $a+a$, $a+b$, $b+a$, $b+b$ の 4 つの組み合わせについて以下の 2 つを確認した
 - 座標のマージがうまくいく (タイプ a または b の形になる)
 - コストのマージがうまくいく (タイプ a または b の形になる)
- 座標とコストを組み合わせたときに座標はタイプ a の形だけどコストはタイプ b の形という事態が発生するとまずい
- そういう事態は起こりうるのか？

マージ – 一貫性

- 実は心配しなくてよい
- 座標がタイプ a の形になるのは
タイプ a の区間を 2 つ足し合わせたとき
- タイプ a を 2 つ足し合わせても
タイプ a, b のどちらかにしかない (最初に確認した)

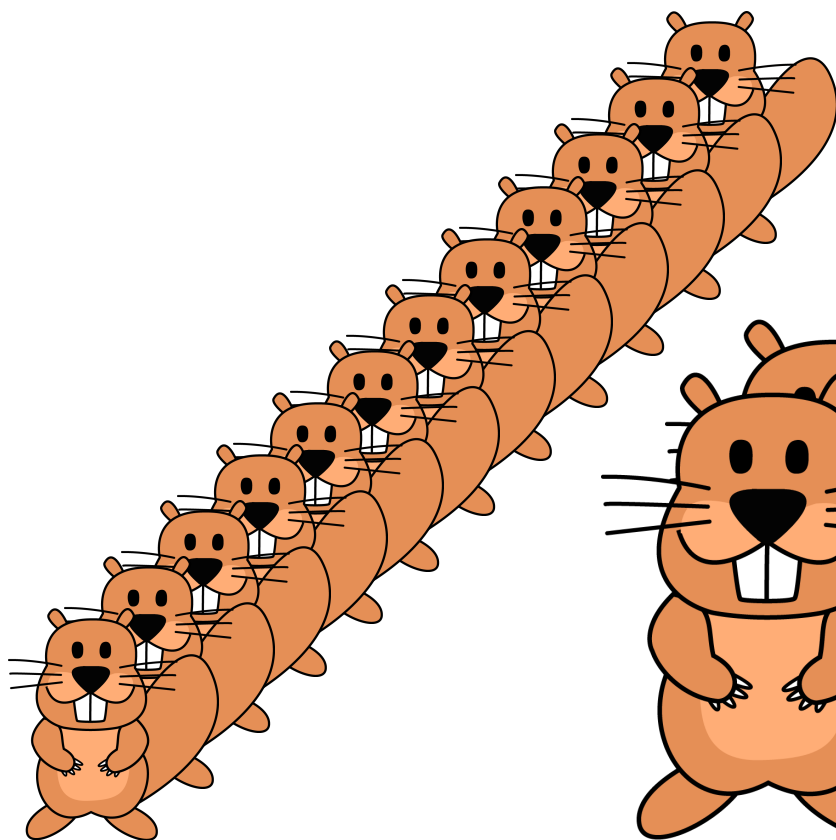
満点解法

- それぞれのブロック対を葉ノードとするセグメントツリーを構築する
- 変更クエリについて
 - セグメントツリーの葉ノードおよびその親たちを更新する
 - $O(\log N)$
- 移動クエリ $(x_1, y_1) \rightarrow (x_2, y_2)$ について
 - $x = x_1$ から $x = x_2$ の間にあるブロック対をすべてマージ
 - 区間を通過した後のコスト・座標を計算
 - $O(\log N)$
- 全体として $O(Q \log N)$ で計算できる

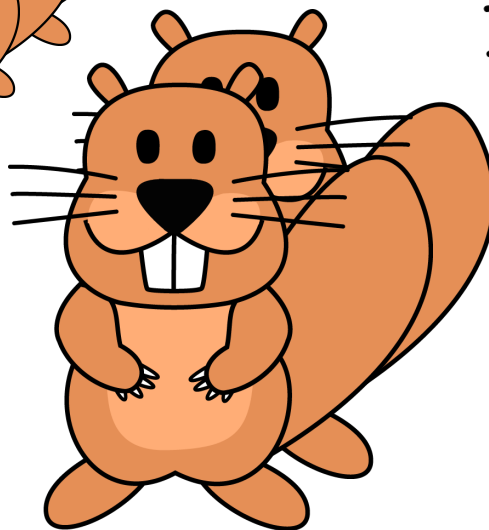
反省

- スライドを作った後に思いましたが
コスト関数と座標関数の合成をするセグメントツリー
だということもできて
そっちのほうに分かりやすかったかな.....。

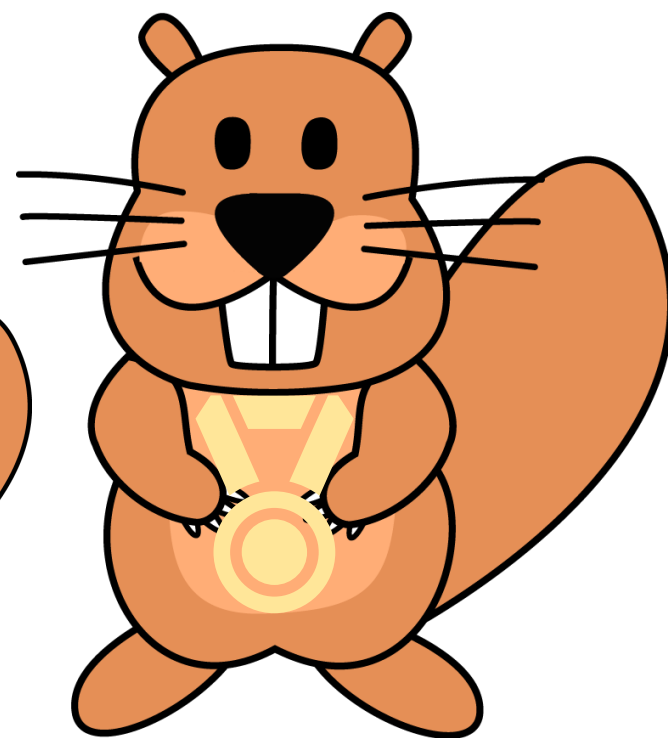
得点分布



4



34



100





JOI2019春合宿 Day 4 -

ふたつのケーキ

解説: 清水郁実



問題概要

- 整数 N, M と N 個のケーキの情報 (V_i, C_i) が与えられる
- このうち M 個のケーキを選んで自由な順に円環状に並べたときの、
(選んだケーキの V の値の総和) - (隣り合う2つのケーキの C の値の
差の絶対値の総和)
の最大値を答えよ
- $3 \leq M \leq N \leq 200,000$
 $0 \leq V_i, C_i \leq 10^9$

自明な考察

- M個のケーキを選び終えたときに、**(隣り合う2つのケーキのCの値の差の絶対値の総和)**を最小化するような並び替えを考える

自明な考察

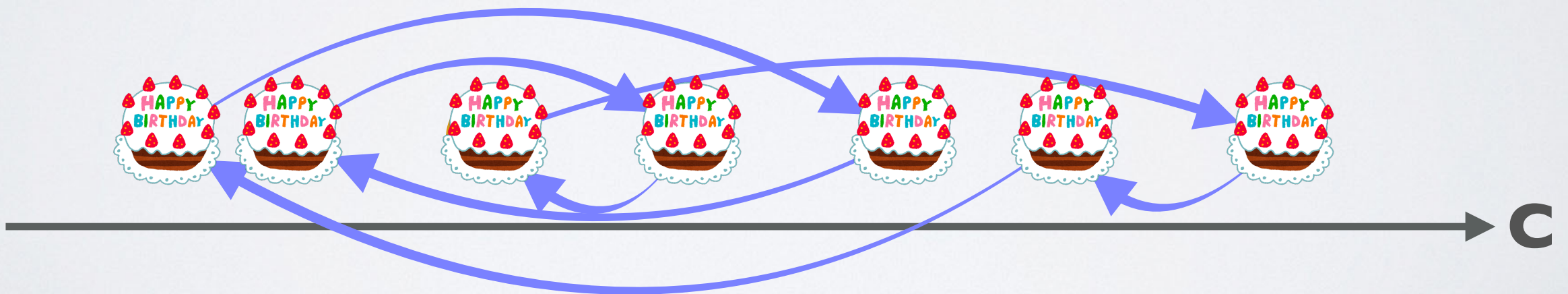
- M個のケーキを選び終えたときに、**(隣り合う2つのケーキのCの値の差の絶対値の総和)**を最小化するような並び替えを考える



C

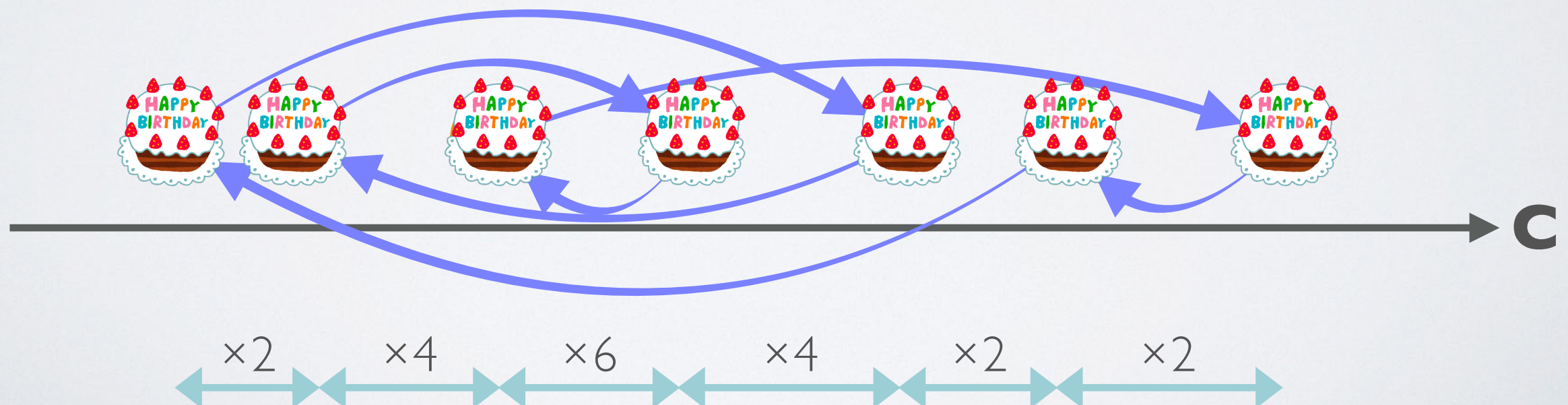
自明な考察

- M個のケーキを選び終えたときに、**(隣り合う2つのケーキのCの値の差の絶対値の総和)**を最小化するような並び替えを考える



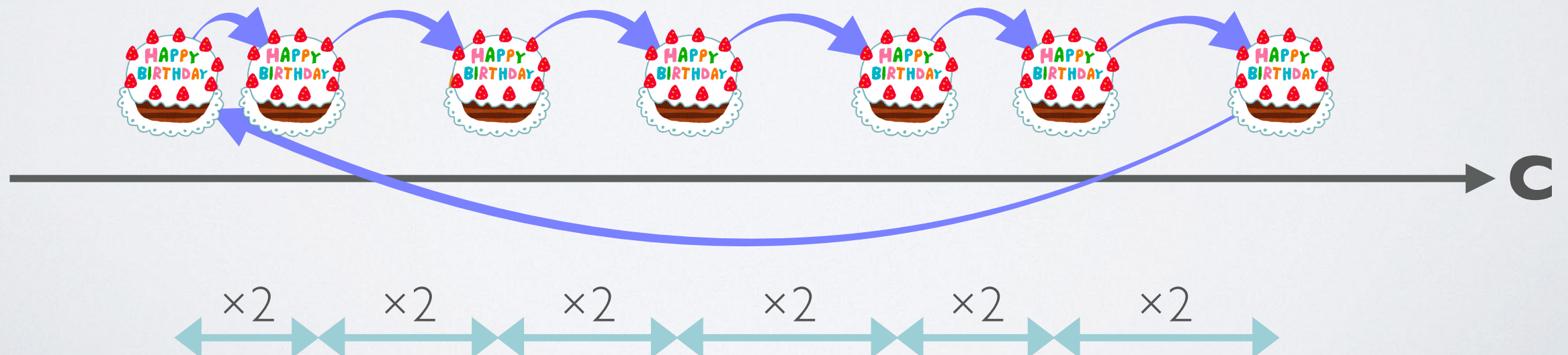
自明な考察

- M個のケーキを選び終えたときに、**(隣り合う2つのケーキのCの値の差の絶対値の総和)**を最小化するような並び替えを考える



自明な考察

- M個のケーキを選び終えたときに、**(隣り合う2つのケーキのCの値の差の絶対値の総和)**を最小化するような並び替えを考える



自明な考察

- Cが小さい(or 大きい)順に並べるのが最適で、
このときの値は **$2 \times (\text{最大値} - \text{最小値})$**

自明な考察

- Cが小さい(or 大きい)順に並べるのが最適で、
このときの値は **$2 \times (\text{最大値} - \text{最小値})$**
- Cの値を予めすべて2倍しておく と2倍の係数がなくなって楽

問題の言い換え

- 整数 N, M と N 個のケーキの情報 (V_i, C_i) が与えられる
- このうち M 個のケーキを選んだときの
(選んだ V の値の総和) - (選んだ C の最大値 - 最小値)
を最大化する

小課題1: $N \leq 100$ (5点)

- Cの最小値と最大値を決めて、その間に収まるケーキのうちVの値が大きい方からM個取る
- 実装方針によって $O(N^3)$ or $O(N^3 \log N)$

小課題2: $N \leq 2,000$ (24点)

- このままだと見通しが悪いので、ケーキを (C_i, V_i) の順にソートしておく
- すべての長さ M 以上の区間について、
区間内部の V の値最大 M 個の和から $(C[\text{右端}] - C[\text{左端}])$ を引いた値の最大値を求めたい

小課題2: $N \leq 2,000$ (24点)

- 区間の左端を固定して、右端を右方向に動かしていく
- (多重)集合に値を追加していった、毎回大きい方からM個の和がわかるようにしたい
- これは優先度付きキュー (priority_queue) 等を使ってよしなに実装するとできて、全体 $O(N^2 \log N)$

小課題2: $N \leq 2,000$ (24点)

- 区間の左端を固定して、右端を右方向に動かしていく
- (多重)集合に値を追加していった、毎回大きい方からM個の和がわかるようにしたい
- これは優先度付きキュー (priority_queue) 等を使ってよしなに実装するとできて、全体 $O(N^2 \log N)$
- ここまでは流石に解けてほしい

ちなみに

- 一応dpをすると $O(N^2)$ でできる
- ここからのdp高速化による改善は多分うまくいかないor
見通しが悪いので深入りしません

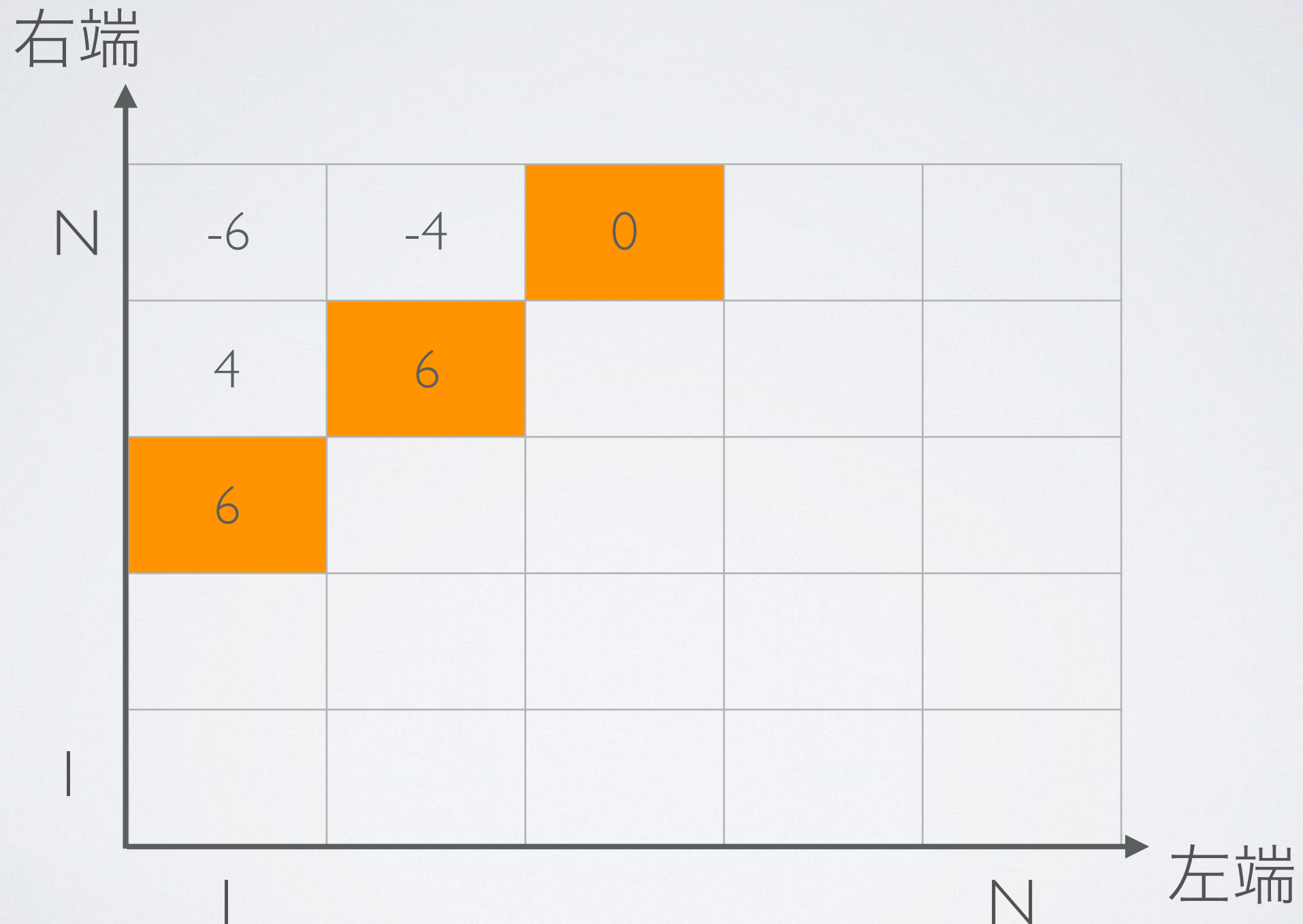
小課題3: $N \leq 200,000$ (100点)

- ここからが本番
- $O(N^2)$ 個ある区間を全部見ようと思うと大変なので、これを $O(N)$ 個ぐらいに減らしたい

小課題3: $N \leq 200,000$ (100点)

- ここからが本番
- $O(N^2)$ 個ある区間を全部見ようと思うと大変なので、これを $O(N)$ 個ぐらいに減らしたい
- 具体的には、各左端に対して、最適値を取る右端の位置のみを調べられると望ましい

觀察 - 入力例 I



観察 - 入力例2

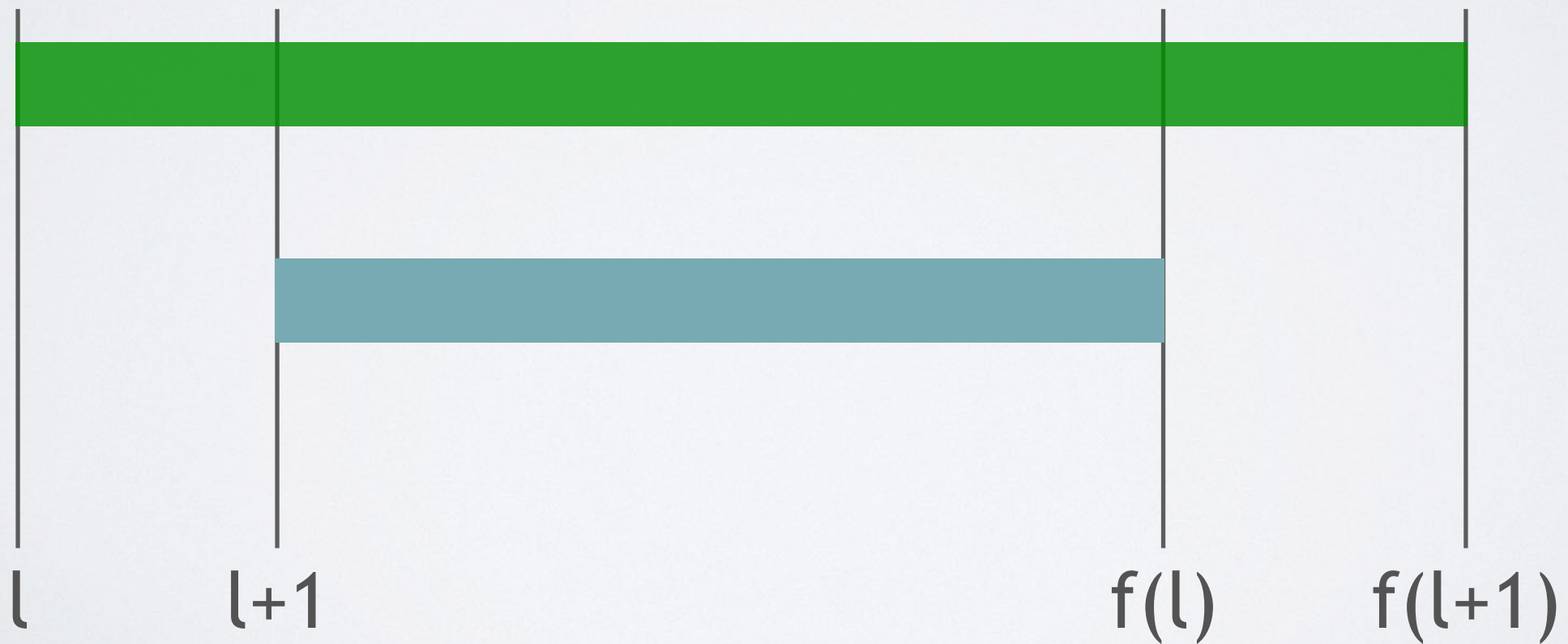


觀察 - 自作入力



観察

- ↓が最適と仮定すると...



観察

- ↓が最適と仮定すると...



観察

- ↓が最適と仮定すると...

■ に ■ を加えるとM個の最大和はx以上増える



観察

- ↓が最適と仮定すると...

■ に ■ を加えるとM個の最大和は x 以上増える

⇒ ■ に ■ を加えても最大和は x 以上増える (■ $\subset$ ■ より)



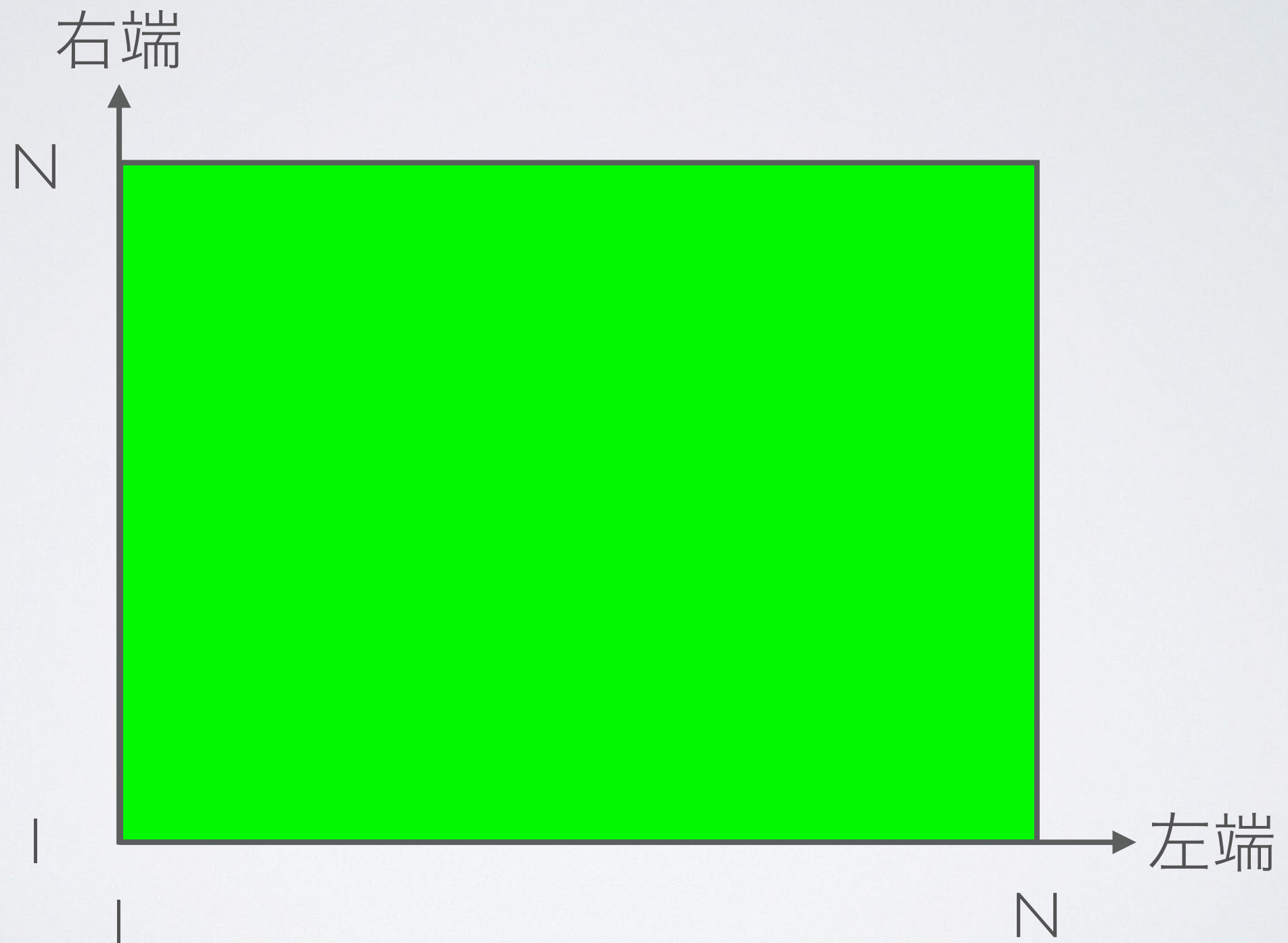
観察

- i を左端としたときの最適な選び方での右端の位置を $f(i)$ とすると、 **$f(i)$ は広義単調増加**と考えてよい

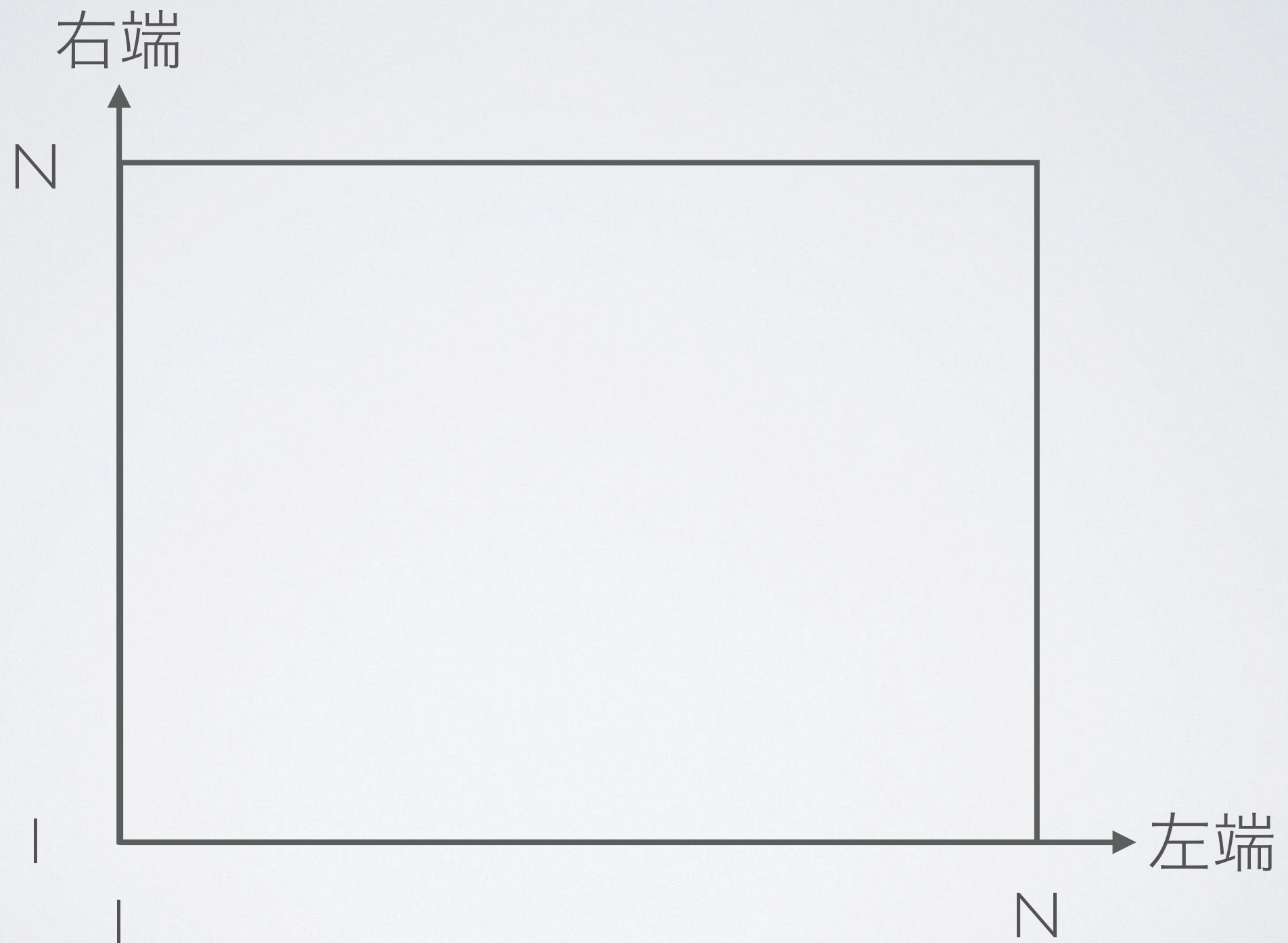
... ?

分割統治

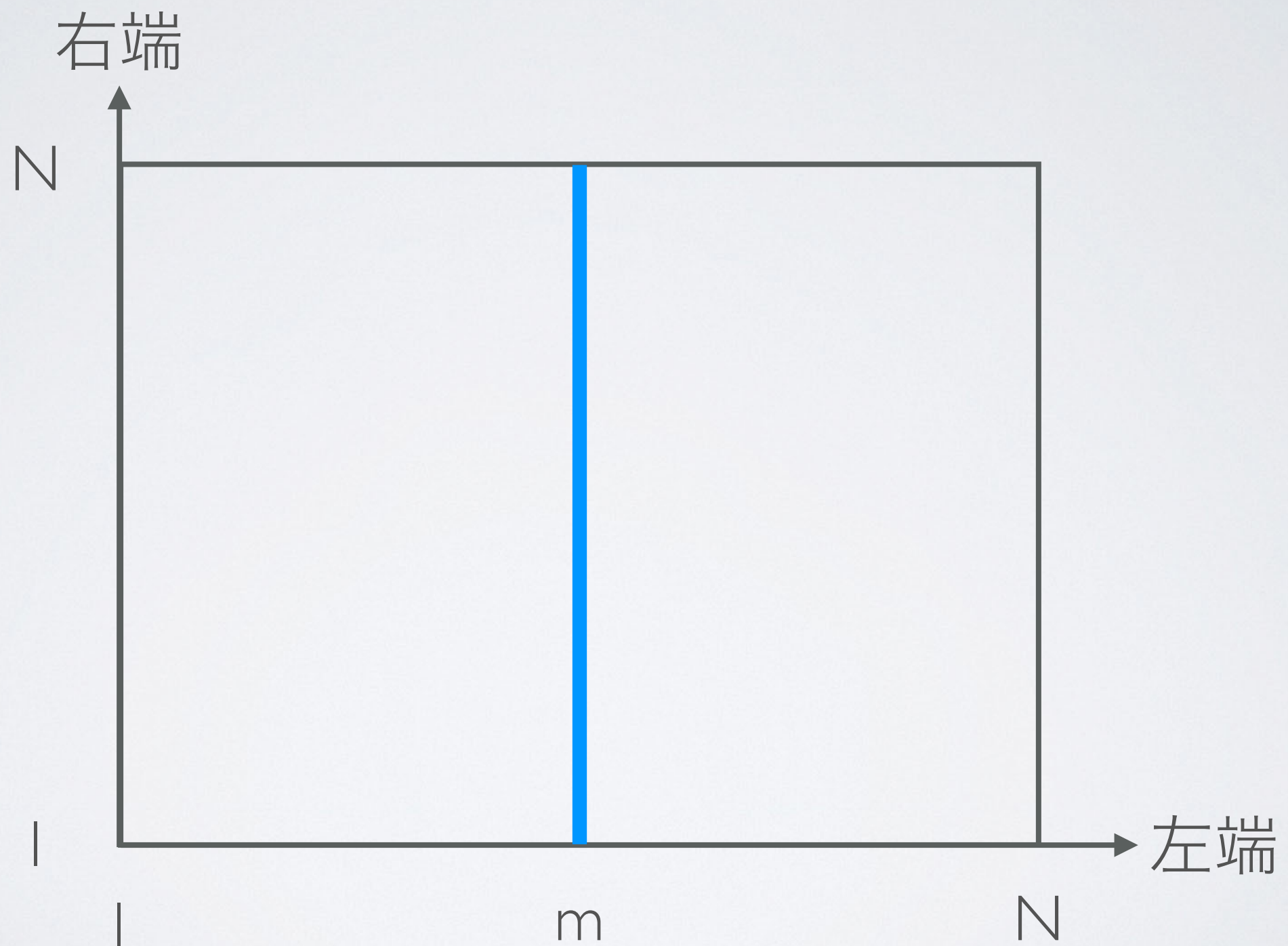
満点解法



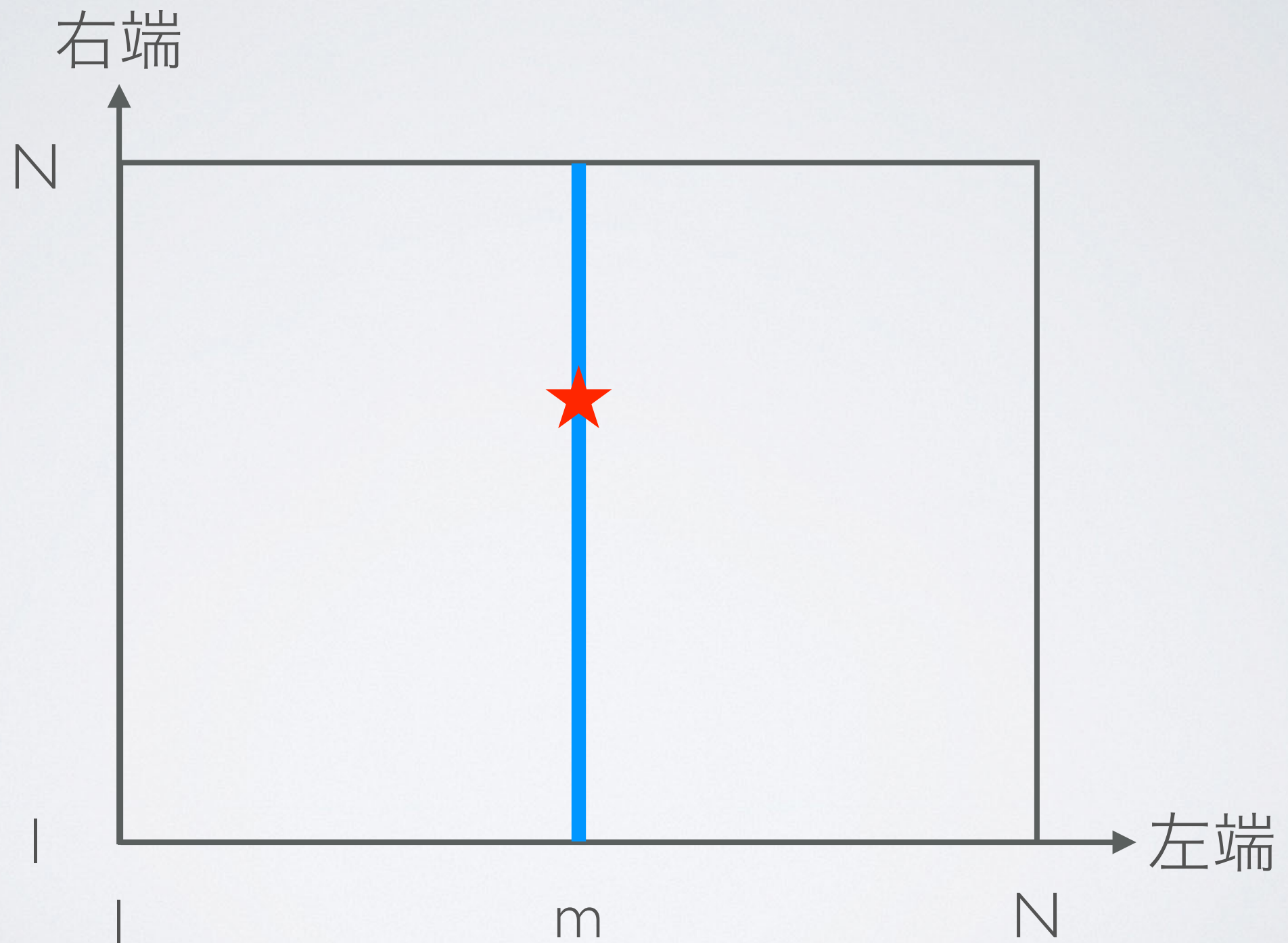
満点解法



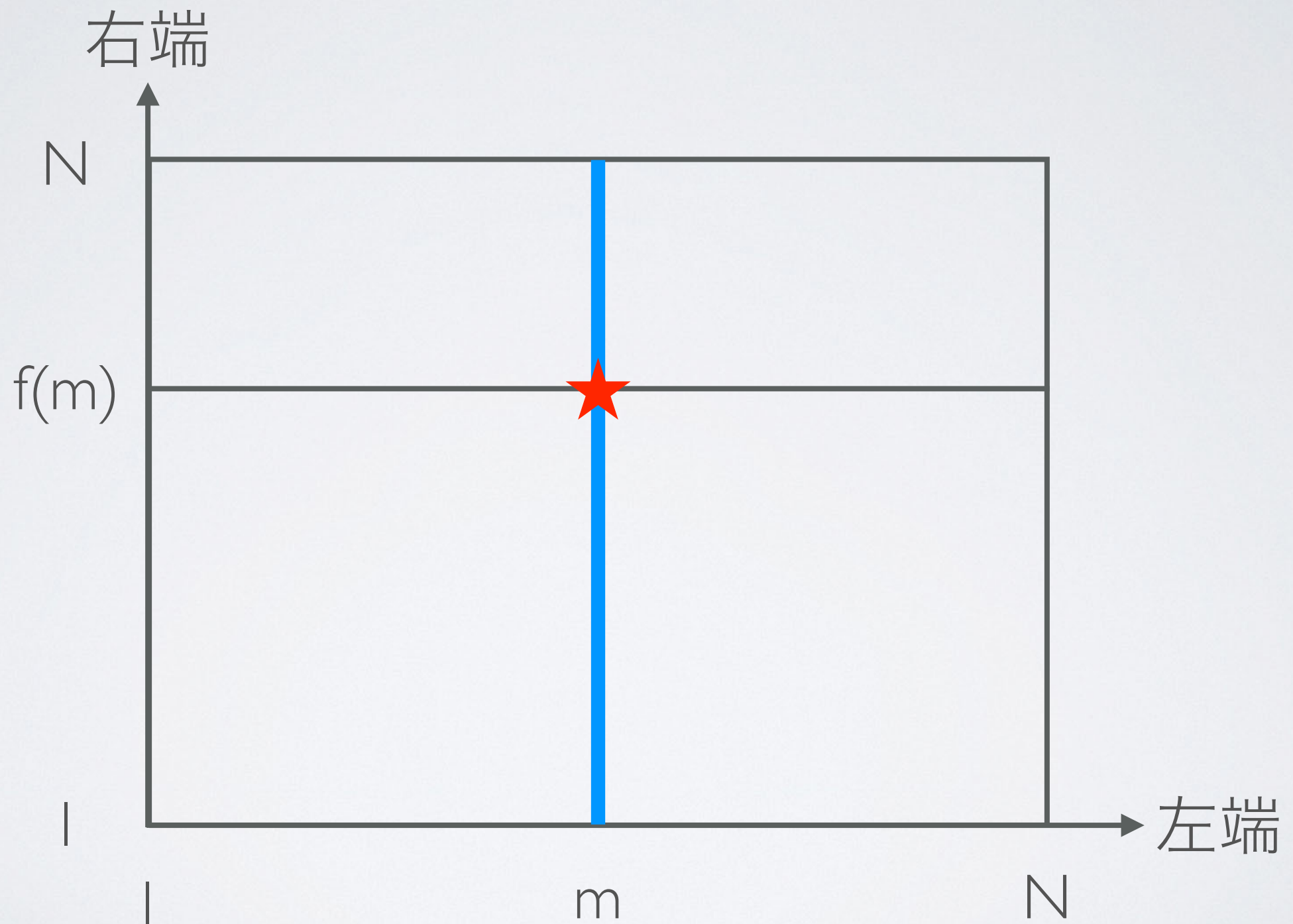
満点解法



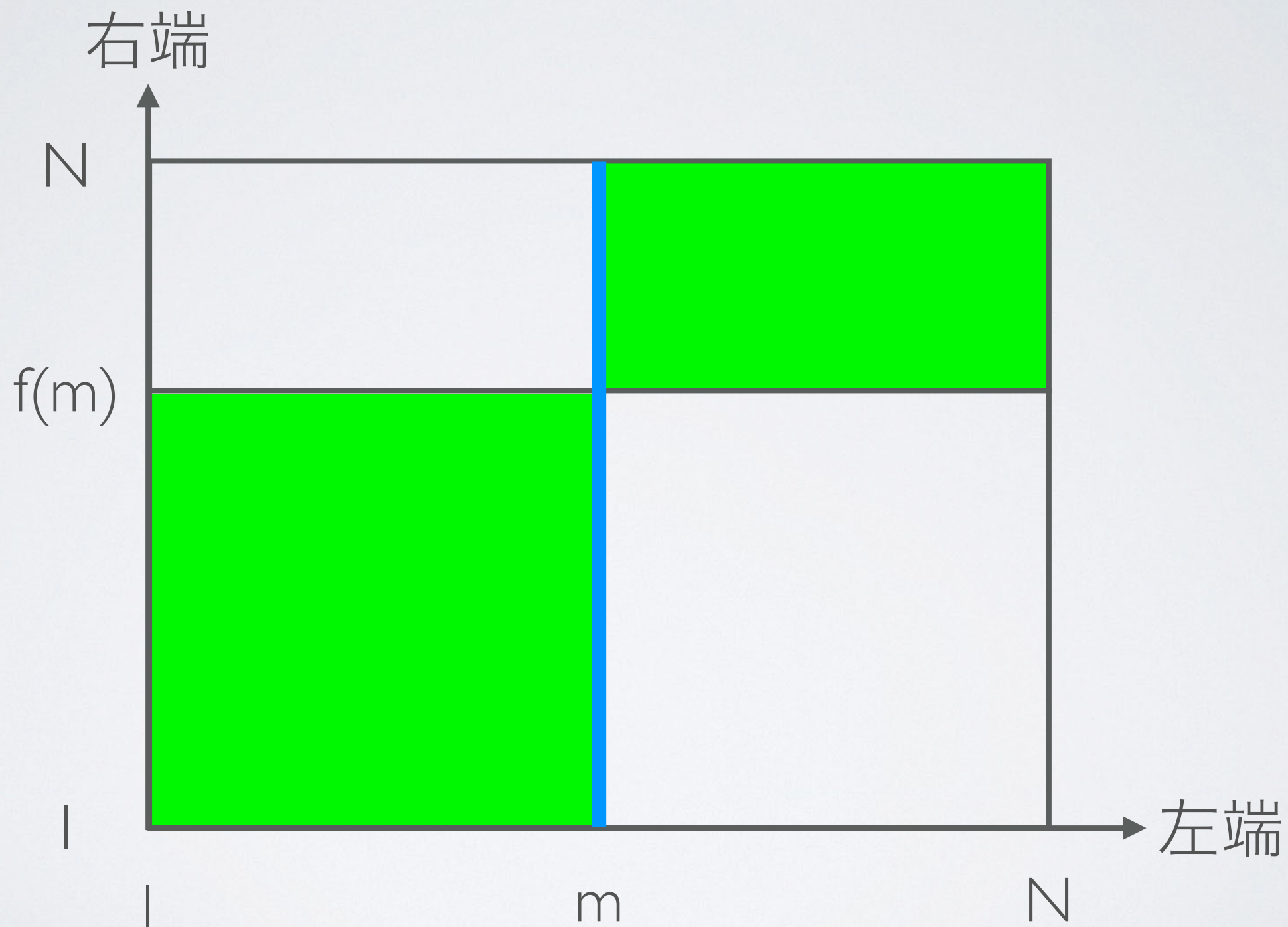
満点解法



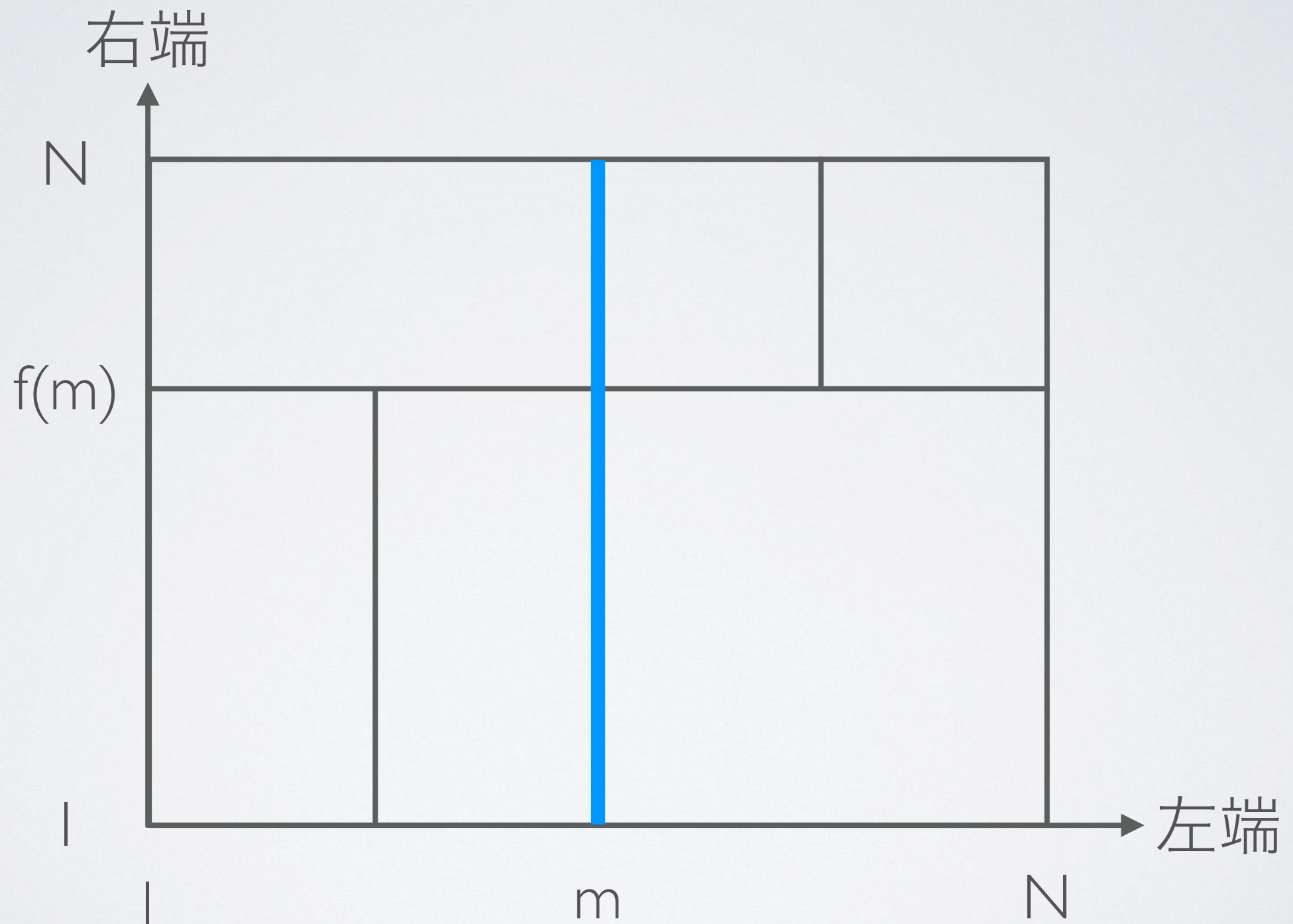
満点解法



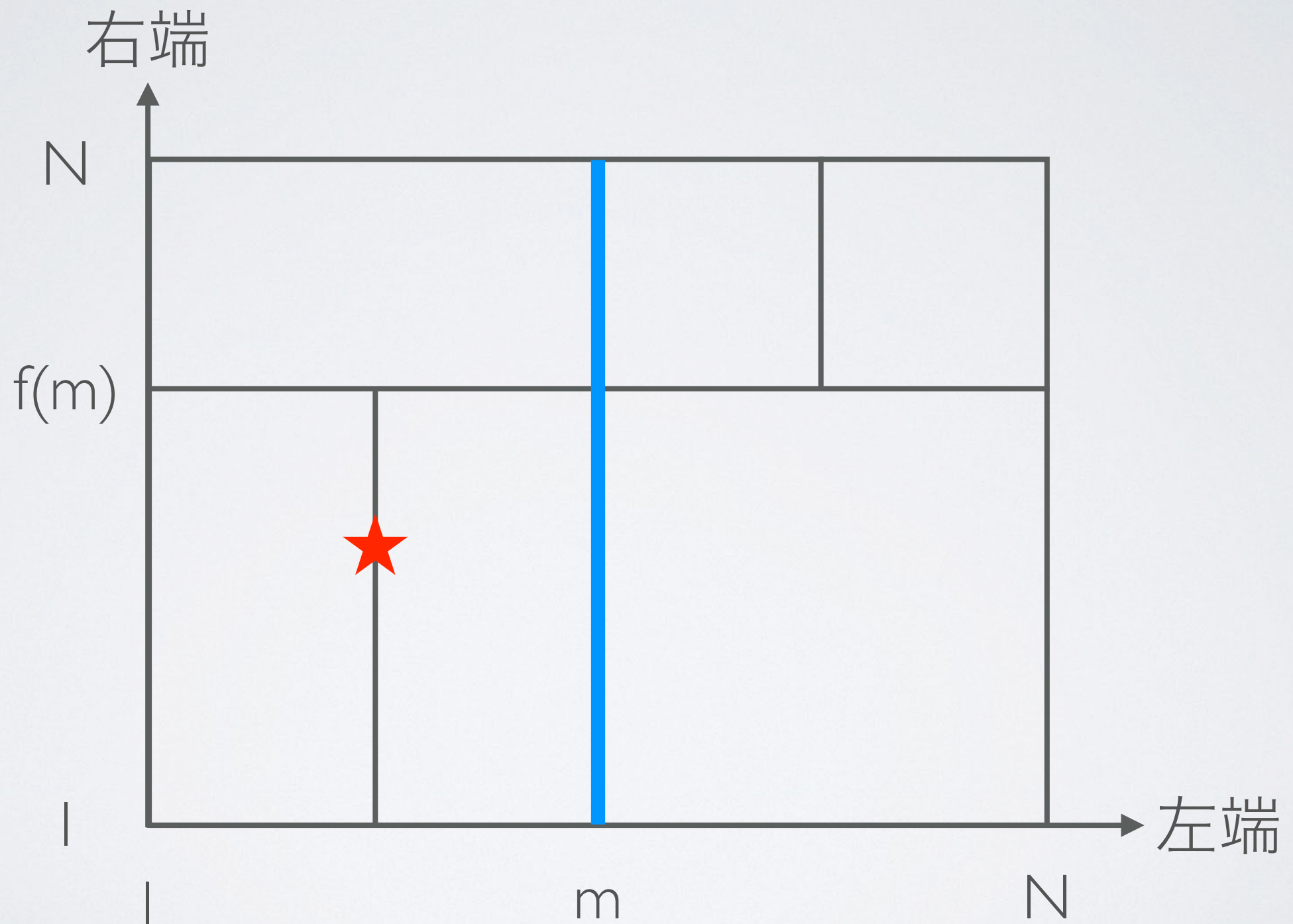
満点解法



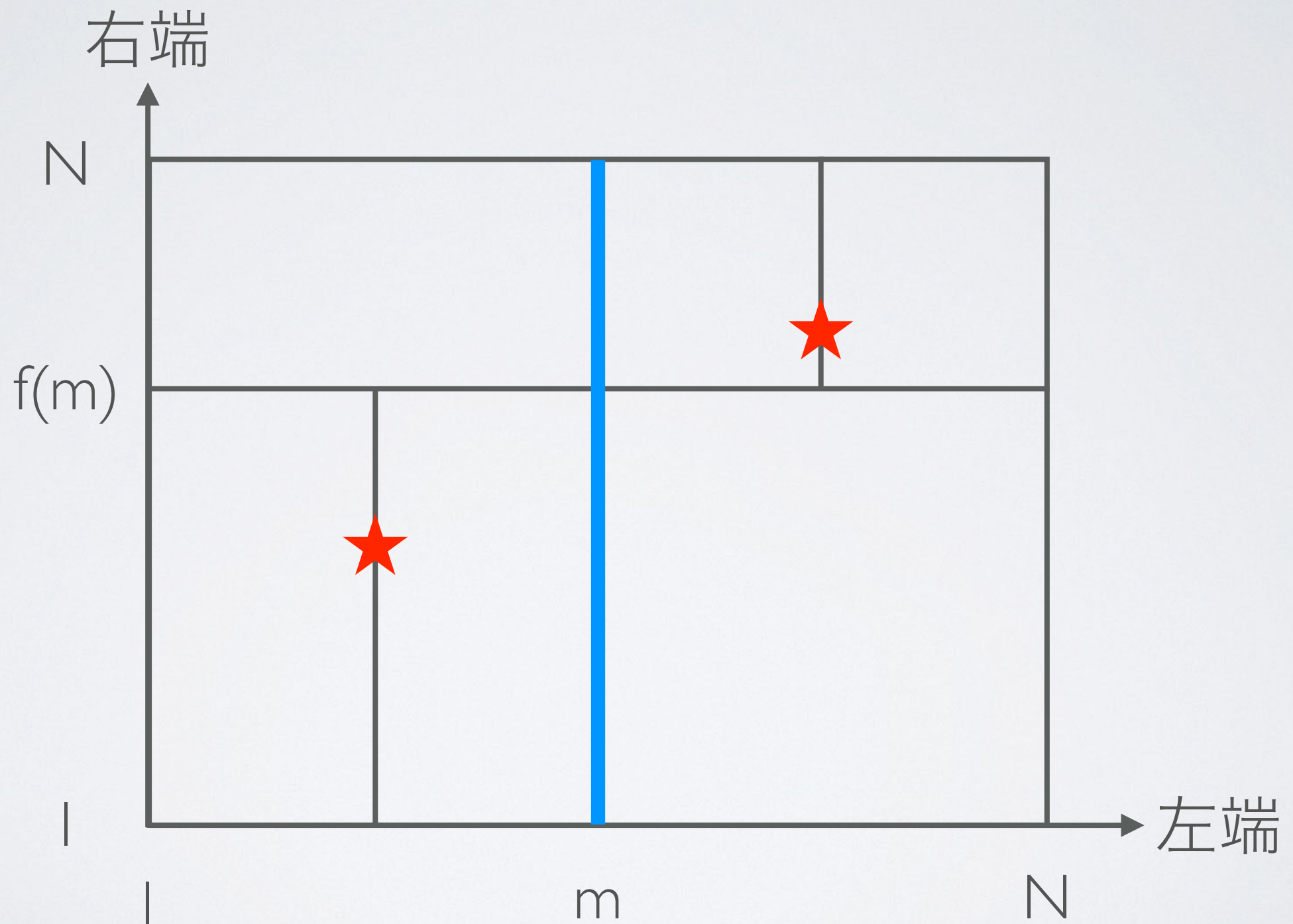
満点解法



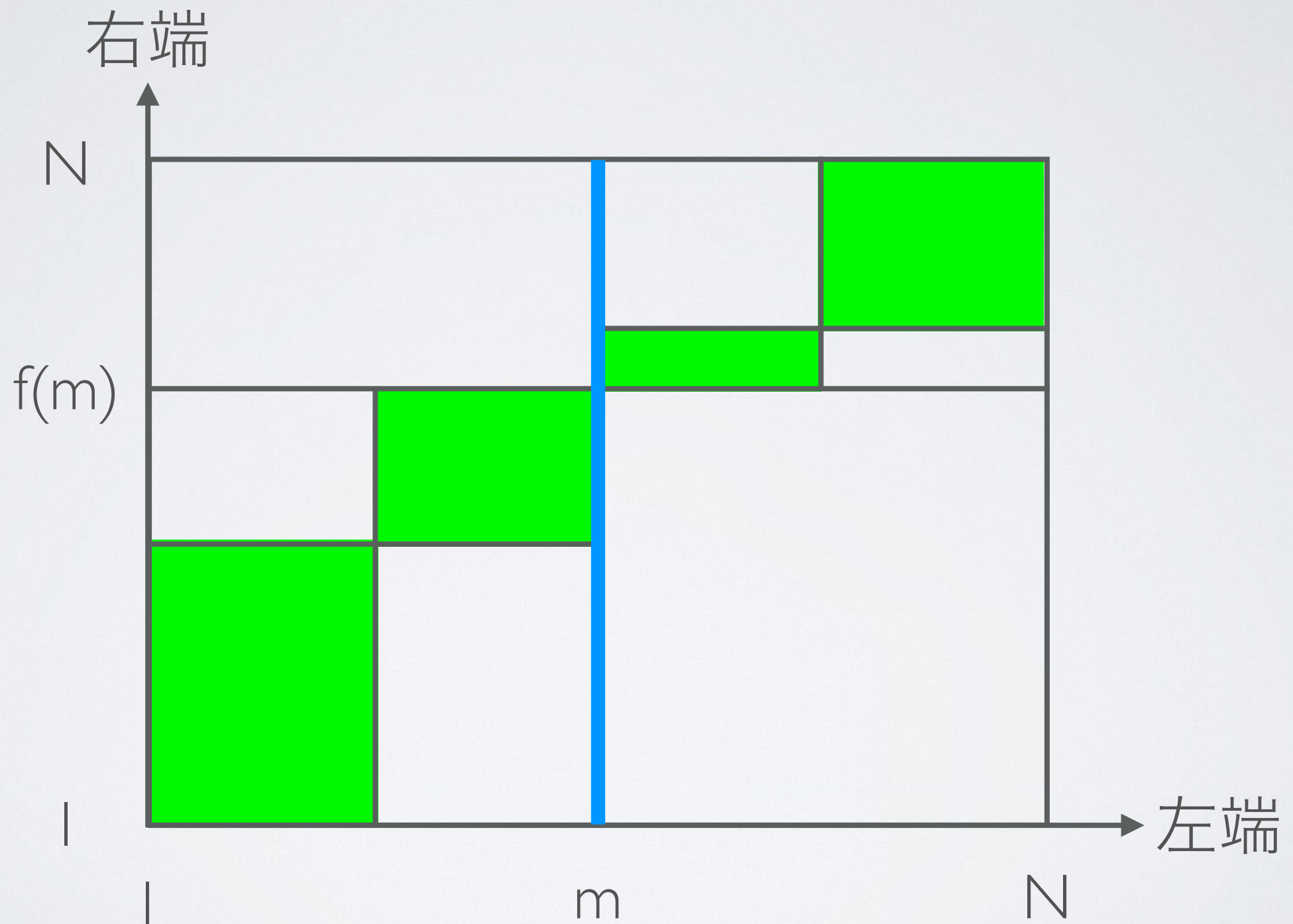
満点解法



満点解法



満点解法



満点解法

- m を左端として固定して、右端を全部試すことで最適な位置を見つける
- →左右の領域それぞれについて再帰的に同じ問題を解く

満点解法

- m を中央で取るようにすると、探索空間の面積は毎回 $1/2$ ずつに減っていくので、
- 左端と右端が決まっているときの答えが $O(X)$ で求まるとき、この問題は全体 $O(NX \log N)$ で解ける

満点解法

- m を中央で取るようにすると、探索空間の面積は毎回 $1/2$ ずつに減っていくので、
- 左端と右端が決まっているときの答えが $O(X)$ で求まるとき、この問題は全体 $O(NX \log N)$ で解ける
- monotone minimaというらしい

類題

- IOI 2014 Holiday
- 代表を目指す皆さんならもちろん2011以降のIOIは埋めていますよね...？

満点解法

- $[l, r]$ 内の値の大きい方から**M**個の総和は？

満点解法

- $[l, r]$ 内の値の大きい方から**M**個の総和は？
- → 完全にオンラインクエリとして処理するのは無理そう

満点解法

- **[l, r]内の値の大きい方からM個の総和は？**
- → 完全にオンラインクエリとして処理するのは無理そう
- WeavletMatrix...? vectorを載せたセグ木...? 二分探索...?

満点解法

- **[l, r]内の値の大きい方からM個の総和は？**
- → 完全にオンラインクエリとして処理するのは無理そう
- WaveletMatrix...? vectorを載せたセグ木...? 二分探索...?
 - ➡ 全体が間に合うためには各クエリ $O(\log N)$ 近くで答えられないとい
けないが、見るからに不可能

満点解法

- **$[l, r]$ 内の値の大きい方からM個の総和は？**
- → 完全にオンラインクエリとして処理するのは無理そう
- WaveletMatrix...? vectorを載せたセグ木...? 二分探索...?
 - ➡ 全体が間に合うためには各クエリ $O(\log N)$ 近くで答えられないとい
けないが、見るからに不可能
- 分割統治で必要になる値の $[l, r]$ の順序をうまくこと利用して効率よく
計算したい

満点解法

- 今使える道具は？
- 小課題2のアプローチを思い出す

満点解法

- 今使える道具は？
- 小課題2のアプローチを思い出す
 - ➡ 区間 $[i, i]$ からスタートして右端を伸ばしていくことなら
 $O(\log N)$ でできる

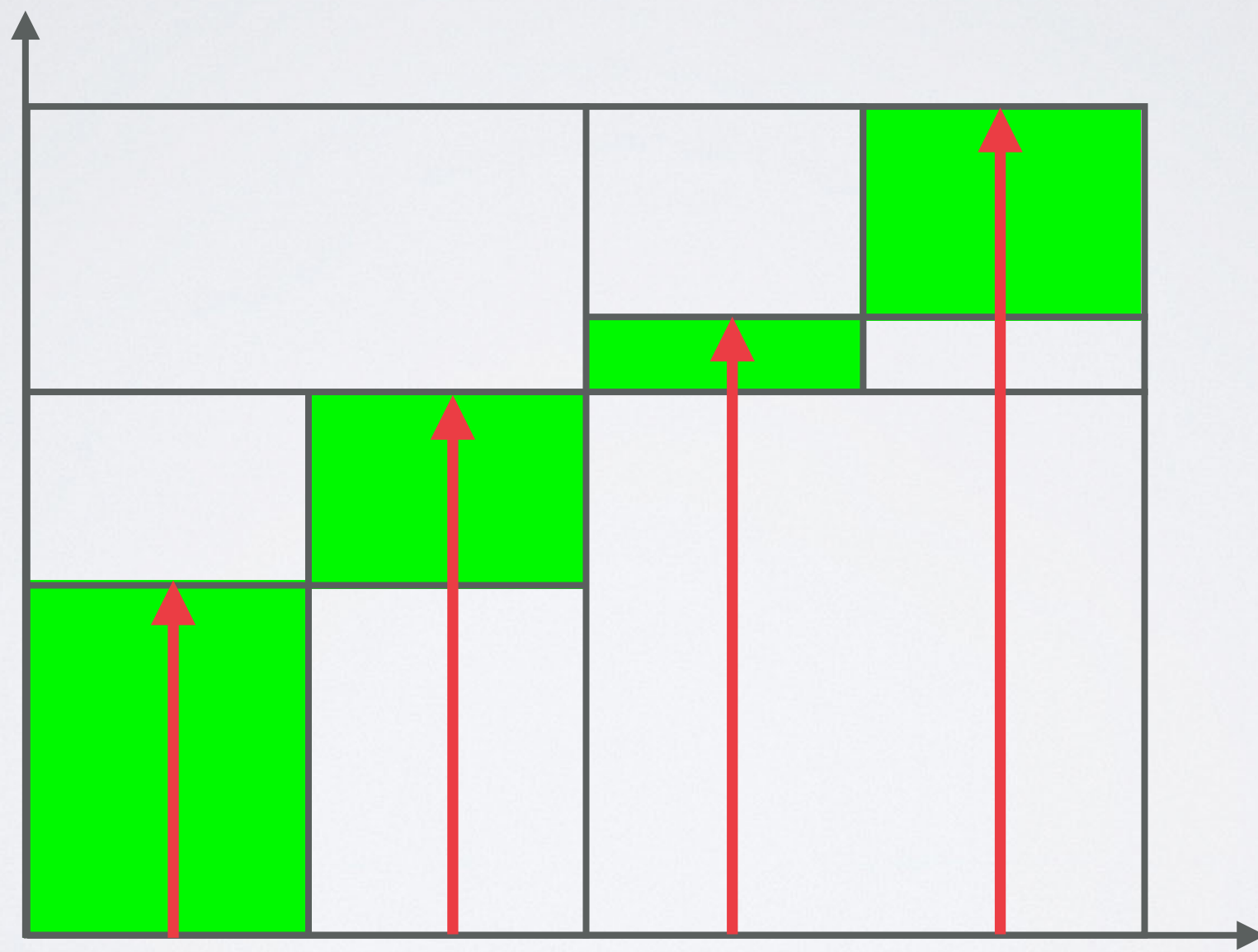
満点解法

- 今使える道具は？
- 小課題2のアプローチを思い出す
 - ➡ 区間 $[i, i]$ からスタートして右端を伸ばしていくことなら $O(\log N)$ でできる
 - ➡ 左端を縮めることも...？

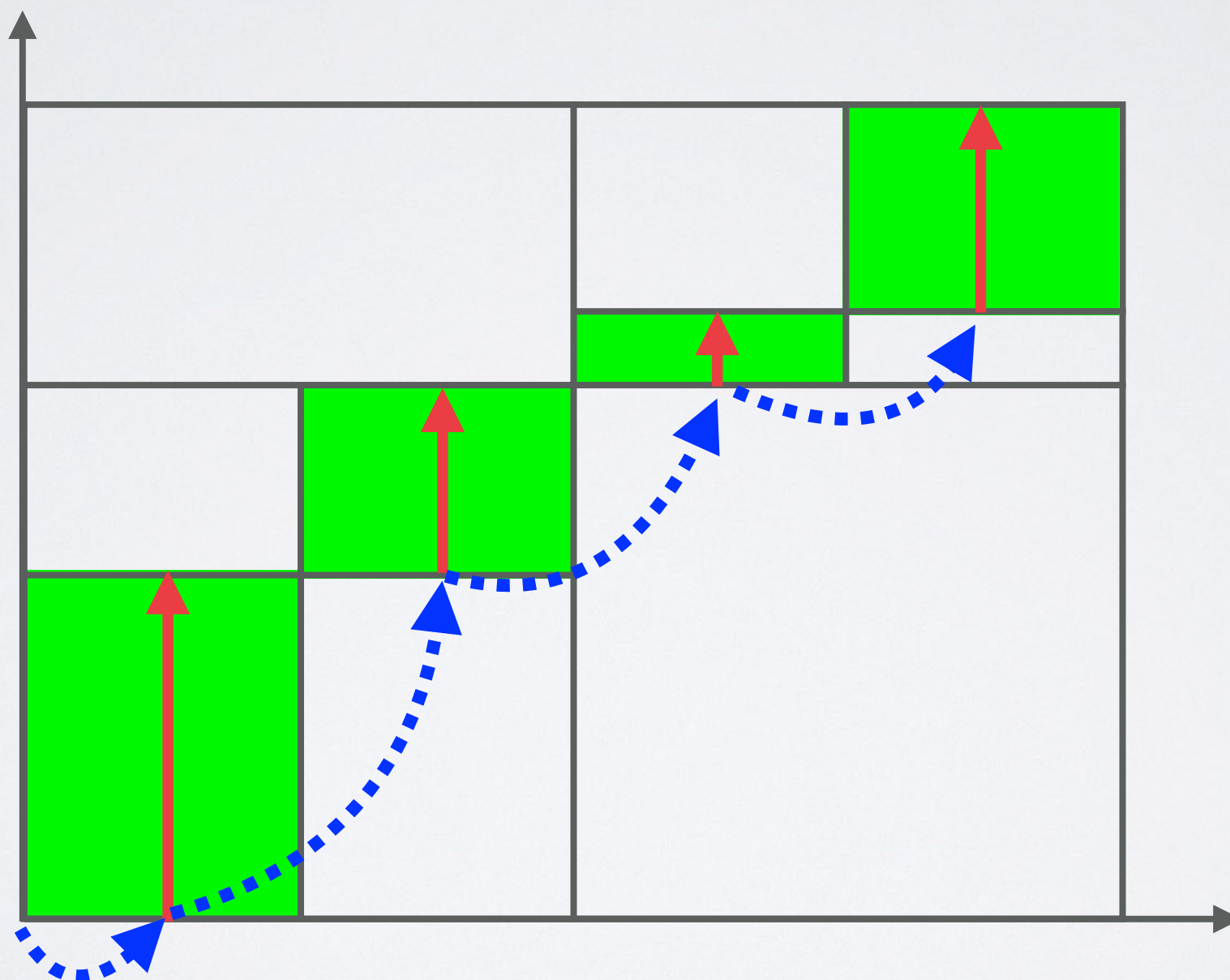
満点解法

- 今使える道具は？
- 小課題2のアプローチを思い出す
 - ➡ 区間 $[i, i]$ からスタートして右端を伸ばしていくことなら $O(\log N)$ ができる
 - ➡ 左端を縮めることも...？ **実はできる（後述）**

↑の長さのオーダーを掛けてしまうとダメ



↑ で右端を伸ばして ↑ で左端を縮める
長さの合計は $O(N)$



満点解法

- 区間[0, 0]からスタートして、右端を伸ばしたり左端を縮めたりしながら、区間内部の最大M個の和を管理しつつ、分割統治で必要な値を計算していく
- 一通り計算が終わったら、計算結果をもとに各探索領域をさらに半分に分割して同じ作業を繰り返す

満点解法

- 毎回横幅が $1/2$ ずつに減っていくので、イテレーションの回数は $\log N$ 回
- 毎回のイテレーションで見なければいけない区間たちが左端,右端ともに単調増加になっていることが嬉しい性質
- parallel binary searchにかなり近いbfsをしたが、再帰でも同じ感じでできる

実装

満点解法

- 今使える道具は？
- 小課題2のアプローチを思い出す
 - ➡ 区間 $[i, i]$ からスタートして右端を伸ばしていくことなら $O(\log N)$ ができる
 - ➡ 左端を縮めることも...？ **実はできる（後述）**

整理すると...

- 値 C_i を（多重）集合に加える
- 値 C_i を（多重）集合から削除する
- 追加されて残っている値のうち最大 M 個の総和を求める
- の3種類のクエリを処理できるデータ構造があればよい

平衡二分木解

- 値を追加/削除するたびに現在のM番目付近の値が特定できればよい
- `std::set`ではk番目の値を求める操作がサポートされていないので、g++拡張の`tree`とかいうやつを使う
(参考: <http://hogloid.hatenablog.com/entry/2014/09/23/132440>)
- または、平衡二分木を自前で書く

平衡二分木解

- g++拡張は覚えなないといけないおまじないの量が多いので、
検索・ライブラリ持ち込み不可のJOIでは少し厳しい

```
#include <ext/pb_ds/assoc_container.hpp>  
#include <ext/pb_ds/tree_policy.hpp>  
#include <ext/pb_ds/tag_and_trait.hpp>
```

```
using namespace __gnu_pbds;
```

```
tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update> ...
```

```
.
```

平衡二分木解

- g++拡張は覚えないといけないおまじないの量が多いので、検索・ライブラリ持ち込み不可のJOIでは少し厳しい

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
#include <ext/pb_ds/tag_and_trait.hpp>

using namespace __gnu_pbds;

tree<T, null_type, less<T>, rb_tree_tag, tree_order_statistics_node_update> ...
.
```

- どのみち定数倍が怪しいのでおすすめしません

セグメント木解

- 一点更新・区間和の一般的なセグメント木を2本用いる
- V_i の順に並べた長さ N の数列を考えて、2本のセグメント木の各節点には、それぞれ区間内の生きている個数と V の総和を持たせる
- あとはセグメント木上の二分探索で右から個数の和が M になる位置を求めて、そこから末尾までの V の総和を普通に取ればよい
- 各クエリ $O(\log N)$ で処理できる

まとめ

- $\log N$ 回のイテレーションで、毎回セグ木初期化→走査しつつ探索する区間を分割統治していけばOK
- セグ木への追加/削除は1回のイテレーションにつき各位置でちょうど1回ずつ行われるのでこれで $O(N \log^2 N)$

まとめ

- $\log N$ 回のイテレーションで、毎回セグ木初期化→走査しつつ探索する区間を分割統治していけばOK
- セグ木への追加/削除は1回のイテレーションにつき各位置でちょうど1回ずつ行われるのでこれで $O(N \log^2 N)$
- 実装はコードを書くことができます

得点分布

得点分布



得点分布



得点分布



合併 解説

笠浦一海

March 23, 2019

問題概要

- ▶ 木が与えられる．それぞれの頂点がいずれかの州に属する．
- ▶ 同じ州に属する頂点が同じ集合に属するように木をふたつの連結部分木に分けることができるとき分裂可能
- ▶ ふたつの集合をひとつにマージすることを繰り返して木を分裂可能でなくしたい
- ▶ マージの最小回数を求めよ

分裂可能性の判定

- ▶ 「同じ州に属する頂点が同じ集合に属するように木をふたつの連結部分木に分けることができる」かを判定する方法を考える

分裂可能性の判定

- ▶ 「同じ州に属する頂点が同じ集合に属するように木をふたつの連結部分木に分けることができる」かを判定する方法を考える
- ▶ 州の数が k のとき頂点の分け方: 2^k 通り

分裂可能性の判定

- ▶ 「同じ州に属する頂点が同じ集合に属するように木をふたつの連結部分木に分けることができる」かを判定する方法を考える
- ▶ 州の数が k のとき頂点の分け方: 2^k 通り
- ▶ 連結判定: $O(N)$

サブタスク 1

- ▶ 州のマージの仕方を全列挙

サブタスク 1

- ▶ 州のマージの仕方を全列挙
- ▶ 簡単な考察で $K!$ を超えないことがわかる

サブタスク 1

- ▶ 州のマージの仕方を全列挙
- ▶ 簡単な考察で $K!$ を超えないことがわかる
- ▶ 全体で $O(K!2^K N)$

サブタスク 1

- ▶ 州のマージの仕方を全列挙
- ▶ 簡単な考察で $K!$ を超えないことがわかる
- ▶ 全体で $O(K!2^K N)$
- ▶ $N \leq 100, K \leq 7$ のサブタスク 1 なら解ける

分裂可能性の判定（もっと賢く）

- ▶ 木をふたつの連結部分木に分ける→ひとつの辺で切る

分裂可能性の判定（もっと賢く）

- ▶ 木をふたつの連結部分木に分ける→ひとつの辺で切る
- ▶ ひとつの辺で切ってその両方の側の頂点を含む州がなければ良い

分裂可能性の判定（もっと賢く）

- ▶ 木をふたつの連結部分木に分ける→ひとつの辺で切る
- ▶ ひとつの辺で切ってその両方の側の頂点を含む州がなければ良い
- ▶ 判定が $O(N^2)$ でできた

分裂可能性の判定（もっと賢く）

- ▶ 木をふたつの連結部分木に分ける→ひとつの辺で切る
- ▶ ひとつの辺で切ってその両方の側の頂点を含む州がなければ良い
- ▶ 判定が $O(N^2)$ でできた
- ▶ 全体で $O(K!N^2)$ → サブタスク 1 のみ

考察

- ▶ 「その辺で切ったときにその両方の側の頂点を含む州がない辺」を生きた辺，それ以外を死んだ辺と呼ぶことにする

考察

- ▶ 「その辺で切ったときにその両方の側の頂点を含む州がない辺」を生きた辺, それ以外を死んだ辺と呼ぶことにする
- ▶ 生きた辺を全部潰したい
- ▶ 両方の側から州を一つずつ選んでマージすれば潰せる

別の問題に還元

- ▶ 死んだ辺をすべて縮約してできるグラフを考える
- ▶ 「ふたつの頂点を選び，それらの間にある辺をすべて縮約する」を繰り返して一点に潰す

別の問題に還元

- ▶ 死んだ辺をすべて縮約してできるグラフを考える
- ▶ 「ふたつの頂点を選び，それらの間にある辺をすべて縮約する」を繰り返して一点に潰す
- ▶ ↑の操作を最低で何回行えば良い…？

答え

- ▶ 答え: $(\text{葉の数} + 1)/2$
- ▶ ここで葉とは次数 1 の頂点のこと

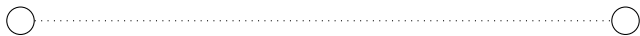
答え

- ▶ 答え: $(\text{葉の数} + 1)/2$
- ▶ ここで葉とは次数 1 の頂点のこと
- ▶ 証明: 一回の操作で葉は高々 2 個しか消えない → 上限

答え

- ▶ 答え: $(\text{葉の数} + 1)/2$
- ▶ ここで葉とは次数 1 の頂点のこと
- ▶ 証明: 一回の操作で葉は高々 2 個しか消えない→上限
- ▶ 逆に葉の数が 3 つでなければ, 一回の操作で葉の数を 2 つ減らせることが証明できる

証明



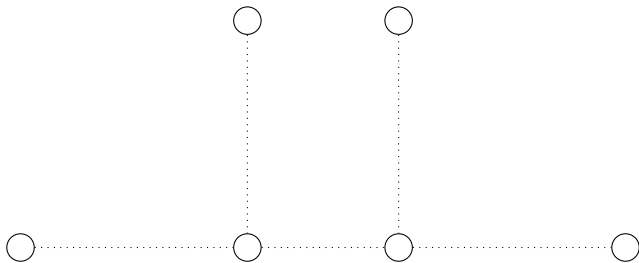
適当に葉を 2 つ選ぶ

証明



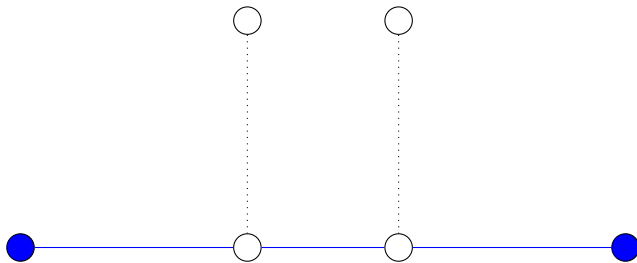
他に葉がないならこれを選べば良い

証明



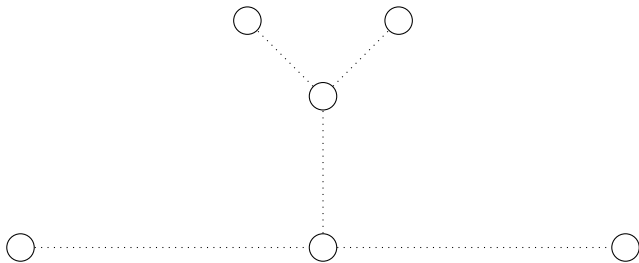
他に葉が 2 つ以上あるとき: パターン 1

証明



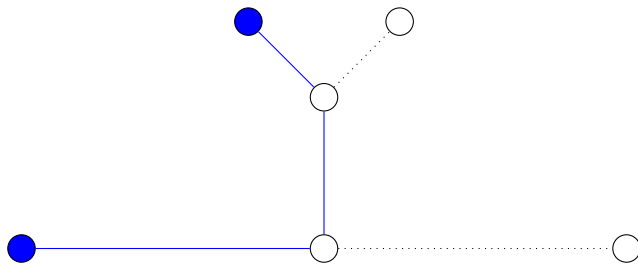
もとの 2 つを選べば良い

証明



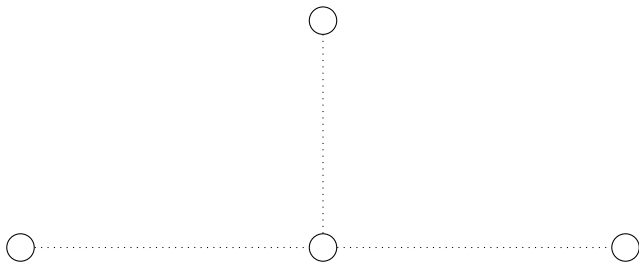
他に葉が 2 つ以上あるとき: パターン 2

証明



この2つを選べば良い

証明



葉の数が 3 つの場合はどう選んでも 1 つだけ減る

サブタスク 2

- ▶ 各辺について生きてるか死んでるか $O(N)$ で求める $O(N^2)$
- ▶ 死んだ辺を潰した木の次数 1 の頂点の数を数える $O(N)$
- ▶ 全体で $O(N^2)$: $N \leq 3000$ なら解ける

サブタスク 3

- ▶ K が小さい

サブタスク 3

- ▶ K が小さい
- ▶ 各州について、「それによって死ぬ辺 i.e. その両側にその州の頂点がある辺」を考える
- ▶ これはその州の頂点をすべて含む最小の部分木内の辺
- ▶ 各州について、これを $O(N)$ で求められればよい

サブタスク 3

- ▶ K が小さい
- ▶ 各州について、「それによって死ぬ辺 i.e. その両側にその州の頂点がある辺」を考える
- ▶ これはその州の頂点をすべて含む最小の部分木内の辺
- ▶ 各州について、これを $O(N)$ で求められればよい
- ▶ 適当に一頂点選んで DFS すればよい

もう少し考える

- ▶ 各州について、それによって死ぬ辺の集合は、その州に属する 2 頂点を結ぶパスすべての Union

もう少し考える

- ▶ 各州について、それによって死ぬ辺の集合は、その州に属する 2 頂点を結ぶパスすべての Union
- ▶ 3 頂点 v_1, v_2, v_3 について、 v_1 と v_3 の間のパスは v_1 と v_2 の間のパスと v_2 と v_3 の間のパスの Union に含まれる

もう少し考える

- ▶ 各州について、それによって死ぬ辺の集合は、その州に属する 2 頂点を結ぶパスすべての Union
- ▶ 3 頂点 v_1, v_2, v_3 について、 v_1 と v_3 の間のパスは v_1 と v_2 の間のパスと v_2 と v_3 の間のパスの Union に含まれる
- ▶ 州に含まれる頂点が $v_1, v_2, \dots, v_n$ なら、 v_1 と v_2 の間のパス、 v_2 と v_3 の間のパス、 $\dots$ v_{n-1} と v_n の間のパスの Union のみ考えれば良い

サブタスク 4

- ▶ パスの長さが小さい→パスに含まれる辺を列挙して良い

サブタスク 4

- ▶ パスの長さが小さい→パスに含まれる辺を列挙して良い
- ▶ 適当に根付き木にして考えてみる
- ▶ 2 頂点の間のパスを求めるには, それらの LCA (最近共通祖先, lowest common ancestor) まで木を上に登れば良い考えれば良い

満点解法

- ▶ パスの長さが長いかもしれない→パスに含まれる辺を列挙できない

満点解法

- ▶ パスの長さが長いかもしれない→パスに含まれる辺を列挙できない
- ▶ LCA 自体は $O(\log N)$ で求められる (Euler tour + Segment tree, Doubling など)
- ▶ パス上の辺すべてを死なせたい

満点解法

- ▶ パスの長さが長いかもしれない→パスに含まれる辺を列挙できない
- ▶ LCA 自体は $O(\log N)$ で求められる (Euler tour + Segment tree, Doubling など)
- ▶ パス上の辺すべてを死なせたい
- ▶ 累積和をつかう

満点解法

- ▶ パスの長さが長いかもしれない→パスに含まれる辺を列挙できない
- ▶ LCA 自体は $O(\log N)$ で求められる (Euler tour + Segment tree, Doubling など)
- ▶ パス上の辺すべてを死なせたい
- ▶ 累積和をつかう
- ▶ 2 頂点に $+1$, LCA の頂点に -2 を加算して、あとで下から累積和を取る
- ▶ 各辺について, それらを殺す州の数が定まる

別解

- ▶ 死ぬことがわかった時点で辺を縮約していく
- ▶ LCA を求めるとき，通った辺はすべて縮約されることに注意

別解

- ▶ 死ぬことがわかった時点で辺を縮約していく
- ▶ LCA を求めるとき，通った辺はすべて縮約されることに注意
- ▶ Union Find を使って，パスの辺をすべて縮約する
- ▶ 一度縮約した辺は次回以降スキップされるので， $N - 1$ 回しか操作を行わない

別解

- ▶ 死ぬことがわかった時点で辺を縮約していく
- ▶ LCA を求めるとき，通った辺はすべて縮約されることに注意
- ▶ Union Find を使って，パスの辺をすべて縮約する
- ▶ 一度縮約した辺は次回以降スキップされるので， $N - 1$ 回しか操作を行わない
- ▶ 全体で $O(N\alpha(N))$ の計算量で解ける (α は逆アッカーマン関数)

得点分布

Minerals 解説

tozangezan

問題概要

- N 種類の鉱物がそれぞれ2個ずつ、合計 $2N$ 個ある
- 以下の操作を1000000回以内行ってどれとどれが同じ鉱物かを特定せよ
- 操作: 装置に1個入れる、もしくは1個出す。その後、装置に入っている鉱物の種類数がわかる

制約

$N \leq 100$

$N \leq 15000$, 切片1~N は全部別の種類

$N \leq 15000$

$N \leq 38000$

$N \leq 39000$

$N \leq 40000$

$N \leq 41000$

$N \leq 42000$

$N \leq 43000$

小課題 1

- 何回かけてもいいのでペアを見つけられますか？
- ${}_{2N}C_2$ 個のペアについて以下を行う
- 1. あるペアを装置に入れる
- 2. 装置が 1 を返したら同じ種類
- 3. ペアを装置から出す
- $O(N^2)$ 回装置を使う必要がある
- 5点

制約

$N \leq 100$

$N \leq 15000$, 切片1~N は全部別の種類

$N \leq 15000$

$N \leq 38000$

$N \leq 39000$

$N \leq 40000$

$N \leq 41000$

$N \leq 42000$

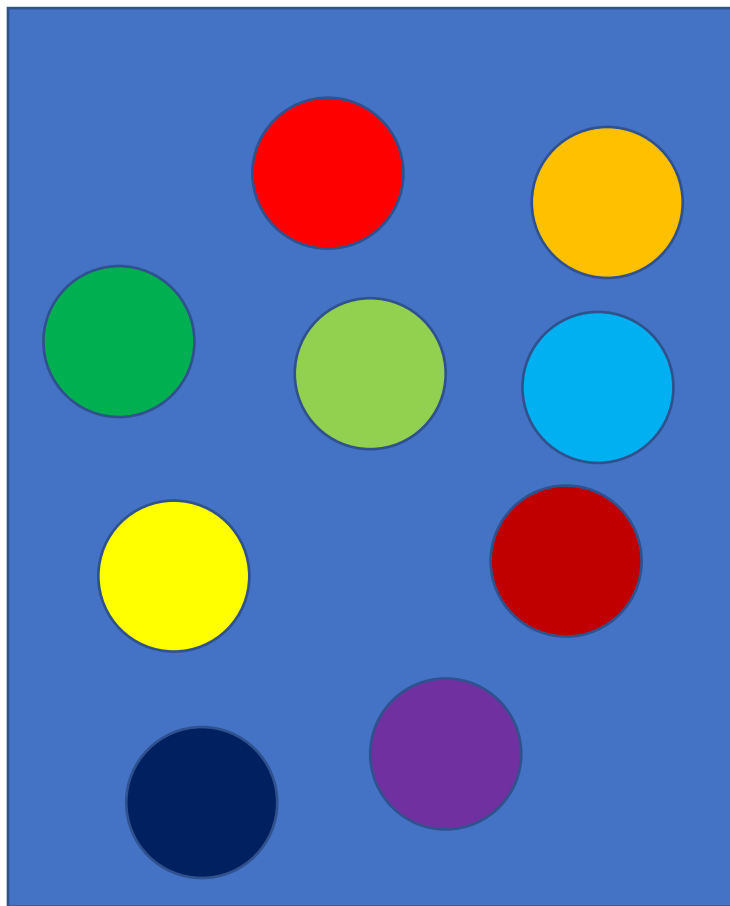
$N \leq 43000$

小課題 2

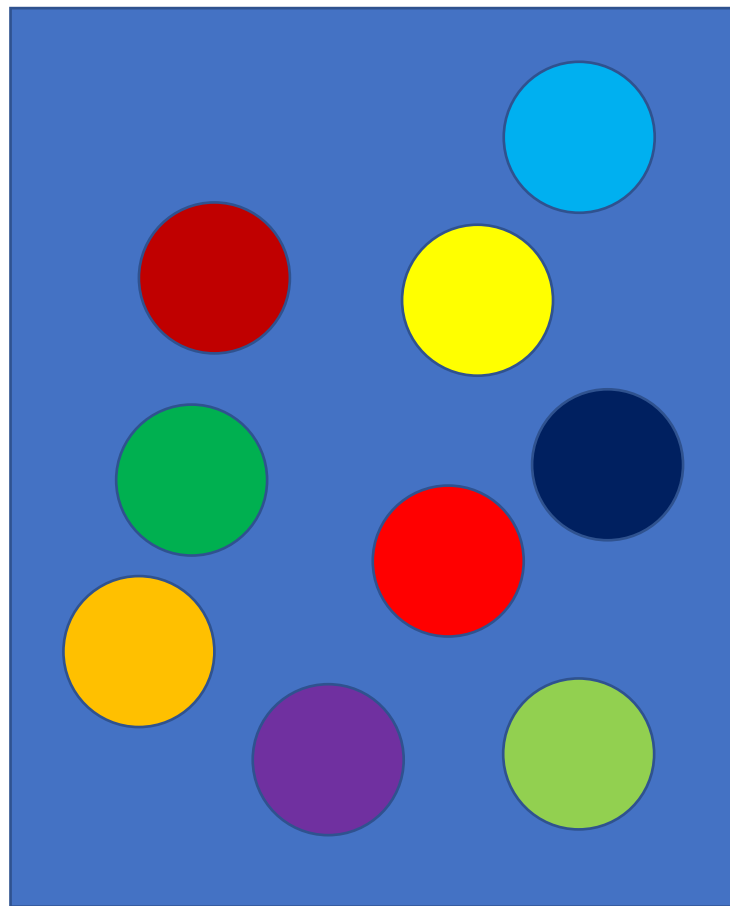
- $N \leq 15000$
- $O(N \log N)$ なら何でも通るかな... ?
- 切片1～N は全部別の種類
- ということだろうか？

こういうことですね

1 ~ N



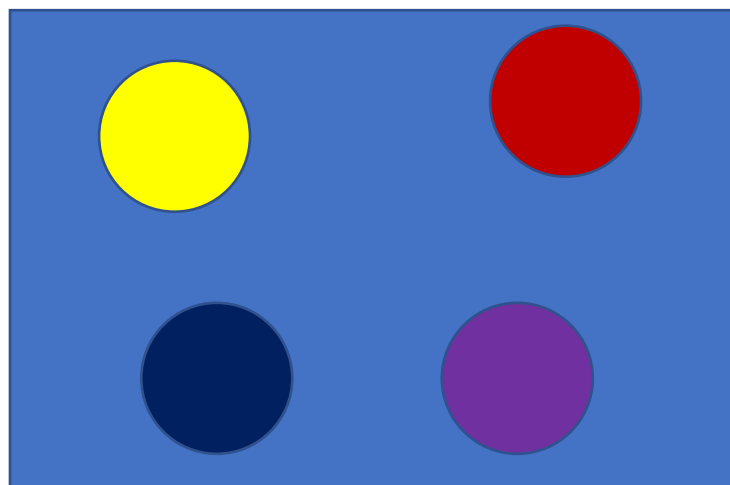
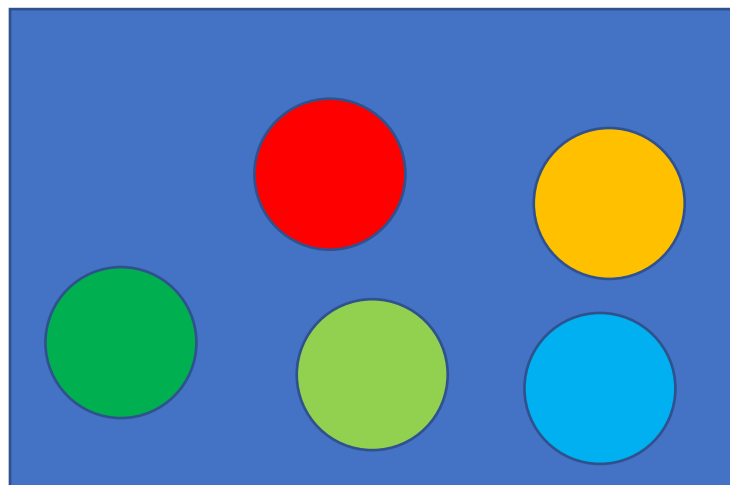
N+1 ~ 2N



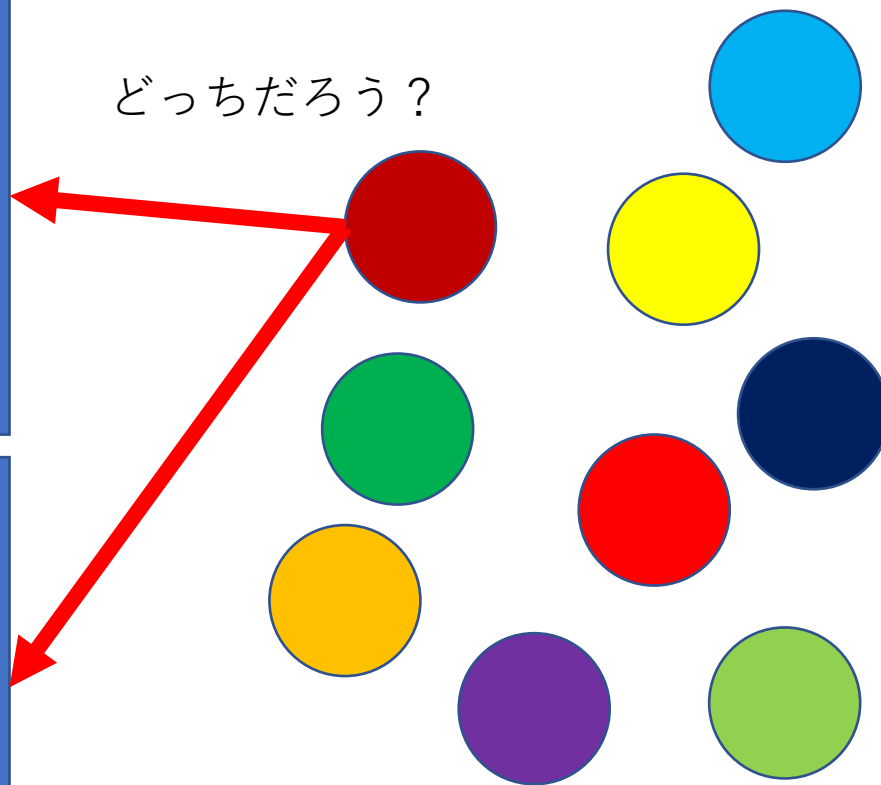
こういうことですね

1 ~ N

N+1 ~ 2N

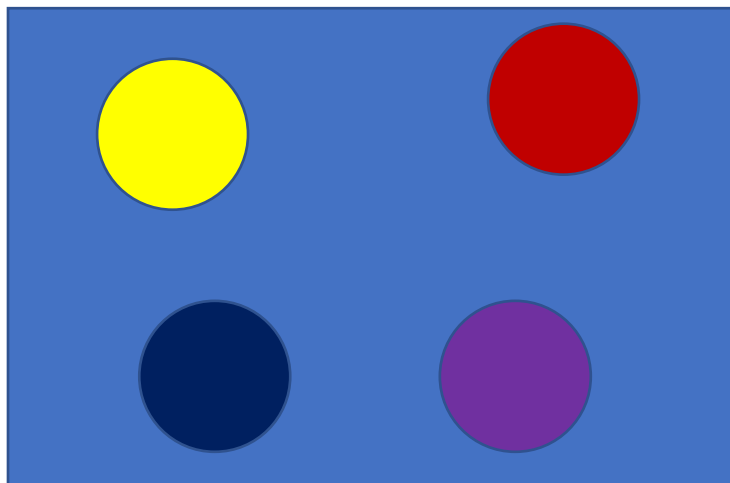
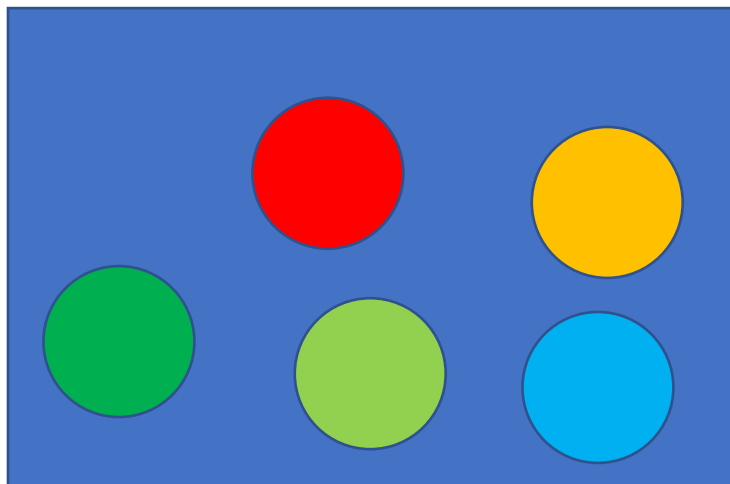


どっちだろう？



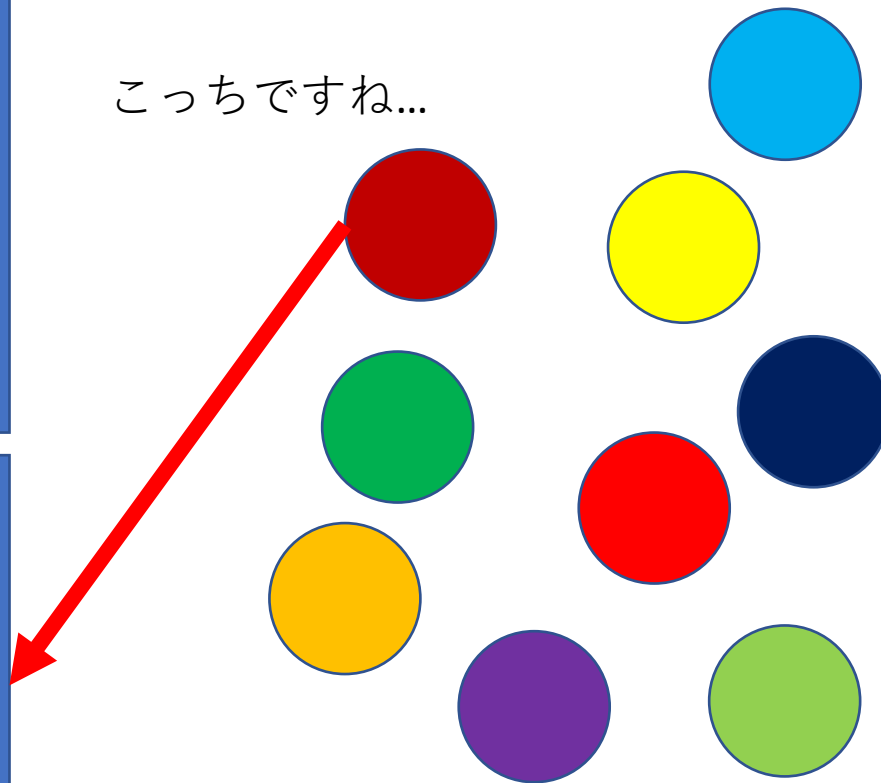
こういうことですね

1 ~ N



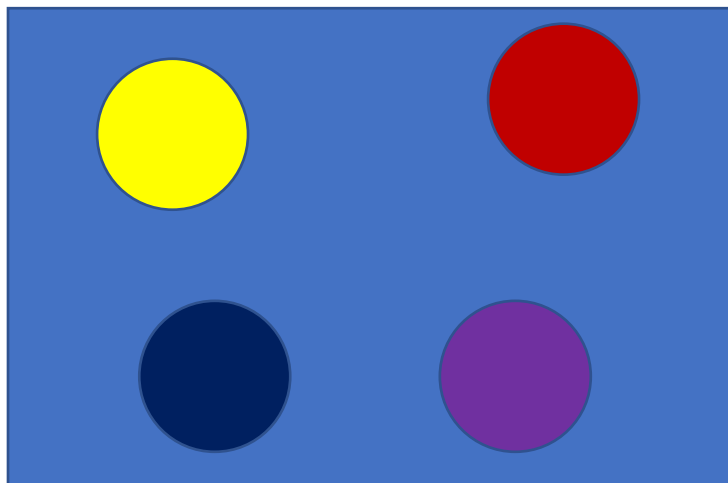
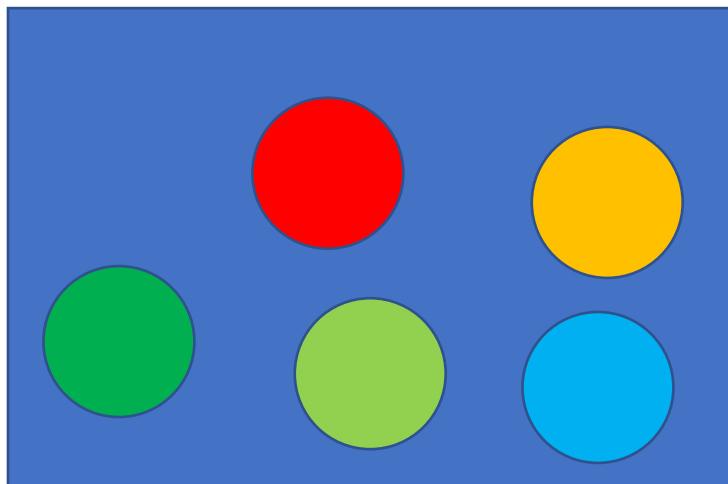
N+1 ~ 2N

こっちですね...



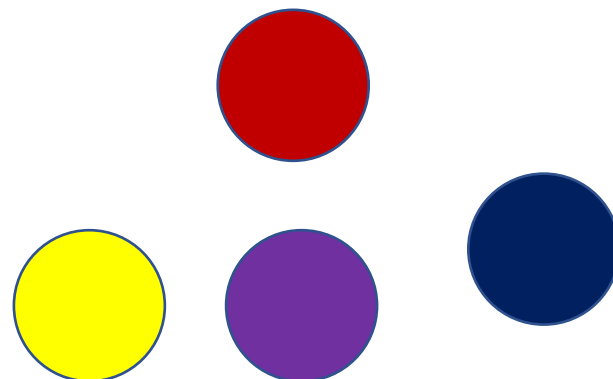
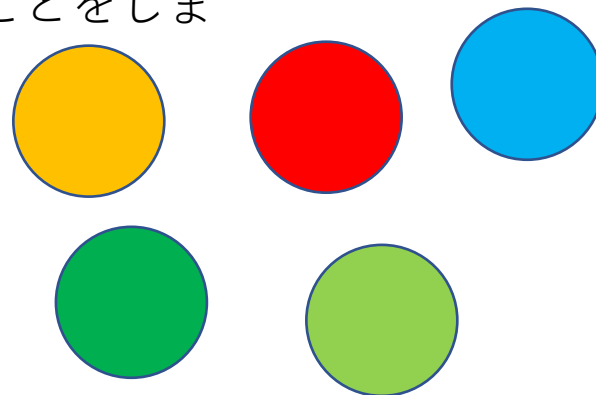
こういうことですね

1 ~ N



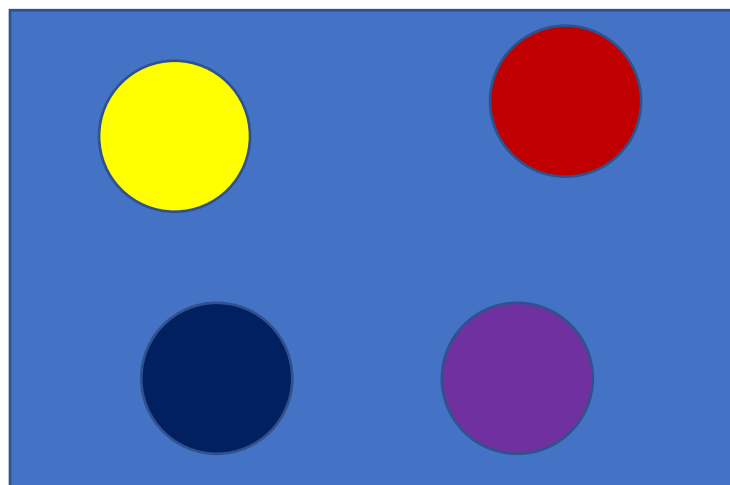
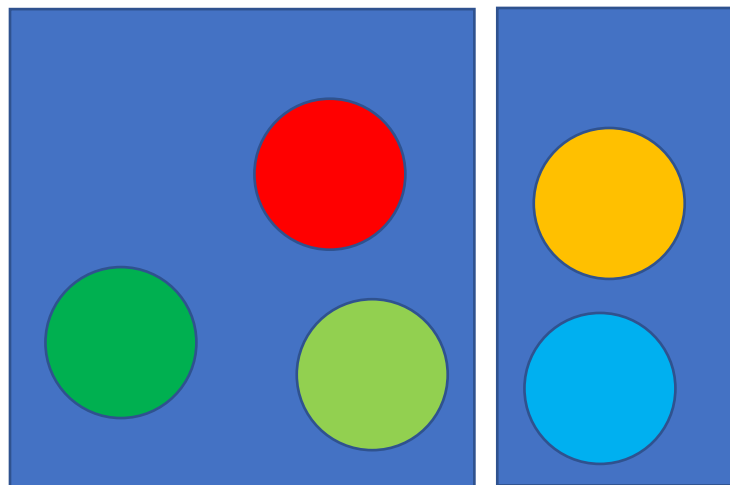
N+1 ~ 2N

同様のことをしま
した

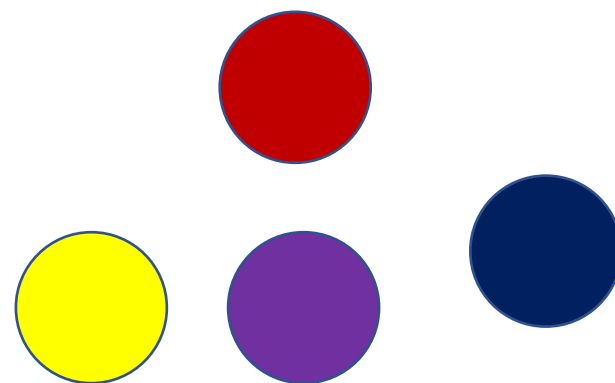
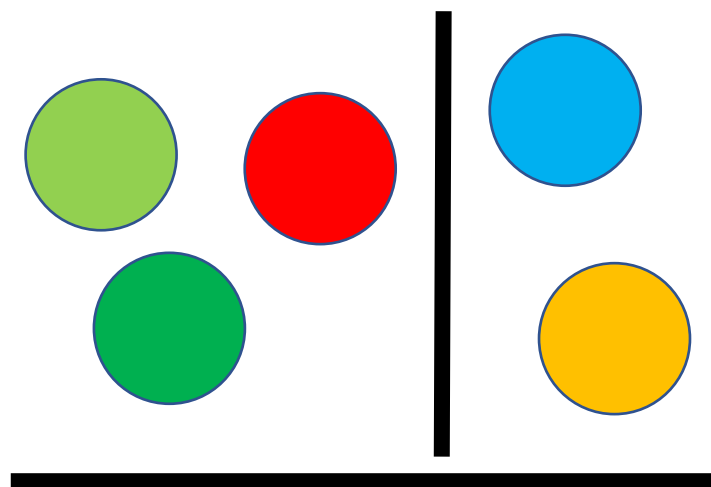


これは... 分割統治

1 ~ N

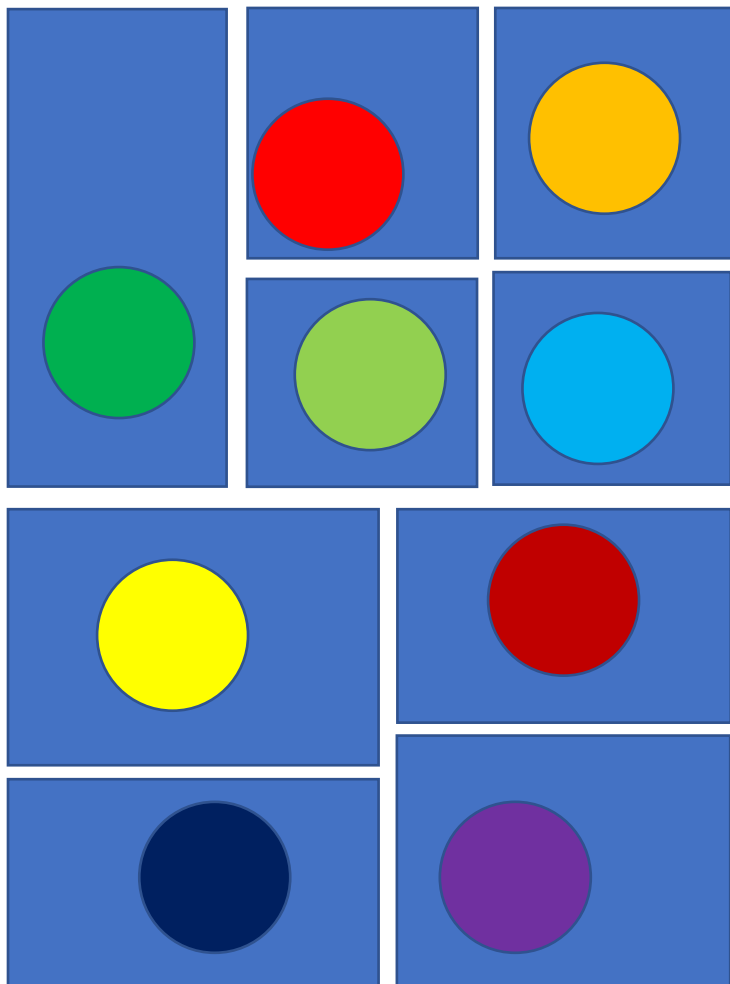


N+1 ~ 2N

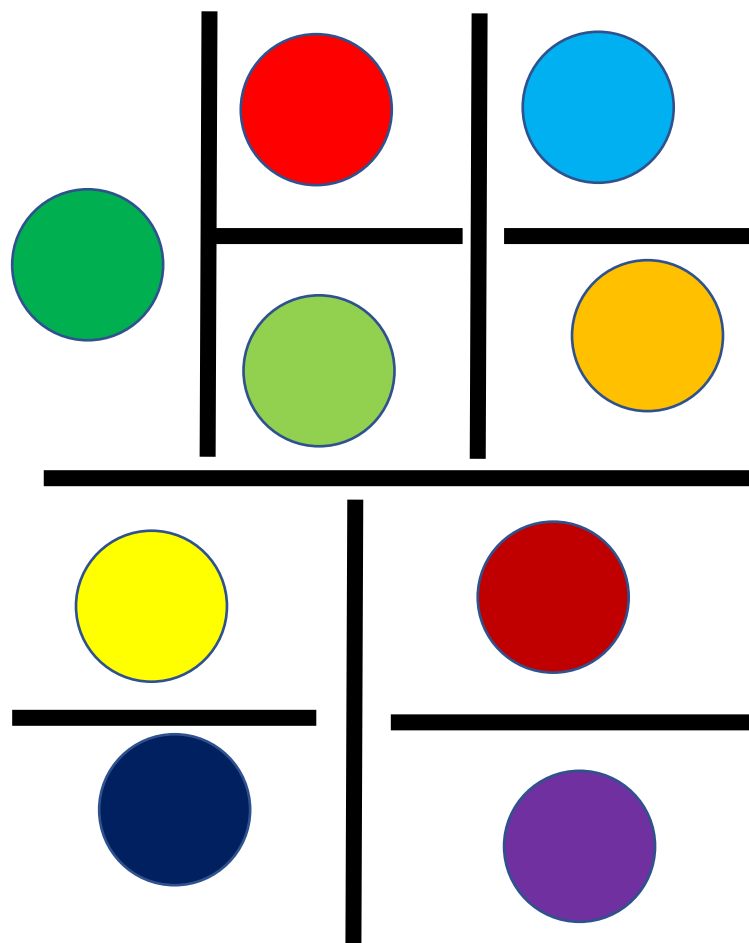


これは... 分割統治

1 ~ N

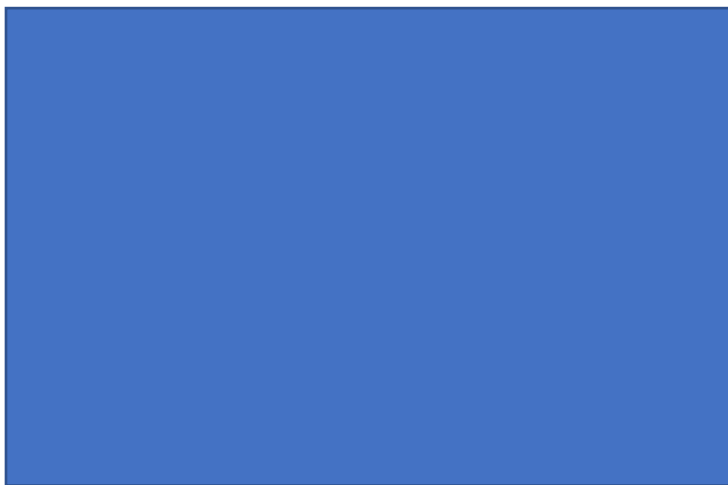


N+1 ~ 2N



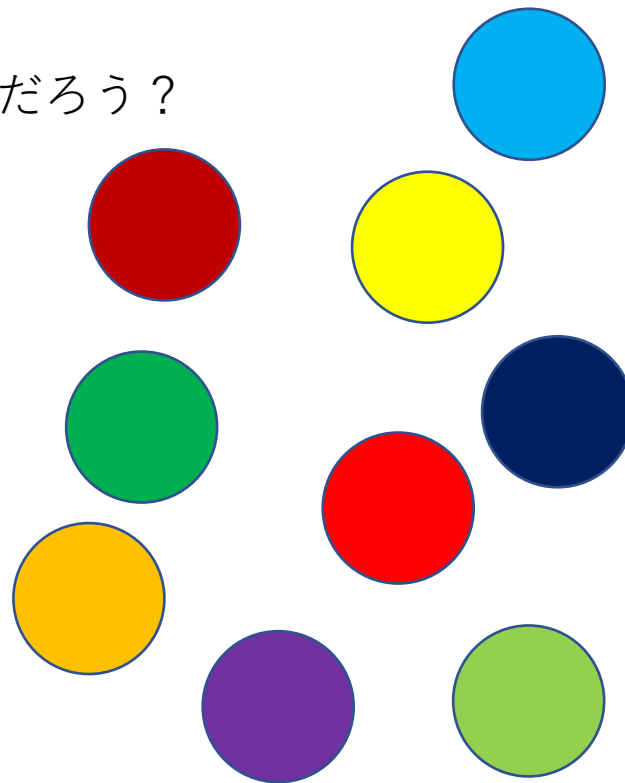
判断の仕方

機械



$N+1 \sim 2N$

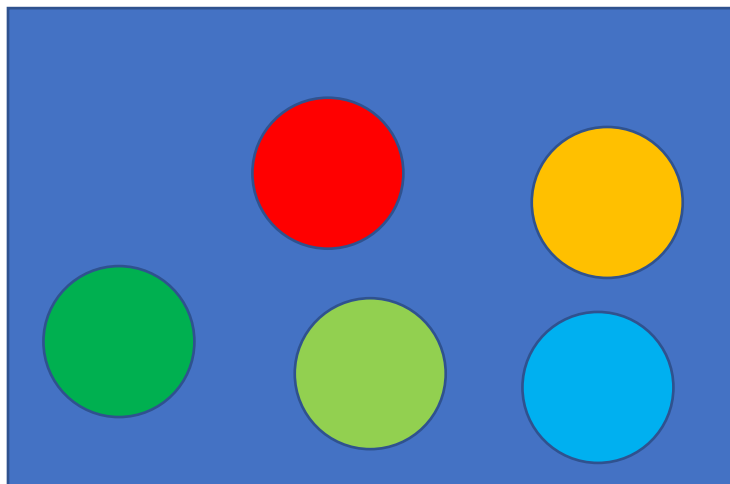
どっちだろう？



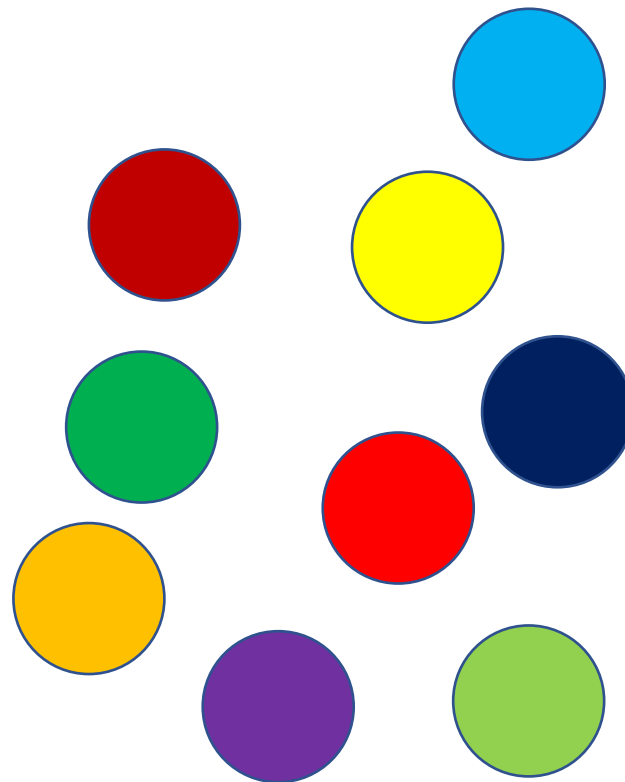
判断の仕方

5

機械



$N+1 \sim 2N$



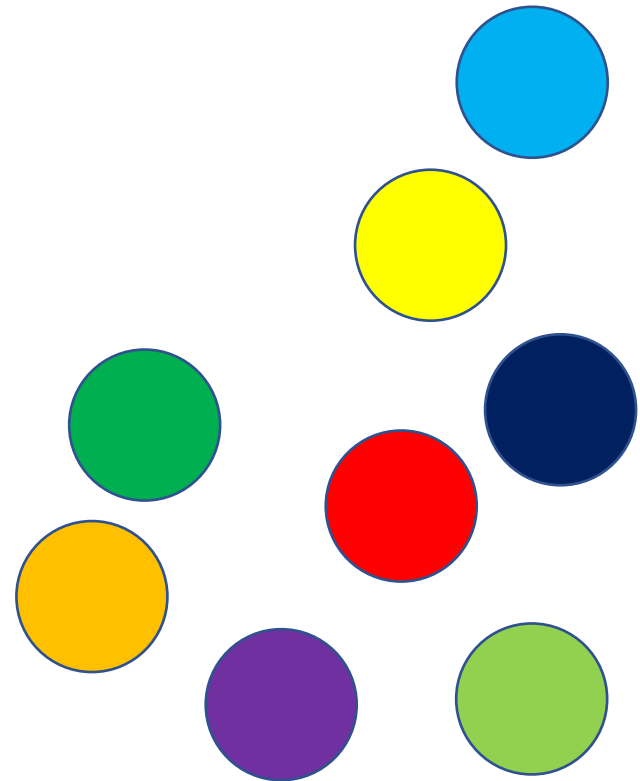
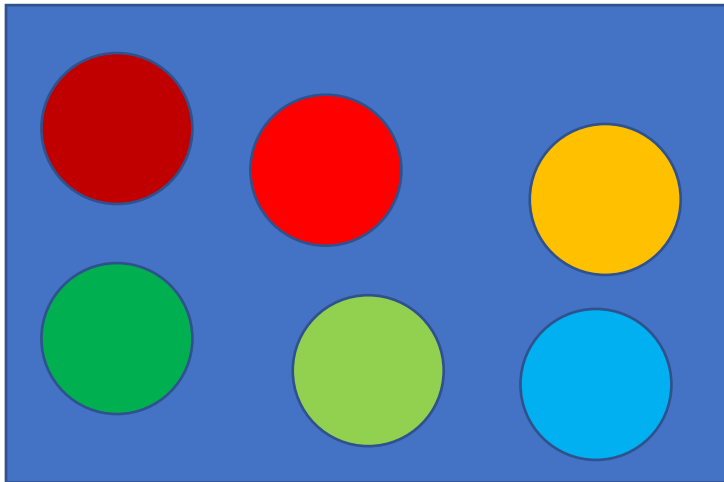
追加しておく

判断の仕方

6

機械

$N+1 \sim 2N$



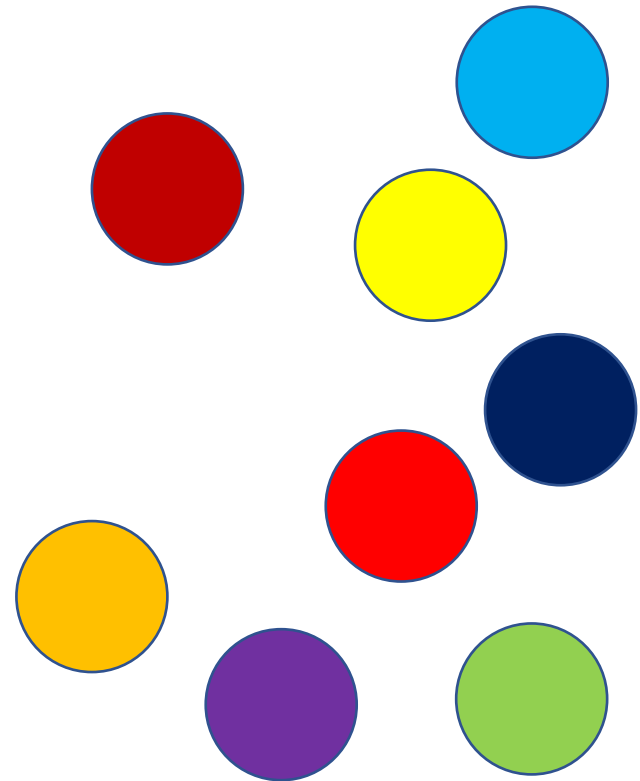
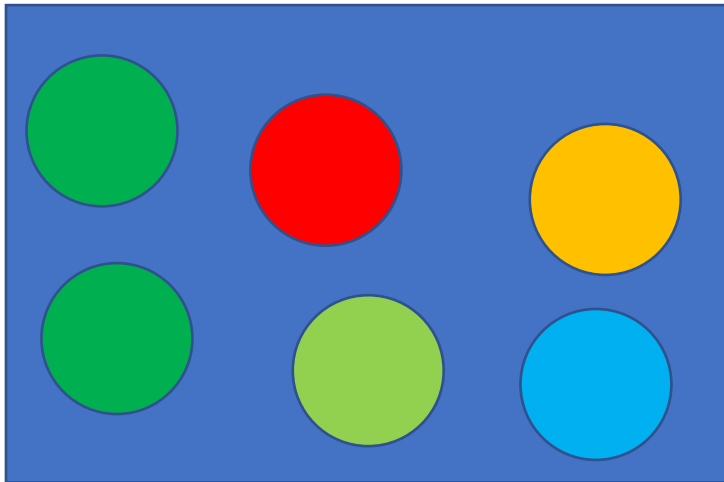
右から持ってくる
種類数が増えたら別の集合で
ある証拠

判断の仕方

5

機械

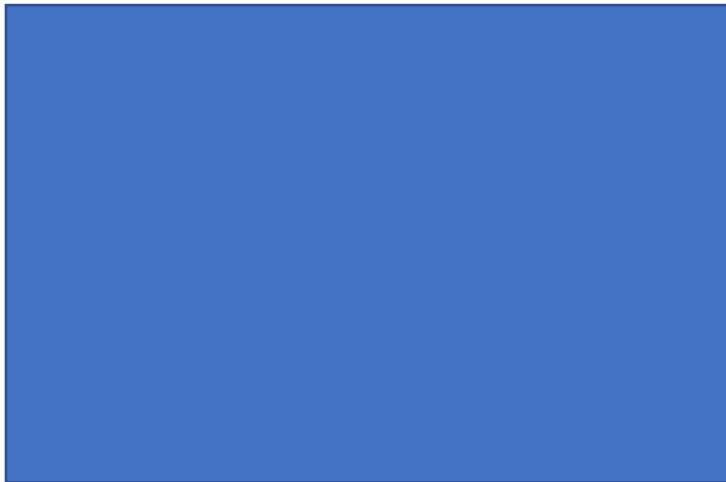
$N+1 \sim 2N$



右から持ってくる
種類数が増えなかったら同じ
集合である証拠

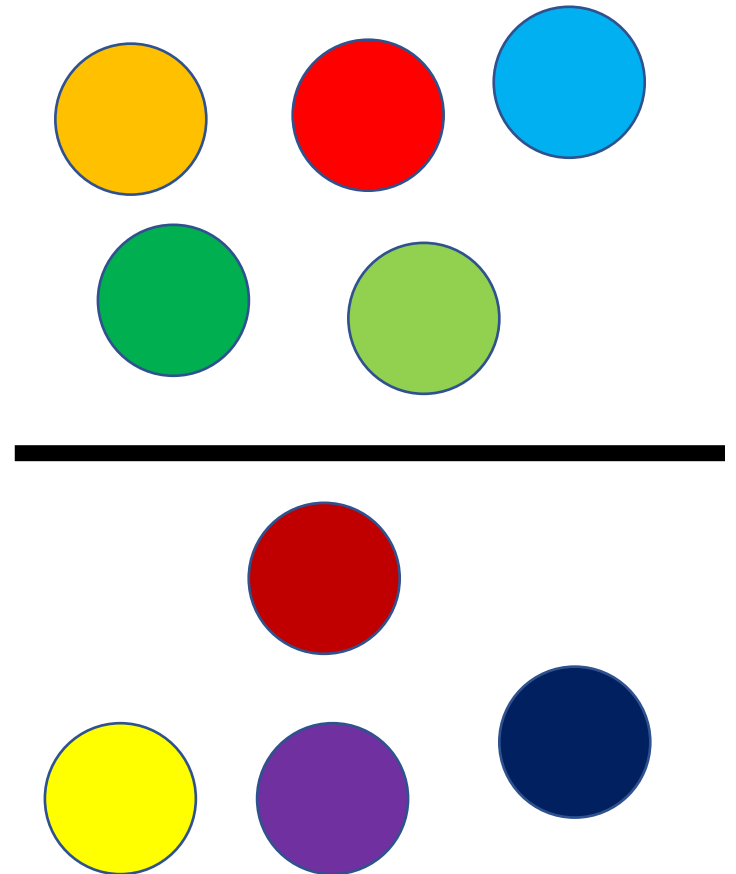
判断の仕方

機械



分割できました
次の人のために機械をきれい
にしておこう

$N+1 \sim 2N$



何回クエリを呼ぶか？

- 集合サイズが **1** になるまでに $\log N$ ステップかかる
 - それぞれに対し、
 - 左側の半分を装置に足す
 - 右側のそれぞれを足す、取り出す
 - 左側の足されたものを取り出す
-
- 各ステップ 左側合計 N 回、右側合計 $2N$ 回クエリを呼んでいるので、およそ $3 N \log N$ 回

制約

$N \leq 100$

$N \leq 15000$, 切片1~N は全部別の種類

$N \leq 15000$

$N \leq 38000$

$N \leq 39000$

$N \leq 40000$

$N \leq 41000$

$N \leq 42000$

$N \leq 43000$

グループ分けがない...

- どうしようかな
- 自分で見つけようか

2つの $\{1, \dots, N\}$ のグループに分類

- $4N$ 回あれば2つに分けられる
- 切片1から順に追加
- 追加した後、前の計測結果より増えてたらその種類は未登場
- そうでなければその種類は以前に登場済み
- 未登場と登場済みでそれぞれグループがつかれる

2つの $\{1, \dots, N\}$ のグループに分類

- $4N$ 回あれば2つに分けられる
- 一つ一つ順に追加するので $2N$ 回
- 後できれいにするので $2N$ 回
- この後も考慮すると、合計 $4N + 3N \log N < 70$ 万

制約

$N \leq 100$

$N \leq 15000$, 切片1~N は全部別の種類

$N \leq 15000$

$N \leq 38000$

$N \leq 39000$

$N \leq 40000$

$N \leq 41000$

$N \leq 42000$

$N \leq 43000$



方針

- あとは方針は同じ、ちまちま改善していこう
- **Output only** ではないけれども **Output only** 的な傾斜配点だと思って

着眼点 1

- いちいちきれいにする必要はあるか？
- ないよね
- 分けたい 2 つの分類の外は放置して良いし、分けたいところも in/out を変えるべきものだけ変えればよい
- 最初の $2N \rightarrow N*2$ にするところも終わった後きれいにする必要はない

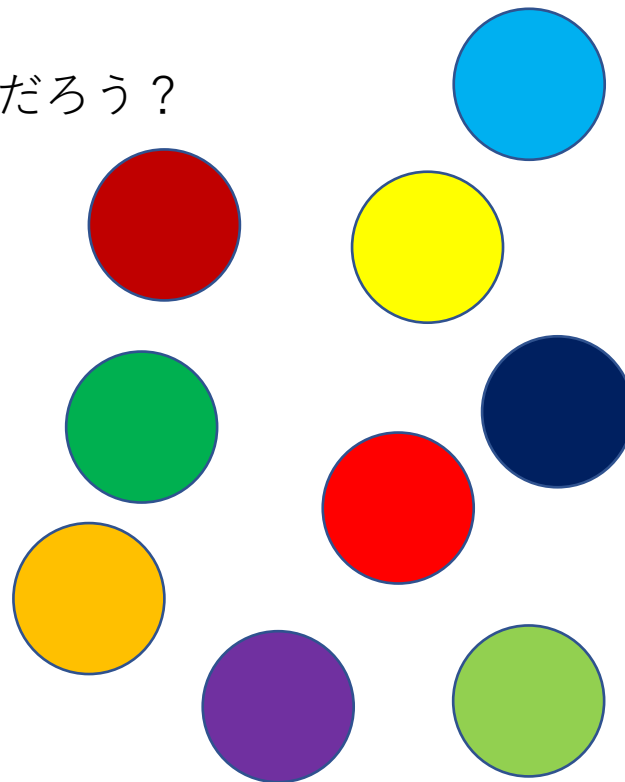
判断の仕方

機械



$N+1 \sim 2N$

どっちだろう？

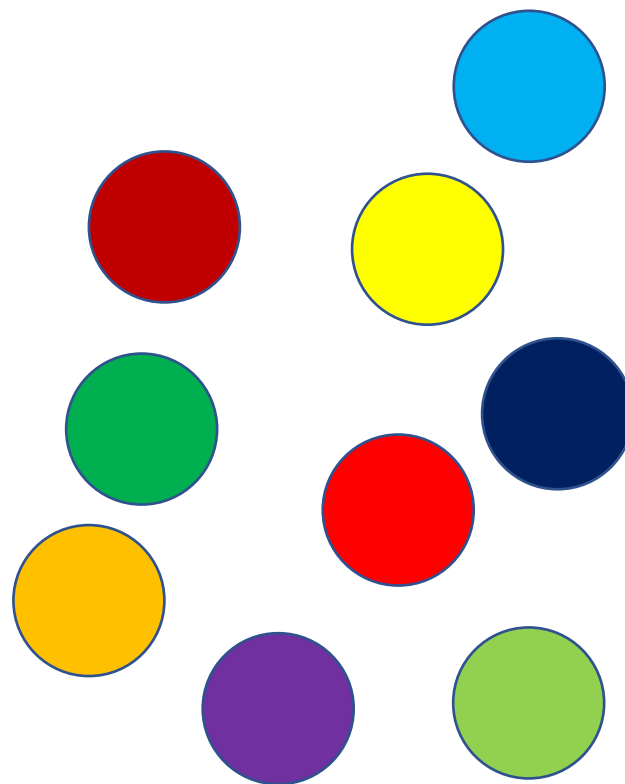
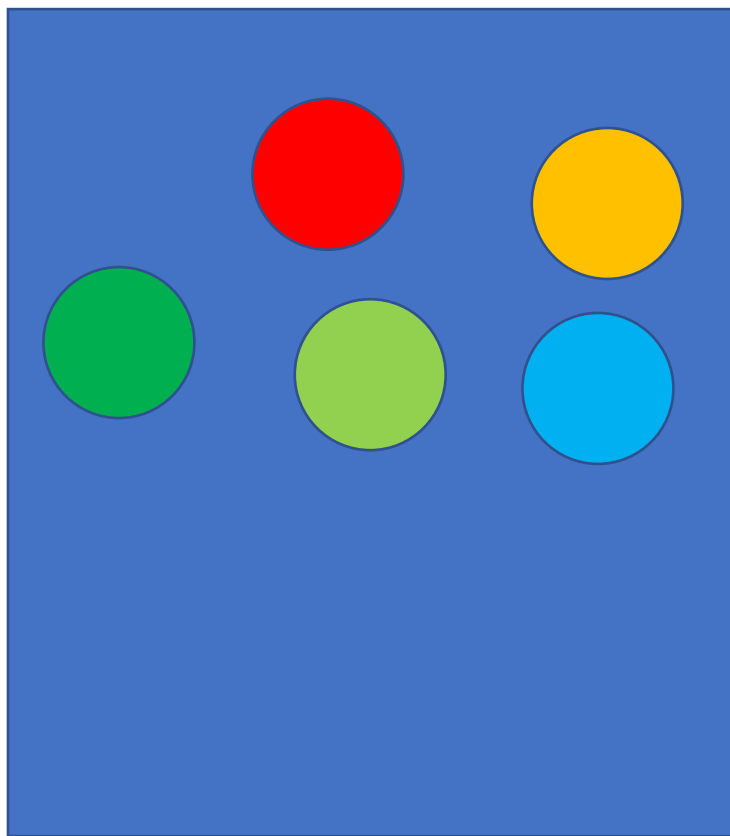


判断の仕方

5

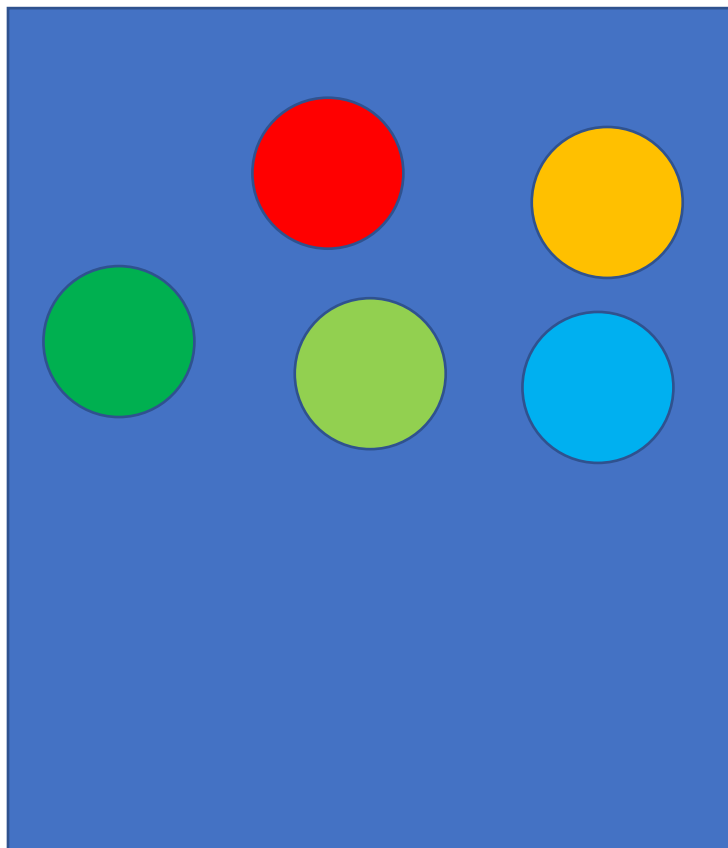
機械

$N+1 \sim 2N$

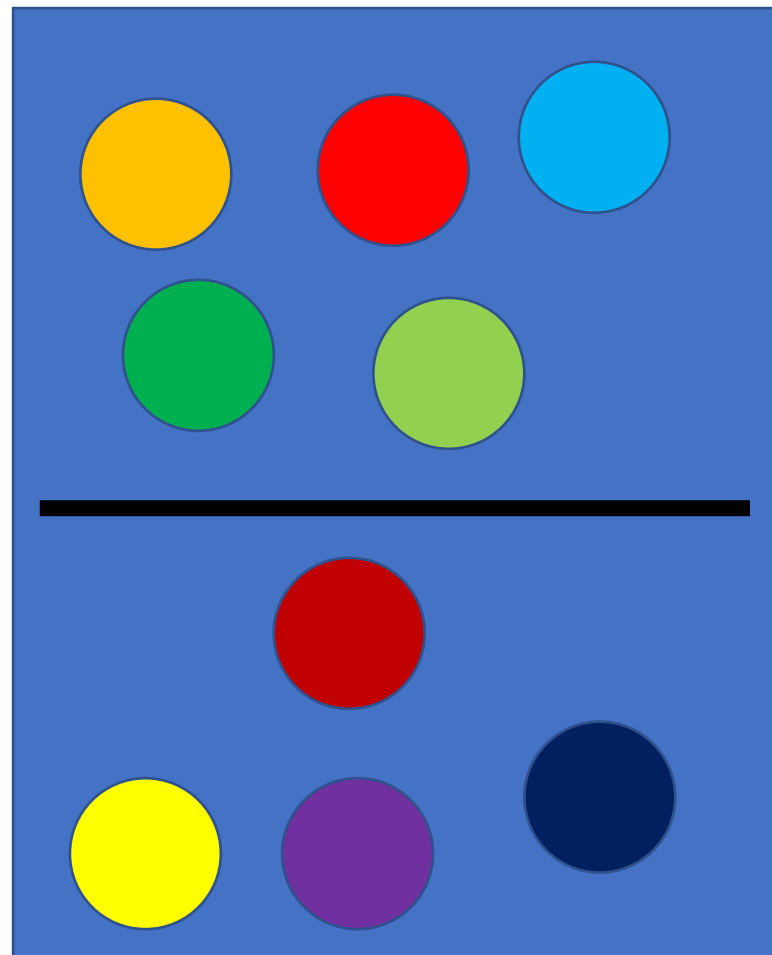


判断の仕方

機械

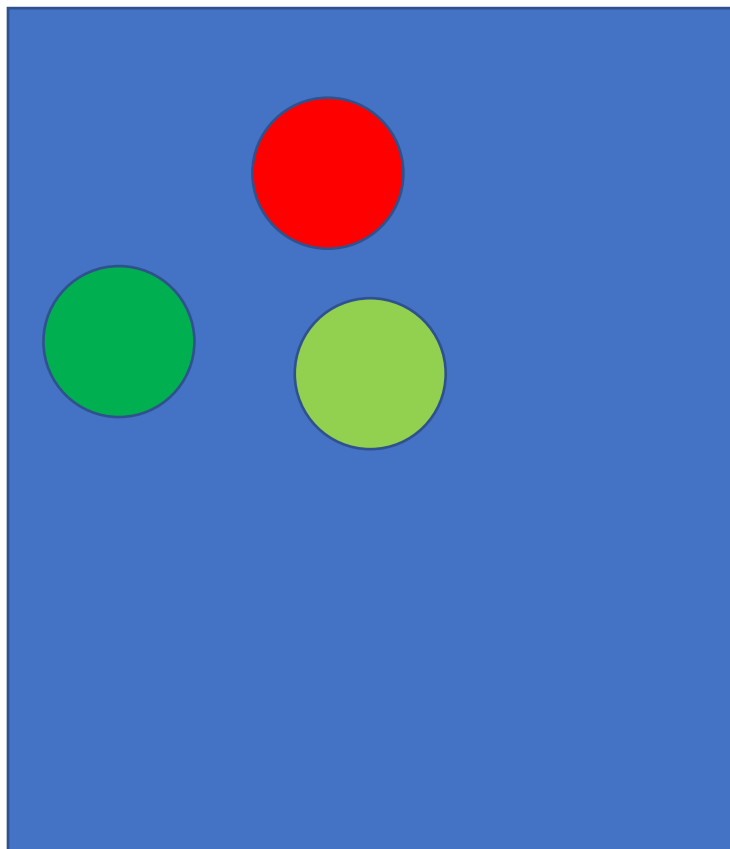


$N+1 \sim 2N$

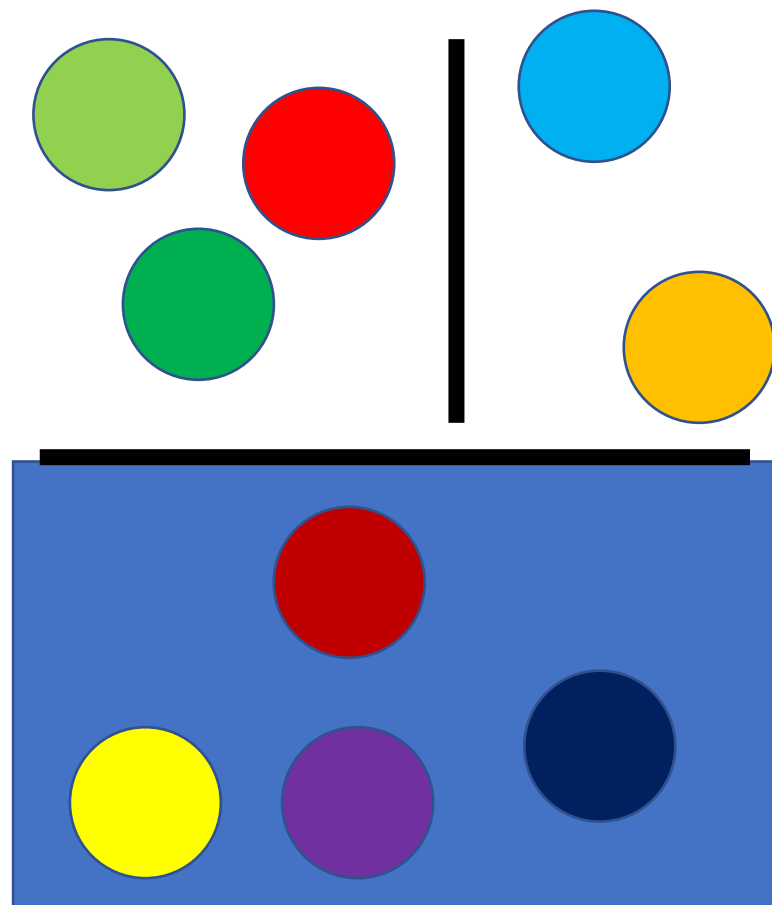


判断の仕方

機械

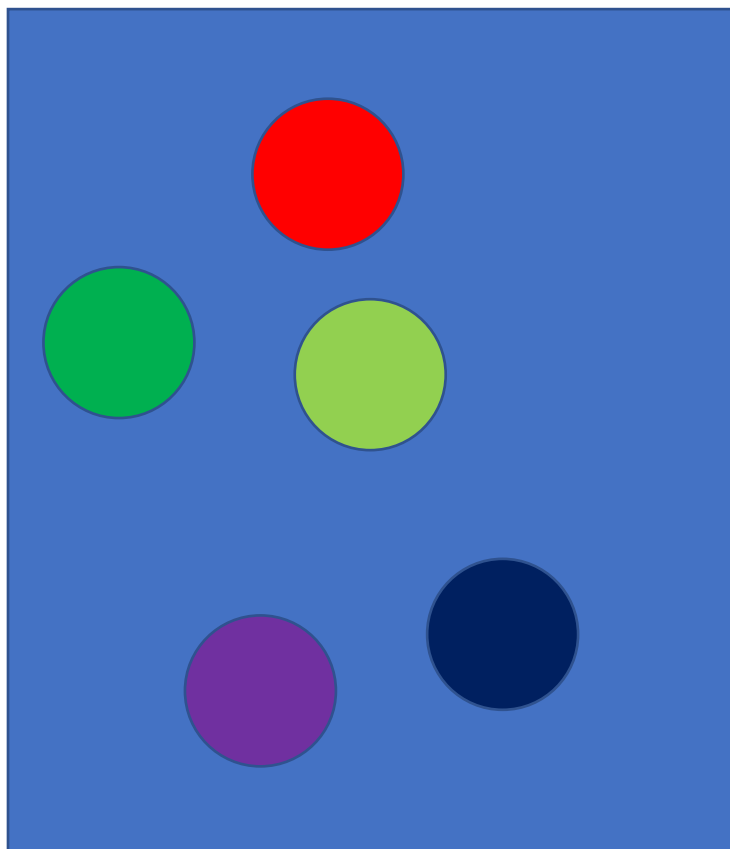


$N+1 \sim 2N$

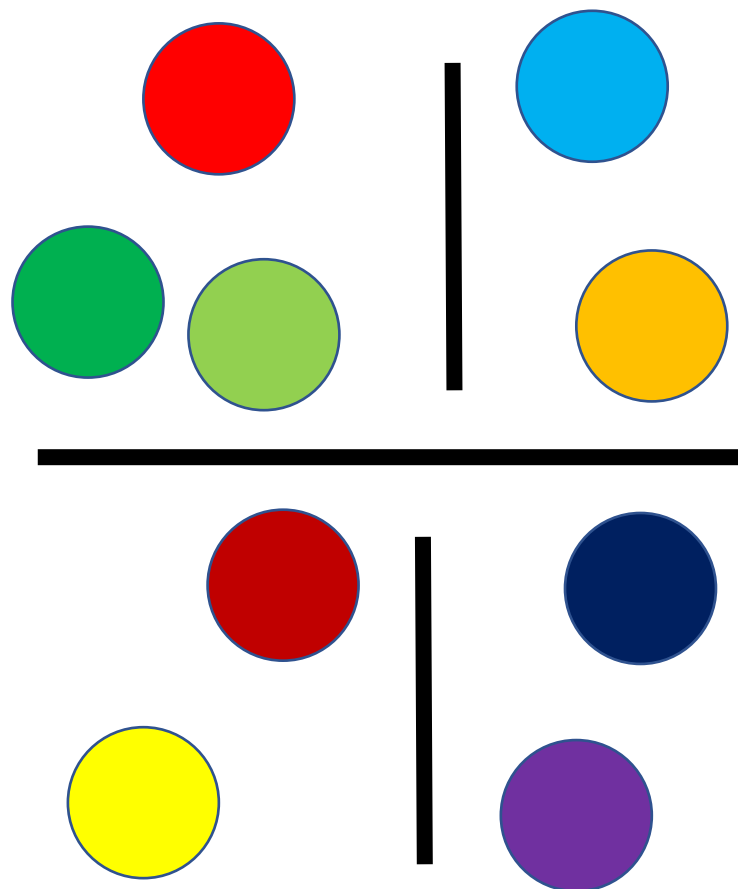


判断の仕方

機械



$N+1 \sim 2N$



着眼点 1

- すると、必要な回数はどのくらいになるか？
- 最初: $2N$ 回
- 分割統治: $\log N$ ステップ
- それぞれ $1 \sim N$: $0.5 N$ 回
- $N+1 \sim 2N$: N 回
- あわせて $2N + 1.5 N \log N$ 回
- ちょっと節約に成功

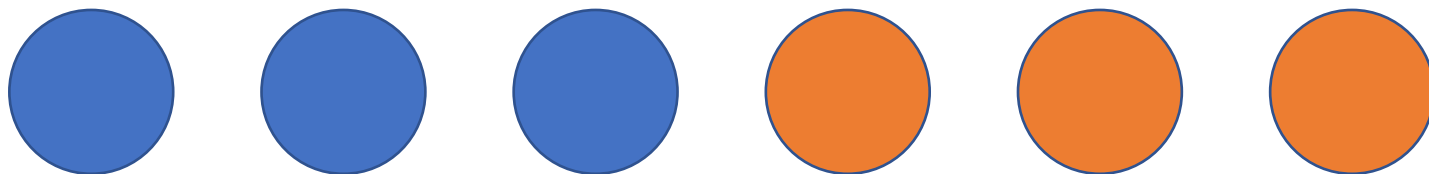


着眼点 2

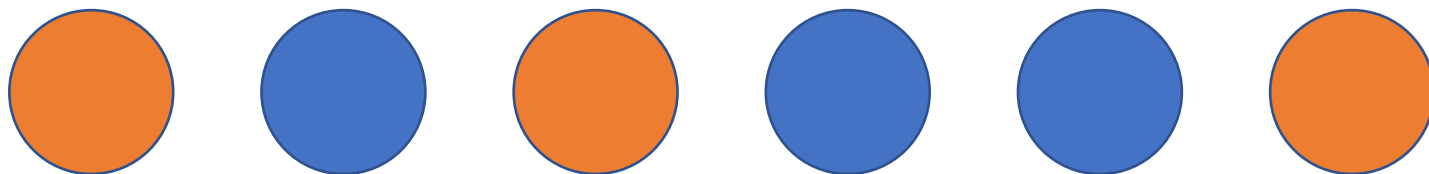
- 本当に $N+1 \sim 2N$ 全部を出し入れして確定する必要があるか？
- 実はいらない
- 最後の何個かは確定したりする
- ということか？

着眼点 2

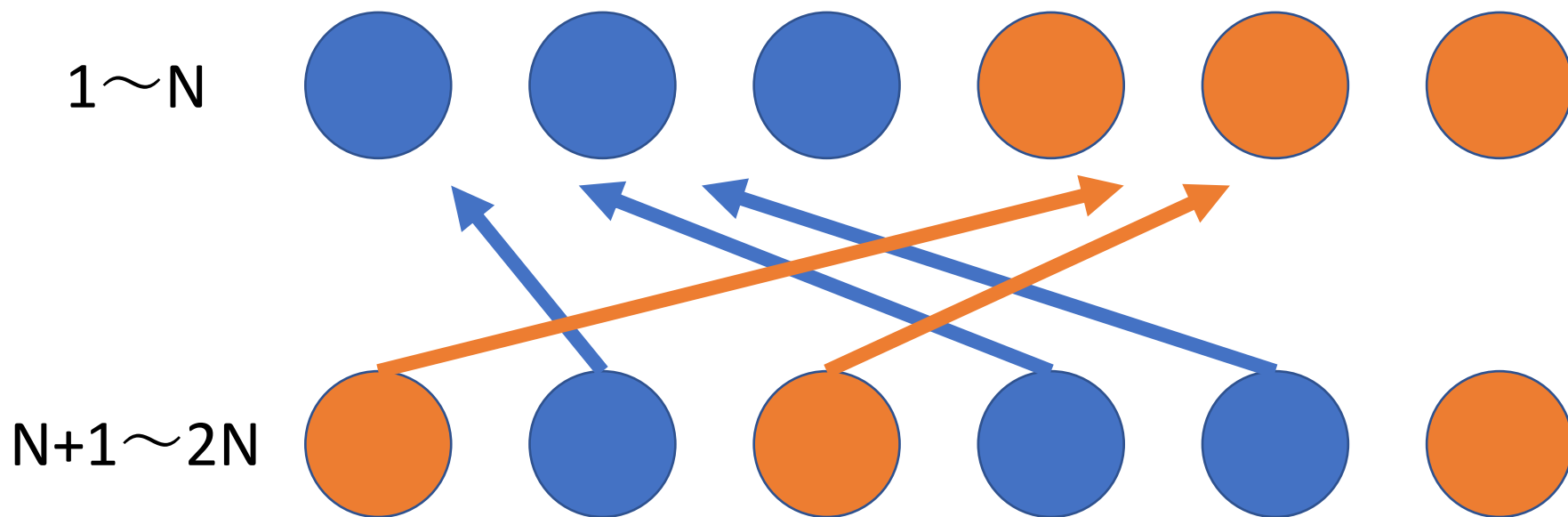
1~N



N+1~2N

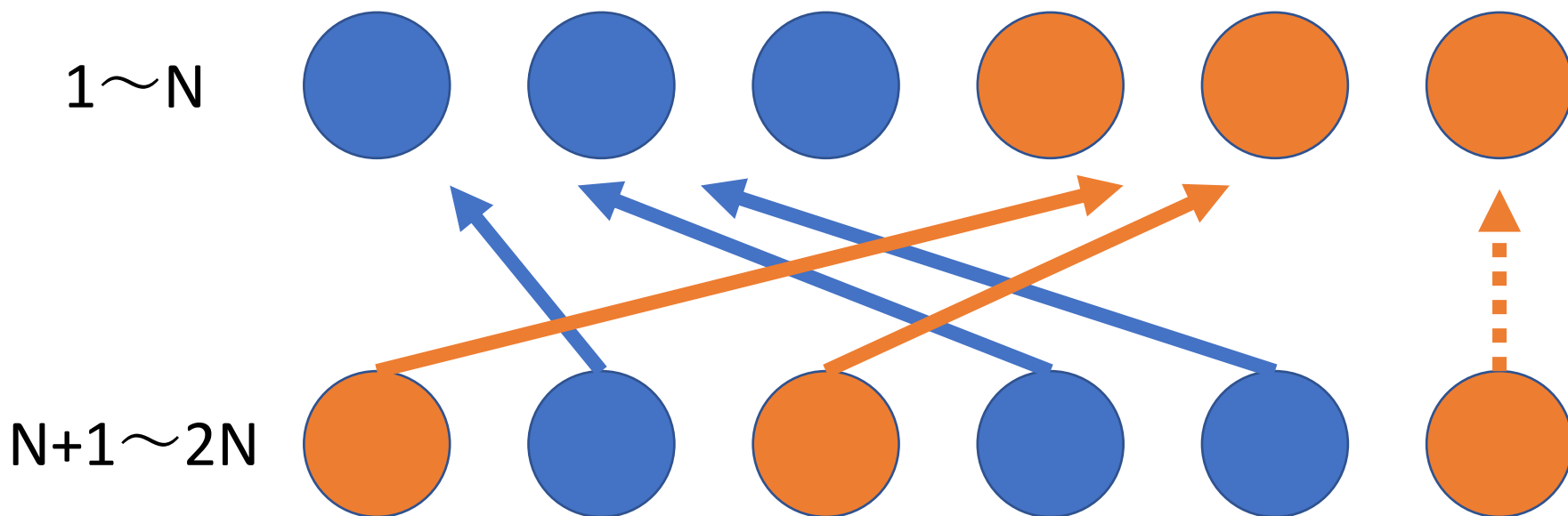


着眼点 2



5個真面目に決めてみた、6個目は...

着眼点 2



5個真面目に決めてみた、6個目は...
どうみても右側ですね...

着眼点 2

- ということで、各分割統治の再帰ステップのうち、最後の 1 個は追加すらする必要がないっぽい (運が良ければ 2 個以上いらなないかも)
- 再帰関数の呼ばれる回数は $2N$ 回くらい
- これで最低 $2N$ 回減ったな (やったぜ。)

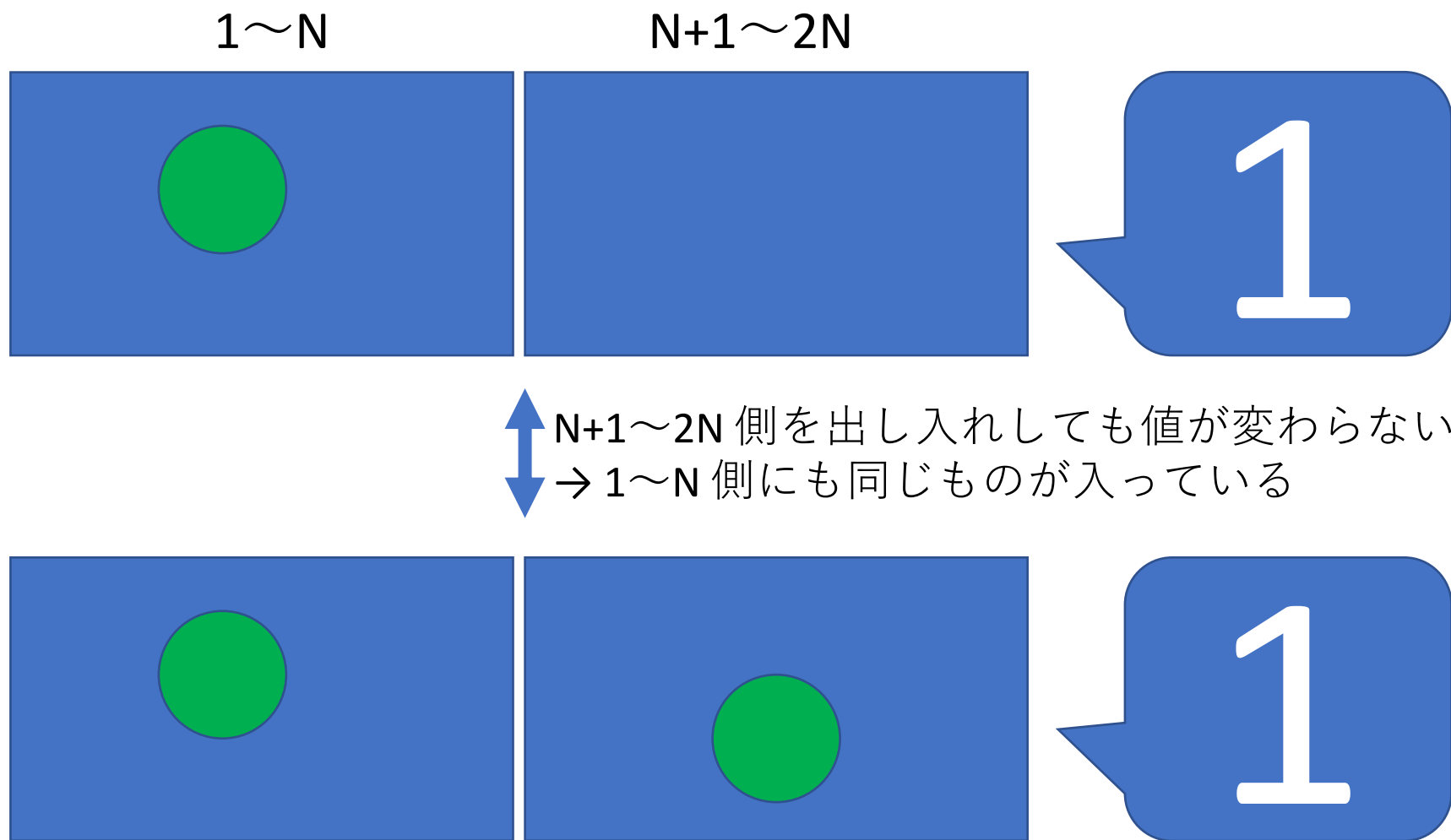
着眼点 2

- ちょっと待った
- 最後の 1 個が切り替わらないと、次そこを切り替える時増えるのか減るのかわからなくなりますか？

着眼点 2

- そもそもクエリの答えの増加と減少に何か意味はあるのか？

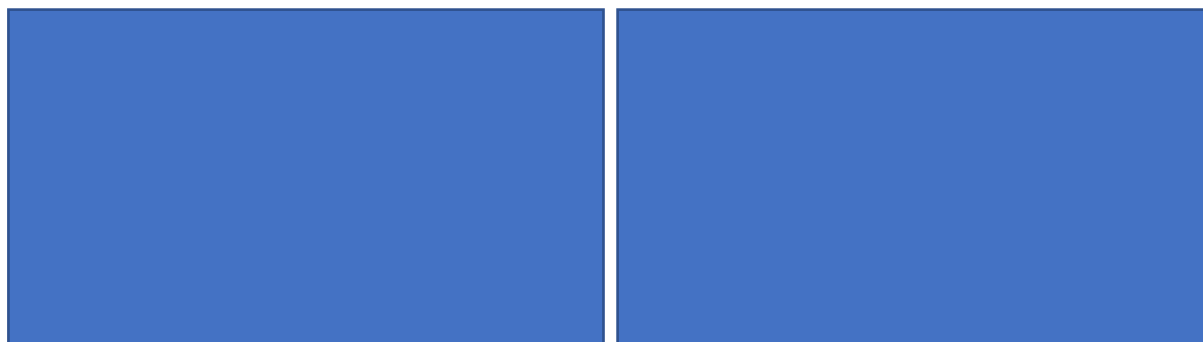
着眼点 2



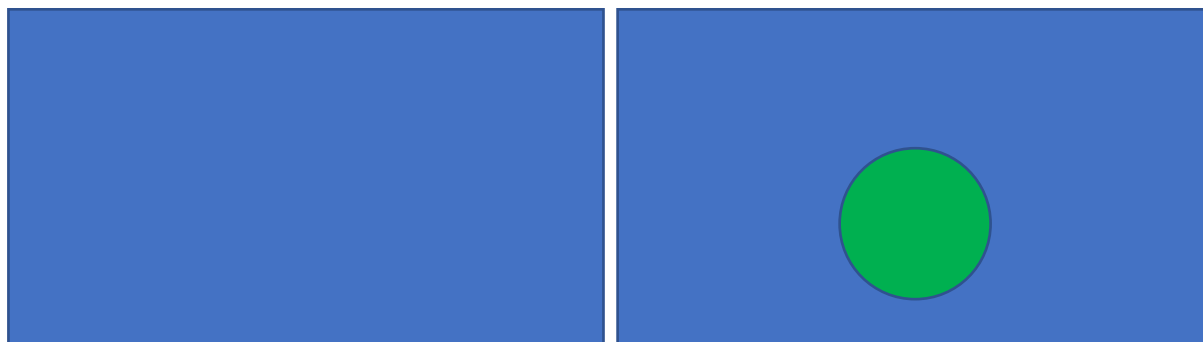
着眼点 2

1~N

N+1~2N



↑ N+1~2N 側を出し入れすると値が変わる
↓ → 1~N 側には同じものが入っていない



着眼点 2

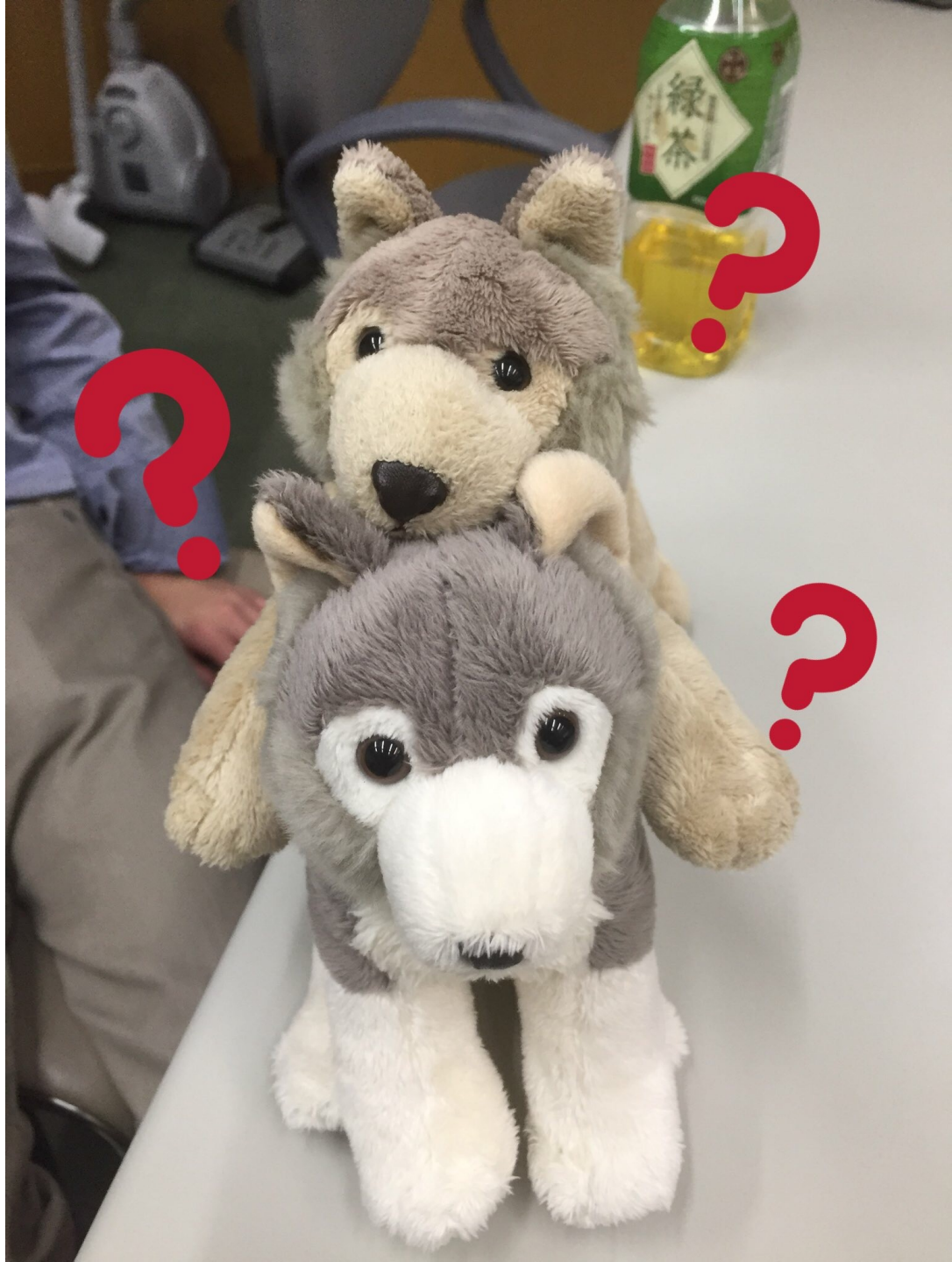
- そもそもクエリの答えの増加と減少に何か意味はあるのか？
- 増加か減少か？ではなく、直前と変化したかどうかには意味があった。
- じゃあ 1 個余ったものもそのまま放置しておいて問題ないね

着眼点 2: 運を良くしたい

- ~~パワーストーンを買きましょう~~
- 最悪ケース(1つも1個しかスキップできない)を回避するには、2つにグループ分けをしたあと `random_shuffle` するのが良い

Random Shuffle ってどうやるの

- `swap(a[rand()%N], a[rand()%N])`
を繰り返して擬似ランダムシャッフルをしていて楽しいか？



Random Shuffle

- `random_shuffle(a,a+N);` を
 - たぶん `<algorithm>` にある
- `for(int i=0;i<N;i++)swap(i,ra`
- いずれにせよ、 $O(N)$ でランダムな順列の一つを等確率で作れて最高

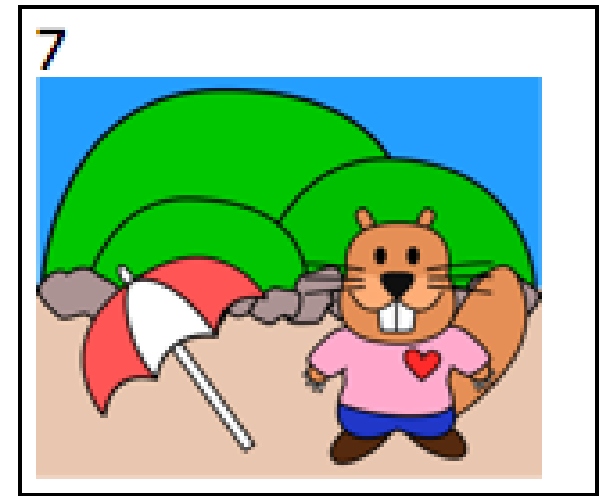


着眼点 3

- 今までは何も言っていなかったが、ステップ数・回数の計算では暗黙の了解で半分ずつに分けていた
- 本当にこれが最適なのか？

着眼点 3

- 分割統治パートを定式化しよう
- 1. $1 \sim N$ 側の候補を $p:1-p$ に分ける
- 2. p 側を全て反転
- 3. $N+1 \sim 2N$ 側を全て出し入れして、どちら側にあるかを定める
- 4. 再帰



着眼点 3

- N 種類を割り当てるのにかかるクエリ回数を $f(N) = tN \log N$ とすると、

$$f(N) = f(pN) + f((1-p)N) + (1+p)N$$

$$\iff tN \log N = tpN \log(pN) + t(1-p)N \log(pN) + (1+p)N$$

$$\iff 0 = tpN \log p + t(1-p)N \log(1-p) + (1+p)N$$

$$\iff t = \frac{-1-p}{p \log p + (1-p) \log(1-p)}$$

- t が最小となる p ってどこだろう？

着眼点 3

- 微分をしましょう

$$\left(\frac{-1-p}{p \log p + (1-p) \log(1-p)} \right)' = 0$$

$$\iff p \log p + (1-p) \log(1-p) - (1+p)(\log p - \log(1-p)) = 0$$

$$\iff 2 \log(1-p) - \log p = 0$$

$$\iff \frac{(1-p)^2}{p} = 1$$

$$\iff p = p^2 - 3p + 1$$

$$\iff p = \frac{3 - \sqrt{5}}{2} \simeq 0.382$$

着眼点 3

- ということで、 $p=0.38$ くらいのところで分けて小さい方を反転すれば回数が最適になることがわかる。
- ちなみに微分しなくても、**DP** で最小回数を達成するための分割方法を求めることができる
- 必要な回数は約 $1.44 N \log N$ になる

まとめ

- 着眼点 1: 一回一回きれいにせず、装置に入れるべきものは置きっぱなしにする
- 着眼点 2: 各再帰呼び出しで最後の1個は見なくてよい
 - 2α : random_shuffle すると減る (なくてもよい)
- 着眼点 3: 半分ではなく、0.38 くらいで分割
- 全部合わせると最悪ケースは何回？
- $N = 43000$ だと ... 999976回！